

spintent

SPANISH PARSE INTENTS

V0.99 2026-09-19*

©2026 by Pablo González L[†]

CTAN: <https://www.ctan.org/pkg/spintent>

 <https://github.com/pablgonz/spintent>

Resumen

El paquete **spintent** proporciona una serie de utilidades para docentes de educación primaria y secundaria que necesiten crear documentos PDF *accesibles* (*tagged* PDF) en español usando Lua[†] $\TeX$.

Índice

1 Descripción	1	3.3 El comando <code>\spmoney</code>	6
1.1 Carga del paquete	1	3.3.1 Llaves para <code>\spmoney</code>	7
1.2 El paquete <code>babel</code>	2	3.3.2 Monedas y alias para <code>\spmoney</code>	7
1.3 Fuentes tipográficas	2	3.4 El comando <code>\spshort</code>	7
1.4 El paquete <code>hyperref</code>	2	3.4.1 Llaves para <code>\spshort</code>	9
2 El comando <code>\setspintent</code>	3	3.4.2 Argumentos para <code>\spshort</code>	9
3 The big four	3	4 Utilidades generales	11
3.1 El comando <code>\spnum</code>	3	5 Utilidades para símbolos	11
3.1.1 Llaves para <code>\spnum</code>	3	6 Utilidades para números	13
3.2 El comando <code>\spunit</code>	4	7 Utilidades para geometría	17
3.2.1 Llaves para <code>\spunit</code>	4	8 Referencias	27
3.2.2 Argumentos para <code>\spunit</code>	4	9 Change history	28
3.2.3 El comando <code>\spnewunit</code>	6	10 Índice de la Documentación	29
3.2.4 El comando <code>\spunitalias</code>	6	11 Implementation	32
		12 Índice de la Implementación	148

Motivación y agradecimientos

Desde que desarrollé `scontents`[7] en 2019 y, más tarde, `enumext`[8] en 2024, me he ido interesando cada vez más por la accesibilidad en los documentos PDF. Al principio era un tema prácticamente desconocido para mí, pero a medida que fui aprendiendo sobre él comprendí la importancia que tiene, especialmente en el ámbito educativo.

El equipo del proyecto de $\TeX$ [19] ha realizado un trabajo extraordinario en este campo. Hoy es posible generar documentos etiquetados, validarlos con herramientas como `veraPDF`[20] o inspeccionar su estructura con `ngpdf`[21]. Sin embargo, siempre me quedó una duda: cuando un estudiante utiliza NVDA[11] junto con MathCAT[12], ¿realmente escucha lo que el o la docente quiso expresar?

Basta con generar un documento accesible siguiendo las recomendaciones de The LaTeX Tagged PDF Project y escucharlo con NVDA/MathCAT para darse cuenta de que, en muchos casos, la *verbalización* no coincide con lo que uno espera. Esto no resulta extraño: las reglas del español, y en particular el lenguaje matemático, dependen en gran medida del contexto en que se utilizan.

El objetivo de **spintent** nace precisamente de esta necesidad: proporcionar herramientas para que el o la docente pueda indicar explícitamente el contexto de determinadas expresiones matemáticas desde el propio código fuente de $\TeX$, de modo que el documento PDF resultante no solo sea *accesible*, sino que también produzca una *verbalización más natural* y adecuada para el estudiante al utilizar NVDA/MathCAT.

El nuevo estándar PDF 2.0[14, 18, 16], las especificaciones de WTPDF[15] y la guía más reciente de la PDF Association, «Best Practice Guide: Math in PDF»[17], coinciden en que el camino hacia una accesibilidad matemática de calidad pasa por el uso de MathML. En ese contexto, Lua[†] $\TeX$ ofrece una plataforma especialmente adecuada para incorporar esta tecnología.

*Este archivo describe la documentación para la versión v0.99 revisada por última vez el 2026-09-19.

[†]E-mail: «pablgonz@educarchile.cl».

Licencia y requerimientos

El paquete `spintent` carga y requiere el paquete `unicode-math`[1] y que se ejecute bajo Lua $\TeX$ por lo que es necesario contar con una distribución moderna de $\TeX$ como $\TeX$ Live 2026. Ha sido probado con las clases estándar proporcionadas por $\LaTeX$: `book`, `report`, `article` y `letter` en tamaños `10pt`, `11pt` y `12pt`.

Se otorga permiso para copiar, distribuir y/o modificar este paquete bajo los términos de $\LaTeX$ Project Public License (LPPL), versión 1.3 o posterior (<https://www.latex-project.org/lppl.txt>). El paquete tiene la condición de mantenido al estilo «*A Work in Progress*»¹.

- El requerimiento mínimo es $\LaTeX$ release 2026-11-01.

Este paquete solo se puede utilizar solo con Lua $\TeX$ y *tagged* PDF activo. Está presente en $\TeX$ Live y MiK $\TeX$, para instalarlo, utilice el gestor de paquetes de su distribución. Si desea una instalación manual, descargue `spintent.zip` y descomprímalo, luego ejecute `luatex spintent.ins`, mueva todos los archivos a las ubicaciones correspondientes y luego ejecute `mktexlsr`. Para generar la documentación, ejecute `arara spintent.dtx`.

```
spintent.sty  ⇒ TDS:tex/luatex/spintent/
spintent.lua  ⇒ TDS:tex/luatex/spintent/
spintent.pdf  ⇒ TDS:doc/luatex/spintent/
README.md    ⇒ TDS:doc/luatex/spintent/
spintent.dtx  ⇒ TDS:source/luatex/spintent/
spintent.ins  ⇒ TDS:source/luatex/spintent/
```

1 Descripción

El nombre `spintent` puede entenderse como «*Spanish Parse Intent*» o también como «*School Parse Intent*». Ambas interpretaciones reflejan el objetivo del paquete y el público para el que fue diseñado: docentes de educación primaria y secundaria.

Está pensado para crear «*apuntes*», «*hojas de ejercicios*», «*clases escritas*» enfocadas en el estudiante. Solo un docente sabe cual es el nivel de aprendizaje del estudiante. El paquete intenta sobrescribir muchas de las reglas dictadas por la heurística de NVDA/MathCAT teniendo esto presente.

- El paquete `spintent` usa y abusa del atributo `:intent` de MathML por medio del comando `\MathMLintent` definido en `latex-lab-mathintent`[30], un encapsulado de `\luamml_attribute:een` proveída por `luamml`[2]. ¡Gracias Marcel!

1.1 Carga del paquete

El paquete `spintent` trabaja solo con Lua $\TeX$ y *tagged* PDF activo usando `tagging=on` dentro del comando `\DocumentMetadata` al inicio del archivo. Las llaves («*keys*») aceptadas por `\DocumentMetadata` y en general para el código de *tagged* PDF están documentadas en los paquetes y `tagpdf`[29] `documentmetadata-support`[30], es deber del usuario revisar la documentación asociada a ellos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage{spintent}
```

1.2 El paquete babel

El paquete `babel-spanish`[10] está marcado como «parcialmente compatible» para *tagged* PDF, pero, es absolutamente necesario para la creación de documentos en español. En caso de encontrar problemas de compatibilidad con otros paquetes se puede desactivar usando:

```
\usepackage[spanish,shorthands=off]{babel}
```

Muchas de las «shorthands» proveídas por `babel-spanish` hoy no son necesarias, más allá de `\sptext` para las abreviaturas y ordinales el resto son solo atajos de teclas para escritura que hoy en día no se ocupan. El paquete `spintent` provee el comando `\spshort` (§3.4) que cubre estos casos, de esta manera podemos cargar `babel`[9] con todas sus funcionalidades sin problemas.

¹A *Work in Progress* es un álbum en vivo de Neil Peart (* 1952–† 2020), baterista y principal letrista de Rush. El nombre no pretende formar parte de la terminología de la LPPL; simplemente describe el estado permanente de este paquete mientras sigo aprendiendo sobre *accesibilidad* en PDF.

1.3 Fuentes tipográficas

Como `spintent` es un paquete Lua $\TeX$ debemos distinguir entre las fuentes tipográficas OpenType/TrueType versus las fuentes de 8bit estándar usadas por $\TeX$ para el *modo texto* y el *modo matemático*. Para evitar problemas con *tagged* PDF preferiremos no combinar fuentes OpenType/TrueType con fuentes 8bit y trabajaremos solo con fuentes OpenType/TrueType que tengan soporte para matemáticas.

La carga de las fuentes tipográficas para Lua $\TeX$ se maneja por medio del paquete `fontspec`[3], si no se especifica ninguna se usarán las fuentes Latin Modern para el *modo texto* y Latin Modern Math para el *modo matemático*.

El tipo de fuente tipográfica escogida es preferencia de cada usuario, en lo personal utilizo las fuente Libertinus para el *modo texto* por medio del paquete `libertinus-otf`[4] y la fuente Erewhon-Math para el modo matemático cargada por el paquete `fourier-otf`[5]. Adicionalmente cargo la fuente Source Code Pro para el texto tipo «máquina de escribir» por medio del paquete `sourcecodepro`[6].

```
% \DocumentMetadata{...}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\usepackage{spintent}
```

1.4 El paquete hyperref

El paquete `hyperref`[23] es «casi obligatorio» cuando se crean documentos *tagged* PDF, se encarga de las referencias cruzadas `\label` y `\ref`, las notas al pie `\footnote`, los hipervínculos `\href` y muchos otros elementos como marcadores, índices y metadatos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\setmathfont{Erewhon-Math}[range={cal,bfcal},StylisticSet=1]
\usepackage{spintent, hyperref}
\hypersetup
{
  colorlinks      = true,
  pdfstartview    = FitH,
  pdfdisplaydoctitle = true,
  pdftitle        = {Test para el paquete spintent},
  pdfinfo         =
  {
    Title   = {Test},
    Subject = {spintent},
  },
}
```

2 El comando \setspintent

`\setspintent` `\setspintent{⟨comando⟩}{⟨key-uno = valor, key-dos = valor, key-tres = valor, ...⟩}`

El comando `\setspintent` establece las llaves (*⟨keys⟩*) de forma global para los comandos provistos por `spintent`. Puede utilizarse tanto en el preámbulo como en el cuerpo del documento tantas veces como se desee. Las llaves (*⟨keys⟩*) definidas en el *argumento opcional* [*⟨key = val⟩*] de los comandos tienen prioridad sobre las establecidas por este, anulando las configuraciones globales de `\setspintent`.

Muchas de las utilidades que provee el paquete `spintent` implementan el sistema [*⟨key = val⟩*] y están pensadas para ser utilizadas en la generación de «*hojas de ejercicios*» o «*clases escritas*». Se recomienda usar `\setspintent` y establecerlas de manera global para cada documento, para mantener la coherencia en la verbalización de NVDA/MathCAT a lo largo del mismo.

El sistema [*⟨key = val⟩*] utilizado por `spintent` se implementa usando el módulo `l3keys` de `expl3`[24]. Por diseño, las llaves que reciben un *valor* después del signo '=' NO aceptan *valores vacíos* y es obligatorio pasar uno válido; las llaves documentadas como *⟨valor prohibido⟩* no reciben *ningun valor*. En caso de omitir un *valor* válido o pasar un *valor vacío* a una llave que acepte *valor* o pasar un *valor* a una llave del tipo *⟨valor prohibido⟩* se devolverá un "error" y se detendrá la compilación del documento.

3 The big four

El paquete `spintent` provee cuatro grandes comandos («*the big four*»²): `\spnum` para el manejo de números, `\spunit` para el manejo de unidades, `\spmoney` para el manejo de dinero y `\spshort` para el manejo de abreviaturas, acrónimos y números ordinales.

- También se incluyen las utilidades `\spnewunit` y `\spunitalias` asociadas a al comando `\spunit`.

3.1 El comando \spnum

`\spnum` `\spnum{⟨número⟩}` `\spnum[⟨key = val⟩]{⟨número,número;número⟩}`
`\spnum[⟨key = val⟩]{⟨+número⟩}` `\spnum[⟨key = val⟩]{⟨número.número;número⟩}`
`\spnum[⟨key = val⟩]{⟨-número⟩}` `\spnum[⟨key = val⟩]{⟨número;número⟩}`
`\spnum[⟨key = val⟩]{⟨número,número⟩}` `\spnum[⟨key = val⟩]{⟨número e exponente⟩}`
`\spnum[⟨key = val⟩]{⟨número.número⟩}` `\spnum[⟨key = val⟩]{⟨número E exponente⟩}`

El comando `\spnum` trabaja en *modo matemático* y recibe el *argumento obligatorio* {*⟨número⟩*}, donde la «*parte entera*» puede iniciar con los signos '+' o '-', la «*parte decimal finita*» **debe** iniciar con una *coma* ',' o con un *punto* '.', y la «*parte decimal infinita (periódica)*» **debe** iniciar con un *punto* y *coma* ';'.

Si no se proporciona el *argumento opcional* [*⟨key = val⟩*] y se ejecuta `\$ \spnum{+23} \$` se imprimirá '+23' en la salida PDF y NVDA/MathCAT verbalizará «*más veintitrés*», si se ejecuta `\$ \spnum{3,14} \$` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce*», si se ejecuta `\$ \spnum{3,1;4} \$` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma uno con cuatro periódico*».

Además, el *argumento obligatorio* {*⟨número⟩*} admite al final de este un sufijo de «*notación científica*». Este se indica mediante las letras 'e' o 'E', acompañadas de los signos opcionales '+' o '-' seguido por los dígitos del exponente. Por ejemplo `\$ \spnum{3,14e3} \$` imprimirá '3,14 · 10³' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- La *coma* ',' o el *punto* '.' son *separadores decimales* y NO de millares, por lo tanto solo pueden aparecer una «única vez» dentro del {*⟨argumento⟩*}. El {*⟨argumento⟩*} puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.1.1 Llaves para \spnum

`cifras-sep = {⟨true | false⟩}`

default: *true*

Establece el *espaciado* utilizado para agrupar los dígitos en bloques de tres, tanto en la parte entera como en la parte decimal finita. Si se establece en *true* se siguen las normas de la RAE y se utiliza un *espacio fino* '\,' entre las cifras; si se establece en *false*, se respetan los espacios matemáticos introducidos en el argumento. Por ejemplo `\$ \spnum{100000} \$` imprimirá '100 000' en la salida PDF y `\$ \spnum[cifras-sep=false]{10\,00\,00} \$` imprimirá '10 00 00' en la salida PDF.

`read-sign = {⟨terse | clear⟩}`

default: *terse*

Establece la *forma* en la que NVDA/MathCAT verbalizará los signos '+' o '-'. Si se establece en *terse*, NVDA/MathCAT verbalizará «*más*» o «*menos*» **antes** de leer el número; si se establece en *clear*, NVDA/MathCAT verbalizará «*positivo*» o «*negativo*» **después** de leer el número. Por ejemplo `\$ \spnum{-23} \$` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*menos veintitrés*» y `\$ \spnum[read-sign=clear]{-23} \$` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés negativo*».

`read-period-sep = {⟨coma | punto⟩}`

default: *coma*

²Los «*cuatro grandes*» del Thrash Metal: Metallica, Megadeth, Slayer y Anthrax, que me acompañan mientras desarrollo `spintent`.

Establece la *forma* en la que NVDA/MathCAT verbalizará e imprimirá el signo de separación ‘;’ de la «*parte decimal infinita (periódica)*» cuando no se tiene la «*parte decimal finita*». Si se establece en *coma*, se imprimirá ‘,’ en la salida PDF y NVDA/MathCAT verbalizará «*coma*»; si se establece en *punto*, se imprimirá ‘.’ en la salida PDF y NVDA/MathCAT verbalizará «*punto*». Por ejemplo `\spnum{3;4}` imprimirá ‘3,4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma cuatro periódico*» y `\spnum[read-period-sep=punto]{3;4}` imprimirá ‘3.4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres punto cuatro periódico*»

- Esta llave es necesaria SOLO cuando se utilizan «*decimales periódicos puros*», es decir, cuando no se ha incluido en el `{(argumento)}` una *coma* ‘,’ o un *punto* ‘.’ antes del signo de separación ‘;’ periódica.

`notation-sym = {(times | cdot)}`

default: *cdot*

Establece el *símbolo* que se utilizará para imprimir la «*notación científica*» al usar ‘e’ o ‘E’. Si se establece en *times* imprimirá ‘x’ en la salida PDF y NVDA/MathCAT verbalizará «*por*»; si se establece en *cdot* imprimirá ‘.’ en la salida PDF y NVDA/MathCAT verbalizará «*por*». Por ejemplo `\spnum{3,14e3}` imprimirá ‘3,14 · 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*» y `\spnum[notation-sym=times]{3,14e3}` imprimirá ‘3,14 × 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- Decir «*notación científica*» aquí es un poco exagerado, en realidad es «*multiplicación por potencias de 10*», no se verifica si la entrada cumple o no con la condición de estar escrita en «*notación científica*».

3.2 El comando \spunit

<code>\spunit{(unidad)}</code>	<code>\spunit[key = val]{(número unidad*unidad)}</code>
<code>\spunit[key = val]{(unidad)}</code>	<code>\spunit[key = val]{(número unidad*unidad/unidad)}</code>
<code>\spunit[key = val]{(número unidad)}</code>	<code>\spunit[key = val]{(número unidad*unidad/unidad*unidad)}</code>

El comando `\spunit` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{(unidad)}`, el cual puede contener una «*parte numérica*» opcional la cual se procesa internamente por el comando `\spnum` al inicio del `{(argumento)}` seguido de una `{(unidad)}` o un `{(alias)}` valido descritos en §3.2.2 o una `{(unidad)}` creada por el comando `\spnewunit` (§3.2.3) o un `{(alias)}` creado por el comando `\spunitalias` (§3.2.4).

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `\spunit{23 m}` se imprimirá ‘23 m’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros*».

La multiplicación entre las `{(unidades)}` se realiza mediante un *asterisco* ‘*’ y las potencias de estas se indican con un *circunflejo seguido del exponente entre llaves* ‘^`{(exponente)}`’. Por ejemplo si se ejecuta `\spunit{23 m*s^{2}}` se imprimirá ‘23 m s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros segundos al cuadrado*».

Se pueden usar *fracciones lineales* con las `{(unidades)}` utilizando una «*única barra*» ‘/’ para la separación entre las `{(unidades)}` del numerador y el denominador, teniendo presente que el denominador NO puede contener números de ningún tipo. Por ejemplo si se ejecuta `\spunit{23 m/s^{2}}` se imprimirá ‘23 m/s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros por segundos al cuadrado*».

- Solo imprimiremos *fracciones lineales*, si se requiere otra forma fraccionaria para explicaciones y otros usos se debe usar el comando por separado. El `{(argumento)}` puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo, NO puede contener más de una «*única barra*» ‘/’ y NO puede contener números en el denominador. Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.2.1 Llaves para \spunit

El comando `\spunit` posee las *meta-keys* *cifras-sep*, *read-period-sep* y *notation-sym* pasadas al comando `\spnum` (§3.1.1).

`print-cdot = {true | false}`

default: *false*

Establece si se *imprime* `\cdot` ‘·’ o un *espacio fino* ‘\,’ para la multiplicación de `{(unidades)}`. Si se establece en *true*, se imprimirá ‘·’ entre las `{(unidades)}` en la salida PDF; si se establece en *false* se imprimirá un *espacio fino* ‘\,’ entre las `{(unidades)}` en la salida PDF.

`read-cdot = {true | false}`

default: *false*

Establece la *forma* en la que NVDA/MathCAT verbalizará la multiplicación de `{(unidades)}`. Si se establece en *true*, NVDA/MathCAT verbalizará «*por*» entre las `{(unidades)}`; si se establece en *false*, NVDA/MathCAT no verbalizará nada entre las `{(unidades)}`.

`read-slash = {por | partido-por}`

default: *por*

Establece la *forma* en la que NVDA/MathCAT verbalizará la «*barra*» ‘/’ de separación entre el numerador y el denominador. Si se establece en *por*, NVDA/MathCAT verbalizará «*por*»; si se establece en *partido-por*, NVDA/MathCAT verbalizará «*partido por*».

3.2.2 Argumentos para \spunit

Resumen de `{(unidades)}` y `{(alias)}` soportados por `\spunit`.

Entrada	Alias soportados	Nombre unidad	Salida	NVDA/MathCAT
m	metro, metros	Metro	m	«metro»
g	gramo, gramos	Gramo	g	«gramo»
s	segundo, segundos	Segundo (Tiempo)	s	«segundo (Tiempo)»
l	litro, l-litro	Litro (minúscula)	l	«litro (minúscula)»
h	hora, horas	Hora	h	«hora»
d	dia, dias	Día	d	«día»
A	amperio, ampere	Amperio	A	«amperio»
V	voltio, volt	Voltio	V	«voltio»
W	vatio, watt	Vatio	W	«vatio»
J	julio, joule	Julio	J	«julio»
N	newton	Newton	N	«newton»
Pa	pascal	Pascal	Pa	«pascal»
Hz	hercio, hertz	Hercio	Hz	«hercio»
Ω	ohmio, ohm, Omega	Ohmio	Ω	«Ohmio»
F	faradio	Faradio	F	«Faradio»
C	culombio	Culombio	C	«Culombio»
K	kelvin, Kelvin	Kelvin	K	«Kelvin»
°C	celsius, degC, °C	Grados Celsius	°C	«Grados Celsius»
°F	fahrenheit, degF, °F	Grados Fahrenheit	°F	«Grados Fahrenheit»
cal	caloria, calorías	Caloría	cal	«Caloría»
b	bit	Bit	b	«Bit»
B	byte, bytes	Byte	B	«Byte»
in	pulgada, pulgadas	Pulgada	in	«Pulgada»
ft	pie, pies	Pie	ft	«Pie»
yd	yarda, yardas	Yarda	yd	«Yarda»
mi	milla, millas	Milla	mi	«Milla»
lb	libra, libras	Libra	lb	«Libra»
oz	onza, onzas	Onza	oz	«Onza»
gal	galon, galones	Galón	gal	«Galón»
Å	angstrom	Angstrom	Å	«Angstrom»
μ	micro	Micro (Prefijo)	μ	«Micro (Prefijo)»
Ω	mho	Mho / Siemens	Ω	«Mho / Siemens»
'	minmin	Minuto (Arco)	'	«Minuto (Arco)»
"	segseg	Segundo (Arco)	"	«Segundo (Arco)»
u	u-masa	Unidad masa atómica	u	«Unidad masa atómica»
Unidades directas (sin alias):				
L	-	Litro (Mayúscula)	L	«Litro (Mayúscula)»
ℓ	-	Litro (Cursiva)	ℓ	«Litro (Cursiva)»
w	-	Vatio (Minúscula)	w	«Vatio (Minúscula)»
°K	-	Grado Kelvin	°K	«Grado Kelvin»
Bq	-	Becquerel	Bq	«Becquerel»
Da	-	Dalton	Da	«Dalton»
Gy	-	Gray	Gy	«Gray»
S	-	Siemens	S	«Siemens»
Sv	-	Sievert	Sv	«Sievert»
T	-	Tesla	T	«Tesla»
Wb	-	Weber	Wb	«Weber»
atm	-	Atmósfera	atm	«Atmósfera»
au	-	Unidad astronómica	au	«Unidad astronómica»
bar	-	Bar	bar	«Bar»
cd	-	Candela	cd	«Candela»
dB	-	Decibelio	dB	«Decibelio»
dyn	-	Dina	dyn	«Dina»
eV	-	Electronvoltio	eV	«Electronvoltio»
erg	-	Ergio	erg	«Ergio»
ha	-	Hectárea	ha	«Hectárea»
kat	-	Katal	kat	«Katal»
kg	-	Kilogramo	kg	«Kilogramo»
km	-	Kilómetro	km	«Kilómetro»
lm	-	Lumen	lm	«Lumen»

Continúa en la siguiente página...

Continuación de la tabla 1				
Entrada	Alias soportados	Nombre unidad	Salida	MathCAT
lx	-	Lux	lx	«Lux»
cm	-	Centímetro	cm	«Centímetro»
min	-	Minuto (Tiempo)	min	«Minuto (Tiempo)»
mm	-	Milímetro	mm	«Milímetro»
mol	-	Mol	mol	«Mol»
pc	-	Pársec	pc	«Pársec»
rad	-	Radián	rad	«Radián»
rpm	-	Rev. por minuto	rpm	«Rev. por minuto»
sr	-	Estereorradián	sr	«Estereorradián»
t	-	Tonelada	t	«Tonelada»
a	-	Año / Área (are)	a	«Año / Área (are)»
y	-	Año (Year)	y	«Año (Year)»
°	-	Grado (Arco)	°	«Grado (Arco)»

3.2.3 El comando \spnewunit

\spnewunit

\spnewunit{<nueva unidad>}{<verbalización NVDA>}

El comando `\spnewunit` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: `{<nueva unidad>}` que establece lo que se imprimirá en la salida PDF y `{<verbalización NVDA>}` que establece como NVDA/MathCAT verbalizará la `{<nueva unidad>}`.

La `{<nueva unidad>}` no debe estar registrada como `{<unidad>}` o `{<alias>}` para `\spunit` descritos en §3.2.2. Si la `{<nueva unidad>}` ya está registrada se devolverá un “error” y se detendrá la compilación del documento.

La declaración de la `{<nueva unidad>}` es «global» y se almacena en el módulo `spintent.lua` como una `{<unidad>}` valida para pasar como `{<argumento>}` a `\spunit`. Por ejemplo: `\spnewunit{px}{píxeles}` creará la `{<nueva unidad>}` ‘px’ y cuando se utilice `\spunit{100 px}` se imprimirá ‘100 px’ en la salida PDF y NVDA/MathCAT verbalizará «cien píxeles».

3.2.4 El comando \spunitalias

\spunitalias

\spunitalias{<nuevo alias>}{<expresión de unidades>}

El comando `\spunitalias` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: `{<nuevo alias>}` que establece es el nuevo `{<alias>}` de `{<argumento>}` para `\spunit` y `{<expresión de unidades>}` que puede contener multiplicación o división de `{<unidades>}` ya registradas que se imprimirán en la salida PDF.

El `{<nuevo alias>}` no debe estar registrado como `{<unidad>}` o `{<alias>}` para `\spunit` descritos en §3.2.2 o estar definido como `{<unidad>}` por medio del comando `\spnewunit`. Si el `{<nuevo alias>}` ya está registrado se devolverá un “error” y se detendrá la compilación del documento.

La declaración de un `{<nuevo alias>}` es «global» y se almacena en el módulo `spintent.lua` como una `{<alias>}` valido para pasar como `{<argumento>}` a `\spunit`. Por ejemplo: `\spunitalias{caudal}{m^3/s}` creará el `{<alias>}` ‘caudal’ y cuando se utilice `\spunit{50 caudal}` se imprimirá ‘50 m³/s’ en la salida PDF y NVDA/MathCAT verbalizará «cincuenta metros al cubo por segundos».

3.3 El comando \spmoney

\spmoney

\spmoney{<cifra>}\spmoney{<cifra moneda>}\spmoney[<key = val>]{<cifra moneda>}

El comando `\spmoney` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<cifra>}` que contiene la «cifra numérica» positiva o negativa la se cual procesa internamente por el comando `\spnum` al inicio del `{<argumento>}` acompañada de un «símbolo de moneda» o «alias de moneda» opcional descritos en §3.3.2.

Si no se proporciona el *argumento opcional* `[<key = val>]` y se ejecuta `\spmoney{500}` se imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos».

El comportamiento de `\spmoney` depende de como se pase el *argumento obligatorio* que tiene prioridad por sobre los valores por defecto. Por ejemplo: `\spmoney{500 dolares}` imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos dolares» aunque la moneda por defecto sean pesos.

Si se pasa un «alias de moneda» del tipo ISO (código de divisas) como CLP o USD (mayúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país y se ajusta la salida PDF al estandar ISO. Por ejemplo: `\spmoney{500 CLP}` imprimirá ‘CLP500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos chilenos».

Si se pasa un «alias de moneda» del tipo ISO (código de divisas) como clp o usd (minúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país, pero, NO se modifica la salida PDF imprimiendo el

«*símbolo de moneda*» correspondiente. Por ejemplo: `\$ \spmoney{500 c\p}` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos chilenos»

Pasar de manera directa en el *argumento obligatorio* un «*símbolo de moneda*» como '\$', '€', '£' o un «*alias de moneda*» como peso, euro o libra produce la salida PDF y verbalización estándar esperada para NVDA/MathCAT.

- La posición del «*símbolo de moneda*» a la izquierda o a la derecha de la «*cifra numérica*» y la verbalización para NVDA/MathCAT se manejan desde el módulo Lua `spintent.lua` y no se proveen llaves para modificar este comportamiento.
- El «*símbolo de moneda*» es tomado de la fuente tipográfica definida para el *modo matemático* al momento de ejecutar el comando, se debe tener presente en caso de querer usar un «*símbolo de moneda*» poco habitual que esté soportada por `spintent`. El `{\argumento}` puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{\#1}` o `\mathbf{\#1}` por ejemplo. La coma ',' o el punto '.' como *separador decimal* solo pueden aparecer una «única vez». Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.3.1 Llaves para \spmoney

El comando `\spmoney` posee la *meta-key* `cifras-sep` pasada al comando `\spnum` (§3.1.1).

`currency = {\moneda | alias}` default: peso

Establece el *símbolo de moneda* por defecto que se imprimirá en la salida PDF y cómo lo verbalizará NVDA/MathCAT cuando SOLO se pasa una «*cifra numérica*» como `{\argumento}` al comando `\spmoney`. El «*símbolo de moneda*» o el «*alias de moneda*» pasado a esta llave debe ser alguno de los descritos en §3.3.2.

`sub-units = {\true | false}` default: false

Establece si se usarán *subunidades* de la moneda en curso para la verbalización en NVDA/MathCAT. La moneda en curso debe aceptar *subunidades*. Por ejemplo: `\$ \spmoney[sub-units=true]{100,5 dolares}` imprimirá '\$100,5' en la salida PDF y NVDA/MathCAT verbalizará «*cient dólares con cincuenta centavos*».

`dolar-math = {\true | false}` default: true

Establece si el *símbolo* '\$' se imprimirá en *modo texto* o en *modo matemático*. Si se establece en `true`, se usará la fuente tipográfica definida para el *modo matemático* en curso para imprimir '\$' en la salida PDF; si se establece en `false`, se usará la fuente tipográfica definida para el *modo texto* para imprimir '\$' en la salida PDF.

3.3.2 Monedas y alias para \spmoney

Resumen de los `{\símbolos de moneda}` y `{\alias de moneda}` soportados como `{\argumento}` para `\spmoney`.

3.4 El comando \spshort

`\spshort` `\spshort{\abreviaturas | acrónimos | números ordinales}`
`\spshort[key = val]{\abreviaturas | acrónimos | números ordinales}`
`\spshort[read-arg={\verbalización NVDA}]{\comandos LATEX}`

El comando `\spshort` trabaja *solo en modo texto* y recibe un *argumento obligatorio* del tipo `{\abreviaturas}`, `{\acrónimos}` o `{\números ordinales}` descritos en §3.4.2.

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `\spshort{dr. a}` se imprimirá 'Dr.^a' en la salida PDF y NVDA/MathCAT verbalizará «*doctora*»; si se ejecuta `\spshort{onu}` se imprimirá 'ONU' en la salida PDF y NVDA/MathCAT verbalizará «*organización de las naciones unidas*» y si se ejecuta `\spshort{mineduc}` se imprimirá 'Mineduc' en la salida PDF y NVDA/MathCAT verbalizará «*ministerio de educación*».

Para la escritura de `{\números ordinales}` se puede ejecutar `\spshort{1. a}` que imprimirá '1.^a' en la salida PDF y NVDA/MathCAT verbalizará «*primera*»; si se ejecuta `\spshort{1. o}` imprimirá '1.^o' en la salida PDF y NVDA/MathCAT verbalizará «*primero*».

Para crear una nueva `{\abreviatura}`, `{\acrónimo}` o `{\número ordinal}` que no se encuentre registrada por `spintent` dentro del módulo Lua `spintent.lua` descritas en §3.4.2 se debe utilizar la llave `read-arg` y definir esta dentro del *argumento obligatorio* que puede contener `{\comandos LATEX}` en *modo texto* los cuales se imprimirán en salida PDF.

- Para la escritura de `{\números ordinales}` se debe tener cuidado y no confundir la «*letra o voladita*» 'o' con el símbolo de grado '°'. Si la fuente tipográfica del *modo texto* está utilizando `Numbers=OldStyle` los números pueden quedar muy separados de la «*letra voladita*», la solución para esto es agregar `\addfontfeatures{Numbers=Lining}` dentro de un grupo junto al comando: `{\addfontfeatures{Numbers=Lining}\spshort{1. o}}`.
- El comando `\spshort` está pensado como un reemplazo para *shorthands* y el comando `\sptext` provistos por el paquete `babel` en español compatible con *tagged* PDF. Es de las pocas utilidades que provee el paquete `spintent` que trabaja *solo en modo texto*. Internamente utiliza la llave `E` (*expansion text*) proveída por el paquete `tagpdf`[29].

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
\$	peso, pesos	peso	\$	«peso / pesos»
€	euro, euros	euro	€	«euro / euros»
£	libra, libras	libra	£	«libra / libras»
¥	yen, yenes	yen	¥	«yen / yenes»
¥	yuan, yuanes	yuan	¥	«yuan / yuanes»
₩	won, wons	won	₩	«won / wons»
₪	shekel, shekels, sequel, sequeles	séquel	₪	«séquel / séqueles»
₹	rupia, rupias	rupia	₹	«rupia / rupias»
₽	rublo, rublos	rublo	₽	«rublo / rublos»
₺	lira, liras	lira	₺	«lira / liras»
₴	grivna, grivnas	grivna	₴	«grivna / grivnas»
¢	centavo, centavos	centavo	¢	«centavo / centavos»

Divisas en formato ISO (Mayúsculas)

CLP	—	peso chileno	CLP	«peso chileno / pesos chilenos»
MXN	—	peso mexicano	MXN	«peso mexicano / pesos mexicanos»
USD	—	dólar	USD	«dólar estadounidense / dólares estadounidenses»
EUR	—	euro	EUR	«euro / euros»
GBP	—	libra	GBP	«libra / libras»
JPY	—	yen	JPY	«yen / yenes»
CNY	—	yuan	CNY	«yuan / yuanes»
KRW	—	won	KRW	«won / wons»
ILS	—	séquel	ILS	«séquel / séqueles»
INR	—	rupia	INR	«rupia / rupias»
RUB	—	rublo	RUB	«rublo / rublos»
TRY	—	lira	TRY	«lira / liras»
UAH	—	grivna	UAH	«grivna / grivnas»

Códigos ISO en minúsculas

clp	—	peso chileno	\$	«peso chileno / pesos chilenos»
mxn	—	peso mexicano	\$	«peso mexicano / pesos mexicanos»
usd	dolar, dolares	dólar	\$	«dólar estadounidense / dólares estadounidenses»
eur	—	euro	€	«euro / euros»
gbp	—	libra	£	«libra / libras»
jpy	—	yen	¥	«yen / yenes»
cny	—	yuan	¥	«yuan / yuanes»
krw	—	won	₩	«won / wons»
ils	—	séquel	₪	«séquel / séqueles»
inr	—	rupia	₹	«rupia / rupias»
rub	—	rublo	₽	«rublo / rublos»
try	—	lira	₺	«lira / liras»
uah	—	grivna	₴	«grivna / grivnas»

3.4.1 Llaves para \spshort

`read-arg = { <verbalización NVDA> }` default: no usado

Establece la *verbalización del argumento* { <comandos $\LaTeX$ > } de la nueva { <abreviatura> }, { <acrónimo> } o { <número ordinal> } para NVDA. El valor de esta llave *siempre* debe pasarse entre ‘{ }’. Por ejemplo: `\spshort[read-arg={octavo}]{8.\textsuperscript{\emph{avo}}}` creará una nueva { <abreviatura> } que imprimirá ‘8.^{avo}’ en la salida PDF y NVDA verbalizará «octavo».

`small-caps = { <true | false> }` default: false

Establece si se usarán *versalitas* para imprimir { <acrónimos> } y para la «letra volada» en { <abreviaturas> } y { <números ordinales> } respetando las reglas de la RAE para el uso de estas. Si se establece en `true`, las abreviaturas con mayúscula como EE. UU. y los acrónimos de cinco o más letras como Unicef no se verán afectados; si se establece en `false`, no se aplicarán *versalitas*. Por ejemplo: `\spshort[small-caps=true]{dr.a}` imprimirá ‘Dr.^a’ en la salida PDF y NVDA verbalizará «doctora».

3.4.2 Argumentos para \spshort

El cuadro 2 resume las abreviaturas, expresiones latinas y títulos registrados en el módulo Lua `spintent.lua`.

Cuadro 2: Abreviaturas soportadas

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Abreviaturas con letras voladas			
dr.a	dr. ^a	Dr. ^a	«doctora»
dr.as	—	Dr. ^{as}	«doctoras»
prof.a	—	Prof. ^a	«profesora»
d.a	d. ^a	D. ^a	«doña»
m.a	—	M. ^a	«maría»
n.o	n. ^o	N. ^o	«número»
n.os	—	N. ^{os}	«números»
c.ia	—	C. ^{ia}	«compañía»
Abreviaturas comunes y expresiones latinas			
pág.	pag.	pág.	«página»
vol.	—	vol.	«volumen»
etc.	—	etc.	«etcétera»
et al.	et. al.	et al.	«y otros»
ibíd.	ibid.	ibíd.	«ibídem»
op. cit.	op.cit.	op. cit.	«obra citada»
loc. cit.	loc.cit.	loc. cit.	«lugar citado»
v. gr.	v.gr.	v. gr.	«verbigracia»
i. e.	i.e.	i. e.	«esto es»
e. g.	e.g.	e. g.	«por ejemplo»
p. ej.	p.ej.	p. ej.	«por ejemplo»
Títulos, profesiones y cargos			
sr.	—	Sr.	«señor»
sra.	—	Sra.	«señora»
srta.	—	Srta.	«señorita»
dr.	—	Dr.	«doctor»
dra.	—	Dra.	«doctora»
dres.	—	Dres.	«doctores»
dras.	—	Dras.	«doctoras»
prof.	—	Prof.	«profesor»
profa.	—	Profa.	«profesora»
profs.	—	Profs.	«profesores»
ing.	—	Ing.	«ingeniero»
ings.	—	Ings.	«ingenieros»
lic.	—	Lic.	«licenciado»
v. b.	v.b.	V. B.	«visto bueno»
Abreviaturas compuestas e institucionales			
s. a.	s.a.	S. A.	«sociedad anónima»
ee. uu.	ee.uu.	EE. UU.	«estados unidos»
dd. hh.	dd.hh.	DD. HH.	«derechos humanos»
jj. oo.	jj.oo.	JJ. OO.	«juegos olímpicos»

Continúa en la siguiente página...

Cuadro 2: (Continuación - Abreviaturas soportadas)

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
ff. aa.	ff.aa.	FF. AA.	«fuerzas armadas»
rr. ee.	rr.ee.	RR. EE.	«relaciones exteriores»
cc. aa.	cc.aa.	CC. AA.	«comunidades autónomas»
p. d.	p.d.	P. D.	«posdata»
a. c.	a.c.	a. C.	«antes de Cristo»
d. c.	d.c.	d. C.	«después de Cristo»
d. n. i.	d.n.i.	D. N. I.	«documento nacional de identidad»
r. u. t.	r.u.t.	R. U. T.	«rol único tributario»
r. u. n.	r.u.n.	R. U. N.	«rol único nacional»

El cuadro 3 detalla las siglas y acrónimos registrados en el módulo Lua `spintent.lua`.

Cuadro 3: Siglas y acrónimos soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Siglas (Deletreo o lectura expandida)			
dni	—	DNI	«documento nacional de identidad»
rut	—	RUT	«rol único tributario»
run	—	RUN	«rol único nacional»
onu	—	ONU	«organización de las naciones unidas»
rae	—	RAE	«real academia española»
ong	ongs	ONG	«organización no gubernamental»
urss	—	URSS	«unión de repúblicas socialistas soviéticas»
oea	—	OEA	«organización de los estados americanos»
oms	—	OMS	«organización mundial de la salud»
fmi	—	FMI	«fondo monetario internacional»
bid	—	BID	«banco interamericano de desarrollo»
otan	—	OTAN	«organización del tratado del atlántico norte»
ue	—	UE	«unión europea»
ru	—	RU	«reino unido»
Acrónimos silábicos			
unicef	—	Unicef	«fondo de las naciones unidas para la infancia»
unesco	—	Unesco	«organización de las naciones unidas para la educación, la ciencia y la cultura»
mercosur	—	Mercosur	«mercado común del sur»
cepal	—	Cepal	«comisión económica para américa latina»
mineduc	—	Mineduc	«ministerio de educación»
conicet	—	Conicet	«consejo nacional de investigaciones»

El cuadro 4 muestra ejemplos de números ordinales registrados en el módulo Lua `spintent.lua`.

Cuadro 4: Números ordinales soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Sufijos para ordinales (Ejemplos)			
1.a	1. ^a	1. ^a	«primera»
2.o	2. ^o	2. ^o	«segundo»
3.er	—	3. ^{er}	«tercer»
4.os	—	4. ^{os}	«cuartos»
5.as	—	5. ^{as}	«quintas»

4 Utilidades generales

Además de los «cuatro grandes» y sus asociados, el paquete **spintent** provee tres utilidades de uso general: `\spsiglo` para la escritura y lectura de siglos, `\sptime` para la escritura y lectura de tiempo y `\spdate` para la escritura y lectura de fechas.

El comando `\spsiglo`

```
\spsiglo \spsiglo{<siglo>}
\spsiglo[<key = val>]{<siglo>}
```

El comando `\spsiglo` trabaja en *modo texto* y en *modo matemático*, recibe el *argumento obligatorio* `{<siglo>}`, el cual puede ser un *número arábigo* ‘21’ o un *número romano* ‘XXI’ mayor que ‘0’ y menor que ‘4000’.

Si no se proporciona el *argumento opcional* `[<key = val>]` y se ejecuta `\spsiglo{21}` o `\spsiglo{XXI}` se imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

- El *argumento obligatorio* `{<siglo>}` «siempre» es convertido a *número romano en versalitas* y es verbalizado por NVDA/MathCAT como *número arábigo*. Esta es la única utilidad que provee el paquete **spintent** que trabaja en *modo texto* y en *modo matemático*. Internamente utiliza la llave `alt` proveída por el paquete **tagpdf** en *modo texto* y usa `\MathMLintent` para el *modo matemático*.

Llaves para `\spsiglo`

```
print-era = {<ac | dc | none>} default: none
```

Establece si se *imprime* o no la abreviatura después de imprimir el *argumento* `{<siglo>}`. Si se establece en *ac*, `\spsiglo[print-era=ac]{XXI}` imprimirá ‘xi a. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno antes de cristo»; si se establece en *dc*, `\spsiglo[print-era=dc]{XXI}` imprimirá ‘xi d. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno después de cristo»; si se establece en *none*, `\spsiglo[print-era=none]{XXI}` imprimirá ‘xi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

El comando `\sptime`

```
\sptime \sptime{<horas : minutos>}
\sptime{<horas : minutos : segundos>}
```

El comando `\sptime` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<horas : minutos>}` o `{<horas : minutos : segundos>}`. Por ejemplo: `\sptime{23:45}` imprimirá ‘23:45’ en *modo texto* la salida PDF y NVDA/MathCAT verbalizará «las veintitrés y cuarenta y cinco minutos».

El comando `\spdate`

```
\spdate \spdate{<DD/ MM/YYYY>} \spdate{<DD- MM-YYYY>}
\spdate{<YYYY/ MM/DD>} \spdate{<YYYY- MM-DD>}
```

El comando `\spdate` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<DD/MM/YYYY>}`, `{<DD-MM-YYYY>}` o `{<YYYY-MM-DD>}`. Por ejemplo: `\spdate{1981-01-02}` y `\spdate{02/01/1981}` imprimirán en *modo texto* la salida PDF ‘1981-01-02’ y ‘02/01/1981’ respectivamente y en ambos casos, NVDA/MathCAT verbalizará «dos de enero de mil novecientos ochenta y uno».

- Si se pasa el *argumento* `{<YYYY/MM/DD>}`, internamente se convertirá en `{<DD/MM/YYYY>}` y se imprimirá de esa forma por compatibilidad con NVDA/MathCAT.

5 Utilidades para símbolos

El paquete **spintent** provee tres comandos `\spread`, `\spdsym` y `\spmsym` para trabajar con «símbolos matemáticos» descritos en esta sección. Están pensando desde el punto de vista docente, para situaciones donde la lectura de un «símbolo» (acorde al contexto) es relevante.

- A diferencia de las otras utilidades implementados por **spintent** (que analizan y dan formato a la salida), estos no procesan los «símbolos matemáticos», solo afectan la *forma* (semántica) en que serán verbalizados por NVDA/MathCAT.

El comando `\spread`

```
\spread \spread{<verbalización NVDA>}{<símbolo matemático>}
```

El comando `\spread` trabaja en *modo matemático* y recibe dos *argumentos obligatorios*: El primer *argumento* `{<verbalización NVDA>}` establece la «forma» en la cual NVDA/MathCAT verbalizará al segundo *argumento* `{<símbolo matemático>}`. Por ejemplo: `\spread{es-paralela}{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralela»; `\spread{es-paralelo}{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralelo».

El comando `\spusep`

```
\spusep <separador>
\spusep[<key = val>]{<separador>}
```

El comando `\spusep` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<separador>}`, el cual debe ser uno de los «*delimitadores*» soportados: ‘(’, ‘)’, ‘[’, ‘]’, ‘{’, ‘}’ o ‘\’. Imprime el *delimitador* indicado con el tamaño establecido por la llave `size` y controla si NVDA/MathCAT lo verbaliza o no mediante la llave `silent`. Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spusep{< >$` imprimirá ‘(’ en la salida PDF y NVDA/MathCAT verbalizará «*paréntesis izquierdo*».

- El motivo de existencia de `\spusep` es que los comandos `\left`, `\right`, `\Uleft` y `\Uright` provistos por LuaTeX crean un *grupo* implícito que dificulta el código para *tagged* PDF cuando se necesita interrumpir algo entre ellos, por ejemplo al construir expresiones complejas con `\spnZ`, `\spmQ` o `\spdiv`. El comando `\spusep` imprime el *delimitador* sin crear ningún grupo.

Llaves para `\spusep`

```
size = {<none | big | Big | bigg | Bigg>} default: none
```

Establece el *tamaño* del `{<delimitador>}` que se imprimirá en la salida PDF. Los valores corresponden a los cuatro tamaños fijos provistos por L^AT_EX: si se establece en `none`, el `{<delimitador>}` se imprime en el tamaño por defecto del modo matemático en curso; si se establece en `big`, se usa `\bigl/\bigr`; si se establece en `Big`, se usa `\Bigl/\Bigr`; si se establece en `bigg`, se usa `\biggl/\biggr`; y si se establece en `Bigg`, se usa `\Biggl/\Biggr`. El tamaño se aplica automáticamente en el lado correcto (izquierdo o derecho) según el `{<delimitador>}` pasado.

```
silent = {<true | false>} default: false
```

Establece si NVDA/MathCAT verbalizará el `{<delimitador>}` impreso por `\spusep`. Si se establece en `true`, el `{<delimitador>}` se marcará con el *intent* `:silent` y NVDA/MathCAT no lo verbalizará; si se establece en `false`, NVDA/MathCAT verbalizará el `{<delimitador>}` de forma normal.

El comando `\spdsym`

```
\spdsym
\spdsym[<key = val>]
```

El comando `\spdsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[<key = val>]` mediante el cual se establece la «*forma*» en la que se imprimirá el *símbolo de división* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spdsym$` imprimirá ‘÷’ en la salida PDF y NVDA/MathCAT verbalizará «*dividido*».

Llaves para `\spdsym`

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «*antes*» de verbalizar el `{<valor>}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
suffix = {<sufijo>} default: no usado
```

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «*después*» de verbalizar el `{<valor>}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
set-sym = {<old-style | two-dots | slash>} default: two-dots
```

Establece el *símbolo de división* que se imprimirá en la salida PDF. Si se establece en `old-style`, imprimirá ‘÷’; si se establece en `two-dots`, imprimirá ‘⋮’ y si se establece `slash`, imprimirá ‘/’.

```
read-sym = {<dividido | dividido-por | dividido-entre>} default: dividido
```

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de división*. Si se establece en `dividido`, NVDA/MathCAT verbalizará «*dividido*»; si se establece en `dividido-por`, NVDA/MathCAT verbalizará «*dividido por*» y si se establece en `dividido-entre`, NVDA/MathCAT verbalizará «*dividido entre*».

El comando `\spmsym`

```
\spmsym
\spmsym[<key = val>]
```

El comando `\spmsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[<key = val>]` mediante el cual se establece la «*forma*» en la que se imprimirá el *símbolo de multiplicación* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spmsym$` imprimirá ‘⋅’ en la salida PDF y NVDA/MathCAT verbalizará «*por*».

Llaves para `\spmsym`

`prefix = {\langle prefijo \rangle}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el `\{ \langle valor \rangle \}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves `\{ \}`.

`suffix = {\langle sufijo \rangle}` default: no usado

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «después» de verbalizar el `\{ \langle valor \rangle \}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves `\{ \}`.

`set-sym = {\langle cross | dot | asterisk \rangle}` default: dot

Establece el *símbolo de multiplicación* que se imprimirá en la salida PDF. Si se establece en `cross`, imprimirá `×`; si se establece en `dot`, imprimirá `·` y si se establece en `asterisk`, imprimirá `*`.

`read-sym = {\langle por | multiplicado-por | veces \rangle}` default: por

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de multiplicación*. Si se establece en `por`, NVDA/MathCAT verbalizará «por»; si se establece en `multiplicado-por`, NVDA/MathCAT verbalizará «multiplicado por» y si se establece en `veces`, NVDA/MathCAT verbalizará «veces».

`not-print = {\langle true | false \rangle}` default: false

Deshabilita la *impresión* del *símbolo de multiplicación*. Cuando se establece esta llave, NVDA/MathCAT verbalizará el *símbolo de multiplicación* acorde al valor establecido por la llave `read-sym`, pero, no se imprimirá este en la salida PDF.

6 Utilidades para números

El paquete `spintent` provee utilidades para trabajar con números descritas en esta sección.

El comando `\spdiv`

`\spdiv` `\spdiv{\langle dividendo \rangle}{\langle divisor \rangle}`
`\spdiv[\langle key = val \rangle]{\langle dividendo \rangle}{\langle divisor \rangle}`

El comando `\spdiv` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* `\{ \langle dividendo \rangle \}` y `\{ \langle divisor \rangle \}`.

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\$ \spdiv{20}{26} \$` imprimirá `20 : 26` en la salida PDF y NVDA/MathCAT verbalizará «veinte dividido veinte y seis»

Llaves para `\spdiv`

El comando `\spdiv` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 12).

`wrap-parens = {\langle true | false \rangle}` default: false

Establece si se *imprimen* paréntesis redondos `(...)` alrededor de los *argumentos* `\{ \langle dividendo \rangle \}` y `\{ \langle divisor \rangle \}` pasados a `\spdiv`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {\langle true | false \rangle}` default: true

Establece si NVDA/MathCAT verbalizará los paréntesis redondos `(...)` alrededor de los *argumentos* `\{ \langle dividendo \rangle \}` y `\{ \langle divisor \rangle \}` pasados a `\spdiv`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spmul`

`\spmul` `\spmul{\langle factor \rangle}{\langle factor \rangle}`
`\spmul[\langle key = val \rangle]{\langle factor \rangle}{\langle factor \rangle}`

El comando `\spmul` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* `\{ \langle factor \rangle \}` y `\{ \langle factor \rangle \}`.

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\$ \spmul{20}{26} \$` imprimirá `20 · 26` en la salida PDF y NVDA/MathCAT verbalizará «veinte por veinte y seis».

Llaves para `\spmul`

El comando `\spmul` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 13).

`wrap-parens = {\langle true | false \rangle}` default: false

Establece si se *imprimen* paréntesis redondos `(...)` alrededor de los *argumentos* `\{ \langle factor \rangle \}` y `\{ \langle factor \rangle \}` pasados a `\spmul`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {\langle true | false \rangle}` default: true

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(…)’ alrededor de los *argumentos* $\{ \langle \textit{factor} \rangle \}$ y $\{ \langle \textit{factor} \rangle \}$ pasados a `\spmul`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spnD`

```
\spnD \spnD
\spnD(\<número natural>)
\spnD[key = val](\<número natural>)
```

El comando `\spnD` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* $(\langle \textit{número natural} \rangle)$, por ejemplo: ‘(6)’. Si se usa sin $(\langle \textit{argumento} \rangle)$, `\spnD` imprimirá ‘D(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de *n*».

Si se usa con $(\langle \textit{argumento} \rangle)$, por ejemplo, `\spnD(6)`, imprimirá ‘D(6)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis».

Llaves para `\spnD`

`prefix = { \<prefijo> }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar `\spnD(\<argumento>)`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { \<true | false> }` default: false

Establece si se *imprime* el resultado de calcular los *divisores* del $(\langle \textit{número natural} \rangle)$ pasado a `\spnD`. Si se establece en `true`, `\spnD[result=true](6)` imprimirá ‘D(6) = {1, 2, 3, 6}’ en la salida PDF y NVDA/MathCAT verbalizará «los divisores de seis son uno, dos, tres y seis»; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los *divisores*.

`result-num = { \<número natural> }` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo `5` valores y luego se añadirá ‘...’ en la salida PDF.

`sample-arg = { \<true | false> }` default: false

Desactiva la *validación* interna del $(\langle \textit{argumento} \rangle)$ pasado a `\spnD`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`, `\spnD[sample-arg=true](m)` imprimirá ‘D(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de *m*»; si se establece en `false`, el $(\langle \textit{argumento} \rangle)$ debe ser un número natural, en caso contrario se devolverá un “error”.

`silent-cmd = { \<true | false> }` default: false

Establece si se *silencia* la verbalización para NVDA/MathCAT de `\spnD(\<argumento>)` cuando se establece `sample-arg=true` o cuando `\spnD` se usa sin $(\langle \textit{argumento} \rangle)$. Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnM`

```
\spnM \spnM
\spnM(\<número natural>)
\spnM[key = val](\<número natural>)
```

El comando `\spnM` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* $(\langle \textit{número natural} \rangle)$, por ejemplo: ‘(6)’. Si se usa sin $(\langle \textit{argumento} \rangle)$, `\spnM` imprimirá ‘M(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de *n*».

Si se usa con $(\langle \textit{argumento} \rangle)$, por ejemplo, `\spnM(6)`, imprimirá ‘M(6)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis».

Llaves para `\spnM`

`prefix = { \<prefijo> }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar `\spnM(\<argumento>)`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { \<true | false> }` default: false

Establece si se *imprime* el resultado de calcular los *múltiplos* del $(\langle \textit{número natural} \rangle)$ pasado a `\spnM`. Si se establece en `true`, `\spnM[result=true](6)` imprimirá ‘M(6) = {6, 12, 18, 24, 30, ...}’ en la salida PDF y NVDA/MathCAT verbalizará «los múltiplos de seis son seis, doce, dieciocho, veinticuatro, treinta puntos suspensivos»; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los *múltiplos*.

`result-num = { \<número natural> }` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF seguidos de ‘...’; si no se establece, se imprimirán por defecto los `5` primeros valores y luego se añadirá ‘...’ en la salida PDF.

`sample-arg = { \<true | false> }` default: false

Desactiva la *validación* interna del (*argumento*) pasado a `\spnM`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`, `\spnM[sample-arg=true](m)` imprimirá ‘M(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de *m*»; si se establece en `false`, el (*argumento*) debe ser un número natural, en caso contrario se devolverá un “error”.

`silent-cmd = { (true | false) }` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spnM`(*argumento*) cuando se establece `sample-arg=true` o cuando `\spnM` se usa sin (*argumento*). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false` NVDA/MathCAT lo verbalizará.

El comando `\spmc`

```
\spmc \spmc
\spmc (números separados por coma)
\spmc[key = val] (números separados por coma)
```

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (*números separados por coma*), el cual debe ser una lista de *números naturales*, por ejemplo: ‘(2, 3, 6)’. Si se usa sin (*argumento*) `\spmc` imprimirá ‘mcd’ en la salida PDF, y NVDA/MathCAT verbalizará «máximo común divisor» o «*m c d*», acorde al {*valor*} establecido por la llave `read-cmd`.

Si se usa con (*argumento*) `\spmc(2, 3, 6)` imprimirá ‘mcd(2, 3, 6)’ en la salida PDF, y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis» o «*m c d* entre dos, tres y seis» acorde al {*valor*} establecido por la llave `read-cmd`.

Llaves para `\spmc`

`upper-case = { (true | false) }` default: false

Establece si se usarán *mayúsculas* o *minúsculas* en la salida PDF para el comando `\spmc`. Si se establece en `true`, se imprimirá ‘MCD’ en la salida PDF; si se establece en `false`, se imprimirá ‘mcd’ en la salida PDF.

`read-cmd = { (terse | clear) }` default: clear

Establece la *forma* en la que NVDA/MathCAT verbalizará el comando `\spmc`. Si se establece en `terse`, NVDA/MathCAT verbalizará «*m c d*»; si se establece en `clear`, NVDA/MathCAT verbalizará «máximo común divisor».

`prefix = { (prefijo) }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el valor establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { (true | false) }` default: false

Establece si se *imprime* el resultado de calcular el «máximo común divisor» del (*argumento*) pasado a `\spmc`. Si se establece en `true`: `\spmc[result=true](2, 3, 6)` imprimirá ‘mcd(2, 3, 6) = 1’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis es igual a uno»; si se establece en `false` no se imprimirá en la salida PDF el resultado de calcular el «máximo común divisor».

`sample-arg = { (true | false) }` default: false

Desactiva la *validación* interna del (*argumento*) pasado a `\spmc`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`: `\spmc[sample-arg=true](m, n)`, imprimirá ‘mcd(*m*, *n*)’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre *m* y *n*»; si se establece en `false` el (*argumento*) debe ser una lista de *números naturales*, en caso contrario se devolverá un “error”.

`silent-cmd = { (true | false) }` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spmc`(*argumento*) cuando se establece `sample-arg=true` o cuando `\spmc` se usa sin (*argumento*). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spmc`

```
\spmc \spmc
\spmc (números separados por coma)
\spmc[key = val] (números separados por coma)
```

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (*números separados por coma*), el cual debe ser una lista de *números naturales*, por ejemplo: ‘(2, 3, 6)’. Si se usa sin (*argumento*) `\spmc` imprimirá ‘mcm’ en la salida PDF, y NVDA/MathCAT verbalizará «mínimo común múltiplo» o «*m c m*», acorde al {*valor*} establecido por la llave `read-cmd`.

Si se usa con (*argumento*) `\spmc(2, 3, 6)` imprimirá ‘mcm(2, 3, 6)’ en la salida PDF, y NVDA/MathCAT verbalizará «mínimo común múltiplo entre dos, tres y seis» o «*m c m* entre dos, tres y seis» acorde al {*valor*} establecido por la llave `read-cmd`.

Llaves para `\spmc`

`upper-case = {⟨true | false⟩}`

default: *false*

Establece si se usarán *mayúsculas* o *minúsculas* en la salida PDF para el comando `\spmc`. Si se establece en *true* se imprimirá ‘MCM’ en la salida PDF; si se establece en *false* se imprimirá ‘mcm’ en la salida PDF.

`read-cmd = {⟨terse | clear⟩}`

default: *clear*

Establece la *forma* en la que NVDA/MathCAT verbalizará el comando `\spmc`. Si se establece en *terse*, NVDA/MathCAT verbalizará «*m c m*»; si se establece en *clear*, NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el valor establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = {⟨true | false⟩}`

default: *false*

Establece si se *imprime* el resultado de calcular el «*mínimo común múltiplo*» del (⟨*argumento*⟩) pasado a `\spmc`. Si se establece en *true*: `\spmc[result=true](2, 3, 6)` imprimirá ‘mcm(2, 3, 6) = 6’ en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis es igual a seis*»; si se establece en *false* no se imprimirá en la salida PDF el resultado de calcular el «*mínimo común múltiplo*».

`sample-arg = {⟨true | false⟩}`

default: *false*

Desactiva la *validación* interna del (⟨*argumento*⟩) pasado a `\spmc`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en *true*: `\spmc[sample-arg=true](m, n)`, imprimirá ‘mcm(*m*, *n*)’ en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre m y n*»; si se establece en *false* el (⟨*argumento*⟩) debe ser una lista de *números naturales*, en caso contrario se devolverá un “error”.

`silent-cmd = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spmc(⟨argumento⟩)` cuando se establece `sample-arg=true` o cuando `\spmc` se usa sin (⟨*argumento*⟩). Si se establece en *true* NVDA/MathCAT NO lo verbalizará; si se establece en *false* NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnZ`

`\spnZ {⟨número entero⟩}`

`\spnZ[key = val]{⟨número entero⟩}`

El comando `\spnZ` trabaja en *modo matemático* y recibe el *argumento* obligatorio {⟨*número entero*⟩} el cual se procesa internamente bajo el comando `\spnum`. Este comando está dedicado exclusivamente al manejo de «*números enteros*», y añade algunas utilidades prácticas para el demarcado e interacción con NVDA/MathCAT.

Llaves para `\spnZ`

El comando `\spnZ` posee las ⟨*meta-keys*⟩ *cifras-sep* y *read-sign* pasadas al comando `\spnum` (§3.1.1).

`wrap-parens = {⟨true | false⟩}`

default: *false*

Establece si se *imprimen* paréntesis redondos ‘(…)’ alrededor del argumento {⟨*número entero*⟩} pasado a `\spnZ`. Si se establece en *true*, se imprimirán paréntesis redondos alrededor del {⟨*número entero*⟩}; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}`

default: *false*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(…)’ alrededor del *argumento* {⟨*número entero*⟩} pasado a `\spnZ`. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura del paréntesis ‘(…)»; si se establece en *false*, NVDA/MathCAT no los verbalizará.

El comando `\spmQ`

`\spmQ {⟨número entero⟩}{⟨numerador⟩}{⟨denominador⟩}`

`\spmQ[key = val]{⟨número entero⟩}{⟨numerador⟩}{⟨denominador⟩}`

El comando `\spmQ` trabaja en *modo matemático* y recibe *tres argumentos obligatorios*: {⟨*número entero*⟩}, {⟨*numerador*⟩} y {⟨*denominador*⟩}, los procesa internamente y devuelve un «*número mixto*».

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩], `\spmQ{2}{5}{7}` imprimirá ‘2⁵/₇’ en la salida PDF y NVDA/MathCAT verbalizará «*dos y cinco séptimos*».

- En el caso de las fracciones, y en especial con los «*números mixtos*», no se pueden manejar muchas cosas desde el lado de *spintent*. Por ejemplo, la entrada `\frac{5}{7}` con *tagged* PDF se verbaliza por NVDA/MathCAT como «*tres más cinco séptimos*», lo cual no está mal, pero pedagógicamente hablando no es del todo correcto.

Llaves para `\spmQ`

`join-word = {⟨entero | enteros | y⟩}`

default: `y`

Establece la *forma* en la que NVDA/MathCAT verbaliza la conexión entre el entero y la fracción que forman el *número mixto*. Si se establece en `entero`, al usar `\spmQ[join-word=entero]{1}{5}{7}` imprimirá $1\frac{5}{7}$ en la salida PDF, y NVDA/MathCAT verbalizará «un entero cinco séptimos»; si se establece en `enteros`, al usar `\spmQ[join-word=enteros]{2}{5}{7}` imprimirá $2\frac{5}{7}$ en la salida PDF, y NVDA/MathCAT verbalizará «dos enteros cinco séptimos»; si se establece en `y`, al usar `\spmQ[join-word=y]{2}{5}{7}` imprimirá $2\frac{5}{7}$ en la salida PDF, y NVDA/MathCAT verbalizará «dos y cinco séptimos».

`use-spnZ = {⟨true | false⟩}`

default: `true`

Establece si el `{⟨numerador⟩}` y el `{⟨denominador⟩}` que forman la fracción del *número mixto* se procesan internamente bajo el comando `\spnZ`, añadiendo así *semántica de entero* en la estructura MathML de la salida PDF. Si se establece en `true`, `{⟨numerador⟩}` y `{⟨denominador⟩}` se pasarán a `\spnZ` y NVDA/MathCAT los verbalizará correctamente como *números ordinales*; si se establece en `false`, los argumentos se tipografían directamente sin procesamiento semántico adicional.

`print-dfrac = {⟨true | false⟩}`

default: `false`

Establece si la fracción del *número mixto* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en `true`, al usar `\spmQ[print-dfrac=true]{2}{5}{7}` imprimirá la fracción en tamaño *display* $2\frac{5}{7}$ en la salida PDF; si se establece en `false`, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`wrap-parens = {⟨true | false⟩}`

default: `false`

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor del *número mixto* completo devuelto por `\spmQ`. Si se establece en `true`, se imprimirán paréntesis redondos cuyo tamaño se ajusta automáticamente al estilo matemático en curso; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}`

default: `true`

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(...)’ alrededor del *número mixto* cuando la llave `wrap-parens` está activa. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

7 Utilidades para geometría

El paquete `spintent` provee comandos para trabajar con *geometría* descritos en esta sección. Todos los comandos trabajan en *modo matemático* y reciben como `{⟨argumento⟩}` base una *única* letra (mayúscula en el caso de puntos) con un modificador del tipo subíndice/prima, un número o comando.

Algunos comandos reciben `{⟨argumento⟩}` único (modo “*single*”) o más de uno (modo “*batch*”) donde opcionalmente se pueden separar por comas ‘,’.

- Las primas se debe pasar de forma directa ‘’ y los subíndices siempre deben escribirse usando llaves ‘A_{...}’ y nunca sin estas ‘A_n’ que funciona solo por coincidencia. .

El comando `\spcoord`

`\spcoord {⟨abscisa, ordenada⟩}`
`\spcoord {⟨abscisa; ordenada⟩}`

`\spcoord [⟨key = val⟩]{⟨abscisa, ordenada⟩}`
`\spcoord [⟨key = val⟩]{⟨abscisa; ordenada⟩}`

El comando `\spcoord` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{⟨abscisa, ordenada⟩}` el cual se procesa internamente bajo el comando `\spnum` cuando es numérico no fraccionario.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spcoord{-3, 4}` imprimirá ‘(-3, 4)’ en la salida PDF y NVDA/MathCAT verbalizará «menos tres coma cuatro»; `\spcoord{x_{1}, y_{1}}` imprimirá ‘(x₁, y₁)’ y verbalizará «*exis subíndice uno y subíndice uno*».

- El comando `\spcoord` solo acepta un *separador* ‘,’ o ‘;’ para la abscisa y la ordenada cuando son valores numéricos no fraccionarios.

Llaves para `\spcoord`

`silent-cmd = {⟨true | false⟩}`

default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPoint`

<code>\spPoint</code>	<code>\spPoint{<punto>}</code>	<code>\spPoint[<key = val>]{<punto>}</code>
	<code>\spPoint{<punto>}(<abscisa, ordenada>)</code>	<code>\spPoint[<key = val>]{<punto>}(<abscisa, ordenada>)</code>

El comando `\spPoint` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<punto>}` el cual es una *única letra mayúscula* con prima `'` o subíndice `'_{...}'` y el *argumento opcional delimitado por paréntesis* `( <abscisa, ordenada> )` el cual se procesa internamente bajo el comando `\spcoord`.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spPoint{A}` imprimirá 'A' en la salida PDF y NVDA/MathCAT verbalizará «punto A»; `\spPoint{A_{1}}` imprimirá 'A₁' y verbalizará «punto A sub uno».

Si se proporciona el *argumento opcional delimitado por paréntesis*, `\spPoint{A}(-3,4)` imprimirá 'A(-3,4)' en la salida PDF y NVDA/MathCAT verbalizará «punto A menos tres coma cuatro»; `\spPoint{A_{1}}(-3,4)` imprimirá 'A₁(-3,4)' y verbalizará «punto A sub uno menos tres coma cuatro».

Llaves para `\spPoint`

`read-cmd = {<school | mathcat>}` default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{<punto>}`. Si se establece en `school`, NVDA/MathCAT verbalizará el argumento `{<punto>}` sin el prefijo «mayúscula», `\spPoint{A}` se verbaliza «punto A»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {<prefijo>}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «punto». El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

`concept = {<true | false>}` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «punto» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el `{<argumento>}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{<argumento>}` acorde al valor establecido por la llave `read-cmd`.

`silent-cmd = {<true | false>}` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «punto» y del `{<argumento>}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spvec`

<code>\spvec</code>	<code>\spvec{<letra>}</code>	<code>\spvec{<punto, punto>}</code>
	<code>\spvec{<letra>}(<abscisa, ordenada>)</code>	<code>\spvec{<punto, punto>}(<abscisa, ordenada>)</code>
	<code>\spvec[<key = val>]{<letra>}(<abscisa, ordenada>)</code>	<code>\spvec[<key = val>]{<punto, punto>}(<abscisa, ordenada>)</code>

El comando `\spvec` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{<letra>}` (minúscula) o `{<punto, punto>}` junto al *argumento opcional delimitado por paréntesis* `( <abscisa, ordenada> )` el cual se procesa internamente bajo el comando `\spcoord`.

Si el *argumento* es de la forma `{<punto, punto>}` con prima(s) `'` o subíndice(s) `'_{...}'` opcionales se aplica `\overrightarrow` en la impresión; si el *argumento* es de la forma `{<letra>}` (minúscula) con subíndice `'_{...}'` opcional se aplica `\vec` en la impresión.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spvec{a}` imprimirá ' $\vec{a}$ ' en la salida PDF y NVDA/MathCAT verbalizará «vector a»; `\spvec{AB}` imprimirá ' $\overrightarrow{AB}$ ' y verbalizará «vector A B».

Si se proporciona el *argumento opcional delimitado por paréntesis*, `\spvec{a}(-3,4)` imprimirá ' $\vec{a} = (-3,4)$ ' en la salida PDF y NVDA/MathCAT verbalizará «vector a igual a menos tres coma cuatro»; `\spvec{AB}(-3,4)` imprimirá ' $\overrightarrow{AB} = (-3,4)$ ' y verbalizará «vector A B igual a menos tres coma cuatro».

Llaves para `\spvec`

`read-cmd = {<school | mathcat>}` default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{<punto, punto>}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spvec{AB}` se verbaliza «vector A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {<prefijo>}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «vector». El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

`autofit = {<true | false>}` default: false

Establece si se *utiliza* `\vphantom{b}` al ejecutar el comando cuando se utiliza el argumento `{<letra>}`. Si se establece en `true`, se aplicará `\vphantom{b}` al inicio del argumento `{<letra>}`; si se establece en `false`, no se aplicará `\vphantom{b}`.

`concept = {\true | false}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «vector» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el $\{\langle \text{argumento} \rangle\}$; si se establece en *false*, NVDA/MathCAT solo verbalizará el $\{\langle \text{argumento} \rangle\}$ acorde al valor establecido por la llave `read-cmd`.

`silent-cmd = {\true | false}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «vector» y del $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\sprecta`

<code>\sprecta</code>	<code>\sprecta{\punto, punto}</code>	<code>\sprecta{\letra}</code>
	<code>\sprecta[key = val]{\punto, punto}</code>	<code>\sprecta[key = val]{\letra}</code>

El comando `\sprecta` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma $\{\langle \text{letra} \rangle\}$ o $\{\langle \text{punto, punto} \rangle\}$ con prima(s) ‘*’* o subíndice(s) ‘*_...*’ opcionales. Si el *argumento* es de la forma $\{\langle \text{punto, punto} \rangle\}$ se aplica `\overleftarrow{\rightarrow}` en la impresión; si el *argumento* es de la forma $\{\langle \text{letra} \rangle\}$ (mayúscula o minúscula), se imprimirá solo la $\{\langle \text{letra} \rangle\}$ sin aplicar `\overleftarrow{\rightarrow}`.

Si no se proporciona el *argumento opcional* $[key = val]$, $\$ \text{\sprecta}\{AB\} \$$ imprimirá ‘ $\overrightarrow{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «recta A B»; $\$ \text{\sprecta}\{m\} \$$ imprimirá ‘*m*’ en la salida PDF y NVDA/MathCAT verbalizará «recta m».

Llaves para `\sprecta`

`read-cmd = {\school | mathcat}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento $\{\langle \text{punto, punto} \rangle\}$ y el argumento $\{\langle \text{letra} \rangle\}$ cuando está en mayúscula. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: $\$ \text{\sprecta}\{AB\} \$$ se verbaliza «recta A B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {\prefijo}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «recta». El valor de esta llave *siempre* debe pasarse entre llaves ‘*{ }*’.

`concept = {\true | false}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «recta» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el $\{\langle \text{argumento} \rangle\}$; si se establece en *false*, NVDA/MathCAT solo verbalizará el $\{\langle \text{argumento} \rangle\}$ acorde al valor establecido por la llave `read-cmd`.

`silent-cmd = {\true | false}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «recta» y del $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\sprayo`

<code>\sprayo</code>	<code>\sprayo{\punto, punto}</code>
	<code>\sprayo[key = val]{\punto, punto}</code>

El comando `\sprayo` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle \text{punto, punto} \rangle\}$ con prima(s) ‘*’* o subíndice(s) ‘*_...*’ opcionales. Representa un *rayo* o *semirrecta* con origen en el primer punto y dirección hacia el segundo, aplicando `\overrightarrow{\rightarrow}` en la impresión.

Si no se proporciona el *argumento opcional* $[key = val]$, $\$ \text{\sprayo}\{AB\} \$$ imprimirá ‘ $\overrightarrow{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «rayo A B».

Llaves para `\sprayo`

`read-cmd = {\school | mathcat}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento $\{\langle \text{punto, punto} \rangle\}$. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: $\$ \text{\sprayo}\{AB\} \$$ se verbaliza «rayo A B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty = {\rayo | semirrecta}`

default: *rayo*

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza $\$ \text{\sprayo}[read-sty=rayo]\{AB\} \$$, NVDA/MathCAT verbalizará «rayo A B»; si se utiliza $\$ \text{\sprayo}[read-sty=semirrecta]\{AB\} \$$, NVDA/MathCAT verbalizará «semirrecta A B».

`prefix = {\prefijo}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «*concepto*» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept = {⟨true | false⟩}` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del «*concepto*» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el {⟨*argumento*⟩}; si se establece en `false`, NVDA/MathCAT solo verbalizará el {⟨*argumento*⟩} acorde al valor establecido por la llave `read-cmd`.

`silent-cmd = {⟨true | false⟩}` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del «*concepto*» establecido por la llave `read-sty` y del {⟨*argumento*⟩} pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

- La impresión ' $\overline{AB}$ ' en la salida PDF es idéntica en ambos casos. La elección depende de la terminología del libro de texto o del currículo nacional: en Chile y España se prefiere «semirrecta»; en México y otros países de América Latina se usa «rayo».

El comando `\spside`

`\spside` `\spside{⟨punto, punto⟩}`
`\spside[⟨key = val⟩]{⟨punto, punto⟩}`

El comando `\spside` trabaja en *modo matemático* y recibe el *argumento obligatorio* {⟨*punto, punto*⟩} con prima(s) ' ' o subíndice(s) '_{...}' opcionales. Representa un *lado*, *segmento* o *trazo* determinado por dos puntos, aplicando `\overline` en la impresión.

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩], `\spside{AB}` imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*lado A B*».

Llaves para `\spside`

`read-cmd = {⟨school | mathcat⟩}` default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento {⟨*punto, punto*⟩}. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spside{AB}` se verbaliza «*lado A B*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty = {⟨lado | segmento | trazo⟩}` default: lado

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\spside[read-sty=lado]{AB}`, NVDA/MathCAT verbalizará «*lado A B*»; si se utiliza `\spside[read-sty=segmento]{AB}`, NVDA/MathCAT verbalizará «*segmento A B*» y si se utiliza `\spside[read-sty=trazo]{AB}`, NVDA/MathCAT verbalizará «*trazo A B*».

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «*concepto*» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept = {⟨true | false⟩}` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del «*concepto*» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el {⟨*argumento*⟩}; si se establece en `false`, NVDA/MathCAT solo verbalizará el {⟨*argumento*⟩} acorde al valor establecido por la llave `read-cmd`.

`read-arg = {⟨verbalización NVDA⟩}` default: no usado

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spside[read-arg={hipotenusa}]{AB}` imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*hipotenusa A B*».

`silent-cmd = {⟨true | false⟩}` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del «*concepto*» establecido por la llave `read-sty` y del {⟨*argumento*⟩} pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmside`

```
\spmside \spmside{<punto, punto>}
\spmside[<key = val>]{<punto, punto>}
```

El comando `\spmside` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<punto, punto>}` con prima(s) ‘`’` o subíndice(s) ‘`_ { . . . }`’ opcionales. Representa la *medida* del *lado*, *segmento* o *trazo* determinado por dos puntos. La distinción entre `\spside` y `\spmside` es semántica: `\spside` representa el *objeto geométrico* ‘ $\overline{AB}$ ’; `\spmside` representa su *medida* (un número real positivo).

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spmside{AB}` imprimirá ‘ $\overline{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del lado A B».

Llaves para `\spmside`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{<punto, punto>}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmside{AB}` se verbaliza «medida del lado A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
read-sty = {<lado | segmento | trazo>} default: lado
```

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «medida del» al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\spmside[read-sty=lado]{AB}`, NVDA/MathCAT verbalizará «medida del lado A B»; si se utiliza `\spmside[read-sty=segmento]{AB}`, NVDA/MathCAT verbalizará «medida del segmento A B» y si se utiliza `\spmside[read-sty=trazo]{AB}`, NVDA/MathCAT verbalizará «medida del trazo A B».

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT antes de verbalizar el término «medida del» seguido por el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

```
concept = {<true | false>} default: true
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» y del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el término «medida del», el concepto y el `{<argumento>}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{<argumento>}` acorde al valor establecido por la llave `read-cmd`.

```
read-arg = {<verbalización NVDA>} default: no usado
```

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’ y se debe usar en conjunto con la llave `concept`.

Por ejemplo: `\spmside[concept=false,read-arg={medida de la cuerda}]{AB}` imprimirá ‘ $\overline{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida de la cuerda A B».

```
print-m = {<true | false>} default: false
```

Establece si se *imprime* la notación visual ‘m’ antes aplicar `\overline` al `{<argumento>}`. Si se establece en `true`, `\spmside[print-m=true]{AB}` imprimirá ‘m $\overline{AB}$ ’ en la salida PDF, si se establece en `false` imprimirá ‘ $\overline{AB}$ ’ en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del lado A B».

```
silent-cmd = {<true | false>} default: false
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», del «concepto» establecido por la llave `read-sty` y del `{<argumento>}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spangle`

```
\spangle \spangle{<punto | letra griega | número>} \spangle{<punto, punto, punto>}
\spangle[<key = val>]{<punto | letra griega | número>} \spangle[<key = val>]{<punto, punto, punto>}
```

El comando `\spangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{<punto | letra griega | número>}` o `{<punto, punto, punto>}` aplicando `\angle` o `\rightanglesqr` en la impresión. Solo los argumentos de la forma `{<punto>}` o `{<punto, punto, punto>}` admiten prima(s) ‘`’` o subíndice(s) ‘`_ { . . . }`’ opcionales.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo en A»; `\spangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo alfa»; `\spangle{1}` imprimirá ‘ $\angle 1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo uno» y `\spangle{AOB}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo A O B».

Si no se tiene instalada la fuente matemática NewCMath-Regular.otf parte del paquete `newcomputermodern` en el sistema o no se provee una fuente matemática con soporte para `\rightanglesqr` se usará `\rightangle` en su reemplazo.

Llaves para `\spangle`

`read-cmd = {\school | mathcat}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{\punto}` o `{\punto, punto, punto}`. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spangle{AOB}` se verbaliza «ángulo A O B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {\prefijo}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

`recto = {\true | false}`

default: *false*

Establece si se *usará* `\rightanglesqr` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en *true*, `\spangle[recto=true]{A}` imprimirá `⊥A` en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto en A», `\spangle[recto=true]{AOB}` imprimirá `⊥AOB` en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto A O B»; si se establece en *false*, imprimirá usando `\angle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

`concept = {\true | false}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «ángulo» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto «ángulo» y el `{\argumento}`; si se establece en *false*, NVDA/MathCAT solo verbalizará el `{\argumento}` acorde al valor establecido por la llave `read-cmd`.

`silent-cmd = {\true | false}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «ángulo» y del `{\argumento}` pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmangle`

`\spmangle{\punto | letra griega | número}`

`\spmangle{\punto, punto, punto}`

`\spmangle[key = val]{\punto | letra griega | número}`

`\spmangle[key = val]{\punto, punto, punto}`

El comando `\spmangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{\punto | letra griega | número}` o `{\punto, punto, punto}` aplicando `\measuredangle` o `\measuredrightangle` en la impresión. Solo los argumentos de la forma `{\punto}` o `{\punto, punto, punto}` admiten prima(s) `'` o subíndice(s) `_{\dots}` opcionales.

Si no se proporciona el *argumento opcional* `[key = val]`, `\spmangle{A}` imprimirá `∠A` en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo en A»; `\spmangle{\alpha}` imprimirá `∠α` en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo alfa»; `\spmangle{1}` imprimirá `∠1` en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo uno» y `\spmangle{AOB}` imprimirá `∠AOB` en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo A O B».

- La distinción entre `\spangle` y `\spmangle` es semántica: `\spangle` representa el *objeto geométrico* `∠AOB`; `\spmangle` representa su *medida* (un valor en grados).

Llaves para `\spmangle`

`read-cmd = {\school | mathcat}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{\punto}` o `{\punto, punto, punto}`. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmangle{AOB}` se verbaliza «medida del ángulo A O B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {\prefijo}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el concepto «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

`recto = {\true | false}`

default: *false*

Establece si se *usará* `\measuredrightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en *true*, `\spmangle[recto=true]{A}` imprimirá `⊥A` en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto en A», `\spmangle[recto=true]{AOB}` imprimirá `⊥AOB` en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto A O B»; si se establece en *false*, imprimirá usando `\measuredangle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

`print-m = {\true | false}`

default: *false*

Establece *si se imprime* la notación visual `'m'` antes del argumento pasado al comando sin modificar la verbalización de este. Si se establece en *true*, `\spmangle[print-m=true]{AOB}` imprimirá `m∠AOB` en la salida PDF, si se establece en *false* imprimirá `∠AOB` en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del ángulo A O B».

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*medida del*» seguido por el concepto «*ángulo*» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «*medida del ángulo*» y el $\langle \textit{argumento} \rangle$; si se establece en *false*, NVDA/MathCAT solo verbalizará el $\langle \textit{argumento} \rangle$ acorde al valor establecido por la llave `read-cmd`.

`silent-cmd = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*medida del*», del concepto «*ángulo*» y del $\langle \textit{argumento} \rangle$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spang`

```
\spang {⟨grados⟩}
\spang {⟨grados : minutos⟩}
\spang {⟨grados : minutos : segundos⟩}
```

El comando `\spang` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\langle \textit{grados} \rangle$, $\langle \textit{grados : minutos} \rangle$ o $\langle \textit{grados : minutos : segundos} \rangle$. Los procesa internamente bajo los comandos `\spnum` y `\spunit` para imprimir y verbalizar. Por ejemplo, `\spang{45:30:15}` imprimirá ‘45°30’15” en la salida PDF y NVDA/MathCAT verbalizará «*cuarenta y cinco grados treinta minutos quince segundos*».

El comando `\sparc`

```
\sparc {⟨punto, punto⟩}
\sparc [⟨key = val⟩] {⟨punto, punto⟩}
\sparc {⟨punto, punto, punto⟩}
\sparc [⟨key = val⟩] {⟨punto, punto, punto⟩}
```

El comando `\sparc` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma $\langle \textit{punto, punto} \rangle$ o $\langle \textit{punto, punto, punto} \rangle$ con prima(s) ‘*’* o subíndice(s) ‘*_*...’ opcionales aplicando `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* $[\langle \textit{key} = \textit{val} \rangle]$, `\sparc{AB}` imprimirá ‘ $\widehat{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*arco A B*».

- El acento `\widearc` es proporcionado por algunos paquetes como `fourier-otf` o `kpfonts-otf`, en caso de no estar disponible se utilizará una versión interna para este.

Llaves para `\sparc`

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento $\langle \textit{punto, punto} \rangle$ o $\langle \textit{punto, punto, punto} \rangle$. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sparc{AB}` se verbaliza «*arco A B*»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «*arco*». El valor de esta llave *siempre* debe pasarse entre llaves ‘*{ }*’.

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*arco*» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «*arco*» y el $\langle \textit{argumento} \rangle$; si se establece en *false*, NVDA/MathCAT solo verbalizará el $\langle \textit{argumento} \rangle$ acorde al valor establecido por la llave `read-cmd`.

`silent-cmd = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*arco*» y del $\langle \textit{argumento} \rangle$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmarc`

```
\spmarc {⟨punto, punto⟩}
\spmarc [⟨key = val⟩] {⟨punto, punto⟩}
\spmarc {⟨punto, punto, punto⟩}
\spmarc [⟨key = val⟩] {⟨punto, punto, punto⟩}
```

El comando `\spmarc` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma $\langle \textit{punto, punto} \rangle$ o $\langle \textit{punto, punto, punto} \rangle$ con prima(s) ‘*’* o subíndice(s) ‘*_*...’ opcionales colocando ‘*m*’ antes de aplicar `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* $[\langle \textit{key} = \textit{val} \rangle]$, `\spmarc{AB}` imprimirá ‘ $m\widehat{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*medida del arco A B*».

- La distinción entre `\sparc` y `\spmarc` es semántica: `\sparc` representa el *objeto geométrico* ‘ $\widehat{AB}$ ’; `\spmarc` representa su *medida* (un valor en grados).

Llaves para `\spmarc`

`read-cmd` = {`<school | mathcat>`} default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento {`<punto, punto>`} o {`<punto, punto, punto>`}. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmarc{AB}` se verbaliza «medida del arco *A B*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = {`<prefijo>`} default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el concepto «arco». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = {`<true | false>`} default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» y el concepto «arco» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará «medida del arco» y el {`<argumento>`}; si se establece en `false`, NVDA/MathCAT solo verbalizará el {`<argumento>`} acorde al valor establecido por la llave `read-cmd`.

`silent-cmd` = {`<true | false>`} default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», el concepto «arco» y el {`<argumento>`} pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPoly`

`\spPoly` {`<punto, punto, punto, ...>`}
`\spPoly` [`<key = val>`] {`<punto, punto, punto, ...>`}

El comando `\spPoly` trabaja en *modo matemático* y recibe el *argumento obligatorio* {`<punto, punto, punto, ...>`}, una lista de al menos tres {`<puntos>`} con prima(s) ' ' o subíndice(s) '_{...}' opcionales. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determinan acorde al número de puntos en el argumento y la llave `concept`.

Si son tres puntos se aplica `\triangle` y añade el concepto «triángulo» a la verbalización; si son cuatro puntos se aplica `\mdlgwhsquare` y añade el concepto «cuadrado» a la verbalización; si son cinco o más puntos no se aplica ningún símbolo ni se añade verbalización.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spPoly{ABC}` imprimirá '△ABC' en la salida PDF y NVDA/MathCAT verbalizará «triángulo *A B C*»; `\spPoly{ABCD}` imprimirá '□ABCD' y verbalizará «cuadrado *A B C D*»; `\spPoly{ABCDE}` imprimirá 'ABCDE' y verbalizará «*A B C D E*».

Llaves para `\spPoly`

`read-cmd` = {`<school | mathcat>`} default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento {`<punto, punto, punto, ...>`}. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPoly{ABC}` se verbaliza «triángulo *A B C*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = {`<prefijo>`} default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el {`<argumento>`} o los conceptos «triángulo» y «cuadrado» establecido por la llave `concept`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = {`<true | false>`} default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT de los conceptos «triángulo» y «cuadrado» acorde al número de puntos en el {`<argumento>`} al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará «triángulo» o «cuadrado» y el {`<argumento>`}; si se establece en `false`, NVDA/MathCAT solo verbalizará el {`<argumento>`} acorde al valor establecido por la llave `read-cmd`.

`print-sym` = {`<true | false>`} default: `true`

Establece si se *usará* `\triangle` o `\mdlgwhsquare` en la impresión sin modificar la verbalización de este. Si se establece en `true`, `\spPoly[print-sym=true]{ABC}` imprimirá '△ABC' en la salida PDF y NVDA/MathCAT verbalizará «triángulo *A B C*»; `\spPoly[print-sym=true]{ABCD}` imprimirá '□ABCD' en la salida PDF y NVDA/MathCAT verbalizará «cuadrado *A B C*»; si se establece en `false`, solo se imprimirá el {`<argumento>`} en la salida PDF y la verbalización será la misma.

`silent-cmd` = {`<true | false>`} default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT de los conceptos establecido por la llave `concept` y el {`<argumento>`} pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spAfig`

<code>\spAfig</code>	<code>\spAfig</code>	<code>\spAfig(\langle F P B L T \rangle)</code>
<code>\spAfig[\langle key = val \rangle]</code>	<code>\spAfig[\langle key = val \rangle](\langle F P B L T \rangle)</code>	
<code>\spAfig(\langle 1 2 3 \dots n \rangle)</code>	<code>\spAfig(\langle punto, punto, punto, \dots \rangle)</code>	
<code>\spAfig[\langle key = val \rangle](\langle 1 2 3 \dots n \rangle)</code>	<code>\spAfig[\langle key = val \rangle](\langle punto, punto, punto, \dots \rangle)</code>	

El comando `\spAfig` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis*, el cual es *completamente opcional*, representando el área de una figura poligonal en la impresión aplicando `\mathcal{A}`.

El argumento admite tres formas: una *lista de vértices* (`\langle punto, punto, punto, \dots \rangle`) (al menos tres); un *número o letra sueltos* (`\langle 1 \rangle`), (`\langle n \rangle`); o una *letra de una tabla fija* (F, P, B, L) con subíndice numérico opcional, (`\langle F_{1} \rangle`), (`\langle F_{\{1\}} \rangle`).

Si no se proporciona el *argumento opcional* [`\langle key = val \rangle`] ni el argumento delimitado por paréntesis, `\spAfig` imprimirá ' $\mathcal{A}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área». Con vértices, `\spAfig(A, B, C)` imprimirá ' $\mathcal{A}_{\triangle ABC}$ ' y verbalizará «área del triángulo A B C». Con un número o letra sueltos, `\spAfig(1)` imprimirá ' $\mathcal{A}_1$ ' y verbalizará «área uno»; `\spAfig(n)` imprimirá ' $\mathcal{A}_n$ ' y verbalizará «área n». Con una letra de la tabla, `\spAfig(F)` imprimirá ' $\mathcal{A}_F$ ' y verbalizará «área de la figura»; `\spAfig(F_{1})` imprimirá ' $\mathcal{A}_{F_1}$ ' y verbalizará «área de la figura uno».

Llaves para `\spAfig`

`read-cmd = { \langle school | mathcat \rangle }` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spAfig(A, B, C)` se verbaliza «área del triángulo A B C»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = { \langle prefijo \rangle }` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «área». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`print-sym = { \langle auto | triangle | square | none \rangle }` default: *auto*

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión. Con vértices, *auto* infiere el símbolo según el número de puntos (tres o cuatro); *none* lo omite y NVDA/MathCAT verbaliza solo los vértices sin nombrar la figura: `\spAfig[print-sym=none](A, B, C)` imprimirá ' $\mathcal{A}_{ABC}$ ' y verbalizará «área A B C». Forzar *triangle/square* exige que coincida con el número real de vértices; en caso contrario se produce un error. Con un número o letra sueltos, *auto* y *none* se comportan igual (sin símbolo); *triangle/square* anida el argumento bajo el símbolo: `\spAfig[print-sym=triangle](1)` imprimirá ' $\mathcal{A}_{\triangle_1}$ ' y verbalizará «área del triángulo uno». Esta llave no puede combinarse con una letra de la tabla (F, P, B, L).

`read-sub = { \langle true | false \rangle }` default: *false*

Establece si NVDA/MathCAT antepone la palabra «sub» al verbalizar el número o letra del argumento único. Si se establece en *true*, `\spAfig[read-sub=true](1)` verbaliza «área sub uno»; si se establece en *false*, verbaliza «área uno». No tiene efecto en la notación de vértices ni cuando el argumento es una letra de la tabla sin subíndice.

`silent-cmd = { \langle true | false \rangle }` default: *false*

Establece si se *silencia por completo* la verbalización en NVDA/MathCAT, sin afectar la impresión. Si se establece en *true*, NVDA/MathCAT NO verbalizará nada; si se establece en *false*, NVDA/MathCAT verbalizará normalmente.

`read-arg = { \langle nombre \rangle }` default: *no usado*

Sobrescribe el *nombre de la figura* determinado automáticamente por el número de puntos en el argumento, que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spAfig[read-arg={trapezio}](A, B, C, D)` imprimirá ' $\mathcal{A}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del trapecio A B C D».

El comando `\spPfig`

<code>\spPfig</code>	<code>\spPfig</code>	<code>\spPfig(\langle F P B L T \rangle)</code>
<code>\spPfig[\langle key = val \rangle]</code>	<code>\spPfig[\langle key = val \rangle](\langle F P B L T \rangle)</code>	
<code>\spPfig(\langle 1 2 3 \dots n \rangle)</code>	<code>\spPfig(\langle punto, punto, punto, \dots \rangle)</code>	
<code>\spPfig[\langle key = val \rangle](\langle 1 2 3 \dots n \rangle)</code>	<code>\spPfig[\langle key = val \rangle](\langle punto, punto, punto, \dots \rangle)</code>	

El comando `\spPfig` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis*, el cual es *completamente opcional*, representando el *perímetro* de una figura poligonal en la impresión aplicando `\mathcal{P}`.

El argumento admite las mismas tres formas que `\spAfig` (§7): una *lista de vértices*, un *número o letra sueltos*, o una *letra de la tabla fija* (F, P, B, L) con subíndice numérico opcional.

Si no se proporciona el *argumento opcional* [`\langle key = val \rangle`] ni el argumento delimitado por paréntesis, `\spPfig` imprimirá ‘ $\mathcal{P}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro*». Con vértices, `\spPfig(A, B, C)` imprimirá ‘ $\mathcal{P}_{\triangle ABC}$ ’ y verbalizará «*perímetro del triángulo A B C*». Con un número o letra sueltos, `\spPfig(2)` imprimirá ‘ $\mathcal{P}_2$ ’ y verbalizará «*perímetro dos*». Con una letra de la tabla, `\spPfig(P_{\{2\}})` imprimirá ‘ $\mathcal{P}_{p_2}$ ’ y verbalizará «*perímetro del polígono dos*».

Llaves para `\spPfig`

`read-cmd = { \langle school | mathcat \rangle }` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPfig(A, B, C)` se verbaliza «*perímetro del triángulo A B C*»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = { \langle prefijo \rangle }` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «*perímetro*». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`print-sym = { \langle auto | triangle | square | none \rangle }` default: *auto*

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión, con la misma semántica que en `\spAfig` (§7).

`read-sub = { \langle true | false \rangle }` default: *false*

Establece si NVDA/MathCAT antepone la palabra «sub» al verbalizar el número o letra del argumento único, con la misma semántica que en `\spAfig` (§7).

`silent-cmd = { \langle true | false \rangle }` default: *false*

Establece si se *silencia por completo* la verbalización en NVDA/MathCAT, sin afectar la impresión. Si se establece en *true*, NVDA/MathCAT NO verbalizará nada; si se establece en *false*, NVDA/MathCAT verbalizará normalmente.

`read-arg = { \langle nombre \rangle }` default: *no usado*

Sobrescribe el *nombre de la figura* determinado automáticamente por el número de puntos en el argumento, que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’. Por ejemplo: `\spPfig[read-arg={trapezio}](A, B, C, D)` imprimirá ‘ $\mathcal{P}_{\square ABCD}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro del trapecio A B C D*».

8 Referencias

- [1] ROBERTSON, WILL . “unicode-math - Unicode mathematics support for X_YTeX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/unicode-math>, 2023.
- [2] KRUEGER, MARCEL. “LuaMML - Automatically generate MathML from LuaTeX math mode material”. Available from CTAN, <https://ctan.org/pkg/luamml>, 2026.
- [3] The L^AT_EX Project. “fontspec - Advanced font selection for X_YL^AT_EX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/fontspec>, 2025.
- [4] VOß, HERBERT. “libertinus-otf - Support for Libertinus OpenType”. Available from CTAN, <https://www.ctan.org/pkg/libertinus-otf>, 2025.
- [5] FLIPO, DANIEL. “erewhon-math - Utopia based OpenType Math font”. Available from CTAN, <https://ctan.org/pkg/erewhon-math>, 2026.
- [6] HOFSTRA, SILKE. “sourcecodepro - Use SourceCodePro with T_EX(-alike) systems”. Available from CTAN, <https://ctan.org/pkg/sourcecodepro>, 2025.
- [7] GONZÁLEZ, PABLO. “scontents - Stores L^AT_EX contents in memory or files”. Available from CTAN, <https://www.ctan.org/pkg/scontents>, 2026.
- [8] GONZÁLEZ, PABLO. “enumext - Enumerate exercise sheets”. Available from CTAN, <https://www.ctan.org/pkg/enumext>, 2026.
- [9] BRAAMS, JOHANNES AND BEZOS, JAVIER. “babel - Multilingual support for L^AT_EX, LuaL^AT_EX, X_YL^AT_EX, and Plain T_EX”. Available from CTAN, <https://www.ctan.org/pkg/babel>, 2026.
- [10] BEZOS, JAVIER. “babel-spanish - Babel support for Spanish”. Available from CTAN, <https://www.ctan.org/pkg/babel-spanish>, 2026.
- [11] NV ACCESS. “NVDA - NonVisual Desktop Access screen reader”. Available from NV ACCESS, <https://www.nvaccess.org/>, 2026.
- [12] SOIFFER, NEIL. “MathCAT - Math to speech and braille translation”. Available from GITHUB, <https://nsoiffer.github.io/MathCAT/>, 2026.
- [13] ADOBE INC. “Adobe Acrobat Reader - Reliable PDF viewer”. Available from ADOBE, <https://get.adobe.com/reader/>, 2026.
- [14] ISO. “ISO 32000-2:2020 - Portable Document Format (PDF 2.0)”. Available from PDF ASSOCIATION, <https://pdfa.org/resource/iso-32000-2/>, 2020.
- [15] PDF ASSOCIATION. “WTPDF - Well-Tagged PDF specification”. Available from PDFA, <https://pdfa.org/wtpdf/>, 2024.
- [16] PDF ASSOCIATION. “PDF 2.0 Errata Collection 3”. Available from PDFA, <https://pdfa.org/PDF-2-0-errata-collection-3-now-available/>, 2026.
- [17] L^AT_EX PROJECT LWG. “Best Practice Guide: Math in PDF”. Available from PDFA, <https://pdfa.org/resource/best-practice-guide-math-in-PDF/>, 2026.
- [18] JOHNSON, DUFF. “PDF 2.0: New Features, Real-World Impact”. Available from PDFA, <https://pdfa.org/PDF-2-0-new-features-real-world-impact/>, 2026.
- [19] The L^AT_EX Project. “LaTeX Tagged PDF Project - Documentation and resources”. Available from L^AT_EX PROJECT, <https://latex3.github.io/tagging-project/>, 2026.
- [20] VERAPDF CONSORTIUM. “veraPDF - Industry supported PDF/A and PDF/UA validator”. Available from VERAPDF, <https://verapdf.org/>, 2026.
- [21] DUAL LAB. “ngPDF - Next Generation PDF demonstrator and validator”. Available from NGPDF, <https://ngpdf.com/>, 2026.
- [22] BERRY, KARL. “L^AT_EX 2_ε: An Unofficial Reference Manual”. Available from CTAN, <https://ctan.org/pkg/latex2e-help-texinfo>, 2026.
- [23] The L^AT_EX Project. “Extensive support for hypertext in L^AT_EX”. Available from CTAN, <https://www.ctan.org/pkg/hyperref>, 2026.
- [24] The L^AT_EX Project. “The expl3 package”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.

- [25] The $\text{\LaTeX}$ Project. “The $\text{\LaTeX}$ 3 Interfaces”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.
- [26] The $\text{\LaTeX}$ Project. “The $\text{\LaTeX}$ 2 ϵ sources”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2026.
- [27] The $\text{\LaTeX}$ Project. “ $\text{\LaTeX}$ News, Issue 41, June 2025”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.
- [28] The $\text{\LaTeX}$ Project. “ $\text{\LaTeX}$ for authors current version”. Available from CTAN, <https://ctan.org/pkg/latex-base>, 2026.
- [29] FISCHER, ULRIKE. “tagpdf – $\text{\LaTeX}$ kernel code for PDF tagging”. Available from CTAN, <https://www.ctan.org/pkg/tagpdf>, 2026.
- [30] The $\text{\LaTeX}$ Project. “latex-lab – $\text{\LaTeX}$ laboratory”. Available from CTAN, <https://www.ctan.org/pkg/latex-lab>, 2026.

9 Change history

v0.99 (ctan), 2026-09-19 – First public release.

10 Índice de la Documentación

Los números en cursiva indican las páginas donde se describe la entrada correspondiente.

C	
Document class:	
article	1
book	1
letter	1
report	1
Commands provide by spintent :	
\setspintent	3
\spAfig	25
\spPfig	26
\spPoint	18
\spPoly	24
\spangle	21
\spang	23
\sparc	23
\spcoord	17
\spdate	11
\spdiv	13
\spdsym	12
\spmQ	16
\spmangle	22
\spmarc	23
\spmcd	15
\spmcm	15
\spmoney	6
\spmside	21
\spmsym	12
\spmul	13
\spnD	14
\spnM	14
\spnZ	16
\spnewunit	6
\spnum	3
\sprayo	19
\spread	11
\sprecta	19
\spshort	7
\spside	20
\spsiglo	11
\sptime	11
\spunitalias	6
\spunit	4
\spusep	12
\spvec	18
K	
Keys for \spAfig provide by spintent :	
prefix	25
print-sym	25
read-arg	25
read-cmd	25
read-sub	25
silent-cmd	25
Keys for \spangle provide by spintent :	
concept	22
prefix	22
read-cmd	22
recto	22
silent-cmd	22
Keys for \sparc provide by spintent :	
concept	23
prefix	23
read-cmd	23
silent-cmd	23
Keys for \spcoord provide by spintent :	
prefix	23
read-cmd	23
silent-cmd	23
Keys for \spdiv provide by spintent :	
read-parens	13
wrap-parens	13
Keys for \spdsym provide by spintent :	
prefix	12
read-sym	12
set-sym	12
suffix	12
Keys for \spmangle provide by spintent :	
concept	23
prefix	22
print-m	22
read-cmd	22
recto	22
silent-cmd	23
Keys for \spmarc provide by spintent :	
concept	24
prefix	24
read-cmd	24
silent-cmd	24
Keys for \spmcd provide by spintent :	
prefix	15
read-cmd	15
result	15
sample-arg	15
silent-cmd	15
upper-case	15
Keys for \spmcm provide by spintent :	
prefix	16
read-cmd	16
result	16
sample-arg	16
silent-cmd	16
upper-case	16
Keys for \spmoney provide by spintent :	
currency	7
dolar-math	7
sub-units	7
Keys for \spmQ provide by spintent :	
join-word	17
print-dfrac	17
read-parens	17
use-spnZ	17
wrap-parens	17
Keys for \spmside provide by spintent :	
concept	21
prefix	21
print-m	21
read-arg	21
read-cmd	21
read-sty	21
silent-cmd	21
Keys for \spmsym provide by spintent :	
not-print	13
prefix	13
read-sym	13

set-sym	13	Keys for \spsiglo provide by spintent :	
suffix	13	print-era	11
Keys for \spmul provide by spintent :		Keys for \spunit provide by spintent :	
read-parens	13	print-cdot	4
wrap-parens	13	read-cdot	4
Keys for \spnD provide by spintent :		read-slash	4
prefix	14	Keys for \spusep provide by spintent :	
result-num	14	silent	12
result	14	size	12
sample-arg	14	Keys for \spvec provide by spintent :	
silent-cmd	14	autofit	18
Keys for \spnM provide by spintent :		concept	19
prefix	14	prefix	18
result-num	14	read-cmd	18
result	14	silent-cmd	19
sample-arg	14	Keys for \spdiv provide by spintent :	
silent-cmd	15	cifras-sep	13
Keys for \spnum provide by spintent :		notation-sym	13
cifras-sep	3	read-period-sep	13
notation-sym	4	read-sign	13
read-period-sep	3	read-sym	13
read-sign	3	set-sym	13
Keys for \spnZ provide by spintent :		Keys for \spdsym provide by spintent :	
read-parens	16	read-sym	12
wrap-parens	16	Keys for \spmcd provide by spintent :	
Keys for \spPfig provide by spintent :		read-cmd	15
prefix	26	Keys for \spmcm provide by spintent :	
print-sym	26	read-cmd	16
read-arg	26	Keys for \spmoney provide by spintent :	
read-cmd	26	cifras-sep	7
read-sub	26	Keys for \spmQ provide by spintent :	
silent-cmd	26	wrap-parens	17
Keys for \spPoint provide by spintent :		Keys for \spmsym provide by spintent :	
concept	18	read-sym	13
prefix	18	Keys for \spmul provide by spintent :	
read-cmd	18	cifras-sep	13
silent-cmd	18	notation-sym	13
Keys for \spPoly provide by spintent :		read-period-sep	13
concept	24	read-sign	13
prefix	24	read-sym	13
print-sym	24	set-sym	13
read-cmd	24	Keys for \spnZ provide by spintent :	
silent-cmd	24	cifras-sep	16
Keys for \sprayo provide by spintent :		read-sign	16
concept	20	Keys for \spside provide by spintent :	
prefix	19	read-sty	20, 21
read-cmd	19	Keys for \spunit provide by spintent :	
read-sty	19	cifras-sep	4
silent-cmd	20	notation-sym	4
Keys for \sprecta provide by spintent :		read-period-sep	4
concept	19	Keys for \spusep provide by spintent :	
prefix	19	silent	12
read-cmd	19	size	12
silent-cmd	19		
Keys for \spshort provide by spintent :			
read-arg	9		
small-caps	9		
Keys for \spside provide by spintent :			
concept	20		
prefix	20		
read-arg	20		
read-cmd	20		
read-sty	20		
silent-cmd	20		

M

Lua module provide by **spintent**:

spintent.lua	6, 7, 9, 10
--------------------	-------------

P

Packages :

babel-spanish	2
babel	2, 7
documentmetadata-support	1
enumext	1
expl3	3
fontspec	2

fourier-otf	2	luamml	1
hyperref	2	scontents	1
l3keys	3	sourcecodepro	2
latex-lab-mathintent	1	tagpdf	1, 7, 11
libertinus-otf	2	unicode-math	1

11 Implementation

The most recent publicly released version of `spintent` is available at CTAN: <https://www.ctan.org/pkg/spintent>. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: <https://github.com/pablgonz/spintent/issues>.

- The documentation presented here is far from professional, it contains a lot of obvious information that to the eye of a TeXpert are superfluous, but, after so many years developing this project is the only way to remember what does what.

11.1 General conventions

Functions of the form `\__spintent_luafun_some:n` or `\__spintent_luafun_some:nn` are defined (created) in the Lua module `spintent.lua` and only their variants are declared in `expl3`.

Similarly, variables of the form `\l__spintent_luaset_some_tl` or `\l__spintent_luaset_some_str` are defined in the Lua module `spintent.lua` and only declared in `expl3`.

- All variables and functions defined in this package are private and are NOT intended to work or be used by another package or module.

11.2 Initial set up

Start the DocStrip guards.

```
1 (*package)
```

Identify the internal prefix (L^AT_EX3 DocStrip convention) for `l3doc` class.

```
2 (@@=spintent)
```

11.3 Declaration of the package

First we will make sure we have a minimum (super updated) release version of L^AT_EX to work correctly, otherwise, we will return an “error”.

```
3 \NeedsTeXFormat{LaTeX2e}[2026-11-01]
4 \IfFormatAtLeastF { 2026-11-01 }
5 {
6   \PackageError { spintent }
7     { LaTeX~kernel~too~old~need~2026-11-01~or~later }
8     {
9       spintent~requires~a~LaTeX2e~release~2026-11-01~or~
10      later.~Update~your~TeX~distribution.
11     }
12 }
```

Now declare the `spintent` package and add only two keys: `mathcal` and `AFghost`, which can be passed as package options.

```
13 \ProvidesExplPackage {spintent} {2026-09-19} {0.99} {Spanish parse intents}
14 \keys_define:nn { spintent }
15 {
16   mathcal .usage:n    = load,
17   mathcal .bool_set:N = \l__spintent_mathcal_bool,
18   mathcal .initial:n  = true,
19   mathcal .value_required:n = true,
20   AFghost .usage:n    = load,
21   AFghost .bool_set:N = \l__spintent_debug_ghost_bool,
22   AFghost .initial:n  = false,
23   AFghost .value_required:n = true,
24 }
25 \ProcessKeyOptions [ spintent ]
```

We check if the requirements for running the `spintent` package are met; that is: `\DocumentMetadata` must be active, Lua_LTeX must be running, and the `unicode-math` or `lua-unicode-math` package must be loaded. If neither is present, we will load `unicode-math`. If the conditions cannot be met, we will return a “fatal error”.

```
26 \IfDocumentMetadataTF
27 {
28   \sys_if_engine luatex:TF
29   {
30     \msg_new:nnn { spintent } { package-not-load }
31     {
32       The~'#1'~package~will~be~loaded~as~a~dependency.
33     }
34     \IfPackageLoadedF { unicode-math }
35     {
36       \IfPackageLoadedF { lua-unicode-math }
37       {
38         \msg_info:nnn { spintent } { package-not-load } { unicode-math }
39         \RequirePackage{unicode-math}

```

```

40     }
41   }
42   \msg_new:nnn { spintent } { env-success }
43   { Everything~looks~correct,~spintent~will~make~the~intent! }
44   \msg_info:nn { spintent } { env-success }
45 }
46 {
47   \msg_new:nnnn { spintent } { fatal-engine-requires-luatex }
48   { The~spintent~package~requires~LuaLaTeX. }
49   { This~package~relies~on~Lua~module.~Compile~with~lualatex. }
50   \msg_fatal:nn { spintent } { fatal-engine-requires-luatex }
51 }
52 }
53 {
54   \msg_new:nnnn { spintent } { fatal-metadata-missing }
55   { \token_to_str:N \DocumentMetadata \c_space_tl is-missing. }
56   {
57     The~spintent~package~requires~tagged~PDF~active.\\
58     You~must~declare~\token_to_str:N \DocumentMetadata { tagging=on } ~
59   }
60   \msg_fatal:nn { spintent } { fatal-metadata-missing }
61 }

```

11.4 Some kernel variants

```

\str_if_eq:nVTF \str_if_eq:nVF
\str_uppercase:V \str_lowercase:V
\seq_set_from_clist:Ne
\cs_generate_variant:Nn \str_if_eq:nnTF { V }
\cs_generate_variant:Nn \str_if_eq:nnF { V }
\cs_generate_variant:Nn \str_uppercase:n { V }
\cs_generate_variant:Nn \str_lowercase:n { V }
\cs_generate_variant:Nn \seq_set_from_clist:Nn { Ne }

```

(End of definition for `\str_if_eq:nVTF` and others.)

11.5 Variables for the key `mathcal` and `\rightanglesqr`

First, we declare the string variable `\l__spintent_luaset_cal_fam_ok_str` used by the function `\__spintent_luafun_classic_mathcal:nn` (controlled by the key `mathcal`) alongside the string variable `\l__spintent_luaset_sym_fam_ok_str` used by the function `\__spintent_luafun_right_angle_sqr:` to copy `\rightanglesqr` before loading the Lua module `spintent.lua`. This is necessary due to the implementation of the functions within the Lua module; otherwise, `expl3` would issue a warning regarding duplicate variable declarations.

```

67 \str_new:N \l__spintent_luaset_cal_fam_ok_str
68 \str_new:N \l__spintent_luaset_sym_fam_ok_str

```

(End of definition for `\l__spintent_luaset_cal_fam_ok_str` and `\l__spintent_luaset_sym_fam_ok_str`.)

11.6 Loading the Lua module and functions

`spintent.lua` Loads the Lua module `spintent.lua`, where every `_luafun_` function used throughout the package is registered.

```

69 \lua_load_module:n { spintent.lua }

```

(End of definition for `spintent.lua`.)

`\__spintent_luafun_set_debug_ghost:n` Passes `AFghost` to Lua as a plain string. When `true`, both ghost characters are marked `sptmp="outer"` or `sptmp="inner"` in the file `-luamml-mathml.html`, so an external script can identify and strip them before generating the Associated~Files structure for *tagged* PDF — `math-AF` mode does not want either ghost present.

```

70 \bool_if:NTF \l__spintent_debug_ghost_bool
71 { \__spintent_luafun_set_debug_ghost:n { true } }
72 { \__spintent_luafun_set_debug_ghost:n { false } }

```

(End of definition for `\__spintent_luafun_set_debug_ghost:n`.)

`\l__spintent_scope_attr_int` A LuaTeX attribute number, allocated collision-free from Lua via `luatexbase.new_attribute` and exposed here as an integer. Every command that uses the shared function `\__spintent_luafun_mrow_intent_arg:nn` activates it, via `\attribute \l__spintent_scope_attr_int = 1`, inside its own group — marking every node typeset within as eligible for that queue to claim. Without this mark, the queue could claim content from an unrelated command sitting in the same formula.

```

73 \int_new:N \l__spintent_scope_attr_int
74 %% \str_new:N \l__spintent_scope_luaset_attr_str
75 \int_set:Nn \l__spintent_scope_attr_int { \l__spintent_scope_luaset_attr_str }

```

(End of definition for `\l__spintent_scope_attr_int` and `\l__spintent_scope_luaset_attr_str`.)

`\__spintent_luafun_spunit_lookup_alias:V` Variants generated for `\__spintent_luafun_spunit_lookup_alias:n` (used by `\spunit`), `\__spintent_luafun_`
`\__spintent_luafun_spmoney_lookup_data:V` (used by `\spmoney`), `\__spintent_luafun_spmQ_scale_int:nn` (used by `\spmQ`) and `\__spintent_luafun_clean`
`\__spintent_luafun_spmQ_scale_int:Vn` (used by `\spnum`).
`\__spintent_luafun_clean_split_and_set:V`

```

76 \cs_generate_variant:Nn \__spintent_luafun_spunit_lookup_alias:n { V }
77 \cs_generate_variant:Nn \__spintent_luafun_spmoney_lookup_data:n { V }
78 \cs_generate_variant:Nn \__spintent_luafun_spmQ_scale_int:nn { Vn }
79 \cs_generate_variant:Nn \__spintent_luafun_clean_split_and_set:n { V }

```

(End of definition for `\__spintent_luafun_spunit_lookup_alias:V` and others.)

11.7 Internal implementation for `\mathcal` and `\rightanglesqr`

To simplify user configuration and obtain the expected output style pdf_{TeX} for `\mathcal`, we will provide a fallback to the font NewCMMath-Regular.otf, which contains the glyphs for these characters. Using this same font, we will obtain the symbol `\rightanglesqr` for the key `recto`.

`mathcal-fallback`
`right-angle-sqr-fallback`

First, we'll define two error messages in case our fallback fails.

```

80 \hook_gput_code:nnn { begindocument } { spintent }
81 {
82   \__spintent_luafun_init_math_fallbacks:
83 }
84 \msg_new:nnn { spintent } { mathcal-fallback }
85 {
86   Using~classic~\token_to_str:N \mathcal \c_space_tl as~fallback~
87   (font~not~found~or~no~free~math~family~available).~\msg_line_context:
88 }
89 \msg_new:nnn { spintent } { right-angle-sqr-fallback }
90 {
91   Using~\token_to_str:N \rightangle \c_space_tl as~fallback~
92   for~the~right-angle~ glyph.~\msg_line_context:
93 }

```

(End of definition for `mathcal-fallback` and `right-angle-sqr-fallback`.)

`\__spintent_mathcal:n`

Now we define the internal function `\__spintent_mathcal:n` for `\mathcal` which checks the state of the variable `\l__spintent_mathcal_bool` set by the package key `mathcal`; if its state is true, we check the string variable `\l__spintent_luaset_cal_fam_ok_str` and if the latter has the value true, we call the function `\__spintent_luafun_classic_mathcal:n` and otherwise (we cannot find the source in the system) we return a warning message and directly use `\mathcal`. Otherwise, that is, `mathcal=false` we simply use `\mathcal`.

```

94 \cs_new_protected:Nn \__spintent_mathcal:n
95 {
96   \bool_if:NTF \l__spintent_mathcal_bool
97   {
98     \str_if_eq:VnTF \l__spintent_luaset_cal_fam_ok_str { true }
99     { \__spintent_luafun_classic_mathcal:n {#1} }
100     {
101       \msg_warning:nn { spintent } { mathcal-fallback }
102       \mathcal {#1}
103     }
104   }
105   { \mathcal {#1} }
106 }

```

(End of definition for `\__spintent_mathcal:n`.)

`\__spintent_right_angle_sqr:`

The internal function `\__spintent_right_angle_sqr:` establishes the internal copy of the command `\rightanglesqr`. First, we check the string variable `\l__spintent_luaset_sym_fam_ok_str`. If it is true, we call the function `\__spintent__luafun_right_angle_sqr:`; otherwise (we do not find the font in the system), we return a warning message and use `\rightangle` instead.

```

107 \cs_new_protected:Nn \__spintent_right_angle_sqr:
108 {
109   \str_if_eq:VnTF \l__spintent_luaset_sym_fam_ok_str { true }
110   { \__spintent_luafun_right_angle_sqr: }
111   {
112     \msg_warning:nn { spintent } { right-angle-sqr-fallback }
113     \rightangle
114   }
115 }

```

(End of definition for `\__spintent_right_angle_sqr:`.)

11.8 Shared annotation mechanisms: the queue and the ghosts

Almost all commands in the `spintent` package share three annotation functions, the mechanism of which is defined in the Lua module `spintent.lua`.

`\__spintent_luafun_mrow_intent_arg:nn`

The internal function `\__spintent_luafun_mrow_intent_arg:nn` — called directly, with no `expl3` wrapper — pushes one entry onto a queue: a concept `intent` (`#1`, possibly empty) and a Lua table literal `arg=/intent=(#2)`. A `pre_mlist_to_hlist_filter` callback (installed once, before `luamml[2]`’s own) walks the formula’s top-level noads in order and, for each one whose nucleus or subscript was typeset inside a `spintent` command’s own group (marked via a LuaTeX `\attribute` activated by every command that uses this queue), claims it for the next pending entry: `arg`/a literal `intent` land on that leaf, and, when the concept `intent` is non-empty, the leaf is wrapped in a native `<mrow>` carrying it — no token, no ghost involved. Entries are consumed strictly in order; any left unclaimed when the formula ends are discarded rather than carried into the next one. A native `\MathMLarg/\MathMLintent` claim is invisible to this queue: mixing the two for the same content risks the queue claiming something already tagged natively, or never being claimed at all.

(End of definition for `\__spintent_luafun_mrow_intent_arg:nn`.)

The other two are “ghost” characters (an invisible U+2063, self-tagged `:silent` through their own `mathml_filter` — so the queue above never mistakes either for one of its own free targets) dropped in as an extra sibling so the resulting `<mrow>` is not the formula’s sole top-level content, which `luamml` would otherwise promote straight to `<math intent=...>` instead of leaving it nested (issue #26).

`\__spintent_invisible_sep:`

Placed once, before opening the formula’s own outer `\MathMLintent` — protects the *whole formula* from collapsing when that outer group is its only top-level content. Fires at most once per document: past the first call, in any command, it does nothing, since one sibling anywhere in the formula is already enough.

```
116 \cs_new_protected:Nn \__spintent_invisible_sep:
117 {
118   \__spintent_luafun_invisible_sep:
119 }
```

Executing this function writes the following line directly to the output file `-luamml-mathml.html`:

```
<mi intent=":silent" mathvariant="normal"></mi>
```

and if the package key `AFghost=true` is used, the output is:

```
<mi intent=":silent" mathvariant="normal" sptmp="outer"></mi>
```

If you wish to use math-AF mode, you can remove the line generated by `AFghost=true` using the Lua script `spiafcls`.

(End of definition for `\__spintent_invisible_sep:`.)

`\__spintent_inner_sep:`

Placed just inside a native `\MathMLintent` group — protects *that specific group* when it too would otherwise hold only one child. Never deduplicates: called once per such group, however many appear, nested or sibling, in the same formula.

```
120 \cs_new_protected:Nn \__spintent_inner_sep:
121 {
122   \__spintent_luafun_inner_sep:
123 }
```

Executing this function writes the following line directly to the output file `-luamml-mathml.html`:

```
<mi intent=":silent" mathvariant="normal"></mi>
```

and if the package key `AFghost=true` is used, the output is:

```
<mi intent=":silent" mathvariant="normal" sptmp="inner"></mi>
```

If you wish to use math-AF mode, you can remove the line generated by `AFghost=true` using the Lua script `spiafcls`.

(End of definition for `\__spintent_inner_sep:`.)

`\c__spintent_invisible_sep_int`

A raw, untagged copy of the same U+2063 character, via the LuaTeX primitive `\Umathchardef` — for use as ordinary content *inside* an explicit `\MathMLintent{<intent>}` call (typically `:silent`), not as a sibling on its own.

```
124 \hook_gput_code:nnn { begindocument } { spintent }
125 {
126   \Umathchardef \c__spintent_invisible_sep_int = "0 "0 "2063
127 }
```

Documented in `source3` on page 887 for `\int_const:Nn`: “As stated, most constants can be defined as `\chardef` or `\mathchardef` but that’s engine dependent.” —so this appears to be the correct option here.

(End of definition for `\c__spintent_invisible_sep_int`.)

11.9 Normalize argument and affixes

We normalize the `{⟨argument⟩}` to support affixes (prefixes/suffixes) by trimming leading and trailing spaces, converting internal spaces to hyphens, collapsing consecutive hyphens, and removing leading or trailing hyphens.

`\__spintent_collapse_hyphens:N` A recursive helper to collapse multiple hyphens into a single one.

```

128 \cs_new_protected:Nn \__spintent_collapse_hyphens:N
129 {
130   \tl_if_in:NnT #1 { -- }
131   {
132     \tl_replace_all:Nnn #1 { -- } { - }
133     \__spintent_collapse_hyphens:N #1
134   }
135 }
```

(End of definition for `\__spintent_collapse_hyphens:N`.)

`\__spintent_sanitize_affix:Nn` The function `\__spintent_sanitize_affix:Nn` checks if the `{⟨argument⟩}` passed in ‘#2’ is *blank*. If so, it clears the target token list variable passed in ‘#1’. Otherwise, trimming bounds, converting spaces ‘`␣`’ to hyphens ‘`-`’, collapsing multiple hyphens, and safely stripping leading/trailing hyphens. For example, if you pass a target variable and raw text:

```
\__spintent_sanitize_affix:Nn \l__spintent_spsomesym_prefix_tl{ foo bar baz }
```

The function will “store” the normalized string `foo-bar-baz` inside `\l__spintent_spsomesym_prefix_tl`, ready to be conditionally appended.

```

136 \cs_new_protected:Nn \__spintent_sanitize_affix:Nn
137 {
138   \tl_if_blank:nTF {#2}
139   { \tl_clear:N #1 }
140   {
141     \tl_set:Ne #1 {#2}
142     \tl_trim_spaces:N #1
143     \tl_replace_all:Nnn #1 { ~ } { - }
144     \__spintent_collapse_hyphens:N #1
145     \tl_put_left:Nn #1 { \q_mark }
146     \tl_put_right:Nn #1 { \q_stop }
147     \tl_replace_all:Nnn #1 { \q_mark - } { \q_mark }
148     \tl_replace_all:Nnn #1 { - \q_stop } { \q_stop }
149     \tl_remove_all:Nn #1 { \q_mark }
150     \tl_remove_all:Nn #1 { \q_stop }
151   }
152 }
153 \cs_generate_variant:Nn \__spintent_sanitize_affix:Nn { cn }
```

(End of definition for `\__spintent_sanitize_affix:Nn`.)

11.10 The command `\setspintent`

The `\setspintent` command is used to configure `⟨keys⟩` “globally” for the utilities provided by `spintent`.

11.10.1 Internal variables and error messages for `\setspintent`

`\l__spintent_setspintent_input_arg_cmd_tl` The variable `\l__spintent_setspintent_input_arg_cmd_tl` will store the command or command name passed to the “first” `{⟨argument⟩}` ‘#1’ of the command `\setspintent`.

```
154 \tl_new:N \l__spintent_setspintent_input_arg_cmd_tl
```

Error message if the `{⟨argument⟩}` ‘#1’ passed to `\setspintent` is a “non-existent” command.

```

155 \msg_new:nnnn { spintent } { unknown-command-setup }
156 {
157   Cannot~configure~unknown~command~'\exp_not:c{#1}'.
158 }
159 {
160   You~attempted~to~use~\token_to_str:N \setspintent{#1}{...},~but~the~
161   command~\exp_not:c{#1}~is~not~defined.~Please~check~for~typos.
162 }
```

(End of definition for `\l__spintent_setspintent_input_arg_cmd_tl` and `unknown-command-setup`.)

```

__spintent_setspintent_set_keys:nn
__spintent_setspintent_set_keys:en

```

The function `__spintent_setspintent_set_keys:nn` will check if the command passed as `{⟨argument⟩}` exists in ‘#1’ and set the `⟨keys⟩` passed as `{⟨argument⟩}` in ‘#2’ to the correct path; otherwise, it will return an error.

```

163 \cs_new_protected:Nn __spintent_setspintent_set_keys:nn
164 {
165   \cs_if_exist:cTF { #1 }
166   {
167     \keys_set:nn { spintent / #1 } { #2 }
168   }
169   {
170     \msg_error:nnn { spintent } { unknown-command-setup } { #1 }
171   }
172 }
173 \cs_generate_variant:Nn __spintent_setspintent_set_keys:nn { en }

```

(End of definition for `__spintent_setspintent_set_keys:nn`.)

```

__spintent_setspintent_parse_args:nn
__spintent_setspintent_parse_args:Vn

```

The function `__spintent_setspintent_parse_args:nn` will check if the argument passed in #1 begins with ‘\’ or is just a *command name*. If it begins with ‘\’, it will remove it using `\cs_to_str:N` and then execute `__spintent_setspintent_set_keys:en`. Otherwise, i.e., if ‘\’ was not passed, it will directly execute `__spintent_setspintent_set_keys:nn`.

```

174 \cs_new_protected:Nn __spintent_setspintent_parse_args:nn
175 {
176   \tl_if_single:nTF { #1 }
177   {
178     \token_if_cs:NTF #1
179     {
180       __spintent_setspintent_set_keys:en { \cs_to_str:N #1 } { #2 }
181     }
182     {
183       __spintent_setspintent_set_keys:nn { #1 } { #2 }
184     }
185   }
186   {
187     __spintent_setspintent_set_keys:nn { #1 } { #2 }
188   }
189 }
190 \cs_generate_variant:Nn __spintent_setspintent_parse_args:nn { Vn }

```

(End of definition for `__spintent_setspintent_parse_args:nn`.)

`\setspintent`

Now we define the user command `\setspintent`.

```

191 \NewDocumentCommand \setspintent { m m }
192 {
193   \tl_if_blank:nTF { #1 }
194   {
195     \msg_error:nnn { spintent } { unknown-command-setup } { <empty> }
196   }
197   {
198     \tl_set:Nn \l__spintent_setspintent_input_arg_cmd_tl { #1 }
199     \tl_trim_spaces:N \l__spintent_setspintent_input_arg_cmd_tl
200     __spintent_setspintent_parse_args:Vn \l__spintent_setspintent_input_arg_cmd_tl { #2 }
201   }
202 }

```

(End of definition for `\setspintent`. This function is documented on page 3.)

11.11 Handling unknown keys

```

\l__spintent_command_name_str
unknown-choice
cmd-key-unknown
cmd-key-value-unknown

```

We define the variable `\l__spintent_command_name_str` which will *store* the “name” of each command within the implementation along with the messages `unknown-choice` used by `⟨keys⟩` that use `.choice:n` in their implementation and the messages `cmd-key-unknown`, `cmd-key-value-unknown` used by the `⟨key⟩` `unknown` in commands that use `[⟨key = val⟩]`.

```

203 \str_new:N \l__spintent_command_name_str
204 \msg_new:nnn { spintent } { unknown-choice }
205 {
206   The~value~'~#3'~for~'~#1'~key~is~invalid~use~('~#2').
207 }
208 \msg_new:nnnn { spintent } { cmd-key-unknown }
209 {
210   The~key~'~#1'~is~unknown~by~command~

```

```

211     \c_backslash_str \l__spintent_command_name_str \c_space_tl
212     and~is~being~ignored~\msg_line_context:.
213 }
214 {
215     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
216     does~not~have~a~key~called~'#1'.\\
217     Check~that~you~have~spelled~the~key~name~correctly.
218 }
219 \msg_new:nnnn { spintent } { cmd-key-value-unknown }
220 {
221     The~key~'#1=#2'~is~unknown~by~command~
222     \c_backslash_str \l__spintent_command_name_str \c_space_tl
223     and~is~being~ignored~\msg_line_context:.
224 }
225 {
226     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
227     does~not~have~a~key~called~'#1'.\\
228     Check~that~you~have~spelled~the~key~name~correctly.
229 }

```

(End of definition for `\l__spintent_command_name_str` and others.)

```

\__spintent_unknown_keys:nn
\__spintent_unknown_keys:Vn
\__spintent_unknown_keys:n

```

The internal functions `\__spintent_unknown_keys:nn` and `\__spintent_unknown_keys:n` used by the `<key>` unknown.

```

230 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:nn
231 {
232     \tl_if_blank:nTF {#2}
233     { \msg_error:nnn { spintent } { cmd-key-unknown } {#1} }
234     { \msg_error:nnnn { spintent } { cmd-key-value-unknown } {#1} {#2} }
235 }
236 \cs_generate_variant:Nn \__spintent_unknown_keys_cmd:nn { Vn }
237 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:n
238 {
239     \__spintent_unknown_keys_cmd:Vn \l_keys_key_str {#1}
240 }

```

(End of definition for `\__spintent_unknown_keys:nn` and `\__spintent_unknown_keys:n`.)

11.12 Definition of core base variables

```

\__spintent_luafun_clean_split_and_set:n
\__spintent_luafun_clean_split_and_set:V

```

The function `\__spintent_luafun_clean_split_and_set:n` defined in the Lua module `spintent.lua`, is common to several commands in this implementation. It analyzes, processes, cleans and splits the `{<argument>}` passed to commands using internal functions defined in the Lua module `spintent.lua`. It isolates signs, integer components, decimal separators, decimal parte, decimal periodic part, exponents and final physical “units”, assigning them directly to `expl3` variables.

This function assigns the states of the variables ‘_str’ or ‘_tl’ described below to `true`, `false`, `error` or `stores` information according to the role it fulfills within the implementation.

(End of definition for `\__spintent_luafun_clean_split_and_set:n`.)

```

\l__spintent_luaset_sign_tl
\l__spintent_luaset_part_int_tl
\l__spintent_luaset_dec_sep_tl
\l__spintent_luaset_part_dec_tl
\l__spintent_luaset_part_period_tl
\l__spintent_luaset_has_integer_str
\l__spintent_luaset_has_natural_str

```

The variable `\l__spintent_luaset_sign_tl` stores the explicit ‘+’ or ‘−’ sign if present. The variable `\l__spintent_luaset_part_int_tl` stores the raw digit sequence of the “integer part”. The variable `\l__spintent_luaset_dec_sep_tl` stores the matched ‘,’ or ‘.’ used as the decimal separator. The variable `\l__spintent_luaset_part_dec_tl` stores the raw digits of the “finite decimal part” that follow the separator, and the variable `\l__spintent_luaset_part_period_tl` stores the digits of the “infinite periodic decimal part” that follow a semicolon ‘;’.

The variable `\l__spintent_luaset_has_integer_str` store the state `true` when it is *only* an “integer”. The variable `\l__spintent_luaset_has_natural_str` store the state `true` when it is *only* an “natural number”.

```

241 \tl_new:N \l__spintent_luaset_sign_tl
242 \tl_new:N \l__spintent_luaset_part_int_tl
243 \tl_new:N \l__spintent_luaset_dec_sep_tl
244 \tl_new:N \l__spintent_luaset_part_dec_tl
245 \tl_new:N \l__spintent_luaset_part_period_tl
246 \str_new:N \l__spintent_luaset_has_integer_str
247 \str_new:N \l__spintent_luaset_has_natural_str

```

(End of definition for `\l__spintent_luaset_sign_tl` and others.)

```

\l__spintent_luaset_only_part_int_str
\l__spintent_luaset_only_part_dec_str
\l__spintent_luaset_only_part_period_str
\l__spintent_luaset_format_part_int_str
\l__spintent_luaset_format_part_dec_str

```

The literal entries of the numbers that we will pass to the *intent function* :number of `\MathMLintent` are *stored* in `\l__spintent_luaset_only_part_int_str` for the “*integer part*”, in `\l__spintent_luaset_only_part_dec_str` for the “*finite decimal part*”, and in `\l__spintent_luaset_only_part_period_str` for the “*infinite periodic decimal part*”.

Formatted versions (`cifras-sep=true`) are *stored* in `\l__spintent_luaset_format_part_int_str` for the “*integer part*” and `\l__spintent_luaset_format_part_dec_str` for the “*finite decimal part*”.

```

248 \str_new:N \l__spintent_luaset_only_part_int_str
249 \str_new:N \l__spintent_luaset_only_part_dec_str
250 \str_new:N \l__spintent_luaset_only_part_period_str
251 \str_new:N \l__spintent_luaset_format_part_int_str
252 \str_new:N \l__spintent_luaset_format_part_dec_str

```

(End of definition for `\l__spintent_luaset_only_part_int_str` and others.)

```

\l__spintent_luaset_millions_str
\l__spintent_luaset_decimal_period_str
\l__spintent_luaset_not_decimal_period_str
\l__spintent_luaset_not_decimal_not_period_str
\l__spintent_luaset_dec_is_all_zeros_str
\l__spintent_luaset_has_exponent_str
\l__spintent_luaset_exponent_tl

```

The flag variable `\l__spintent_luaset_millions_str` indicates whether the integer part translates exactly to a multiple of one million when passed to `\MathMLintent`. To properly structure the reading and writing of numbers in MathML, we use the flag variables `\l__spintent_luaset_decimal_period_str`, `\l__spintent_luaset_not_decimal_period_str`, and `\l__spintent_luaset_not_decimal_not_period_str`. The flag variable `\l__spintent_luaset_dec_is_all_zeros_str` monitors whether the “*finite decimal part*” is completely zero (useful for “*pure periodic decimals*”).

The flag variable `\l__spintent_luaset_has_exponent_str` indicates whether active “*scientific notation*” was detected at the end of the entered number, passed with ‘e’ or ‘E’. If true, it will store the digits of the exponent in `\l__spintent_luaset_exponent_tl`.

```

253 \str_new:N \l__spintent_luaset_millions_str
254 \str_new:N \l__spintent_luaset_decimal_period_str
255 \str_new:N \l__spintent_luaset_not_decimal_period_str
256 \str_new:N \l__spintent_luaset_not_decimal_not_period_str
257 \str_new:N \l__spintent_luaset_dec_is_all_zeros_str
258 \str_new:N \l__spintent_luaset_has_exponent_str
259 \tl_new:N \l__spintent_luaset_exponent_tl

```

(End of definition for `\l__spintent_luaset_millions_str` and others.)

```

\l__spintent_luaset_has_units_str
\l__spintent_luaset_status_str
\l__spintent_luaset_arg_above_tl
\l__spintent_luaset_arg_below_tl
\l__spintent_luaset_denom_has_number_str

```

Since the function `\__spintent_luafun_clean_split_and_set:n` is common to `\spnum` (§11.14) and `\spunit` (§11.15) (as well as others), it determines the following unit-related variables after processing the `{⟨arguments⟩}` via the Lua module `spintent.lua`.

First, it checks if there is any residual text *after* the processed number that is a candidate for physical “*units*”. It sets the flag variable `\l__spintent_luaset_has_units_str` accordingly. Following this, it verifies if this text contains more than one “/”. It then sets the variable `\l__spintent_luaset_status_str` to `valid` or `multislash`. If the structure is `valid`, it splits the unit into `\l__spintent_luaset_arg_above_tl` and `\l__spintent_luaset_arg_below_tl`. Finally, it analyzes `\l__spintent_luaset_arg_below_tl`; if the denominator starts with a number, it sets `\l__spintent_luaset_denom_has_number_str` to `true`, otherwise to `false`.

```

260 \str_new:N \l__spintent_luaset_has_units_str
261 \str_new:N \l__spintent_luaset_status_str
262 \tl_new:N \l__spintent_luaset_arg_above_tl
263 \tl_new:N \l__spintent_luaset_arg_below_tl
264 \str_new:N \l__spintent_luaset_denom_has_number_str

```

(End of definition for `\l__spintent_luaset_has_units_str` and others.)

11.13 Common messages for text mode and math mode

Messages shared by the different commands for text mode or math mode.

```

265 \msg_new:nnnn { spintent } { need-math-mode }
266 {
267   Math~mode~required~for~'#1'~\msg_line_context:.
268 }
269 {
270   Use~inside~$...$~or~in~math~environment.
271 }
272 \msg_new:nnnn { spintent } { need-text-mode }
273 {
274   Text~mode~required~for~'#1'~\msg_line_context:.
275 }
276 {
277   Use~outside~of~$...$~or~math~environment.
278 }

```

11.14 The command `\spnum`

The user command `\spnum` is *first* of “the big four” provided by the `spintent` package and is fundamental to the implementation of other commands. It handles the processing of numerical input, scientific notation, and units of measurement, using the Lua module `spintent.lua`, which is responsible for separating and assigning variables defined in `expl3`.

From the `expl3` side, we handle the printing logic and the *intent functions* passed to `\MathMLintent` to achieve a valid MathML structure that is accepted by MathCAT.

11.14.1 Definition of keys and variables for `\spnum`

The variable `\l__spintenc_spnum_read_terse_bool` handled by the key `read-sign`, controls the *intent* passed to `\MathMLintent` to spoken the ‘+’ or ‘-’. If set to `clear`, it will say “*positivo*” or “*negativo*” after reading the number; if set to `terse`, it will say “*más*” or “*menos*” before reading the number.

The variable `\l__spintenc_spnum_read_period_sep_str` handled by the key `read-period-sep`, controls how the decimal separator ‘.’ (periodic part) is printed and spoken when the `{(argument)}` is a *pure decimal periodic* and is passed the *intent* to `\MathMLintent`. If set to `coma`, it will say “*coma*” and print ‘.’, if set to `punto` it will say “*punto*” and print ‘.’.

The variable `\l__spintenc_spnum__notation_sym_tl`, handled by the key `notation-sym`, determines whether ‘x’ will be printed when set to `times` or ‘.’ when set to `cdot`.

The variable `\l__spintenc_spnum_intent_read_sign_str` will *store* the spoken translation used in the *intent* function passed to `\MathMLintent` for the ‘+’ or ‘-’ according to the value passed to the key `read-sign`.

The variable `\l__spintenc_spnum_intent_read_dec_sep_str` *stores* the spoken translation that will be used in the *intent* function passed to `\MathMLintent` for reading the decimal separation as a “*coma*” or a “*punto*” according to the value found in the variable `\l__spintenc_luaset_dec_sep_tl`.

```
279 \bool_new:N \l__spintenc_spnum_read_terse_bool
280 \str_new:N \l__spintenc_spnum_read_period_sep_str
281 \tl_new:N \l__spintenc_spnum_notation_sym_tl
282 \str_new:N \l__spintenc_spnum_intent_read_sign_str
283 \str_new:N \l__spintenc_spnum_intent_read_dec_sep_str
```

(End of definition for `\l__spintenc_spnum_read_terse_bool` and others.)

```
cifras-sep      Now we add the keys cifras-sep, read-sign, read-period-sep and notation-sym.
read-sign
read-period-sep
notation-sym

284 \keys_define:nn { spintenc/spnum }
285 {
286   cifras-sep          .bool_set:N = \l__spintenc_spnum_cifras_sep_bool,
287   cifras-sep          .initial:n = true,
288   cifras-sep          .value_required:n = true,
289   read-sign           .choices:nn =
290     { terse , clear }
291     {
292       \int_case:nn { \l_keys_choice_int }
293       {
294         {1} { \bool_set_true:N \l__spintenc_spnum_read_terse_bool }
295         {2} { \bool_set_false:N \l__spintenc_spnum_read_terse_bool }
296       }
297     },
298   read-sign           .initial:n = terse,
299   read-sign           .value_required:n = true,
300   read-sign / unknown .code:n =
301     \msg_error:nneee { spintenc } { unknown-choice }
302     { read-sign } { terse , ~clear } { \exp_not:n {#1} },
303   read-period-sep     .choices:nn =
304     { coma , punto }
305     {
306       \int_case:nn { \l_keys_choice_int }
307       {
308         {1} { \str_set:Nn \l__spintenc_spnum_read_period_sep_str { coma } }
309         {2} { \str_set:Nn \l__spintenc_spnum_read_period_sep_str { punto } }
310       }
311     },
312   read-period-sep     .initial:n = coma,
313   read-period-sep     .value_required:n = true,
314   read-period-sep / unknown .code:n =
315     \msg_error:nneee { spintenc } { unknown-choice }
316     { read-period-sep } { coma , ~punto } { \exp_not:n {#1} },
317   notation-sym        .choices:nn =
318     { times , cdot }
```

```

319     {
320       \int_case:nn { \l_keys_choice_int }
321       {
322         {1} { \tl_set:Nn \__spintent_snum_notation_sym_tl { \times } }
323         {2} { \tl_set:Nn \__spintent_snum_notation_sym_tl { \cdot } }
324       }
325     },
326     notation-sym .initial:n = cdot,
327     notation-sym .value_required:n = true,
328     notation-sym / unknown .code:n =
329     \msg_error:nneee { spintent } { unknown-choice }
330     { notation-sym } { times , ~ cdot } { \exp_not:n {#1} },
331     unknown .code:n =
332     {
333       \str_set:Nn \__spintent_command_name_str { snum }
334       \__spintent_unknown_keys_cmd:n {#1}
335     },
336   }

```

(End of definition for *cifras-sep* and others.)

11.14.2 Internal functions for \snum

```

\__spintent_snum_render_part:n
\__spintent_snum_render_int:
\__spintent_snum_render_dec:

```

The function `\__spintent_snum_render_part:n` processes the $\langle argument \rangle$ passed in ‘#1’ used by the functions `\__spintent_snum_render_int:` and `\__spintent_snum_render_dec:` to define the *intent function* :number for the integer and decimal part passed to `\MathMLintenc`. It evaluates whether the key *cifras-sep* is active or not.

```

337 \cs_new_protected:Nn \__spintent_snum_render_part:n
338 {
339   \MathMLintenc { \str_use:c { l__spintenc_luaset_only_part_#1_str } :number }
340   {
341     \bool_if:NTF \__spintenc_snum_cifras_sep_bool
342     { \str_use:c { l__spintenc_luaset_format_part_#1_str } }
343     { \tl_use:c { l__spintenc_luaset_part_#1_tl } }
344   }
345 }
346 \cs_new_protected:Nn \__spintenc_snum_render_int:
347 {
348   \__spintenc_snum_render_part:n { int }
349 }
350 \cs_new_protected:Nn \__spintenc_snum_render_dec:
351 {
352   \__spintenc_snum_render_part:n { dec }
353 }

```

(End of definition for `\__spintenc_snum_render_part:n`, `\__spintenc_snum_render_int:`, and `\__spintenc_snum_render_dec:`.)

```

\__spintenc_snum_render_only_period:
\__spintenc_snum_print_period_bar:
\__spintenc_snum_render_period_sep:

```

The function `\__spintenc_snum_render_only_period:` isolates the “infinite periodic decimal part” from the “finite decimal part”, assigning the *intent function* :number passed to `\MathMLintenc` which will be used by `\MathMLarg` in the function `\__spintenc_snum_print_period_bar:`.

```

354 \cs_new_protected:Nn \__spintenc_snum_render_only_period:
355 {
356   \MathMLintenc { \l__spintenc_luaset_only_part_period_str :number }
357   {
358     \l__spintenc_luaset_part_period_tl
359   }
360 }

```

The function `\__spintenc_snum_print_period_bar:` inserts the function `\__spintenc_snum_render_only_period:` as an argument within the *intent function* `_periódico:postfix($dp)` passed to `\MathMLintenc`, generating the correct spoken for the “infinite periodic decimal part”.

```

361 \cs_new_protected:Nn \__spintenc_snum_print_period_bar:
362 {
363   \MathMLintenc { _periódico:postfix($dp) }
364   {
365     {
366       \overline
367       {
368         \MathMLarg { dp } { \__spintenc_snum_render_only_period: }
369       }
370     }
371   }

```

```
372 }
```

The function `\__spintent_snum_render_period_sep:` sets how the decimal separator handled by the key `read-period-sep` will be printed and spoken when a “*pure periodic decimal*” is passed.

```
373 \cs_new_protected:Nn \__spintent_snum_render_period_sep:
374 {
375   \str_if_eq:VnTF \l__spintent_snum_read_period_sep_str { coma }
376   {
377     \MathMLintent { _coma:prefix($pp) }
378     {
379       \mathord{,}
380       \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
381     }
382   }
383   {
384     \MathMLintent { _punto:prefix($pp) }
385     {
386       \mathord{.}
387       \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
388     }
389   }
390 }
```

(End of definition for `\__spintent_snum_render_only_period:`, `\__spintent_snum_print_period_bar:`, and `\__spintent_snum_render_period_sep:`.)

```
\__spintent_snum_render_decimal_sep:
```

The function `\__spintent_snum_render_decimal_sep:` parses the value of the variable `\l__spintent_luaset` and sets the variable `\l__spintent_snum_intent_read_dec_sep_str` with the function `_coma:prefix($dp)` for reading a “*coma*” or `_punto:prefix($dp)` for reading a “*punto*” to `\MathMLintent`. We print the decimal separator stored in `\l__spintent_luaset_dec_sep_tl` using `\mathord` to remove spaces around it and then use the command `\MathMLarg` where we insert `\__spintent_invisible_sep:` for compatibility with MathML and finally call the function `\__spintent_snum_render_dec:` to print the finite decimal part.

```
391 \cs_new_protected:Nn \__spintent_snum_render_decimal_sep:
392 {
393   \str_if_eq:VnTF \l__spintent_luaset_dec_sep_tl { , }
394   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _coma } }
395   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _punto } }
396   \MathMLintent { \l__spintent_snum_intent_read_dec_sep_str :prefix($dp) }
397   {
398     \mathord { \l__spintent_luaset_dec_sep_tl }
399     \MathMLarg { dp }
400     {
401       \__spintent_inner_sep:
402       \__spintent_snum_render_dec:
403     }
404   }
405   % period part
406   \str_if_eq:VnTF \l__spintent_luaset_decimal_period_str { true }
407   {
408     \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
409     {
410       \__spintent_inner_sep:
411       \__spintent_snum_print_period_bar:
412     }
413     {
414       \MathMLintent { _con:prefix($pnum) }
415       {
416         \__spintent_inner_sep:
417         \MathMLarg { pnum }
418         {
419           \__spintent_snum_print_period_bar:
420         }
421       }
422     }
423   }
424 }
```

(End of definition for `\__spintent_snum_render_decimal_sep:`.)

```
\__spintent_snum_print_part_dec:
\__spintent_snum_print_part_period:
```

The function `\__spintent_snum_print_part_dec:` checks if a decimal part exists. If it does, it determines whether the decimal separator is a comma or a period to set the correct speech intent (“*coma*” or “*punto*”).

It then prints the separator, renders the decimal digits, and if a periodic sequence follows, it decides how to attach the periodic bar.

The function `\__spinttent_snum_print_part_period:` handles cases where a periodic part exists. It evaluates if it should render the periodic separator directly, particularly when the number has a periodic part but lacks a standard decimal part.

```

425 \cs_new_protected:Nn \__spinttent_snum_print_part_dec:
426 {
427   \tl_if_empty:NF \l__spinttent_luaset_part_dec_tl
428   {
429     \__spinttent_snum_render_decimal_sep:
430   }
431 }
432 \cs_new_protected:Nn \__spinttent_snum_print_part_period:
433 {
434   \tl_if_empty:NF \l__spinttent_luaset_part_period_tl
435   {
436     \str_if_eq:VnTF \l__spinttent_luaset_not_decimal_period_str { true }
437     { \__spinttent_snum_render_period_sep: }
438     {
439       \tl_if_empty:NT \l__spinttent_luaset_part_dec_tl
440       {
441         \__spinttent_snum_render_period_sep:
442       }
443     }
444   }
445 }

```

(End of definition for `\__spinttent_snum_print_part_dec:` and `\__spinttent_snum_print_part_period:`.)

```

\__spinttent_snum_print_integer:
\__spinttent_snum_print_num_start:
\__spinttent_snum_print_number_core:

```

The function `\__spinttent_snum_print_integer:` checks if the number consists only of an integer. If so, it simply renders it. If a decimal or periodic part exists, it sets up a `MathML` intent to link the integer part (respecting digit separators if enabled) to the subsequent decimal or periodic functions.

The function `\__spinttent_snum_print_num_start:` checks if an explicit sign is present. If it finds one, it evaluates the terse reading flag to decide whether the sign should be read as a prefix (“más” or “menos”) or as a postfix (“positivo” or “negativo”), and wraps the intent around the rest of the number.

The function `\__spinttent_snum_print_number_core:` acts as the main wrapper. It triggers the sign and base number evaluation, and then checks if a scientific exponent is present. If it is, it appends the correct base-10 notation symbol and exponent to complete the sequence.

 Note for `\__spinttent_invisible_sep:`.

```

446 \cs_new_protected:Nn \__spinttent_snum_print_integer:
447 {
448   \str_if_eq:VnTF \l__spinttent_luaset_not_decimal_not_period_str { true }
449   { \__spinttent_snum_render_int: }
450   {
451     \MathMLintent { \l__spinttent_luaset_only_part_int_str :number:prefix($next) }
452     {
453       \bool_if:NTF \l__spinttent_snum_cifras_sep_bool
454       { \l__spinttent_luaset_format_part_int_str }
455       { \l__spinttent_luaset_part_int_tl }
456       \MathMLarg { next }
457       {
458         \tl_if_empty:NTF \l__spinttent_luaset_part_dec_tl
459         { \__spinttent_snum_print_part_period: }
460         { \__spinttent_snum_print_part_dec: }
461       }
462     }
463   }
464 }
465 \cs_new_protected:Nn \__spinttent_snum_print_num_start:
466 {
467   \tl_if_empty:NTF \l__spinttent_luaset_sign_tl
468   {
469     \__spinttent_invisible_sep: % need
470     \__spinttent_snum_print_integer:
471   }
472   {
473     \bool_if:NTF \l__spinttent_snum_read_terse_bool
474     {
475       \str_if_eq:VnTF \l__spinttent_luaset_sign_tl { + }

```

```

476         { \str_set:Nn \__spinttent_snum_intent_read_sign_str { _más:prefix($n)} }
477         { \str_set:Nn \__spinttent_snum_intent_read_sign_str { _menos:prefix($n)} }
478     }
479     {
480         \str_if_eq:VnTF \__spinttent_luaset_sign_tl { + }
481         { \str_set:Nn \__spinttent_snum_intent_read_sign_str { _positivo:postfix($n) } }
482         { \str_set:Nn \__spinttent_snum_intent_read_sign_str { _negativo:postfix($n) } }
483     }
484     \__spinttent_invisible_sep: % need
485     \MathMLintent { \__spinttent_snum_intent_read_sign_str }
486     {
487         \__spinttent_luaset_sign_tl
488         \MathMLarg { n } { \__spinttent_snum_print_integer: }
489     }
490 }
491 }
492 \cs_new_protected:Nn \__spinttent_snum_print_number_core:
493 {
494     \__spinttent_snum_print_num_start:
495     \str_if_eq:VnTF \__spinttent_luaset_has_exponent_str { true }
496     {
497         \MathMLintent { _por } { \__spinttent_snum_notation_sym_tl }
498         10^{\__spinttent_luaset_exponent_tl }
499     }
500 }

```

(End of definition for `\__spinttent_snum_print_integer:`, `\__spinttent_snum_print_num_start:`, and `\__spinttent_snum_print_number_core:`.)

11.14.3 Definition of error messages

spnum-has-units
spnum-no-digits

We define two error messages for the `\spnum` command. The first “error” triggers if the user includes “units” in the $\langle argument \rangle$ passed in ‘#1’, advising them to use the `\spunit` command instead. The second “error” is raised if the provided $\langle argument \rangle$ contains no numerical digits.

```

501 \msg_new:nnnn { spinttent } { spnum-has-units }
502 {
503     The~\token_to_str:N \spnum \c_space_tl command~does~not~accept~units~('#1').
504 }
505 {
506     Use~\token_to_str:N \spunit \c_space_tl if~you~need~to~print~numbers~with~units.
507 }
508 \msg_new:nnnn { spinttent } { spnum-no-digits }
509 {
510     The~\token_to_str:N \spnum \c_space_tl command~requires~a~valid~numerical~value.
511 }
512 {
513     You~provided~'#1',~which~contains~no~digits.
514 }

```

(End of definition for `spnum-has-units` and `spnum-no-digits`.)

11.14.4 Implementation of the `\spum` command

`\spnum` The public user command `\spnum` accepts an optional $[\langle key = val \rangle]$ argument in ‘#1’ and a mandatory $\langle rational\ number \rangle$ in ‘#2’. It first checks if it is executed inside math mode, raising an error if it is not. If correct, it opens a local group to apply the options and passes the input to Lua for parsing via `\__spinttent_luafun_clean_split_and_set:n`.

Before printing, it validates the extracted data. It throws an error if it finds trailing units in the input or if the integer part is completely empty (no valid digits). If all checks pass, it calls `\__spinttent_snum_print_number_core:` to render the final number, and finally closes the group.

```

515 \NewDocumentCommand \spnum { O{} m }
516 {
517     \mode_if_math:TF
518     {
519         \group_begin:
520         \keys_set:nn { spinttent/spnum } { #1 }
521         \__spinttent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
522         \str_if_eq:VnTF \__spinttent_luaset_has_units_str { true }
523         { \msg_error:nnn { spinttent } { spnum-has-units } { #2 } }
524         {
525             \tl_if_empty:NTF \__spinttent_luaset_part_int_tl
526             { \msg_error:nnn { spinttent } { spnum-no-digits } { #2 } }
527             { \__spinttent_snum_print_number_core: }

```

```

528         }
529     \group_end:
530 }
531 { \msg_error:nnn { spintent } { need-math-mode } { \spnum } }
532 }

```

(End of definition for `\spnum`. This function is documented on page 3.)

11.15 The command `\spunit`

The user command `\spunit` is the *second* of “the big four” provided by the `spintent` package and is used to parse and print physical units and angular systems. It sends the $\langle\textit{argument}\rangle$ to Lua module `spintent.lua` via `__spintent_luafun_clean_split_and_set:n` to separate and clean the $\langle\textit{argument}\rangle$.

This ensures that all “units” (such as SI fractions, products, or sexagesimal angles) are processed consistently by the `spintent.lua`.

11.15.1 Definition of variables

```

\__spintent_spunit_luaset_canonical_str
\__spintent_spunit_luaset_lookup_status_str
\__spintent_spunit_luaset_read_str
\__spintent_spunit_luaset_is_compact_str
\__spintent_spunit_luaset_compact_exp_str
\__spintent_spunit_luaset_is_sexagesimal_str
\__spintent_spunit_luaset_status_str

```

These string variables store the data returned by the Lua module `spintent.lua`. They hold the canonical “unit symbol”, the dictionary lookup status, the correct speech text for NVDA/MathCAT, and specific flags that indicate if the “unit” is compact or represents a sexagesimal angle.

```

533 \str_new:N \__spintent_spunit_luaset_canonical_str
534 \str_new:N \__spintent_spunit_luaset_lookup_status_str
535 \str_new:N \__spintent_spunit_luaset_read_str
536 \str_new:N \__spintent_spunit_luaset_is_compact_str
537 \str_new:N \__spintent_spunit_luaset_compact_exp_str
538 \str_new:N \__spintent_spunit_luaset_is_sexagesimal_str
539 \str_new:N \__spintent_spunit_luaset_status_str

```

(End of definition for `\__spintent_spunit_luaset_canonical_str` and others.)

```

\__spintent_spunit_exponent_tl
\__spintent_spunit_unit_name_tl
\__spintent_spunit_read_slash_str
\__spintent_spunit_exp_seq
\__spintent_spunit_mult_seq
\__spintent_spunit_is_mult_bool
\__spintent_spunit_read_cdot_bool

```

These variables are used to process and format the “unit”. The sequences `\__spintent_spunit_mult_seq` and `\__spintent_spunit_exp_seq` split compound “units” at multiplication ‘`*`’ or exponent ‘`^`’ boundaries.

The boolean variables control specific rendering options, such as whether the multiplication dot ‘`\cdot`’ is explicitly read aloud or silently skipped.

```

540 \tl_new:N \__spintent_spunit_exponent_tl
541 \tl_new:N \__spintent_spunit_unit_name_tl
542 \str_new:N \__spintent_spunit_read_slash_str
543 \seq_new:N \__spintent_spunit_exp_seq
544 \seq_new:N \__spintent_spunit_mult_seq
545 \bool_new:N \__spintent_spunit_is_mult_bool
546 \bool_new:N \__spintent_spunit_read_cdot_bool

```

(End of definition for `\__spintent_spunit_exponent_tl` and others.)

11.15.2 Definition of error messages

```

spunit-invalid-unit
spunit-no-units
inline-numeric-denominator

```

We define three error messages for the `\spunit` command. The first “error” triggers if the “unit expression” provided in ‘`#1`’ is unknown or malformed. The second “error” occurs if no “units” are found in the $\langle\textit{argument}\rangle$, advising the user to use `\spnum` for pure numbers. The third “error” is raised if a numeric coefficient is illegally placed in the denominator of an inline fraction (after a slash).

```

547 \msg_new:nnnn { spintent } { spunit-invalid-unit }
548 {
549     The~expression~'#1'~contains~an~unknown~or~malformed~unit~structure.
550 }
551 {
552     Check~spelling~or~verify~the~unit~is~defined~in~the~dictionary.
553 }
554 \msg_new:nnnn { spintent } { spunit-no-units }
555 {
556     The~\token_to_str:N \spunit \c_space_tl command~requires~at~least~one~unit~('#1').
557 }
558 {
559     Use~\token_to_str:N \spnum \c_space_tl if~you~only~need~to~format~a~pure~number.
560 }
561 \msg_new:nnnn { spintent } { inline-numeric-denominator }
562 {
563     Illegal~numeric~coefficient~in~inline~denominator~'#1'.
564 }
565 {
566     The~inline~mode~of~\token_to_str:N \spunit \c_space_tl
567     does~not~allow~numbers~after~the~slash~(/).
568 }

```

(End of definition for `spunit-invalid-unit`, `spunit-no-units`, and `inline-numeric-denominator`.)

11.15.3 Definition of keys for \spunit

Now we add the keys `print-cdot`, `read-cdot`, `read-slash`, `cifras-sep`, `read-period-sep` and `notation-sym`.

```

569 \keys_define:nn { spintent/spunit }
570 {
571   print-cdot      .bool_set:N = \l__spintent_spunit_print_cdot_bool,
572   print-cdot      .initial:n   = false,
573   print-cdot      .value_required:n = true,
574   read-cdot       .bool_set:N = \l__spintent_spunit_read_cdot_bool,
575   read-cdot       .initial:n   = false,
576   read-cdot       .value_required:n = true,
577   read-slash      .choices:nn =
578     { por , partido-por }
579   {
580     \int_case:nn { \l_keys_choice_int }
581     {
582       {1} { \str_set:Nn \l__spintent_spunit_read_slash_str { por } }
583       {2} { \str_set:Nn \l__spintent_spunit_read_slash_str { partido-por } }
584     }
585   },
586   read-slash      .initial:n   = por,
587   read-slash      .value_required:n = true,
588   read-slash / unknown .code:n =
589     \msg_error:nneee { spintent } { unknown-choice }
590     { read-slash } { por , ~partido-por } { \exp_not:n {#1} },
591   cifras-sep      .meta:nn = { spintent/spnum } { cifras-sep = {#1} },
592   cifras-sep      .value_required:n = true,
593   read-period-sep .meta:nn = { spintent/spnum } { read-period-sep = {#1} },
594   read-period-sep .value_required:n = true,
595   notation-sym    .meta:nn = { spintent/spnum } { notation-sym = {#1} },
596   notation-sym    .value_required:n = true,
597   unknown         .code:n =
598     {
599       \str_set:Nn \l__spintent_command_name_str { spunit }
600       \__spintent_unknown_keys_cmd:n {#1}
601     },
602 }

```

(End of definition for `read-cdot` and others.)

11.15.4 Internal functions for \spunit

`\__spintent_spunit_format_canonical:` The function `\__spintent_spunit_format_canonical:` handles the visual formatting of the “unit symbol”. It specifically checks for the *liter symbol* ‘*ℓ*’ to print it as `\ell`, and formats all other valid “units” in standard upright text using `\mathrm`.

```

603 \cs_new_protected:Nn \__spintent_spunit_format_canonical:
604 {
605   \str_case:VnF \l__spintent_spunit_luaset_canonical_str
606   {
607     { ℓ } { \ell }
608   }
609   { \mathrm { \l__spintent_spunit_luaset_canonical_str } }
610 }

```

(End of definition for `\__spintent_spunit_format_canonical:`.)

`\__spintent_spunit_process_base_exp:n` The function `\__spintent_spunit_process_base_exp:n` evaluates a “single unit” along with its exponent, ‘*#1*’. It first looks up the unit in the dictionary; if not found, flags it as invalid. Otherwise, unless this is the first unit in a product (`\__spintent_spunit_is_mult_bool`), it handles the spacing and multiplication dot: `\cdot` when `print-cdot` is `true` (read as `por` when `read-cdot` is also `true`, silent otherwise), or a thin space `\,` when `print-cdot` is `false` (still read as `por` under `read-cdot`).

```

611 \cs_new_protected:Nn \__spintent_spunit_process_base_exp:n
612 {
613   \tl_clear:N \l__spintent_spunit_exponent_tl
614   \__spintent_luafun_spunit_lookup_alias:n { #1 }
615   \str_if_eq:VnTF \l__spintent_spunit_luaset_lookup_status_str { notfound }
616   {
617     \str_set:Nn \l__spintent_luaset_status_str { invalid }
618   }
619   {
620     \bool_if:NF \l__spintent_spunit_is_mult_bool

```

```

621     {
622       \bool_if:NTF \l__spinttent_spunit_print_cdot_bool
623       {
624         \bool_if:NTF \l__spinttent_spunit_read_cdot_bool
625         { \MathMLintent { por } { \cdot } }
626         { \MathMLintent { :silent } { \cdot } }
627       }
628       {
629         \bool_if:NTF \l__spinttent_spunit_read_cdot_bool
630         { \MathMLintent { por } { \, } }
631         { \, }
632       }
633     }

```

Then renders the base unit and its exponent: either a pre-defined compact exponent from Lua (`\l__spinttent_spunit_luaset_is_compact_str`) or by manually splitting ‘#1’ at the ‘^’ symbol otherwise.

```

634     \str_if_eq:VnTF \l__spinttent_spunit_luaset_is_compact_str { true }
635     {
636       {
637         \MathMLintent { \l__spinttent_spunit_luaset_read_str }
638         {
639           \l__spinttent_spunit_format_canonical:
640         }
641       }
642       ^ { \l__spinttent_spunit_luaset_compact_exp_str }
643     }
644     {
645       \seq_set_split:Nnn \l__spinttent_spunit_exp_seq { ^ } { #1 }
646       \seq_pop_left:NN \l__spinttent_spunit_exp_seq \l__spinttent_spunit_unit_name_tl
647       \seq_if_empty:NF \l__spinttent_spunit_exp_seq
648       {
649         \seq_pop_left:NN \l__spinttent_spunit_exp_seq \l__spinttent_spunit_exponent_tl
650       }
651       \MathMLintent { \l__spinttent_spunit_luaset_read_str }
652       {
653         \l__spinttent_spunit_format_canonical:
654       }
655       \tl_if_empty:NF \l__spinttent_spunit_exponent_tl
656       {
657         ^ { \l__spinttent_spunit_exponent_tl }
658       }
659     }
660     \bool_set_false:N \l__spinttent_spunit_is_mult_bool
661   }
662 }

```

(End of definition for `\l__spinttent_spunit_process_base_exp:n`)

```

\l__spinttent_spunit_process_mult:n
\l__spinttent_spunit_process_mult:V

```

The function `\l__spinttent_spunit_process_mult:n` handles compound “units” joined by an explicit multiplication star ‘*’. It splits the `{(argument)}` at each “star symbol” and applies `\l__spinttent_spunit_process_base_exp:n` to evaluate and render each resulting unit individually.

```

663 \cs_new_protected:Nn \l__spinttent_spunit_process_mult:n
664 {
665   \bool_set_true:N \l__spinttent_spunit_is_mult_bool
666   \seq_set_split:Nnn \l__spinttent_spunit_mult_seq { * } { #1 }
667   \seq_map_function:NN \l__spinttent_spunit_mult_seq \l__spinttent_spunit_process_base_exp:n
668 }
669 \cs_generate_variant:Nn \l__spinttent_spunit_process_mult:n { V }

```

(End of definition for `\l__spinttent_spunit_process_mult:n`)

```
\l__spinttent_print_spunit_hierarchy:
```

The function `\l__spinttent_print_spunit_hierarchy:` handles the layout of inline “unit” fractions. It first processes the “units” before the slash (the numerator). If a denominator exists, it prints the slash / ‘/’ with its corresponding speech intent, and then processes the remaining “units”.

```

670 \cs_new_protected:Nn \l__spinttent_print_spunit_hierarchy:
671 {
672   \tl_if_empty:NF \l__spinttent_luaset_arg_above_tl
673   {
674     \l__spinttent_spunit_process_mult:V \l__spinttent_luaset_arg_above_tl
675     \tl_if_empty:NF \l__spinttent_luaset_arg_below_tl
676     {
677       \MathMLintent { \l__spinttent_spunit_read_slash_str } { / }

```

```

678         \__spinttent_spunit_process_mult:V \l__spinttent_luaset_arg_below_tl
679     }
680 }
681 }

```

(End of definition for `\__spinttent_print_spunit_hierarchy:`.)

`\__spinttent_spunit_safe_exec:n` The function `\__spinttent_spunit_safe_exec:n` performs the main validation and layout for “units”. It first checks if the `{⟨argument⟩}` has units, is a valid expression, and does not contain illegal numbers in the denominator, throwing the appropriate “errors” if any check fails.

If the unit is valid, it checks if it is a sexagesimal angle (degrees, minutes, or seconds). For sexagesimal “units”, it prints the number and the symbol “without any space in between”, applying a slight negative space ‘\!’ for arcseconds. For standard units, it prints the number followed by a thin space ‘\,’ and then processes the unit sequence.

```

682 \cs_new_protected:Nn \__spinttent_spunit_safe_exec:n
683 {
684     \str_if_eq:VnTF \l__spinttent_luaset_has_units_str { false }
685     { \msg_error:nnn { spinttent } { spunit-no-units } { #1 } }
686     {
687         \str_if_eq:VnTF \l__spinttent_luaset_status_str { valid }
688         {
689             \str_if_eq:VnTF
690             \l__spinttent_luaset_denom_has_number_str { true }
691             {
692                 \msg_error:nnn { spinttent }
693                 { inline-numeric-denominator } { #1 }
694             }
695             {
696                 \__spinttent_luafun_spunit_lookup_alias:V \l__spinttent_luaset_arg_above_tl
697                 \str_if_eq:VnTF \l__spinttent_spunit_luaset_is_sexagesimal_str { true }
698                 {
699                     \tl_if_empty:NF \l__spinttent_luaset_part_int_tl
700                     {
701                         \__spinttent_spnum_print_number_core:
702                     }
703                     \MathMLintent { \l__spinttent_spunit_luaset_read_str }
704                     {
705                         \mathord
706                         {
707                             \str_case:Vn \l__spinttent_spunit_luaset_canonical_str
708                             {
709                                 { " } { \prime \! \prime }
710                                 { ' } { \prime
711                                 { ° } { °
712                             }
713                         }
714                     }
715                 }
716                 {
717                     \tl_if_empty:NF \l__spinttent_luaset_part_int_tl
718                     {
719                         \__spinttent_spnum_print_number_core: \,
720                     }
721                     \__spinttent_print_spunit_hierarchy:
722                 }
723             }
724         }
725         { \msg_error:nnn { spinttent } { spunit-invalid-unit } { #1 } }
726     }
727 }

```

(End of definition for `\__spinttent_spunit_safe_exec:n`.)

11.15.5 Implementation of the `\spunit` command

`\spunit` The public user command `\spunit` accepts an optional `[⟨key = val⟩]` argument in ‘#1’ and a mandatory `{⟨unit expression⟩}` in ‘#2’. It first checks if it is executed inside *math mode*, raising an “error” if it is not. If correct, it opens a local group to apply the options and passes the `{⟨argument⟩}` to Lua module for parsing via `\__spinttent_luafun_clean_split_and_set:n` function.

Finally, it delegates the execution to the function `\__spinttent_spunit_safe_exec:n` to perform the safety checks and render the parsed data, before closing the group.

```

728 \NewDocumentCommand \spunit { 0{ } m }

```

```

729   {
730     \mode_if_math:TF
731     {
732       \group_begin:
733         \keys_set:nn { spintent/spunit } { #1 }
734         \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
735         \__spintent_spunit_safe_exec:n { #2 }
736       \group_end:
737     }
738     { \msg_error:nnn { spintent } { need-math-mode } { \spunit } }
739   }

```

(End of definition for `\spunit`. This function is documented on page 4.)

11.16 The commands `\spnewunit` and `\spunitalias`

The user commands `\spnewunit` and `\spunitalias` allow users to add new custom “units” or “aliases” to the package’s dictionary. Similar to `\spunit`, these commands delegate the processing and validation to the `spintent.lua` via `\__spintent_luafun_define_custom_unit:nn` and `\__spintent_luafun_define_spunit`. During the definition, the Lua module checks if the new “unit” or “alias” is valid and ensures it does not conflict with existing “units”. It returns the result of this operation to TeX using the `\l__spintent_spunit_luaset_status_str` variable. If a collision is detected or the $\langle argument \rangle$ is invalid, the package throws the corresponding “error” message.

11.16.1 Definition of error messages

spnewunit-duplicate
spunitalias-duplicate
spunitalias-invalid-expr

We define three error messages for the creation of “units” and “aliases”. The first two “errors” trigger if the user attempts to define a “new unit” or alias passed in ‘#1’ that already exists in the dictionary. The third “error” is raised by `\spunitalias` if the target expression provided in ‘#2’ is invalid (for example, if it contains unknown “units” or incorrect syntax).

```

740 \msg_new:nnnn { spintent } { spnewunit-duplicate }
741 {
742   The~unit~symbol~or~alias~'#1'~is~already~defined!
743 }
744 {
745   You~cannot~redefine~with~\token_to_str:N \spnewunit.
746 }
747 \msg_new:nnnn { spintent } { spunitalias-duplicate }
748 {
749   The~alias~'#1'~is~already~defined~or~reserved!
750 }
751 {
752   You~cannot~redefine~an~existing~unit~or~another~alias.
753 }
754 \msg_new:nnnn { spintent } { spunitalias-invalid-expr }
755 {
756   The~expression~'#2'~for~alias~'#1'~is~invalid.
757 }
758 {
759   It~must~contain~only~defined~units,~exponents~and~one~'/'~.
760 }

```

(End of definition for `spnewunit-duplicate`, `spunitalias-duplicate`, and `spunitalias-invalid-expr`.)

`\spnewunit`

The public command `\spnewunit` defines a new custom unit. It passes the new unit symbol ‘#1’ and its speech text ‘#2’ to Lua via `\__spintent_luafun_define_custom_unit:nn`. If the module reports that the unit already exists, it throws the corresponding duplicate error.

```

761 \NewDocumentCommand \spnewunit { m m }
762 {
763   \mode_if_math:TF
764   {
765     \msg_error:nnn { spintent } { need-text-mode } { \spnewunit }
766   }
767   {
768     \__spintent_luafun_define_custom_unit:nn { #1 } { #2 }
769     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
770     {
771       \msg_error:nnn { spintent } { spnewunit-duplicate } { #1 }
772     }
773   }
774 }

```

(End of definition for `\spnewunit`. This function is documented on page 6.)

`\spunitalias` The public command `\spunitalias` creates a shortcut for a complex unit expression. It passes the new alias name ‘`#1`’ and the target expression ‘`#2`’ to Lua via `\__spintent_luafun_define_spunit_alias:nn`. It throws an error if the alias already exists, or if the provided target expression contains invalid syntax or unknown units.

```

775 \NewDocumentCommand \spunitalias { m m }
776 {
777   \mode_if_math:TF
778   {
779     \msg_error:nnn { spintent } { need-text-mode } { \spunitalias }
780   }
781   {
782     \__spintent_luafun_define_spunit_alias:nn { #1 } { #2 }
783     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
784     {
785       \msg_error:nnn { spintent } { spunitalias-duplicate } { #1 }
786     }
787     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { invalid-expr }
788     {
789       \msg_error:nnnn { spintent } { spunitalias-invalid-expr } { #1 } { #2 }
790     }
791   }
792 }

```

(End of definition for `\spunitalias`. This function is documented on page 6.)

11.17 The command `\spmoney`

The user command `\spmoney` is the *third* of “the big four” provided by the `spintent` package, designed to format monetary values and currencies. It delegates `{\argument}` parsing to the Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n`. The module then retrieves the correct “currency symbol”, pluralization rules, and layout structures from the database.

The command accepts *optional arguments* to override the default “currency symbol” or enable “subunit” rendering. Finally, it constructs the MathML intent for MathCAT.

These string variables store the currency data returned by the Lua module `spintent.lua`. Upon a successful database lookup, the module populates these variables with the “currency symbol”, its layout position (pre or post), the appropriate singular and plural names for both the main unit and subunits, and the execution status flags. $\TeX$ then uses this information to render the visual layout and construct the correct MathML speech intents.

```

793 \str_new:N \l__spintent_spmoney_luaset_symbol_str
794 \str_new:N \l__spintent_spmoney_luaset_position_str
795 \str_new:N \l__spintent_spmoney_luaset_sing_str
796 \str_new:N \l__spintent_spmoney_luaset_plur_str
797 \str_new:N \l__spintent_spmoney_luaset_conde_str
798 \str_new:N \l__spintent_spmoney_luaset_subsing_str
799 \str_new:N \l__spintent_spmoney_luaset_subplur_str
800 \str_new:N \l__spintent_spmoney_luaset_print_iso_str
801 \str_new:N \l__spintent_spmoney_luaset_currency_arg_str
802 \str_new:N \l__spintent_spmoney_luaset_status_str

```

(End of definition for `\l__spintent_spmoney_luaset_symbol_str` and others.)

The boolean variable tracks whether subunit rendering is enabled. The string variables assemble the specific speech text (intents) for NVDA/MathCAT, including the sign, the main currency, and the subunits. Finally, the remaining variables temporarily store the numeric value and currency code extracted from the user’s input.

```

803 \bool_new:N \l__spintent_spmoney_sub_units_bool
804 \str_new:N \l__spintent_spmoney_intent_money_str
805 \str_new:N \l__spintent_spmoney_intent_sign_str
806 \str_new:N \l__spintent_spmoney_intent_sub_unit_str
807 \str_new:N \l__spintent_spmoney_res_main_voice_str
808 \int_new:N \l__spintent_spmoney_subnum_int
809 \str_new:N \l__spintent_spmoney_extracted_curr_str
810 \tl_new:N \l__spintent_spmoney_extracted_num_tl

```

(End of definition for `\l__spintent_spmoney_sub_units_bool` and others.)

11.17.1 Definition of error messages

currency-no-subunits
invalid-currency

We define two messages to handle invalid currency configurations. The first “error” warns the user if subunit rendering is requested for a currency that does not support fractional divisions (such as JPY or CLP), safely ignoring the flag. The second “error” is triggered if the provided currency identifier is not registered in the Lua database.

```

811 \msg_new:nnnn { spintent } { currency-no-subunits }
812 {
813     The~currency~'#1'~does~not~support~subunits.~
814     The~option~'sub-units=true'~is~ignored.
815 }
816 {
817     Monies~like~the~Japanese~Yen~(JPY)~or~Chilean~Peso~(CLP)~
818     do~not~have~fractional~subunits~registered~in~Lua~module.~
819 }
820 \msg_new:nnnn { spintent } { invalid-currency }
821 {
822     Invalid~currency~identifier~'#1'~in~\token_to_str:N \spmoney .
823 }
824 {
825     The~provided~symbol,~alias,~or~ISO~code~does~not~exist~in~
826     the~Lua~module.
827 }
```

(End of definition for currency-no-subunits and invalid-currency.)

11.17.2 Auxiliary functions for \spmoney

__spintenc_spmoney_set_currency:n
__spintenc_spmoney_normalize_key:n
__spintenc_spmoney_normalize_key:V

The function `\__spintenc_spmoney_set_currency:n` validates the currency identifier provided in ‘#1’ against the database. It delegates the lookup to the helper function `\__spintenc_spmoney_normalize_key:n`, which interfaces with the Lua module `spintenc.lua`. If the module returns a not found status, an “error” is triggered. Otherwise, it stores the valid currency identifier in `\__spintenc_spmoney_luaset_currency_arg_str` for subsequent processing.

```

828 \cs_new_protected:Nn \__spintenc_spmoney_set_currency:n
829 {
830     \__spintenc_spmoney_normalize_key:n {#1}
831     \str_if_eq:VnTF \__spintenc_spmoney_luaset_status_str { notfound }
832     {
833         \msg_error:nnn { spintent } { invalid-currency } {#1}
834     }
835     { \str_set:Nn \__spintenc_spmoney_luaset_currency_arg_str {#1} }
836 }
837 \cs_new_protected:Nn \__spintenc_spmoney_normalize_key:n
838 {
839     \__spintenc_luafun_spmoney_normalize_key:n { #1 }
840 }
841 \cs_generate_variant:Nn \__spintenc_spmoney_normalize_key:n { V }
```

(End of definition for __spintenc_spmoney_set_currency:n and __spintenc_spmoney_normalize_key:n.)

11.17.3 Definition of keys for \spmoney

currency
sub-units
dolar-math
cifras-sep

Now we add the keys `currency`, `sub-units`, `dolar-math` and `cifras-sep`.

```

842 \keys_define:nn { spintenc / spmoney }
843 {
844     currency      .code:n      = { \__spintenc_spmoney_set_currency:n {#1} },
845     currency      .initial:n    = { pesos },
846     currency      .value_required:n = true,
847     sub-units     .bool_set:N    = \__spintenc_spmoney_sub_units_bool,
848     sub-units     .value_required:n = true,
849     dolar-math    .bool_set:N    = \__spintenc_spmoney_dolar_math_bool,
850     dolar-math    .initial:n    = { true },
851     dolar-math    .value_required:n = true,
852     cifras-sep    .meta:nn      = { spintenc/spnum } { cifras-sep = {#1} },
853     cifras-sep    .value_required:n = true,
854     unknown       .code:n      =
855     {
856         \str_set:Nn \__spintenc_command_name_str { spmoney }
857         \__spintenc_unknown_keys_cmd:n {#1}
858     },
859 }
```

(End of definition for currency and others.)

11.17.4 Internal functions for \spmoney

__spintenc_spmoney_validate_money:

The function `\__spintenc_spmoney_validate_money`: verifies if the parsed input contains an embedded currency identifier by checking `\l__spintenc_luaset_has_units_str`. If a currency is detected, it extracts and normalizes the key from `\l__spintenc_luaset_arg_above_tl`. It throws an “error” if the currency is not found in the database. If valid, it updates `\l__spintenc_spmoney_luaset_currency_arg_str` and retrieves the full currency metadata from the `spintenc.lua` via `\__spintenc_luafun_spmoney_lookup_data:V`.

```

860 \cs_new_protected:Nn \__spintenc_spmoney_validate_money:
861 {
862   \str_if_eq:VnT \l__spintenc_luaset_has_units_str { true }
863   {
864     \__spintenc_spmoney_normalize_key:V \l__spintenc_luaset_arg_above_tl
865     \str_if_eq:VnTF \l__spintenc_spmoney_luaset_status_str { notfound }
866     { \msg_error:nne { spintenc } { invalid-currency } { \l__spintenc_luaset_arg_above_tl } }
867     {
868       \str_set:NV
869         \l__spintenc_spmoney_luaset_currency_arg_str
870         \l__spintenc_luaset_arg_above_tl
871     }
872   }
873   \str_set:Ne \l__spintenc_spmoney_luaset_status_str { found }
874   \__spintenc_luafun_spmoney_lookup_data:V \l__spintenc_spmoney_luaset_currency_arg_str
875 }

```

(End of definition for __spintenc_spmoney_validate_money:.)

__spintenc_spmoney_print_symbol:

The function `\__spintenc_spmoney_print_symbol`: renders the “*currency symbol*”. If `\l__spintenc_spmoney_luaset_print_iso_bool` is `true`, it outputs the ISO code using `\mathrm`. Otherwise, it prints the “*standard symbol*”. For dollar signs ‘\$’, it checks `\l__spintenc_spmoney_dolar_math_bool` to determine whether to render it directly as a math character ‘\$’ or wrap it inside `\text`.

```

876 \cs_new_protected:Nn \__spintenc_spmoney_print_symbol:
877 {
878   \str_if_eq:VnTF \l__spintenc_spmoney_luaset_print_iso_str { true }
879   { \mathrm { \l__spintenc_spmoney_luaset_currency_arg_str } }
880   {
881     \str_if_eq:VnTF \l__spintenc_spmoney_luaset_symbol_str { $ }
882     {
883       \bool_if:NTF \l__spintenc_spmoney_dolar_math_bool
884       { \$ } { \text { \$ } }
885     }
886     { \l__spintenc_spmoney_luaset_symbol_str }
887   }
888 }

```

(End of definition for __spintenc_spmoney_print_symbol:.)

__spintenc_spmoney_render_layout:

The function `\__spintenc_spmoney_render_layout`: determines the layout order of the “*currency symbol*” relative to the numeric value. It evaluates `\l__spintenc_spmoney_luaset_position_str`; if set to `pre`, it renders the “*currency symbol*” *before* the number node. Otherwise, it places the number first, followed by the “*currency symbol*”.

```

889 \cs_new_protected:Nn \__spintenc_spmoney_render_layout:
890 {
891   \str_if_eq:VnTF \l__spintenc_spmoney_luaset_position_str { pre }
892   {
893     \__spintenc_spmoney_print_symbol:
894     \MathMLarg { n } { \__spintenc_spmoney_render_number_node: }
895   }
896   {
897     \MathMLarg { n } { \__spintenc_spmoney_render_number_node: }
898     \__spintenc_spmoney_print_symbol:
899   }
900 }

```

(End of definition for __spintenc_spmoney_render_layout:.)

__spintenc_spmoney_render_number_node:
 __spintenc_spmoney_render_only_integer_node:

The function `\__spintenc_spmoney_render_number_node`: renders the full numeric value, encompassing both the integer and decimal components. Conversely, the function `\__spintenc_spmoney_render_only_integer_node` outputs exclusively the integer portion, omitting any decimal data.

```

901 \cs_new_protected:Nn \__spintenc_spmoney_render_number_node:
902 {
903   \__spintenc_spmoney_render_int:

```

```

904     \__spintent_snum_print_part_dec:
905   }
906 \cs_new_protected:Nn \__spintent_spmoney_render_only_integer_node:
907 {
908     \__spintent_snum_render_int:
909 }

```

(End of definition for `\__spintent_spmoney_render_number_node:` and `\__spintent_spmoney_render_only_integer_node:`.)

`\__spintent_spmoney_print_int:`

The function `\__spintent_spmoney_print_int:` evaluates the presence of an explicit sign in `\l__spintent_luaset_sign_tl`. If a sign is detected, it assigns a prefix speech intent ‘más’ or ‘menos’ and embeds the sign and subsequent layout inside a `\MathMLintent`. If no sign is present, it routes execution based on whether subunits are enabled.

```

910 \cs_new_protected:Nn \__spintent_spmoney_print_int:
911 {
912     \tl_if_empty:NTF \l__spintent_luaset_sign_tl
913     {
914         \bool_if:NTF \l__spintent_spmoney_sub_units_bool
915         { \__spintent_spmoney_set_and_print_sub_units: }
916         { \__spintent_spmoney_intent_int: }
917     }
918     {
919         \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
920         {
921             \str_set:Nn \l__spintent_spmoney_intent_sign_str { _más:prefix ($n) }
922         }
923         {
924             \str_set:Nn \l__spintent_spmoney_intent_sign_str { _menos:prefix ($n) }
925         }
926         \__spintent_invisible_sep:
927         \MathMLintent { \l__spintent_spmoney_intent_sign_str }
928         {
929             \l__spintent_luaset_sign_tl
930             \MathMLarg { n }
931             {
932                 \bool_if:NTF \l__spintent_spmoney_sub_units_bool
933                 { \__spintent_spmoney_set_and_print_sub_units: }
934                 { \__spintent_spmoney_intent_int: }
935             }
936         }
937     }
938 }

```

(End of definition for `\__spintent_spmoney_print_int:`.)

`\__spintent_spmoney_intent_int:`

The function `\__spintent_spmoney_intent_int:` determines the correct grammatical inflection for the currency string (singular, plural, or the prepositional ‘de’ variant for exact millions). It evaluates the integer component, decimal boundaries, and zero-fills to select the appropriate string register, applying it as a postfix `MathML` intent over the layout rendered by `\__spintent_spmoney_render_layout:`.

```

939 \cs_new_protected:Nn \__spintent_spmoney_intent_int:
940 {
941     \tl_if_empty:NTF \l__spintent_luaset_dec_sep_tl
942     {
943         \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
944         {
945             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_sing_str
946         }
947         {
948             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
949         }
950         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
951         {
952             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde_s
953         }
954     }
955     {
956         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
957         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
958         {
959             \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
960             {

```

```

961         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_con
962     }
963     {
964         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plu
965     }
966 }
967 }
968 \__spintent_invisible_sep:
969 \MathMLint { \l__spintent_spmoney_intent_money_str :postfix($n) }
970 {
971     \__spintent_spmoney_render_layout:
972 }
973 }

```

(End of definition for `\__spintent_spmoney_intent_int:`)

`\__spintent_spmoney_check_sub_units:`
`spintent_spmoney_set_and_print_sub_units:`

The function `\__spintent_spmoney_check_sub_units:` verifies whether the selected currency supports “*subunits*”. If the database returns empty subunit data, it triggers a warning and automatically disables the “*subunit*” rendering flag.

The function `\__spintent_spmoney_set_and_print_sub_units:` processes and formats the speech intents when “*subunits*” are active. It normalizes the decimal component (for instance, padding a single-digit decimal like ‘5’ to ‘50’), evaluates the correct singular or plural inflections for both the main “*currency*” and the “*subunits*”, and delegates the final MathML tree assembly to `\__spintent_spmoney_build_tree:`.

```

974 \cs_new_protected:Nn \__spintent_spmoney_check_sub_units:
975 {
976     \str_if_empty:NT \l__spintent_spmoney_luaset_subsing_str
977     {
978         \msg_warning:nne { spintent } { currency-no-subunits }
979         { \l__spintent_spmoney_luaset_currency_arg_str }
980         \bool_set_false:N \l__spintent_spmoney_sub_units_bool
981     }
982 }
983 \cs_new_protected:Nn \__spintent_spmoney_set_and_print_sub_units:
984 {
985     \int_compare:nTF { \str_count:N \l__spintent_luaset_only_part_dec_str == 1 }
986     {
987         \int_set:Nn \l__spintent_spmoney_subnum_int
988         { \l__spintent_luaset_only_part_dec_str * 10 }
989         \str_set:Ne \l__spintent_luaset_only_part_dec_str
990         { \l__spintent_luaset_only_part_dec_str 0 }
991     }
992     {
993         \int_set:Nn \l__spintent_spmoney_subnum_int
994         { \l__spintent_luaset_only_part_dec_str }
995     }
996     \int_compare:nTF { \l__spintent_spmoney_subnum_int == 1 }
997     {
998         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
999         { \l__spintent_spmoney_luaset_subsing_str :postfix($sub) }
1000     }
1001     {
1002         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
1003         { \l__spintent_spmoney_luaset_subplur_str :postfix($sub) }
1004     }
1005     \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
1006     {
1007         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1008         \l__spintent_spmoney_luaset_sing_str
1009     }
1010     {
1011         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1012         \l__spintent_spmoney_luaset_plur_str
1013     }
1014     \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
1015     {
1016         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1017         \l__spintent_spmoney_luaset_conde_str
1018     }
1019     \__spintent_spmoney_build_tree:
1020 }

```

(End of definition for `\__spintent_spmoney_check_sub_units:` and `\__spintent_spmoney_set_and_print_sub_units:`.)

```
\__spintent_spmoney_build_tree:
\__spintent_spmoney_print_sep_con:
\__spintent_spmoney_print_dec_sub_unit:
```

The function `\__spintent_spmoney_build_tree:` structures the final MathML tree assembly for fractional “*currencies*”. It arranges the integer component, the decimal separator, and the “*subunit*” component based on the layout position (pre or post) of the “*currency symbol*”.

Furthermore, `\__spintent_spmoney_print_sep_con:` applies the specific speech intent ‘con’ to the decimal separator, while `\__spintent_spmoney_print_dec_sub_unit:` binds the computed “*subunit*” intent to the decimal digits, ensuring accurate accessibility output.

```
1021 \cs_new_protected:Nn \__spintent_spmoney_build_tree:
1022 {
1023   \str_if_eq:VnTF \__spintent_spmoney_luaset_position_str { pre }
1024   {
1025     \MathMLintent { \__spintent_spmoney_res_main_voice_str :postfix($n) }
1026     {
1027       \__spintent_spmoney_print_symbol:
1028       \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1029     }
1030     \__spintent_spmoney_print_sep_con:
1031     \__spintent_spmoney_print_dec_sub_unit:
1032   }
1033   {
1034     \MathMLintent { \__spintent_spmoney_res_main_voice_str :postfix($n) }
1035     {
1036       \__spintent_inner_sep:
1037       \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1038     }
1039     \__spintent_spmoney_print_sep_con:
1040     \__spintent_spmoney_print_dec_sub_unit:
1041     \MathMLintent {:silent} { \__spintent_spmoney_print_symbol: }
1042   }
1043 }
1044 \cs_new_protected:Nn \__spintent_spmoney_print_sep_con:
1045 {
1046   \MathMLintent { _con }
1047   {
1048     \mathord { \__spintent_luaset_dec_sep_tl }
1049   }
1050 }
1051 \cs_new_protected:Nn \__spintent_spmoney_print_dec_sub_unit:
1052 {
1053   \MathMLintent { \__spintent_spmoney_intent_sub_unit_str }
1054   {
1055     \__spintent_inner_sep:
1056     \MathMLarg { sub }
1057     {
1058       \__spintent_spmoney_render_dec:
1059     }
1060   }
1061 }
```

(End of definition for `\__spintent_spmoney_build_tree:`, `\__spintent_spmoney_print_sep_con:`, and `\__spintent_spmoney_print_dec_sub_unit:`.)

```
\__spintent_spmoney_parse_and_route:n
```

The internal function `\__spintent_spmoney_parse_and_route:n` processes the $\{\langle argument \rangle\}$ passed in ‘#1’ using Lua module to extract the numeric value and any “*currency*” identifier. If a “*currency*” is found (`\__spintent_spmoney_extracted_curr_str`), it sets the corresponding variables. It then calls `\__spintent_spmoney_validate_money:` to verify the “*currency*”. If “*subunit*” rendering is enabled (`\__spintent_spmoney_sub_units_bool`), it calls `\__spintent_spmoney_check_sub_units:`. Finally, it calls `\__spintent_spmoney_print_int:` to format the output.

```
1062 \cs_new_protected:Nn \__spintent_spmoney_parse_and_route:n
1063 {
1064   \__spintent_luafun_spmoney_prepare_input:n { #1 }
1065   \__spintent_luafun_clean_split_and_set:V \__spintent_spmoney_extracted_num_tl
1066   \str_if_empty:NF \__spintent_spmoney_extracted_curr_str
1067   {
1068     \str_set:Ne \__spintent_luaset_has_units_str { true }
1069     \str_set:NV \__spintent_luaset_arg_above_tl \__spintent_spmoney_extracted_curr_str
1070   }
1071   \__spintent_spmoney_validate_money:
1072   \bool_if:NT \__spintent_spmoney_sub_units_bool
```

```

1073     { \__spintent_spmoney_check_sub_units: }
1074     \__spintent_spmoney_print_int:
1075 }

```

(End of definition for __spintent_spmoney_parse_and_route:n.)

\spmoney The public document command **\spmoney** formats monetary values with accessible speech intents. It accepts an optional argument ‘#1’ for local key-value configurations and a mandatory argument ‘#2’ containing the numeric value and optional embedded currency identifier. It first evaluates the typesetting context, triggering a fatal error if executed outside of math mode. Upon validation, it establishes a local $\TeX$ group to safely isolate the key assignments, delegates the raw input string to **__spintent_spmoney_parse_and_route:n** for parsing and rendering, and subsequently closes the group to prevent scope leakage.

```

1076 \NewDocumentCommand \spmoney { 0{} m }
1077 {
1078     \mode_if_math:TF
1079     {
1080         \group_begin:
1081         \keys_set:nn { spintent / spmoney } { #1 }
1082         \__spintent_spmoney_parse_and_route:n { #2 }
1083         \group_end:
1084     }
1085     { \msg_error:nnn { spintent } { need-math-mode } { \spmoney } }
1086 }

```

(End of definition for \spmoney. This function is documented on page 6.)

11.18 The command \spshort

The user command **\spshort** is the *fourth* and *final* of “the big four”. It manages the semantic injection and typographical formatting of *text mode* abbreviations, ordinals, and acronyms. Unlike *math mode* commands, which rely on **MathML** attributes, this command uses the PDF *tagging tools* provided by the **tagpdf**[29] package. Specifically, it uses the **E** (expanded) key to ensure that NVDA interpret the full form of the abbreviation, ordinal, or acronym in *text mode*.

These internal string variables serve as the primary data interface with the Lua module **spintent.lua**. During processing, the Lua module “stores” within these registers the execution status, the targeted typographical layout parameters, the phonetic replacement text for the **E** key intended for NVDA, the final sanitized output string, and the isolated structural components for ordinals (base and suffix).

```

1087 \str_new:N \__spintent_spshort_luaset_status_str
1088 \str_new:N \__spintent_spshort_luaset_layout_str
1089 \str_new:N \__spintent_spshort_luaset_expanded_str
1090 \str_new:N \__spintent_spshort_luaset_output_str
1091 \str_new:N \__spintent_spshort_luaset_base_str
1092 \str_new:N \__spintent_spshort_luaset_suffix_str

```

(End of definition for __spintent_spshort_luaset_status_str and others.)

11.18.1 Definition of error messages for \spshort

unknown-abbreviation The internal “error” message **unknown-abbreviation** is triggered when the command **\spshort** encounters an invalid lookup state from the Lua module. It acts as a strict fallback exception when the provided $\langle argument \rangle$ cannot be mapped to any registered database entry in **spintent.lua** nor successfully resolved through the grammatical ordinal expansion rules.

```

1093 \msg_new:nnnn { spintent } { unknown-abbreviation }
1094 { The~abbreviation~or~ordinal~'~#1'~is~invalid~or~not~recognized. }
1095 { Verify~the~spelling~or~check~the~grammar~rules~for~ordinals. }

```

(End of definition for unknown-abbreviation.)

11.18.2 Accessibility interface

__spintent_spshort_tag_span:nn The internal function **__spintent_spshort_tag_span:nn** processes the $\langle argument \rangle$ passed in ‘#2’ using the tools provided by **tagpdf**. By utilizing the **tag** and **E** keys, it creates a native PDF /E (#1) structure and expands the $\langle argument \rangle$ passed in ‘#1’ for proper reading with NVDA.

```

1096 \cs_new_protected:Nn \__spintent_spshort_tag_span:nn
1097 {
1098     \tag_mc_end_push:
1099     \tag_struct_begin:n { tag=Span, E={ #1 } }
1100     \tag_mc_begin:n { tag=Span }
1101     \tag_suspend:n { \spshort }
1102     #2
1103     \tag_resume:n { \spshort }
1104     \tag_mc_end:

```

```

1105     \tag_struct_end:
1106     \tag_mc_begin_pop:n {}
1107 }
1108 \cs_generate_variant:Nn \__spintent_spshort_tag_span:nn { en }

```

(End of definition for __spintent_spshort_tag_span:nn.)

11.18.3 Definition of keys for \spshort

Now we add the keys `small-caps` and `read-arg`.

```

small-caps read-arg 1109 \keys_define:nn { spintent / spshort }
1110 {
1111     small-caps .bool_set:N = \__spintent_spshort_smallcaps_bool,
1112     small-caps .initial:n = false,
1113     small-caps .value_required:n = true,
1114     read-arg .str_set:N = \__spintent_spshort_read_arg_str,
1115     read-arg .initial:n = {},
1116     read-arg .value_required:n = true,
1117     unknown .code:n =
1118     {
1119         \str_set:Nn \__spintent_command_name_str { spshort }
1120         \__spintent_unknown_keys_cmd:n {#1}
1121     },
1122 }

```

(End of definition for `small-caps` and `read-arg`.)

11.18.4 Internal functions for \spshort

The internal function `\__spintent_spshort_print:nn` acts as the primary formatter for text abbreviations. It clears the string variable `\__spintent_spshort_read_arg_str` and evaluates the optional `<keys>` passed in ‘#1’. It then checks if a manual reading override was provided via those keys. If that string variable remains empty, it calls `\__spintent_spshort_process_and_render:n` to process the `{<argument>}` passed in ‘#2’. Otherwise, if an override is detected, it directly delegates the execution to `\__spintent_spshort_render_bypass:n`.

```

1123 \cs_new_protected:Nn \__spintent_spshort_print:nn
1124 {
1125     \str_clear:N \__spintent_spshort_read_arg_str
1126     \keys_set:nn { spintent / spshort } {#1}
1127     \str_if_empty:NTF \__spintent_spshort_read_arg_str
1128     {
1129         \__spintent_spshort_process_and_render:n {#2}
1130     }
1131     {
1132         \__spintent_spshort_render_bypass:n {#2}
1133     }
1134 }

```

(End of definition for `\__spintent_spshort_print:nn`.)

The internal function `\__spintent_spshort_process_and_render:n` interfaces with the Lua module `spintent.lua` function `\__spintent_luafun_spshort_process:n` to resolve the abbreviation or ordinal `{<argument>}` passed in ‘#1’.

It subsequently evaluates the execution state stored within the string variable `\__spintent_spshort_luaset_status` and dynamically routes execution to the corresponding rendering subroutine `success`, `fallback` or `error`. Any unrecognized status automatically defaults to the “error” handler.

```

1135 \cs_new_protected:Nn \__spintent_spshort_process_and_render:n
1136 {
1137     \__spintent_luafun_spshort_process:n {#1}
1138     \str_case:VnF \__spintent_spshort_luaset_status_str
1139     {
1140         { success } { \__spintent_spshort_render_success: }
1141         { fallback } { \__spintent_spshort_render_fallback: }
1142         { error } { \__spintent_spshort_render_error:n {#1} }
1143     }
1144     { \__spintent_spshort_render_error:n {#1} }
1145 }

```

(End of definition for `\__spintent_spshort_process_and_render:n`.)

```

\__spintent_spshort_render_success:
\__spintent_spshort_render_fallback:
\__spintent_spshort_render_bypass:n
\__spintent_spshort_render_error:n

```

The function `\__spintent_spshort_render_success:` injects the key `E` and executes the targeted typographical layout. The `\__spintent_spshort_render_fallback:` function handles default cases, applying optional formatting such as `SMALL CAPS` to unrecognized or partially resolved terms.

Conversely, the function `\__spintent_spshort_render_bypass:n` directly maps the user-defined “*speech*” to the $\{\langle argument \rangle\}$, bypassing the automated lookup Lua module. Finally, the function `\__spintent_spshort_rende` triggers a structural exception while yielding the unformatted $\{\langle argument \rangle\}$ to prevent fatal compilation.

```

1146 \cs_new_protected:Nn \__spintent_spshort_render_success:
1147 {
1148   \__spintent_spshort_tag_span:en
1149   { \l__spintent_spshort_luaset_expanded_str }
1150   { \__spintent_spshort_execute_layout: }
1151 }
1152 \cs_new_protected:Nn \__spintent_spshort_render_fallback:
1153 {
1154   \__spintent_spshort_tag_span:en
1155   { \l__spintent_spshort_luaset_expanded_str }
1156   {
1157     \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1158     { \textsc{ \l__spintent_spshort_luaset_output_str } }
1159     { \l__spintent_spshort_luaset_output_str }
1160   }
1161 }
1162 \cs_new_protected:Nn \__spintent_spshort_render_bypass:n
1163 {
1164   \__spintent_spshort_tag_span:en { \l__spintent_spshort_read_arg_str } { #1 }
1165 }
1166 \cs_new_protected:Nn \__spintent_spshort_render_error:n
1167 {
1168   \msg_error:nnn { spintent } { unknown-abbreviation } {#1}
1169 }

```

(End of definition for `\__spintent_spshort_render_success:` and others.)

```

\__spintent_spshort_execute_layout:

```

The internal function `\__spintent_spshort_execute_layout:` executes the typographical layout. It evaluates the descriptor “*stored*” within the string variable `\l__spintent_spshort_luaset_layout_str` (such as `linear_regular`, `superscript`, or `small_caps_pure`) and maps it to the corresponding $\text{\LaTeX}$ formatting. Furthermore, it manages complex nested configurations, dynamically applying combinations such as superscripts integrated with conditional `SMALL CAPS` based on the active boolean variables.

```

1170 \cs_new_protected:Nn \__spintent_spshort_execute_layout:
1171 {
1172   \str_case:VnF \l__spintent_spshort_luaset_layout_str
1173   {
1174     { linear_regular }
1175     {
1176       \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1177       { \textsc{ \l__spintent_spshort_luaset_output_str } }
1178       { \l__spintent_spshort_luaset_output_str }
1179     }
1180     { linear_caps }
1181     { \l__spintent_spshort_luaset_output_str }
1182     { linear_mixed }
1183     { \l__spintent_spshort_luaset_output_str }
1184     { superscript }
1185     {
1186       \l__spintent_spshort_luaset_base_str
1187       \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1188       {
1189         \textsuperscript{
1190           \textsc{
1191             \str_lowercase:V \l__spintent_spshort_luaset_suffix_str
1192           }
1193         }
1194       }
1195       {
1196         \textsuperscript{ \l__spintent_spshort_luaset_suffix_str }
1197       }
1198     }
1199     { small_caps_pure }
1200     {
1201       \bool_if:NTF \l__spintent_spshort_smallcaps_bool

```

```

1202         {
1203             \textsc{
1204                 \str_lowercase:V \l__spintent_spsshort_luaset_output_str
1205             }
1206         }
1207         { \l__spintent_spsshort_luaset_output_str }
1208     }
1209     { acronym_long }
1210     { \l__spintent_spsshort_luaset_output_str }
1211 }
1212 { \l__spintent_spsshort_luaset_output_str }
1213 }

```

(End of definition for `\l__spintent_spsshort_execute_layout:`.)

`\spsshort` The public document command `\spsshort` manages the accessible formatting of *text mode* abbreviations, acronyms, and ordinals. It first triggers an error if invoked within a *math mode*, upon validation, puts `\mode_leave_vertical:` and establishes a local group to call the function `\l__spintent_spsshort_print:nn` for processing and printing.

```

1214 \NewDocumentCommand \spsshort { 0{} m }
1215 {
1216     \mode_if_math:TF
1217     {
1218         \msg_error:nnn { spintent } { need-text-mode } { \spsshort }
1219     }
1220     {
1221         \mode_leave_vertical:
1222         \group_begin:
1223         \l__spintent_spsshort_print:nn {#1} {#2}
1224         \group_end:
1225     }
1226 }

```

(End of definition for `\spsshort`. This function is documented on page 7.)

11.19 The command `\spsiglo`

The public document command `\spsiglo` provides a semantic interface for typesetting century designations (e.g., siglo XXI). It accepts both Arabic and Roman numerals as `{\argument}`, interfacing with the Lua module `spintent.lua` to validate, parse, and normalize chronological values.

This command work in dual-mode context awareness, operating uniformly across both *text mode* and *math mode*. When executed within *math mode*, it constructs an accessible **MathML** intent expression tree. Conversely, when invoked in standard *text mode*, it encapsulates the rendered output within a native PDF tagging structure (such as `/Alt (#1)`) containing an `alt` key.

`\l__spintent_spsiglo_luaset_error_str` These internal string variables handle data transfer and synchronization with the Lua module `spintent.lua`.
`\l__spintent_spsiglo_luaset_arabic_str` They isolate the exception evaluation states, the Arabic numeral required for speech synthesis, the lowercase
`\l__spintent_spsiglo_luaset_roman_min_str` Roman numeral designated for small caps typesetting via `\textsc`, and the localized chronological era modifier.
`\l__spintent_spsiglo_print_era_str`

```

1227 \str_new:N \l__spintent_spsiglo_luaset_error_str
1228 \str_new:N \l__spintent_spsiglo_luaset_arabic_str
1229 \str_new:N \l__spintent_spsiglo_luaset_roman_min_str
1230 \str_new:N \l__spintent_spsiglo_print_era_str

```

(End of definition for `\l__spintent_spsiglo_luaset_error_str` and others.)

11.19.1 Definition of error messages for `\spsiglo`

`invalid-century-format` This internal error message is issued when the provided century identifier fails both Arabic and Roman numeral validation. It is triggered when the Lua module `spintent.lua` cannot map the `{\argument}` to an integer within the valid range of 1 to 4000, preventing incorrect speech synthesis.

```

1231 \msg_new:nnnn { spintent } { invalid-century-format }
1232 {
1233     Invalid~century~format~in~'#1'.
1234 }
1235 {
1236     The~argument~'#2'~is~not~a~valid~Arabic~or~Roman~numeral.~
1237     Ensure~the~value~is~between~1~and~4000.
1238 }

```

(End of definition for `invalid-century-format`.)

11.19.2 Definition of keys for \spsiglo

`print-era` Now we add the key `print-era`. User configuration determining if an era designation is appended, available values are `none`, `ac` (antes de cristo) and `dc` (después de cristo).

```

1239 \keys_define:nn { spintent / spsiglo }
1240 {
1241   print-era           .choices:nn =
1242     { none , ac , dc }
1243     {
1244       \int_case:nn { \l_keys_choice_int }
1245       {
1246         {1} { \str_set:Nn \l__spintent_spsiglo_print_era_str { none } }
1247         {2} { \str_set:Nn \l__spintent_spsiglo_print_era_str { ac } }
1248         {3} { \str_set:Nn \l__spintent_spsiglo_print_era_str { dc } }
1249       }
1250     },
1251   print-era           .initial:n = none,
1252   print-era           .value_required:n = true,
1253   print-era / unknown .code:n =
1254     \msg_error:nneee { spintent } { unknown-choice }
1255     { print-era } { none, ~ac, ~dc } { \exp_not:n {#1} },
1256   unknown             .code:n =
1257     {
1258       \str_set:Nn \l__spintent_command_name_str { spsiglo }
1259       \__spintent_unknown_keys_cmd:n {#1}
1260     },
1261 }

```

(End of definition for `print-era`.)

11.19.3 Internal functions for \spsiglo

`\__spintent_spsiglo_render_simple:nn` The functions `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn` generate MathML structures when century designations are evaluated within *math mode*. They construct MathML intent structures, mapping the Arabic numerals to their phonetic equivalents. Furthermore, `\__spintent_spsiglo_render_with_era:nnn` integrates the era modifiers `ac` or `dc`. It adds invisible spacing and postfix intents to ensure correct NVDA/MathCAT pronunciation.

```

1262 \cs_new_protected:Nn \__spintent_spsiglo_render_simple:nn
1263 {
1264   \MathMLintent { #1:number } { \text { \textsc {#2} } } }
1265 }
1266 \cs_generate_variant:Nn \__spintent_spsiglo_render_simple:nn { VV }
1267 \cs_new_protected:Nn \__spintent_spsiglo_render_with_era:nnn
1268 {
1269   \str_if_eq:nnTF { #1 } { ac }
1270   {
1271     \MathMLintent { antes-de-cristo:postfix($z) }
1272     {
1273       \__spintent_inner_sep:
1274       \MathMLarg { z }
1275       {
1276         \MathMLintent { #2:number }
1277         {
1278           \text { \textsc {#3}~a.~C. }
1279         }
1280       }
1281     }
1282   }
1283   {
1284     \MathMLintent { después-de-cristo:postfix($z) }
1285     {
1286       \__spintent_inner_sep:
1287       \MathMLarg { z }
1288       {
1289         \MathMLintent { #2:number }
1290         {
1291           \text { \textsc {#3}~d.~C. }
1292         }
1293       }
1294     }
1295   }
1296 }

```

```
1297 \cs_generate_variant:Nn \__spintent_spsiglo_render_with_era:nnn { VVV }
```

(End of definition for `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn`.)

```
\__spintent_spsiglo_tag_span:nn
\__spintent_spsiglo_tag_span:en
```

The function `\__spintent_spsiglo_tag_span:nn` handles century formatting within *text mode*. It utilizes the `tagpdf` package to enclose the content inside a PDF /Alt (`#1`) structure. By passing the `{⟨argument⟩}` to the `alt` key, it embeds the chronological value directly into the PDF, ensuring correct NVDA/MathCAT pronunciation without relying on MathML intents.

```
1298 \cs_new_protected:Nn \__spintent_spsiglo_tag_span:nn
1299 {
1300   \tag_mc_end_push:
1301   \tag_struct_begin:n { tag=Span, alt={ #1 } }
1302   \tag_mc_begin:n { tag=Span }
1303   \tag_suspend:n { \spsiglo }
1304   #2
1305   \tag_resume:n { \spsiglo }
1306   \tag_mc_end:
1307   \tag_struct_end:
1308   \tag_mc_begin_pop:n {}
1309 }
1310 \cs_generate_variant:Nn \__spintent_spsiglo_tag_span:nn { en }
```

(End of definition for `\__spintent_spsiglo_tag_span:nn`.)

```
\__spintent_spsiglo_print_math:n
\__spintent_spsiglo_print_text:n
```

The functions `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n` format century designations based on the active mode. Both first check the “error” status in the variable `\l__spintent_spsiglo_luaset_error_str` returned by the Lua module `spintent.lua` and issue an “error” if validation fails.

If the `{⟨argument⟩}` is valid, the function `\__spintent_spsiglo_print_math:n` operates in *math mode*, applying MathML structures depending on whether an era modifier is present. Conversely, the function `\__spintent_spsiglo_print_text:n` operates in *text mode*, utilizing the `tag` and `alt` keys to expand the value of the `print-era` option into full text (e.g., “antes de cristo”) for NVDA/MathCAT.

```
1311 \cs_new_protected:Nn \__spintent_spsiglo_print_math:n
1312 {
1313   \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1314   {
1315     \msg_error:nnnn { spintent } { invalid-century-format }
1316     { \spsiglo } { #1 }
1317   }
1318   {
1319     \str_if_eq:VnTF \l__spintent_spsiglo_print_era_str { none }
1320     {
1321       \__spintent_spsiglo_render_simple:VV
1322       \l__spintent_spsiglo_luaset_arabic_str
1323       \l__spintent_spsiglo_luaset_roman_min_str
1324     }
1325     {
1326       \__spintent_spsiglo_render_with_era:VVV
1327       \l__spintent_spsiglo_print_era_str
1328       \l__spintent_spsiglo_luaset_arabic_str
1329       \l__spintent_spsiglo_luaset_roman_min_str
1330     }
1331   }
1332 }
1333 \cs_new_protected:Nn \__spintent_spsiglo_print_text:n
1334 {
1335   \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1336   {
1337     \msg_error:nnnn { spintent } { invalid-century-format }
1338     { \spsiglo } { #1 }
1339   }
1340   {
1341     \str_case:Vn \l__spintent_spsiglo_print_era_str
1342     {
1343       { none }
1344       {
1345         \__spintent_spsiglo_tag_span:en
1346         { \l__spintent_spsiglo_luaset_arabic_str }
1347         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } }
1348       }
1349       { ac }
```

```

1350         {
1351             \__spintent_spsiglo_tag_span:en
1352             { \l__spintent_spsiglo_luaset_arabic_str ~ antes ~ de ~ cristo }
1353             { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ a.~C. }
1354         }
1355         { dc }
1356         {
1357             \__spintent_spsiglo_tag_span:en
1358             { \l__spintent_spsiglo_luaset_arabic_str ~ después ~ de ~ cristo }
1359             { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ d.~C. }
1360         }
1361     }
1362 }
1363 }

```

(End of definition for __spintent_spsiglo_print_math:n and __spintent_spsiglo_print_text:n)

\spsiglo The public command `\spsiglo` is the main interface for century typesetting. It opens a local group and evaluates any optional *(keys)* passed in ‘#1’, then executes the function `\__spintent_luafun_spsiglo_parse:n` defined in the Lua module `spintent.lua` passed in ‘#2’ and formats the output for *math mode* or *text mode*.

```

1364 \NewDocumentCommand \spsiglo { 0{} m }
1365 {
1366     \group_begin:
1367     \keys_set:nn { spintent / spsiglo } { #1 }
1368     \__spintent_luafun_spsiglo_parse:n { #2 }
1369     \mode_if_math:TF
1370     { \__spintent_spsiglo_print_math:n { #2 } }
1371     {
1372         \mode_leave_vertical:
1373         \__spintent_spsiglo_print_text:n { #2 }
1374     }
1375     \group_end:
1376 }

```

(End of definition for \spsiglo. This function is documented on page 11.)

11.20 The command \sptime

The command `\sptime` is the main public interface for typesetting and tagging time expressions. It parses time string formats (such as HH:MM or HH:MM:SS) using the Lua function `\__spintent_luafun_sptime_parse:n`. This separates the hours, minutes, seconds, and fractions to construct a MathML *intent* tree for NVDA/MathCAT.

```

\l__spintent_sptime_luaset_seconds_str
\l__spintent_sptime_luaset_has_seconds_str
\l__spintent_sptime_luaset_error_str
\l__spintent_sptime_luaset_is_fraction_str
\l__spintent_sptime_luaset_base_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted time units, error flags, and boolean indicators (such as the presence of seconds or decimal fractions) used during formatting.

```

1377 \str_new:N \l__spintent_sptime_luaset_seconds_str
1378 \str_new:N \l__spintent_sptime_luaset_has_seconds_str
1379 \str_new:N \l__spintent_sptime_luaset_error_str
1380 \str_new:N \l__spintent_sptime_luaset_is_fraction_str
1381 \str_new:N \l__spintent_sptime_luaset_base_str

```

(End of definition for \l__spintent_sptime_luaset_seconds_str and others.)

11.20.1 Definition of error messages

invalid-time-format

The error message `invalid-time-format` defines the text displayed when an invalid time input is detected. It is triggered when an argument fails the format checks or contains values outside normal ranges, such as hours exceeding 23 or minutes and seconds exceeding 59.

```

1382 \msg_new:nnnn { spintent } { invalid-time-format }
1383 {
1384     Invalid~time~format~in~#1. \\\
1385     The~argument~'~#2'~does~not~match~any~supported~time~format.
1386 }
1387 {
1388     Allowed~formats:~HH:MM,~HH:MM:SS,~or~HH:MM:SS.ss. \\\
1389     Please~verify~that~hours~are~0-23~and~minutes/seconds~are~0-59.
1390 }

```

(End of definition for invalid-time-format.)

11.20.2 Internal functions for \sptime

__spintent_sptime_build_seconds:nn

The internal function `\__spintent_sptime_build_seconds:nn` builds the MathML structure for the seconds component of a time expression. It wraps the numeric value ‘#2’ in speech intent tags, applying a connector prefix (passed in ‘#1’) and the “segundos” postfix so it is read correctly by NVDA/MathCAT.

```

1391 \cs_new_protected:Nn \__spintent_sptime_build_seconds:nn
1392 {
1393   \MathMLintent { #1:prefix($z) }
1394   {
1395     \text { : }
1396     \MathMLarg { z }
1397     {
1398       \MathMLintent { _segundos:postfix($m) }
1399       {
1400         \__spintent_inner_sep:
1401         \MathMLarg { m }
1402         {
1403           \text { #2 }
1404         }
1405       }
1406     }
1407   }
1408 }

```

(End of definition for __spintent_sptime_build_seconds:nn.)

__spintent_sptime_build_tree:

The internal function `\__spintent_sptime_build_tree:` builds the complete MathML structure for the time expression. It checks the string variables `\l__spintent_sptime_luaset_has_seconds_str` and `\l__spintent_sptime_luaset_is_fraction_str` to determine if the time includes seconds or decimal fractions. Based on these values, it inserts the base time string and either calls `\__spintent_sptime_build_seconds:` with the appropriate prefix (‘con’ or ‘minutos-con’), or simply appends the ‘minutos’ intent tag if no seconds are present.

```

1409 \cs_new_protected:Nn \__spintent_sptime_build_tree:
1410 {
1411   \str_if_eq:VnTF \l__spintent_sptime_luaset_has_seconds_str { true }
1412   {
1413     \MathMLintent { :time }
1414     {
1415       \text { \l__spintent_sptime_luaset_base_str }
1416     }
1417     \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { true }
1418     {
1419       \__spintent_sptime_build_seconds:nn { con }
1420       { \l__spintent_sptime_luaset_seconds_str }
1421     }
1422     {
1423       \__spintent_sptime_build_seconds:nn { _minutos-con }
1424       { \l__spintent_sptime_luaset_seconds_str }
1425     }
1426   }
1427   {
1428     \MathMLintent { :time }
1429     {
1430       \text { \l__spintent_sptime_luaset_base_str }
1431     }
1432     \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { false }
1433     {
1434       \MathMLintent { _minutos }
1435       {
1436         \c__spintent_invisible_sep_int
1437       }
1438     }
1439   }
1440 }

```

(End of definition for __spintent_sptime_build_tree:.)

\sptime

The command `\sptime` is the public interface for formatting time expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it parses the input using `\__spintent_luafun_sptime_parse:n` and checks the error status in `\l__spintent_sptime_luaset_error_str`. If

an invalid format is detected, it issues the `invalid-time-format` error; otherwise, it calls `\__spintent_sptime_build_tree:` to assemble the MathML output.

```

1441 \NewDocumentCommand \sptime { m }
1442 {
1443   \mode_if_math:TF
1444   {
1445     \group_begin:
1446     \__spintent_luafun_sptime_parse:n { #1 }
1447     \str_if_eq:VnTF \__spintent_sptime_luaset_error_str { true }
1448     {
1449       \msg_error:nnnn { spintent } { invalid-time-format }
1450       { \sptime } { #1 }
1451     }
1452     {
1453       \__spintent_sptime_build_tree:
1454     }
1455   \group_end:
1456 }
1457 { \msg_error:nnn { spintent } { need-math-mode } { \sptime } }
1458 }

```

(End of definition for `\sptime`. This function is documented on page 11.)

11.21 The command `\spdate`

The command `\spdate` is the main public command for typesetting and tagging date expressions. It parses date string formats (such as DD/MM/YYYY or YYYY-MM-DD) using the Lua function `\__spintent_luafun_spdate_parse:n`. This verifies the input and constructs a MathML `intent` tree for NVDA/MathCAT.

```

\__spintent_spdate_luaset_output_str
\__spintent_spdate_luaset_error_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the formatted date string and the error status used during formatting.

```

1459 \str_new:N \__spintent_spdate_luaset_output_str
1460 \str_new:N \__spintent_spdate_luaset_error_str

```

(End of definition for `\__spintent_spdate_luaset_output_str` and `\__spintent_spdate_luaset_error_str`.)

11.21.1 Definition of error messages

```
invalid-date-format
```

The error message `invalid-date-format` defines the text displayed when an invalid date input is detected. It is triggered when an argument fails the format checks or represents an invalid calendar date, such as `31/02/2024` or using an unsupported pattern like `YYYY/MM/DD`.

```

1461 \msg_new:nnnn { spintent } { invalid-date-format }
1462 {
1463   Invalid~date~or~unsupported~format~in~#1. \\\
1464   The~argument~'#2'~does~not~match~a~supported~format.
1465 }
1466 {
1467   Allowed~formats:~DD\slash MM\slash YYYY,~DD-MM-YYYY,~or~YYYY-MM-DD. \\\
1468   Ensure~the~day~and~month~are~valid~(e.g.,~avoid~31\slash 02\slash 2024).
1469 }

```

(End of definition for `invalid-date-format`.)

11.21.2 Internal functions for `\spdate`

```
\__spintent_spdate_process:n
```

The internal function `\__spintent_spdate_process:n` parses the input using `\__spintent_luafun_spdate_parse:n` and checks the error status in `\__spintent_spdate_luaset_error_str`. If an invalid format is detected, it issues the `invalid-date-format` error. Otherwise, it builds the MathML `intent` structure using the formatted date stored in `\__spintent_spdate_luaset_output_str`.

```

1470 \cs_new_protected:Nn \__spintent_spdate_process:n
1471 {
1472   \__spintent_luafun_spdate_parse:n { #1 }
1473   \str_if_eq:VnTF \__spintent_spdate_luaset_error_str { true }
1474   {
1475     \msg_error:nnnn { spintent } { invalid-date-format }
1476     { \spdate } { #1 }
1477   }
1478   {
1479     \MathMLintent { :date }
1480     {
1481       \text { \__spintent_spdate_luaset_output_str }
1482     }
1483   }
1484 }

```

(End of definition for `\__spintent_sdate_process:n`.)

`\sdate` The command `\sdate` is the public interface for formatting date expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_sdate_process:n` to process the input.

```

1485 \NewDocumentCommand \sdate { m }
1486 {
1487   \mode_if_math:TF
1488   {
1489     \group_begin:
1490       \__spintent_sdate_process:n { #1 }
1491     \group_end:
1492   }
1493   { \msg_error:nnn { spintent } { need-math-mode } { \sdate } }
1494 }

```

(End of definition for `\sdate`. This function is documented on page 11.)

11.22 Semantic Operators

This section provides purely semantic operators. Unlike their formatting counterparts (which parse and format numbers), these commands do not process mandatory arguments. Instead, they solely inject a mathematical glyph wrapped in its corresponding regional `MathCAT` intent. This grants teachers absolute control when building complex algebraic expressions, mixed numbers, or fraction divisions.

11.23 The command `\spusep`

For the use of parentheses, we will provide the command `\spusep` with only two `<keys>` `size` and `silent`. Most of the time we can use `\left` and `\right` or `\Uleft` and `\Uright`, but both forms create a group, and this hinders the code for *tagged* PDF when we want to interrupt something between them.

`spusep-empty-arg`
`spusep-invalid-arg`

The first “error” message `spusep-empty-arg` is raised when `\spusep` receives an empty or blank `{<argument>}`, the second this is when an input is invalid.

```

1495 \msg_new:nnnn { spintent } { spusep-empty-arg }
1496 {
1497   Empty~argument~'#1'~for~\token_to_str:N \spusep.
1498 }
1499 {
1500   \token_to_str:N \spusep~requires~a~non-empty~argument.
1501 }
1502 \msg_new:nnn { spintent } { spusep-invalid-arg }
1503 {
1504   Invalid~argument~'#1'~for~\token_to_str:N \spusep.
1505 }

```

(End of definition for `spusep-empty-arg` and `spusep-invalid-arg`.)

`\__spintent_spusep_arg_tl`
`\__spintent_spusep_left_tl`
`\__spintent_spusep_right_tl`

The variable `\__spintent_spusep_arg_tl` stores the `{<argument>}`, the variable `\__spintent_spusep_left_tl` and `\__spintent_spusep_right_tl` stores the parentheses.

```

1506 \tl_new:N \__spintent_spusep_arg_tl
1507 \tl_new:N \__spintent_spusep_left_tl
1508 \tl_new:N \__spintent_spusep_right_tl

```

(End of definition for `\__spintent_spusep_arg_tl`, `\__spintent_spusep_left_tl`, and `\__spintent_spusep_right_tl`.)

`size`
`silent`
`unknown`

We define the `<keys>` `size` and `silent`.

```

1509 \keys_define:nn { spintent / spusep }
1510 {
1511   size .choices:nn =
1512     { none , big , Big , bigg , Bigg }
1513     {
1514       \int_case:nn { \l_keys_choice_int }
1515       {
1516         { 1 }
1517         {
1518           \tl_clear:N \__spintent_spusep_left_tl
1519           \tl_clear:N \__spintent_spusep_right_tl
1520         }
1521         { 2 }
1522         {
1523           \tl_set:Nn \__spintent_spusep_left_tl { \bigl }
1524           \tl_set:Nn \__spintent_spusep_right_tl { \bigr }
1525         }
1526       }
1527     }

```

```

1526         { 3 }
1527         {
1528             \tl_set:Nn \l__spintent_spusep_left_tl { \Bigl }
1529             \tl_set:Nn \l__spintent_spusep_right_tl { \Bigr }
1530         }
1531         { 4 }
1532         {
1533             \tl_set:Nn \l__spintent_spusep_left_tl { \biggl }
1534             \tl_set:Nn \l__spintent_spusep_right_tl { \biggr }
1535         }
1536         { 5 }
1537         {
1538             \tl_set:Nn \l__spintent_spusep_left_tl { \Biggl }
1539             \tl_set:Nn \l__spintent_spusep_right_tl { \Biggr }
1540         }
1541     }
1542 },
1543 size .initial:n = none,
1544 size .value_required:n = true,
1545 size / unknown .code:n =
1546     \msg_error:nneee { spintent } { unknown-choice }
1547     { size } { none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
1548 silent .bool_set:N = \l__spintent_spusep_silent_bool,
1549 silent .initial:n = false,
1550 silent .value_required:n = true,
1551 unknown .code:n =
1552     {
1553         \str_set:Nn \l__spintent_command_name_str { spusep }
1554         \__spintent_unknown_keys_cmd:n {#1}
1555     },
1556 }

```

(End of definition for `size`, `silent`, and `unknown`.)

`\__spintent_spusep_set_parens:nn`

The function `\__spintent_spusep_set_parens:nn` will place the argument passed in ‘#2’ to the right of the variables `\l__spintent_spusep_left_tl` and `\l__spintent_spusep_right_tl` and then check the state of the variable `\l__spintent_spusep_silent_bool` handled by the key `silent` to print.

```

1557 \cs_new_protected:Nn \__spintent_spusep_set_parens:nn
1558 {
1559     \tl_put_right:cn { \l__spintent_spusep_ #1 _tl } { #2 }
1560     \bool_if:NTF \l__spintent_spusep_silent_bool
1561     {
1562         \MathMLint { :silent } { \tl_use:c { \l__spintent_spusep_ #1 _tl } }
1563     }
1564     {
1565         \tl_use:c { \l__spintent_spusep_ #1 _tl }
1566     }
1567 }

```

(End of definition for `\__spintent_spusep_set_parens:nn`.)

`\__spintent_spusep_convert:n`

The function `\__spintent_spusep_convert:n` captures and stores the $\langle argument \rangle$ passed in ‘#1’ in the variable `\l__spintent_spusep_arg_tl`, validates the cases by converting them to the macros provided by `unicode-math`, and passes them to the function `\__spintent_spusep_set_parens:nn` for printing. If the input is not supported or is empty, we return an “error”.

```

1568 \cs_new_protected:Nn \__spintent_spusep_convert:n
1569 {
1570     \tl_set:Nn \l__spintent_spusep_arg_tl { \tl_trim_spaces:n {#1} }
1571     \tl_if_empty:NT \l__spintent_spusep_arg_tl
1572     {
1573         \msg_error:nnn { spintent } { spusep-empty-arg } {#1}
1574     }
1575     \str_case:VnF \l__spintent_spusep_arg_tl
1576     {
1577         { ( }
1578         {
1579             \__spintent_spusep_set_parens:nn { left } { \lparen }
1580         }
1581         { ) }
1582         {
1583             \__spintent_spusep_set_parens:nn { right } { \rparen }

```

```

1584     }
1585     { [ ] }
1586     {
1587         \__spintent_spusep_set_parens:nn { left } { \lbrack }
1588     }
1589     { ] }
1590     {
1591         \__spintent_spusep_set_parens:nn { right } { \rbrack }
1592     }
1593     { \{ }
1594     {
1595         \__spintent_spusep_set_parens:nn { left } { \lbrace }
1596     }
1597     { \} }
1598     {
1599         \__spintent_spusep_set_parens:nn { right } { \rbrace }
1600     }
1601 }
1602 {
1603     \msg_error:nnn { spintent } { spusep-invalid-arg } { #1 }
1604 }
1605 }

```

(End of definition for __spintent_spusep_convert:n.)

\spusep The command `\spusep` checks that it is running in math mode, opens a local group, sets the keys passed in ‘#1’ and processes the $\langle argument \rangle$ passed in ‘#2’ by means of the function `\__spintent_spusep_convert:n`.

```

1606 \NewDocumentCommand \spusep { O{} m }
1607 {
1608     \mode_if_math:TF
1609     {
1610         \group_begin:
1611         \keys_set:nn { spintent / spusep } { #1 }
1612         \__spintent_spusep_convert:n { #2 }
1613         \group_end:
1614     }
1615     { \msg_error:nnn { spintent } { need-math-mode } { \spusep } }
1616 }

```

(End of definition for \spusep. This function is documented on page 12.)

11.23.1 The command \spread

The command `\spread` provides a semantic read for MathCAT of mathematical symbols for situations in which it is necessary to specify gender or plurality in Spanish.

spread-empty-args The error message `spread-empty-args` is raised when `\spread` receives an empty or blank $\langle argument \rangle$.

```

1617 \msg_new:nnnn { spintent } { spread-empty-args }
1618 { Empty~argument~in~\token_to_str:N \spread. }
1619 { \token_to_str:N \spread~requires~two~non-empty~arguments. }

```

(End of definition for spread-empty-args.)

__spintent_spread_intent_tl The variable `\__spintent_spread_intent_tl` stores the sanitized semantic reading $\langle argument \rangle$ for symbol to be injected into the `\MathMLintent`.

```

1620 \tl_new:N \__spintent_spread_intent_tl

```

(End of definition for __spintent_spread_intent_tl.)

__spintent_spread_exec:nn The function `\__spintent_spread_exec:nn` implements the core logic for `\spread`. It verifies that neither $\langle argument \rangle$ is empty or blank. If valid, it evaluates the *reading* $\langle argument \rangle$ ‘#1’ using `\__spintent_sanitize_affix:Nn` and embeds it via `\MathMLintent` into the *math symbol* ‘#2’.

```

1621 \cs_new_protected:Nn \__spintent_spread_exec:nn
1622 {
1623     \bool_lazy_or:nnTF { \tl_if_blank_p:n { #1 } } { \tl_if_blank_p:n { #2 } }
1624     { \msg_error:nn { spintent } { spread-empty-args } }
1625     {
1626         \__spintent_sanitize_affix:Nn \__spintent_spread_intent_tl { #1 }
1627         \MathMLintent { \__spintent_spread_intent_tl } { #2 }
1628     }
1629 }

```

(End of definition for __spintent_spread_exec:nn.)

`\spread` The command `\spread` assigns a custom semantic *reading* ‘#1’ to any mathematical *symbol* ‘#2’. It enforces math mode and passes execution to `\__spintent_spread_exec:nn`.

```

1630 \NewDocumentCommand \spread { m m }
1631 {
1632   \mode_if_math:TF
1633   {
1634     \group_begin:
1635     \__spintent_spread_exec:nn { #1 } { #2 }
1636     \group_end:
1637   }
1638   { \msg_error:nnn { spintent } { need-math-mode } { \spread } }
1639 }

```

(End of definition for `\spread`. This function is documented on page 11.)

11.23.2 The command `\spdsym`

`\__spintent_spdsym_read_sym_tl` Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spdsym`.

`\__spintent_spdsym_symbol_tl`

`\__spintent_spdsym_prefix_tl`

`\__spintent_spdsym_suffix_tl`

```

1640 \tl_new:N \__spintent_spdsym_read_sym_tl
1641 \tl_new:N \__spintent_spdsym_symbol_tl
1642 \tl_new:N \__spintent_spdsym_prefix_tl
1643 \tl_new:N \__spintent_spdsym_suffix_tl

```

(End of definition for `\__spintent_spdsym_read_sym_tl` and others.)

prefix We define the *(keys)* prefix, suffix, set-sym and read-sym for `\spdsym`.

suffix

set-sym

read-sym

unknown

```

1644 \keys_define:nn { spintent / spdsym }
1645 {
1646   prefix .code:n =
1647   { \__spintent_sanitizize_affix:Nn \__spintent_spdsym_prefix_tl {#1} },
1648   prefix .initial:n = ,
1649   prefix .value_required:n = true,
1650   suffix .code:n =
1651   { \__spintent_sanitizize_affix:Nn \__spintent_spdsym_suffix_tl {#1} },
1652   suffix .initial:n = ,
1653   suffix .value_required:n = true,
1654   set-sym .choices:nn =
1655   { old-style , two-dots , slash }
1656   {
1657     \int_case:nn { \l_keys_choice_int }
1658     {
1659       {1} { \tl_set:Nn \__spintent_spdsym_symbol_tl { \div } }
1660       {2} { \tl_set:Nn \__spintent_spdsym_symbol_tl { : } }
1661       {3} { \tl_set:Nn \__spintent_spdsym_symbol_tl { / } }
1662     }
1663   },
1664   set-sym .initial:n = two-dots,
1665   set-sym .value_required:n = true,
1666   set-sym / unknown .code:n =
1667   \msg_error:nnee { spintent } { unknown-choice }
1668   { set-sym } { old-style, ~two-dots, ~slash } { \exp_not:n {#1} },
1669   read-sym .choices:nn =
1670   { dividido , dividido-por , dividido-entre }
1671   {
1672     \int_case:nn { \l_keys_choice_int }
1673     {
1674       {1} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido } }
1675       {2} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido-por } }
1676       {3} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido-entre } }
1677     }
1678   },
1679   read-sym .initial:n = dividido,
1680   read-sym .value_required:n = true,
1681   read-sym / unknown .code:n =
1682   \msg_error:nnee { spintent } { unknown-choice }
1683   { read-sym } { dividido, ~dividido-por, ~dividido-entre }
1684   { \exp_not:n {#1} },
1685   unknown .code:n =
1686   {
1687     \str_set:Nn \__spintent_command_name_str { spdsym }
1688     \__spintent_unknown_keys_cmd:n {#1}

```

```

1689     },
1690 }

```

(End of definition for `prefix` and others.)

\spdsym Defines the `\spdsym` operator. It takes a single optional argument for local setup and outputs the configured intent structure, safely attaching prefixes and suffixes if present.

```

1691 \NewDocumentCommand \spdsym { 0{} }
1692 {
1693   \mode_if_math:TF
1694   {
1695     \group_begin:
1696     \keys_set:nn { spintent / spdsym } { #1 }
1697     \MathMLintent
1698     {
1699       \tl_if_empty:NF \l__spintent_spdsym_prefix_tl
1700       { \l__spintent_spdsym_prefix_tl - }
1701       \l__spintent_spdsym_read_sym_tl
1702       \tl_if_empty:NF \l__spintent_spdsym_suffix_tl
1703       { - \l__spintent_spdsym_suffix_tl }
1704     }
1705     { \l__spintent_spdsym_symbol_tl }
1706   \group_end:
1707   }
1708   { \msg_error:nnn { spintent } { need-math-mode } { \spdsym } }
1709 }

```

(End of definition for `\spdsym`. This function is documented on page 12.)

11.23.3 The command `\spmsym`

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spmsym`.

```

\l__spintent_spmsym_read_sym_tl
\l__spintent_spmsym_symbol_tl
\l__spintent_spmsym_prefix_tl
\l__spintent_spmsym_suffix_tl
1710 \tl_new:N \l__spintent_spmsym_read_sym_tl
1711 \tl_new:N \l__spintent_spmsym_symbol_tl
1712 \tl_new:N \l__spintent_spmsym_prefix_tl
1713 \tl_new:N \l__spintent_spmsym_suffix_tl

```

(End of definition for `\l__spintent_spmsym_read_sym_tl` and others.)

```

prefix  We define the (keys) prefix, suffix, set-sym, read-sym and not-print for \spmsym.
suffix
set-sym
read-sym
not-print
unknown
1714 \keys_define:nn { spintent / spmsym }
1715 {
1716   prefix .code:n =
1717   { \__spintent_sanitizize_affix:Nn \l__spintent_spmsym_prefix_tl {#1} },
1718   prefix .initial:n = ,
1719   prefix .value_required:n = true,
1720   suffix .code:n =
1721   { \__spintent_sanitizize_affix:Nn \l__spintent_spmsym_suffix_tl {#1} },
1722   suffix .initial:n = ,
1723   suffix .value_required:n = true,
1724   set-sym .choices:nn =
1725   { cross , dot , asterisk }
1726   {
1727     \int_case:nn { \l_keys_choice_int }
1728     {
1729       {1} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \times } }
1730       {2} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \cdot } }
1731       {3} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \ast } }
1732     }
1733   },
1734   set-sym .initial:n = dot,
1735   set-sym .value_required:n = true,
1736   set-sym / unknown .code:n =
1737   \msg_error:nnee { spintent } { unknown-choice }
1738   { set-sym } { cross, ~dot, ~asterisk } { \exp_not:n {#1} },
1739   read-sym .choices:nn =
1740   { por , multiplicado-por , veces }
1741   {
1742     \int_case:nn { \l_keys_choice_int }
1743     {
1744       {1} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { por } }
1745       {2} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { multiplicado-por } }

```

```

1746         {3} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { veces } }
1747     }
1748 },
1749 read-sym      .initial:n = por,
1750 read-sym      .value_required:n = true,
1751 read-sym / unknown .code:n =
1752     \msg_error:nneee { spintent } { unknown-choice }
1753     { read-sym } { por, ~multiplicado-por, ~veces } { \exp_not:n {#1} },
1754 not-print     .bool_set:N = \l__spintent_spmsym_not_print_bool,
1755 not-print     .initial:n = false,
1756 not-print     .value_required:n = true,
1757 unknown      .code:n =
1758     {
1759         \str_set:Nn \l__spintent_command_name_str { spmsym }
1760         \l__spintent_unknown_keys_cmd:n {#1}
1761     },
1762 }

```

(End of definition for `prefix` and others.)

\spmsym Defines the `\spmsym` operator. Similar to `\spdsym`, it outputs the correct mathematical glyph based on the local `<keys>`, safely attaching prefixes and suffixes if present.

```

1763 \NewDocumentCommand \spmsym { 0{} }
1764 {
1765     \mode_if_math:TF
1766     {
1767         \group_begin:
1768         \keys_set:nn { spintent / spmsym } { #1 }
1769         \MathMLintent
1770         {
1771             \tl_if_empty:NF \l__spintent_spmsym_prefix_tl
1772             { \l__spintent_spmsym_prefix_tl - }
1773             \l__spintent_spmsym_read_sym_tl
1774             \tl_if_empty:NF \l__spintent_spmsym_suffix_tl
1775             { - \l__spintent_spmsym_suffix_tl }
1776         }
1777         {
1778             \bool_if:NTF \l__spintent_spmsym_not_print_bool
1779             {
1780                 \invisibletimes
1781             }
1782             { \l__spintent_spmsym_symbol_tl }
1783         }
1784         \group_end:
1785     }
1786     { \msg_error:nnn { spintent } { need-math-mode } { \spmsym } }
1787 }

```

(End of definition for `\spmsym`. This function is documented on page 12.)

11.24 The command `\spdiv`

The user-level command `\spdiv` formats division representations. It takes two mandatory arguments: the `{<dividend>}` and the `{<divisor>}`. Both `{<arguments>}` are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the `{<dividend>}` nor the `{<divisor>}` is *blank*.

11.24.1 Definition of keys for `\spdiv`

We define the `<keys>` `wrap-parens` and `read-parens` for `\spdiv` command. Number-related `<keys>` `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator `<keys>` `set-sym` and `read-sym` are forwarded to the `\spdsym` command using `.meta:nn`.

```

1788 \keys_define:nn { spintent / spdiv }
1789 {
1790     cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1791     read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1792     read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1793     notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1794     set-sym         .meta:nn = { spintent / spdsym } { set-sym         = {#1} },
1795     read-sym        .meta:nn = { spintent / spdsym } { read-sym        = {#1} },
1796     wrap-parens     .bool_set:N = \l__spintent_spdiv_wrap_parens_bool,
1797     wrap-parens     .initial:n = false,

```

```

1798     wrap-parens      .value_required:n = true,
1799     read-parens      .bool_set:N = \l__spintent_spdiv_read_parens_bool,
1800     read-parens      .initial:n = true,
1801     read-parens      .value_required:n = true,
1802     unknown          .code:n =
1803     {
1804         \str_set:Nn \l__spintent_command_name_str { spdiv }
1805         \__spintent_unknown_keys_cmd:n {#1}
1806     },
1807 }

```

(End of definition for *cifras-sep* and others.)

11.24.2 Definition of error messages

`spdiv-empty-argument` Raised when either the $\{\langle\textit{dividend}\rangle\}$ or the $\{\langle\textit{divisor}\rangle\}$ is missing or *blank*.

```

1808 \msg_new:nnnn { spintent } { spdiv-empty-argument }
1809 { The~arguments~for~\token_to_str:N \spdiv~cannot~be~empty. }
1810 {
1811     You~must~provide~both~a~dividend~and~a~divisor.\\
1812 }

```

(End of definition for *spdiv-empty-argument*.)

11.24.3 Internal functions for `\spdiv`

`\__spintent_spdiv_render_op:nn` The function `\__spintent_spdiv_render_op:nn` parses both $\{\langle\textit{arguments}\rangle\}$ through the `\spnum` command and embeds the semantic operator from `\spdsym`.

```

1813 \cs_new_protected:Nn \__spintent_spdiv_render_op:nn
1814 {
1815     \spnum {#1}
1816     \MathMLintent { \l__spintent_spdsym_read_sym_tl }
1817     {
1818         \l__spintent_spdsym_symbol_tl
1819     }
1820     \spnum {#2}
1821 }

```

(End of definition for `\__spintent_spdiv_render_op:nn`.)

`\__spintent_spdiv_render_parens:nn` The function `\__spintent_spdiv_render_parens:nn` handles the structural parentheses around the division.

```

1822 \cs_new_protected:Nn \__spintent_spdiv_render_parens:nn
1823 {
1824     \bool_if:NTF \l__spintent_spdiv_read_parens_bool
1825     {
1826         ( \__spintent_spdiv_render_op:nn {#1} {#2} )
1827     }
1828     {
1829         \MathMLintent { :silent } { ( }
1830         \__spintent_spdiv_render_op:nn {#1} {#2}
1831         \MathMLintent { :silent } { ) }
1832     }
1833 }

```

(End of definition for `\__spintent_spdiv_render_parens:nn`.)

`\__spintent_spdiv_process:nnn` The function `\__spintent_spdiv_process:nnn` sets $\langle\textit{keys}\rangle$, validates $\{\langle\textit{arguments}\rangle\}$, and decides on parenthesis wrapping.

```

1834 \cs_new_protected:Nn \__spintent_spdiv_process:nnn
1835 {
1836     \keys_set:nn { spintent / spdiv } { #1 }
1837     \tl_if_blank:NTF { #2 }
1838     { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1839     {
1840         \tl_if_blank:NTF { #3 }
1841         { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1842         {
1843             \bool_if:NTF \l__spintent_spdiv_wrap_parens_bool
1844             { \__spintent_spdiv_render_parens:nn {#2} {#3} }
1845             { \__spintent_spdiv_render_op:nn {#2} {#3} }
1846         }
1847     }
1848 }

```

(End of definition for `\__spintent_spdiv_process:nnn`.)

`\spdiv` Defines the user command `\spdiv`. It strictly relies on *math mode*.

```

1849 \NewDocumentCommand \spdiv { O{} m m }
1850 {
1851   \mode_if_math:TF
1852   {
1853     \group_begin:
1854       \__spintent_spdiv_process:nnn { #1 } { #2 } { #3 }
1855     \group_end:
1856   }
1857   { \msg_error:nnn { spintent } { need-math-mode } { \spdiv } }
1858 }

```

(End of definition for `\spdiv`. This function is documented on page 13.)

11.25 The command `\spmul`

The user-level command `\spmul` formats multiplication representations. It takes two mandatory arguments: the `{\langle first factor \rangle}` and the `{\langle second factor \rangle}`. Both `{\langle arguments \rangle}` are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the `{\langle first factor \rangle}` nor the `{\langle second factor \rangle}` is *blank*.

11.25.1 Definition of keys for `\spmul`

We define the `\langle keys \rangle` wrap-parens and read-parens for `\spmul` command. Number-related `\langle keys \rangle` `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator `\langle keys \rangle` `set-sym` and `read-sym` are forwarded to the `\spmsym` command using `.meta:nn`.

```

1859 \keys_define:nn { spintent / spmul }
1860 {
1861   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1862   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1863   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1864   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1865   set-sym         .meta:nn = { spintent / spmsym } { set-sym         = {#1} },
1866   read-sym        .meta:nn = { spintent / spmsym } { read-sym        = {#1} },
1867   wrap-parens     .bool_set:N = \__spintent_spmul_wrap_parens_bool,
1868   wrap-parens     .initial:n  = false,
1869   wrap-parens     .value_required:n = true,
1870   read-parens     .bool_set:N = \__spintent_spmul_read_parens_bool,
1871   read-parens     .initial:n  = true,
1872   read-parens     .value_required:n = true,
1873   unknown         .code:n =
1874   {
1875     \str_set:Nn \__spintent_command_name_str { spmul }
1876     \__spintent_unknown_keys_cmd:n {#1}
1877   },
1878 }

```

(End of definition for `cifras-sep` and others.)

11.25.2 Definition of error messages

`spmul-empty-argument` Raised when either the `{\langle first factor \rangle}` or the `{\langle second factor \rangle}` is missing or *blank*.

```

1879 \msg_new:nnnn { spintent } { spmul-empty-argument }
1880 { The~arguments~for~\token_to_str:N \spmul~cannot~be~empty. }
1881 {
1882   You~must~provide~both~first~factor~and~second~factor.\
1883 }

```

(End of definition for `spmul-empty-argument`.)

11.25.3 Internal functions for `\spmul`

`\__spintent_spmul_render_op:nn` The function `\__spintent_spmul_render_op:nn` parses both `{\langle arguments \rangle}` through the `\spnum` command and embeds the semantic operator from `\spmsym`.

```

1884 \cs_new_protected:Nn \__spintent_spmul_render_op:nn
1885 {
1886   \spnum {#1}
1887   \MathMLintent { \__spintent_spmsym_read_sym_tl }
1888   {
1889     \__spintent_spmsym_symbol_tl
1890   }
1891   \spnum {#2}
1892 }

```

(End of definition for `\__spintent_spmul_render_op:nn`.)

`\__spintent_spmul_render_parens:nn`

The function `\__spintent_spmul_render_parens:nn` handles the structural parentheses around the multiplication.

```

1893 \cs_new_protected:Nn \__spintent_spmul_render_parens:nn
1894 {
1895   \bool_if:NTF \__spintent_spmul_read_parens_bool
1896   {
1897     ( \__spintent_spmul_render_op:nn {#1} {#2} )
1898   }
1899   {
1900     \MathMLintent { :silent } { ( }
1901     \__spintent_spmul_render_op:nn {#1} {#2}
1902     \MathMLintent { :silent } { ) }
1903   }
1904 }
```

(End of definition for `\__spintent_spmul_render_parens:nn`.)

`\__spintent_spmul_process:nnn`

The function `\__spintent_spmul_process:nnn` sets `\keys`, validates `\{arguments\}`, and decides on parenthesis wrapping.

```

1905 \cs_new_protected:Nn \__spintent_spmul_process:nnn
1906 {
1907   \keys_set:nn { spintent / spmul } { #1 }
1908   \tl_if_blank:nTF { #2 }
1909   { \msg_error:nnn { spintent } { spmul-empty-argument } }
1910   {
1911     \tl_if_blank:nTF { #3 }
1912     { \msg_error:nnn { spintent } { spmul-empty-argument } }
1913     {
1914       \bool_if:NTF \__spintent_spmul_wrap_parens_bool
1915       { \__spintent_spmul_render_parens:nn {#2} {#3} }
1916       { \__spintent_spmul_render_op:nn {#2} {#3} }
1917     }
1918   }
1919 }
```

(End of definition for `\__spintent_spmul_process:nnn`.)

`\spmul`

Defines the user command `\spmul`. It strictly relies on *math mode*.

```

1920 \NewDocumentCommand \spmul { O{} m m }
1921 {
1922   \mode_if_math:TF
1923   {
1924     \group_begin:
1925     \__spintent_spmul_process:nnn { #1 } { #2 } { #3 }
1926     \group_end:
1927   }
1928   { \msg_error:nnn { spintent } { need-math-mode } { \spmul } }
1929 }
```

(End of definition for `\spmul`. This function is documented on page 13.)

11.26 The commands `\spmcd` and `\spmcm`

The user-level commands `\spmcd` and `\spmcm` provide a semantic interface for the Greatest Common Divisor (MCD in Spanish) and the Least Common Multiple (MCM in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.26.1 Variables for `\spmcd` and `\spmcm`

Internal variables generated dynamically via a comma-separated list to avoid code duplication for both symmetric commands.

`\__spintent_spmcm_spmcd_luaset_error_str`
`\__spintent_mc_luaset_types_str`
`\__spintent_mc_luaset_all_numeric_str`
`\__spintent_mc_args_seq`
`\__spintent_mc_raw_args_seq`
`\__spintent_spmcd_print_result_bool`
`\__spintent_spmcm_print_result_bool`
`\__spintent_spmcd_read_terse_bool`
`\__spintent_spmcm_read_terse_bool`
`\__spintent_spmcd_print_upper_bool`
`\__spintent_spmcm_print_upper_bool`
`\__spintent_spmcd_prefix_tl`
`\__spintent_spmcm_prefix_tl`

```

1930 \str_new:N \__spintent_spmcm_spmcd_luaset_error_str
1931 \clist_new:N \__spintent_mc_luaset_types_clist
1932 \str_new:N \__spintent_mc_luaset_all_numeric_str
1933 \seq_new:N \__spintent_mc_args_seq
1934 \seq_new:N \__spintent_mc_raw_args_seq
1935 \seq_new:N \__spintent_mc_types_seq
1936 \int_new:N \__spintent_mc_idx_int
1937 \clist_map_inline:nn { mcd , mcm }
1938 {
1939   \tl_new:c { \__spintent_sp #1 _luaset_ #1 _value_tl }
```

```

1940     \bool_new:c { \__spintent_sp #1 _read_terse_bool }
1941     \tl_new:c { \__spintent_sp #1 _prefix_tl }
1942 }

```

(End of definition for __spintent_spmcm_spmcd_luaset_error_str and others.)

11.26.2 Definition of error messages

mc-invalid-arguments

Exception triggered when the comma-separated sequence contains negative integers, floats, or alphabetical characters.

```

1943 \msg_new:nnnn { spintent } { mc-invalid-arguments }
1944 {
1945     Invalid~arguments~provided~to~#1.
1946 }
1947 {
1948     The~arguments~must~be~a~comma-separated~list. \\\
1949 }

```

(End of definition for mc-invalid-arguments.)

11.26.3 Keys for \spmcd and \spmcm

result
prefix
read-cmd
upper-case
concept
silent-cmd
unknown

We define the shared *⟨keys⟩* result, prefix, read-cmd, upper-case, concept and silent-cmd.

```

1950 \clist_map_inline:nn { mcd , mcm }
1951 {
1952     \keys_define:nn { spintent / sp #1 }
1953     {
1954         result      .bool_set:c = { \__spintent_sp #1 _print_result_bool },
1955         result      .initial:n = false,
1956         result      .value_required:n = true,
1957         prefix      .code:n =
1958             { \__spintent_sanitise_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
1959         prefix      .initial:n = ,
1960         prefix      .value_required:n = true,
1961         read-cmd    .choices:nn =
1962             { terse , clear }
1963             {
1964                 \int_case:nn { \l_keys_choice_int }
1965                 {
1966                     {1} { \bool_set_true:c { \__spintent_sp #1 _read_terse_bool } }
1967                     {2} { \bool_set_false:c { \__spintent_sp #1 _read_terse_bool } }
1968                 }
1969             },
1970         read-cmd    .initial:n = clear,
1971         read-cmd    .value_required:n = true,
1972         read-cmd / unknown .code:n =
1973             \msg_error:nneee { spintent } { unknown-choice }
1974             { read-cmd } { terse , ~clear } { \exp_not:n {##1} },
1975         upper-case  .bool_set:c = { \__spintent_sp #1 _print_upper_bool },
1976         upper-case  .initial:n = false,
1977         upper-case  .value_required:n = true,
1978         concept     .bool_set:c = { \__spintent_sp #1 _concept_bool },
1979         concept     .initial:n = true,
1980         concept     .value_required:n = true,
1981         silent-cmd  .bool_set:c = { \__spintent_sp #1 _silent_cmd_bool },
1982         silent-cmd  .initial:n = false,
1983         silent-cmd  .value_required:n = true,
1984         unknown     .code:n =
1985             {
1986                 \str_set:Nn \l__spintent_command_name_str { sp #1 }
1987                 \__spintent_unknown_keys_cmd:n {##1}
1988             },
1989     }
1990 }

```

(End of definition for result and others.)

11.26.4 Internal functions for \spmcd and \spmcm

__spintent_mc_process_args:nn

The function `\__spintent_mc_process_args:nn` parses the operation ‘#1’ and its raw argument list ‘#2’. Calls `\__spintent_luafun_mc_classify_args:n`, splits ‘#2’ into `\__spintent_mc_raw_args_seq` and `\__spintent_mc_luaset_types_clist` into `\__spintent_mc_types_seq`, then walks the former by position, fetching the matching type from the latter through `\seq_item:Nn`, to build `\__spintent_mc_args_seq`:

each argument wrapped through `\spnum` when its type is `number`, verbatim otherwise. Injects MathML intents around the already-formatted `\l__spintenc_mc_args_seq`.

```

1991 \cs_new_protected:Nn \__spintenc_mc_process_args:nn
1992 {
1993   \__spintenc_luafun_mc_classify_args:n {#2}
1994   \seq_set_from_clist:Nn \l__spintenc_mc_raw_args_seq {#2}
1995   \seq_set_from_clist:NN \l__spintenc_mc_types_seq \l__spintenc_mc_luaaset_types_clist
1996   \seq_clear:N \l__spintenc_mc_args_seq
1997   \int_zero:N \l__spintenc_mc_idx_int
1998   \seq_map_inline:Nn \l__spintenc_mc_raw_args_seq
1999   {
2000     \int_incr:N \l__spintenc_mc_idx_int
2001     \str_if_eq:eeTF { \seq_item:Nn \l__spintenc_mc_types_seq { \l__spintenc_mc_idx_int } } {
2002       { \seq_put_right:Nn \l__spintenc_mc_args_seq { \spnum {##1} } }
2003       { \seq_put_right:Nn \l__spintenc_mc_args_seq {##1} }
2004     }
2005   }

```

Injects MathML intents around the already-formatted `\l__spintenc_mc_args_seq`, exactly as before.

```

2005   \bool_if:cTF { l__spintenc_sp #1 _silent_cmd_bool }
2006   {
2007     \seq_set_map:Nnn \l__spintenc_mc_args_seq \l__spintenc_mc_args_seq
2008     { \MathMLintent { :silent } { ##1 } }
2009     \MathMLintent { _entre } { \c__spintenc_invisible_sep_int }
2010     \MathMLintent { :silent } { ( }
2011     \seq_use:Nnnn \l__spintenc_mc_args_seq
2012     { \MathMLintent { _y:silent } { , } }
2013     { \MathMLintent { :silent } { , } }
2014     { \MathMLintent { _y:silent } { , } }
2015   }
2016   {
2017     \MathMLintent { _entre } { \c__spintenc_invisible_sep_int }
2018     \MathMLintent { :silent } { ( }
2019     \seq_use:Nnnn \l__spintenc_mc_args_seq
2020     { \MathMLintent { _y } { , } }
2021     { \MathMLintent { :silent } { , } }
2022     { \MathMLintent { _y } { , } }
2023   }
2024   \MathMLintent { :silent } { ) }
2025 }

```

(End of definition for `\__spintenc_mc_process_args:nn`.)

`\__spintenc_spmc_process:nnnn`

The internal function `\__spintenc_spmc_process:nnnn` is the shared handler for both `\spcmd` and `\spmcn`. Takes the short identifier in ‘#1’, its full reading string in ‘#2’, the local `(keys)` in ‘#3’, and the optional arguments in ‘#4’.

The operator name (through `\operatorname`, per upper-case) is silenced with `silent-cmd=true` — `\__spintenc_mc_process_args:nn` silences the arguments too, in that case — or with `concept=false` alone, leaving the arguments untouched. With `concept=true`, it carries the literal `intent` instead — #1 or #2 per `read-cmd`, prefixed with `prefix` when given.

With ‘#4’ given, classifies it through `\__spintenc_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting `mc-invalid-arguments` on failure, and, when `result` is `true`, printing the computed value. `\__spintenc_mc_process_args:n` prints the argument list itself either way.

```

2026 \cs_new_protected:Nn \__spintenc_spmc_process:nnnn
2027 {
2028   \keys_set:nn { spintenc / sp #1 } { #3 }
2029   \bool_if:cTF { l__spintenc_sp #1 _silent_cmd_bool }
2030   {
2031     \MathMLintent { :silent }
2032     {
2033       \operatorname
2034       {
2035         \bool_if:cTF { l__spintenc_sp #1 _print_upper_bool }
2036         { \text_uppercase:n {##1} } { #1 }
2037       }
2038     }
2039   }
2040   {
2041     \bool_if:cTF { l__spintenc_sp #1 _concept_bool }
2042     {

```

```

2043 \MathMLintent
2044 {
2045   \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
2046   {
2047     \tl_use:c { \__spintent_sp #1 _prefix_tl } -
2048   }
2049   \bool_if:cTF { \__spintent_sp #1 _read_terse_bool } { #1 :nofix } { #2 :nofix }
2050 }
2051 {
2052   \operatorname
2053   {
2054     \bool_if:cTF { \__spintent_sp #1 _print_upper_bool }
2055     { \text_uppercase:n {#1} } { #1 }
2056   }
2057 }
2058 }
2059 {
2060   \MathMLintent { :silent }
2061   {
2062     \operatorname
2063     {
2064       \bool_if:cTF { \__spintent_sp #1 _print_upper_bool }
2065       { \text_uppercase:n {#1} } { #1 }
2066     }
2067   }
2068 }
2069 }

```

With ‘#4’ given, classifies it through `\__spintent_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting mc-invalid-arguments on failure — which should not happen, since every argument already passed classification — and, when `result` is `true`, printing the computed value. With any non-numeric argument, no calculation is attempted at all. Either way, `\__spintent_mc_process_args:n` prints the argument list itself.

```

2070 \tl_if_novalue:nF { #4 }
2071 {
2072   \__spintent_luafun_mc_classify_args:n {#4}
2073   \str_if_eq:VnT \__spintent_mc_luaset_all_numeric_str { true }
2074   {
2075     \use:c { __spintent_luafun_calculate_ #1 :n } { #4 }
2076     \str_if_eq:VnT \__spintent_spmcm_spmcd_luaset_error_str { true }
2077     {
2078       \msg_error:nne { spintent } { mc-invalid-arguments } { \exp_not:c { sp #1 } }
2079     }
2080   }
2081   \__spintent_mc_process_args:nn {#1} {#4}
2082   \str_if_eq:VnT \__spintent_mc_luaset_all_numeric_str { true }
2083   {
2084     \bool_if:cT { \__spintent_sp #1 _print_result_bool }
2085     {
2086       \MathMLintent { es-igual-a } { = }
2087       \tl_use:c { \__spintent_sp #1 _luaset_ #1 _value_tl }
2088     }
2089   }
2090 }
2091 }

```

(End of definition for `\__spintent_spmc_process:nnnn`.)

`\spmc` The commands `\spmc` and `\spmc` act as the public interface wrappers for typesetting the greatest common divisor and least common multiple. They enforce math mode, open a local group, and defer execution to the main processing macro `\__spintent_spmc_process:nnnn`, passing along the appropriate reading strings and arguments.

```

2092 \NewDocumentCommand \spmc { 0 } d() {
2093   {
2094     \mode_if_math:TF
2095     {
2096       \group_begin:
2097       \__spintent_spmc_process:nnnn { mcd } { máximo-común-divisor } {#1} {#2}
2098       \group_end:
2099     }
2100     { \msg_error:nnn { spintent } { need-math-mode } { \spmc } }

```

```

2101 }
2102 \NewDocumentCommand \spmc { 0{} d() }
2103 {
2104   \mode_if_math:TF
2105   {
2106     \group_begin:
2107     \__spinttent_spmc_process:nnnn { mcm } { mínimo-común-múltiplo } {#1} {#2}
2108     \group_end:
2109   }
2110   { \msg_error:nnn { spinttent } { need-math-mode } { \spmc } }
2111 }

```

(End of definition for \spmc and \spmc. These functions are documented on page 15.)

11.27 The commands \spnM and \spnD

The user-level commands `\spnM` and `\spnD` provide a semantic interface for the Multiples of a number (“*múltiplos*” in Spanish) and the Divisors of a number (“*divisores*” in Spanish). They delegate argument validation and calculation to the Lua module `spinttent.lua`, constructing a valid MathCAT intent.

11.27.1 Definition of variables for \spnM and \spnD

Internal variables generated dynamically for multiples and divisors.

```

\__spinttent_md_out_seq
\__spinttent_spmmd_luaset_is_number_str
\__spinttent_spm_luaset_result_tl
\__spinttent_spnD_luaset_result_tl
\__spinttent_spm_luaset_is_truncated_str
\__spinttent_spnD_luaset_is_truncated_str
\__spinttent_spm_prefix_tl
\__spinttent_spnD_prefix_tl
\__spinttent_spm_print_result_num_tl
\__spinttent_spnD_print_result_num_tl
2112 \seq_new:N \__spinttent_md_out_seq
2113 \str_new:N \__spinttent_spmmd_luaset_is_number_str
2114 \clist_map_inline:nn { nM , nD }
2115 {
2116   \tl_new:c { \__spinttent_sp #1_luaset_result_tl }
2117   \str_new:c { \__spinttent_sp #1_luaset_is_truncated_str }
2118   \tl_new:c { \__spinttent_sp #1_prefix_tl }
2119 }

```

(End of definition for __spinttent_md_out_seq and others.)

11.27.2 Keys for \spnM and \spnD

We define the shared `<keys>` `result`, `silent-cmd`, `result-num`, `prefix` and `concept`.

```

result
silent-cmd
result-num
prefix
concept
unknown
2120 \clist_map_inline:nn { nM , nD }
2121 {
2122   \keys_define:nn { spinttent / sp #1 }
2123   {
2124     result .bool_set:c = { \__spinttent_sp #1_print_result_bool },
2125     result .initial:n = false,
2126     result .value_required:n = true,
2127     silent-cmd .bool_set:c = { \__spinttent_sp #1_silent_cmd_bool },
2128     silent-cmd .initial:n = false,
2129     silent-cmd .value_required:n = true,
2130     result-num .tl_set:c = { \__spinttent_sp #1_print_result_num_tl },
2131     result-num .initial:n = ,
2132     prefix .code:n =
2133     { \__spinttent_sanitize_affix:cn { \__spinttent_sp #1_prefix_tl } {##1} },
2134     prefix .initial:n = ,
2135     prefix .value_required:n = true,
2136     concept .bool_set:c = { \__spinttent_sp #1_concept_bool },
2137     concept .initial:n = true,
2138     concept .value_required:n = true,
2139     unknown .code:n =
2140     {
2141       \str_set:Nn \__spinttent_command_name_str { sp #1 }
2142       \__spinttent_unknown_keys_cmd:n {##1}
2143     },
2144   }
2145 }

```

(End of definition for result and others.)

11.27.3 Definition of error messages for \spnM and \spnD

`md-invalid-argument` Exception triggered when the argument for multiples or divisors is invalid.

```

2146 \msg_new:nnnn { spinttent } { md-invalid-argument }
2147 { Invalid~argument~provided~to~#1. }
2148 {
2149   The~argument~must~be~a~single~positive~integer. \
2150 }

```

(End of definition for md-invalid-argument.)

11.27.4 Internal functions for \spnM and \spnD

__spintenc_spnM_spnD_process:nnnn

The internal function `\__spintenc_spnM_spnD_process:nnnn` acts as the primary entry point for the multiples and divisors commands. It takes the command identifier ‘#1’, its printed mathematical symbol ‘#2’, its base reading string ‘#3’, the local key-value options ‘#4’, and the optional argument ‘#5’. Its sole purpose is to evaluate and set the provided keys within the current scope before delegating the core branching and typesetting logic to `\__spintenc_spnM_and_D_exec:nnnn`.

```
2151 \cs_new_protected:Nn \__spintenc_spnM_spnD_process:nnnnn
2152 {
2153   \keys_set:nn { spintenc / sp #1 } { #4 }
2154   \__spintenc_spnM_spnD_exec:nnnn { #1 } { #2 } { #3 } { #5 }
2155 }
```

(End of definition for __spintenc_spnM_spnD_process:nnnnn.)

__spintenc_spnM_spnD_intent:nn

The internal function `\__spintenc_spnM_spnD_intent:nn` resolves the main reading intent string. It takes the command identifier ‘#1’ (such as “nM” or “nD”) and the base reading string ‘#2’. It automatically prepends the localized prefix if defined, normalizes accentuation for “*múltiplos*”, and appends ‘-de’.

```
2156 \cs_new:Nn \__spintenc_spnM_spnD_intent:nn
2157 {
2158   \tl_if_empty:cF { \__spintenc_sp #1 _prefix_tl }
2159   { \tl_use:c { \__spintenc_sp #1 _prefix_tl } - }
2160   \str_case:nnF { #2 }
2161   { { múltiplos } { múltiplos } }
2162   { #2 }
2163   -de
2164 }
```

(End of definition for __spintenc_spnM_spnD_intent:nn.)

__spintenc_spnM_spnD_silent:nn

The internal function `\__spintenc_spnM_and_D_silent:nn` prints the atomized notation structure in full silent mode. It takes the mathematical symbol ‘#1’ (such as “D” or “M”) and the variable or explicit argument ‘#2’. Every component is wrapped in a silent intent, and an invisible separator is injected between the parentheses to prevent assistive technologies from applying default contextual reading rules.

```
2165 \cs_new_protected:Nn \__spintenc_spnM_spnD_silent:nn
2166 {
2167   \MathMLintent { :silent } { \operatorname { #1 } }
2168   \MathMLintent { :silent } { ( }
2169   \MathMLintent { :silent } { \c__spintenc_invisible_sep_int }
2170   \MathMLintent { :silent } { #2 }
2171   \MathMLintent { :silent } { ) }
2172 }
```

(End of definition for __spintenc_spnM_spnD_silent:nn.)

__spintenc_spnM_spnD_result:n

The internal function `\__spintenc_spnM_and_D_result:n` handles the calculation and accessible formatting of the computed mathematical set. It takes the command identifier ‘#1’. It invokes the corresponding Lua backend calculation, maps the resulting clist into a sequence, and formats the final printed output utilizing custom accessibility intents for both separators and conjunctions. It also safely manages truncation indicators.

```
2173 \cs_new_protected:Nn \__spintenc_spnM_spnD_result:n
2174 {
2175   \use:c { __spintenc_luafun_calcular_ #1 :nn }
2176   { \__spintenc_luaset_part_int_tl }
2177   { \tl_use:c { \__spintenc_sp #1 _print_result_num_tl } }
2178   \MathMLintent { _son } { = \{ }
2179   \seq_set_from_clist:Ne
2180   \__spintenc_md_out_seq { \tl_use:c { \__spintenc_sp #1 _luaset_result_tl } }
2181   \seq_use:Nnnn \__spintenc_md_out_seq
2182   { \MathMLintent { _y } { , } }
2183   { \MathMLintent { :silent } { , } }
2184   { \MathMLintent { _y } { , } }
2185   \str_if_eq:vnT
2186   { \__spintenc_sp #1 _luaset_is_truncated_str } { true }
2187   {
2188     \MathMLintent { :silent } { , } ~ \dots
2189   }
2190   \MathMLintent { :silent } { \} }
2191 }
```

(End of definition for __spintenc_spnM_spnD_result:n.)

`\__spintent_spnM_spnD_exec:nnnn`

The function `\__spintent_spnM_spnD_exec:nnnn` takes the command name in ‘#1’, the symbol in ‘#2’, the base reading string in ‘#3’, and the optional argument in ‘#4’. The parentheses are always silenced.

With ‘#4’ absent, the placeholder `n` stands in for the argument. With ‘#4’ given, `\__spintent_luafun_spnmd_classify` decides whether it is wrapped through `\spnum` — and, when it is, whether `\__spintent_spnM_spnD_result:n` also prints the computed set, per `result` — or left verbatim, with no result attempted.

Either way, `silent-cmd` silences the whole thing through `\__spintent_spnM_spnD_silent:nn`; otherwise `concept true` carries the literal `intent` from `\__spintent_spnM_spnD_intent:nn`, `false` silences only the operator symbol, leaving the argument to read normally.

```

2192 \cs_new_protected:Nn \__spintent_spnM_spnD_exec:nnnn
2193 {
2194   \tl_if_novalue:nTF { #4 }
2195   {
2196     \bool_if:cTF { \__spintent_sp #1 _silent_cmd_bool }
2197     { \__spintent_spnM_spnD_silent:nn { #2 } { n } }
2198     {
2199       \bool_if:cTF { \__spintent_sp #1 _concept_bool }
2200       {
2201         \MathMLintent
2202         { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
2203         { \operatorname { #2 } }
2204       }
2205       { \MathMLintent { :silent } { \operatorname { #2 } } }
2206       \MathMLintent { :silent } { ( }
2207       n
2208       \MathMLintent { :silent } { ) }
2209     }
2210   }
2211   {
2212     \__spintent_luafun_spnmd_classify:n { \exp_not:n { #4 } }
2213     \bool_if:cTF { \__spintent_sp #1 _silent_cmd_bool }
2214     {
2215       \str_if_eq:VnTF \__spintent_spnmd_luaset_is_number_str { true }
2216       { \__spintent_spnM_spnD_silent:nn { #2 } { \spnum { #4 } } }
2217       { \__spintent_spnM_spnD_silent:nn { #2 } { #4 } }
2218     }
2219     {
2220       \bool_if:cTF { \__spintent_sp #1 _concept_bool }
2221       {
2222         \MathMLintent
2223         { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
2224         { \operatorname { #2 } }
2225       }
2226       { \MathMLintent { :silent } { \operatorname { #2 } } }
2227       \MathMLintent { :silent } { ( }
2228       \str_if_eq:VnTF \__spintent_spnmd_luaset_is_number_str { true }
2229       { \spnum { #4 } }
2230       { #4 }
2231       \MathMLintent { :silent } { ) }
2232     }
2233     \str_if_eq:VnTF \__spintent_spnmd_luaset_is_number_str { true }
2234     {
2235       \bool_if:cT { \__spintent_sp #1 _print_result_bool }
2236       { \__spintent_spnM_spnD_result:n { #1 } }
2237     }
2238   }
2239 }

```

(End of definition for `\__spintent_spnM_spnD_exec:nnnn`.)

`\spnM` The commands `\spnM` and `\spnD` act as the public interface wrappers for typesetting multiples and divisors, respectively. They enforce math mode, open a local group, and defer execution to the function `\__spintent_spnM_spnD_process:nnnnn`, passing along the optional *(keys)* ‘#1’ and the numeric *{(argument)}* ‘#2’.

```

2240 \NewDocumentCommand \spnM { 0{} d() }
2241 {
2242   \mode_if_math:TF
2243   {
2244     \group_begin:
2245     \__spintent_spnM_spnD_process:nnnnn { nM } { M } { multiplos } { #1 } { #2 }
2246     \group_end:

```

```

2247     }
2248     { \msg_error:nnn { spintent } { need-math-mode } { \spnM } }
2249   }
2250   \NewDocumentCommand \spnD { 0{} d() }
2251   {
2252     \mode_if_math:TF
2253     {
2254       \group_begin:
2255         \__spintent_spnM_spnD_process:nnnnn { nD } { D } { divisores } { #1 } { #2 }
2256       \group_end:
2257     }
2258     { \msg_error:nnn { spintent } { need-math-mode } { \spnD } }
2259   }

```

(End of definition for `\spnM` and `\spnD`. These functions are documented on page 14.)

11.28 The command `\sprazon`

The user command `\sprazon` typesets a ratio $a:b$ as a fraction (`\dfrac` or `\frac`, per `use-dfrac`), read natively — no Lua parsing involved.

11.28.1 Definition of keys for `\sprazon`

We define the *(keys)* `silent-cmd` and `use-dfrac`.

```

silent-cmd 2260 \keys_define:nn { spintent / sprazon }
use-dfrac   2261 {
unknown     2262   silent-cmd .bool_set:N = \__spintent_sprazon_silent_cmd_bool,
            2263   silent-cmd .initial:n = false,
            2264   silent-cmd .value_required:n = true,
            2265   use-dfrac .bool_set:N = \__spintent_sprazon_use_dfrac_bool,
            2266   use-dfrac .initial:n = false,
            2267   use-dfrac .value_required:n = true,
            2268   unknown .code:n =
            2269   {
            2270     \str_set:Nn \__spintent_command_name_str { sprazon }
            2271     \__spintent_unknown_keys_cmd:n { #1 }
            2272   },
            2273 }

```

(End of definition for `silent-cmd`, `use-dfrac`, and `unknown`.)

`\__spintent_sprazon_exec:nnn` The function `\__spintent_sprazon_exec:nnn` receives the optional *(keys)* in ‘#1’, the numerator in ‘#2’ and the denominator in ‘#3’. Under `silent-cmd`, the whole fraction is silenced as one unit. Otherwise, numerator and denominator are each tagged through a native `\MathMLarg` (num/ denom), and the outer `\MathMLintent` reads `es-a:infix($num,$denom)`.

```

2274 \cs_new_protected:Nn \__spintent_sprazon_exec:nnn
2275 {
2276   \keys_set:nn { spintent / sprazon } { #1 }
2277   \__spintent_invisible_sep:
2278   \bool_if:NTF \__spintent_sprazon_silent_cmd_bool
2279   {
2280     \MathMLintent { _- }
2281     {
2282       \__spintent_inner_sep:
2283       \bool_if:NTF \__spintent_sprazon_use_dfrac_bool
2284       { \dfrac { #2 } { #3 } }
2285       { \frac { #2 } { #3 } }
2286     }
2287   }
2288   {
2289     \MathMLintent { es-a:infix($num,$denom) }
2290     {
2291       \__spintent_inner_sep:
2292       \bool_if:NTF \__spintent_sprazon_use_dfrac_bool
2293       {
2294         \dfrac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } }
2295       }
2296       { \frac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } } }
2297     }
2298   }
2299 }

```

(End of definition for `\__spintent_sprazon_exec:nnn`.)

`\sprazon` The public command `\sprazon` enforces *math mode* and delegates execution to `\__spintent_sprazon_exec:nnn`.

```

2300 \NewDocumentCommand \sprazon { 0{} m m }
2301 {
2302   \mode_if_math:TF
2303   {
2304     \group_begin:
2305       \__spintent_sprazon_exec:nnn { #1 } { #2 } { #3 }
2306     \group_end:
2307   }
2308   { \msg_error:nnn { spintent } { need-math-mode } { \sprazon } }
2309 }

```

(End of definition for `\sprazon`. This function is documented on page ??.)

11.29 The command `\spProp`

The user command `\spProp` typesets a proportion $a:b = c:d$ as two fractions joined by $=$, reusing `\__spintent_sprazon_exec:nnn` directly (not the public `\sprazon` command) for each ratio.

`\spProp` The public command `\spProp` enforces *math mode*, and forwards the same *keys* in ‘#1’ to both ratios: a/b (‘#2’/‘#3’) and c/d (‘#4’/‘#5’), joined by a native `\MathMLintent` reading `_como` over the $=$ sign.

```

2310 \NewDocumentCommand \spProp { 0{} m m m m }
2311 {
2312   \mode_if_math:TF
2313   {
2314     \group_begin:
2315       \__spintent_sprazon_exec:nnn { #1 } { #2 } { #3 }
2316       \bool_if:NTF \l__spintent_sprazon_silent_cmd_bool
2317       {
2318         \MathMLintent { :silent } { = }
2319       }
2320       { \MathMLintent { _como } { = } }
2321       \__spintent_sprazon_exec:nnn { #1 } { #4 } { #5 }
2322     \group_end:
2323   }
2324   { \msg_error:nnn { spintent } { need-math-mode } { \spProp } }
2325 }

```

(End of definition for `\spProp`. This function is documented on page ??.)

11.30 The command `\spang`

The command `\spang` is the main public command for typesetting and tagging sexagesimal angle expressions (degrees, minutes, and seconds). It parses the input using the Lua function `\__spintent_luafun_spang_parse:n`. This separates the degrees, minutes, and seconds to construct a `MathML intent` tree with the correct postfixes for `NVDA/MathCAT`.

```

\l__spintent_spang_luaset_grado_str
\l__spintent_spang_luaset_minuto_str
\l__spintent_spang_luaset_segundo_str
\l__spintent_spang_luaset_error_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted numerical values for degrees, minutes, and seconds, as well as the error status used during formatting.

```

2326 \str_new:N \l__spintent_spang_luaset_grado_str
2327 \str_new:N \l__spintent_spang_luaset_minuto_str
2328 \str_new:N \l__spintent_spang_luaset_segundo_str
2329 \str_new:N \l__spintent_spang_luaset_error_str

```

(End of definition for `\l__spintent_spang_luaset_grado_str` and others.)

11.30.1 Definition of error messages for `\spang`

`invalid-angle-format`

The error message `invalid-angle-format` defines the text displayed when an invalid angle input is detected. It is triggered when an argument fails the format checks, does not match the expected sexagesimal pattern, or contains invalid measurement symbols.

```

2330 \msg_new:nnnn { spintent } { invalid-angle-format }
2331 { Invalid~angle~format~in~#1. }
2332 {
2333   The~argument~'#2'~could~not~be~parsed. \
2334 }

```

(End of definition for `invalid-angle-format`.)

11.30.2 Internal functions for `\spang`

`__spintent_spang_process:n`

The internal function `__spintent_spang_process:n` parses the angle input using `__spintent_luafun_spang_parse:n` after resetting the error flag in `__spintent_spang_luaaset_error_str`. If an invalid format is detected, it issues the `invalid-angle-format` error. Otherwise, it checks the extracted string variables `__spintent_spang_luaaset_grado_str`, `__spintent_spang_luaaset_minuto_str`, and `__spintent_spang_luaaset_segundo_str`. For each non-empty variable, it constructs the corresponding MathML `intent` structure with the correct measurement symbols and postfixes for NVDA/MathCAT.

```

2335 \cs_new_protected:Nn __spintent_spang_process:n
2336 {
2337   \str_set:Nn __spintent_spang_luaaset_error_str { false }
2338   __spintent_luafun_spang_parse:n { #1 }
2339   \str_if_eq:NnTF __spintent_spang_luaaset_error_str { true }
2340   {
2341     \msg_error:nnnn { spintent } { invalid-angle-format } { \spang } { #1 }
2342   }
2343   {
2344     \str_if_empty:NF __spintent_spang_luaaset_grado_str
2345     {
2346       \spunit{ __spintent_spang_luaaset_grado_str °}
2347     }
2348     \str_if_empty:NF __spintent_spang_luaaset_minuto_str
2349     {
2350       \MathMLintent { minutos : postfix($m) }
2351       {
2352         \MathMLLarg { m }
2353         {
2354           \spunit{ __spintent_spang_luaaset_minuto_str ' }
2355         }
2356       }
2357     }
2358     \str_if_empty:NF __spintent_spang_luaaset_segundo_str
2359     {
2360       \MathMLintent { segundos : postfix($s) }
2361       {
2362         \MathMLLarg { s }
2363         {
2364           \spunit { __spintent_spang_luaaset_segundo_str " }
2365         }
2366       }
2367     }
2368   }
2369 }

```

(End of definition for `__spintent_spang_process:n`.)

`\spang`

The command `\spang` is the public interface for formatting sexagesimal angle expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `__spintent_spang_process:n` to process the `{(argument)}`.

```

2370 \NewDocumentCommand \spang { m }
2371 {
2372   \mode_if_math:TF
2373   {
2374     \group_begin:
2375     __spintent_spang_process:n { #1 }
2376     \group_end:
2377   }
2378   { \msg_error:nnn { spintent } { need-math-mode } { \spang } }
2379 }

```

(End of definition for `\spang`. This function is documented on page 23.)

11.31 The command `\spnZ`

The user-level command `\spnZ` formats integers (the set $\mathbb{Z}$). It includes specific options to automatically enclose numbers (particularly negative ones) in visual parentheses and allows you to control whether these parentheses are read by MathCAT.

This command verifies that the `{(argument)}` is indeed an *integer*, checking for the complete absence of decimal separators or attached units of measurement.

11.31.1 Definition of keys for \spnZ

Now we define the $\langle \text{keys} \rangle$ `cifras-sep` and `read-sign`. These are `.meta:nn` $\langle \text{keys} \rangle$ passed directly to the `\spnum` command. We also introduce `wrap-parens`, which determines whether the number is enclosed in parentheses, and `read-parens`, which controls if these parentheses are read by MathCAT.

```

2380 \keys_define:nn { spintent / spnZ }
2381 {
2382   cifras-sep .meta:nn = { spintent / spnum } { cifras-sep = {#1} },
2383   read-sign .meta:nn = { spintent / spnum } { read-sign = {#1} },
2384   wrap-parens .bool_set:N = \l__spintent_spnz_wrap_parens_bool,
2385   wrap-parens .initial:n = false,
2386   wrap-parens .value_required:n = true,
2387   read-parens .bool_set:N = \l__spintent_spnz_read_parens_bool,
2388   read-parens .initial:n = true,
2389   read-parens .value_required:n = true,
2390   unknown .code:n =
2391     {
2392       \str_set:Nn \l__spintent_command_name_str { spnZ }
2393       \__spintent_unknown_keys_cmd:n {#1}
2394     },
2395 }
```

(End of definition for `cifras-sep` and others.)

11.31.2 Definition of error messages

`spnz-not-integer` Raised when the $\{ \langle \text{argument} \rangle \}$ contains decimal separators or attached units, which conflict with the constraints of a pure integer representation.

```

2396 \msg_new:nnnn { spintent } { spnz-not-integer }
2397 { The~value~'~\the~#1~'~is~not~a~valid~integer. }
2398 {
2399   Command~\token_to_str:N \spnZ~only~accepts~integers.\
2400 }
```

(End of definition for `spnz-not-integer`.)

11.31.3 Internal functions for \spnZ

`\__spintent_spnz_render_parens:` The function `\__spintent_spnz_render_parens:` first checks the state of the boolean variable `\l__spintent_spnz_read_parens` (controlled by the `read-parens` key). If it is `true`, it prints the number surrounded by parentheses. Otherwise, it applies the `:silent function intent` to the parentheses while printing the number.

```

2401 \cs_new_protected:Nn \__spintent_spnz_render_parens:
2402 {
2403   \bool_if:NTF \l__spintent_spnz_read_parens_bool
2404   {
2405     ( \__spintent_spnum_print_num_start: )
2406   }
2407   {
2408     \MathMLintent { :silent } { ( }
2409     \__spintent_spnum_print_num_start:
2410     \MathMLintent { :silent } { ) }
2411   }
2412 }
```

(End of definition for `\__spintent_spnz_render_parens:`.)

`\__spintent_spnz_process:nn` The function `\__spintent_spnz_process:nn` first processes the $\langle \text{keys} \rangle$ passed in ‘#1’ and then calls `\__spintent_luafun_clean_split_and_set:n` on ‘#2’. It verifies that the input is a valid integer by checking if the variable `\l__spintent_luaset_has_integer_str` is `true`. If not `true`, an “error” is raised.

If it is `true`, it checks whether `\l__spintent_luaset_part_int_tl` is *empty* and raises an “error” if so. Otherwise, it checks `\l__spintent_spnz_wrap_parens_bool` set by `wrap-parens` and calls either the function `\__spintent_spnz_render_parens:` or `\__spintent_spnum_print_num_start:`.

```

2413 \cs_new_protected:Nn \__spintent_spnz_process:nn
2414 {
2415   \keys_set:nn { spintent / spnZ } {#1}
2416   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#2} }
2417   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
2418   {
2419     \tl_if_empty:NTF \l__spintent_luaset_part_int_tl
2420     { \msg_error:nnn { spintent } { spnum-no-digits } {#2} }
2421     {
2422       \bool_if:NTF \l__spintent_spnz_wrap_parens_bool
```

```

2423         { \__spintent_spnz_render_parens: }
2424         { \__spintent_spnun_print_num_start: }
2425     }
2426 }
2427 { \msg_error:nnn { spintent } { spnz-not-integer } {#2} }
2428 }

```

(End of definition for __spintent_spnz_process:nn.)

\spnZ Defines the user command **\spnZ**, which is executed within a *group* and strictly enforces *math mode*.

```

2429 \NewDocumentCommand \spnZ { O{} m }
2430 {
2431     \mode_if_math:TF
2432     {
2433         \group_begin:
2434         \__spintent_spnz_process:nn { #1 } { #2 }
2435         \group_end:
2436     }
2437     { \msg_error:nnn { spintent } { need-math-mode } { \spnZ } }
2438 }

```

(End of definition for \spnZ. This function is documented on page 16.)

11.32 The command \spmQ

The command **\spmQ** typesets “mixed numbers” within Spanish mathematical contexts (e.g., $3\frac{1}{2}$). It delegates the scaling of the integer part to the Lua module **spintent.lua** via **__spintent_luafun_spmQ_scale_int:Vn**, which measures both the integer glyph and the fraction box at the node level and computes a scaling factor to align their heights visually. Finally, it constructs the correct MathML intent for MathCAT.

11.32.1 Definition of variables for \spmQ

The variable **__spintent_spmQ_join_word_str** stores the value set by the key **join-word** which will be verbalized by MathCAT as a grammatical connector between the integer and fractional parts.

The variable **__spintent_spnQ_intent_read_sign_str** stores the **:intent** passed to **\MathMLintent** when the signs ‘+’ or ‘−’ are present in the integer part, used by the function **__spintent_spmQ_print_scale_int:n**.

The variables **__spintent_spnQ_wrap_parens_l_tl** and **__spintent_spnQ_wrap_parens_r_tl** store the sized parentheses set by the function **__spintent_spmQ_set_wrap_parens_size:** and used by the key **wrap-parens**.

The integer variable **__spintent_saved_luamml_flag_int** temporarily stores the current value of **__luamml_flag_int** before disabling it during the injection of the scaled integer node, and restores it afterwards. This prevents **luamml** from generating a spurious **<math>** element around the **sub_box** processed in isolation, before **\frac** has expanded.

```

2439 \str_new:N \__spintent_spmQ_join_word_str
2440 \str_new:N \__spintent_spnQ_intent_read_sign_str
2441 \tl_new:N \__spintent_spnQ_wrap_parens_l_tl
2442 \tl_new:N \__spintent_spnQ_wrap_parens_r_tl
2443 \int_new:N \__spintent_saved_luamml_flag_int

```

(End of definition for __spintent_spmQ_join_word_str and others.)

11.32.2 Definition of error messages for \spmQ

The error message **spmQ-zero-denominator** is raised when a zero denominator is passed to **\spmQ**. The second error **spmQ-not-integer** is returned when the integer part passed in **{⟨argument⟩}** is not actually an integer.

```

2444 \msg_new:nnnn { spintent } { spmQ-zero-denominator }
2445 {
2446     Zero~denominator~in~\token_to_str:N \spmQ~\msg_line_context:.
2447 }
2448 {
2449     The~denominator~of~a~fraction~cannot~be~zero.\\
2450 }
2451 \msg_new:nnnn { spintent } { spmQ-not-integer }
2452 {
2453     The~value~'#1'~is~not~a~valid~integer~\msg_line_context:.
2454 }
2455 {
2456     Command~\token_to_str:N \spmQ \c_space_tl only~accepts~integers.\\
2457 }

```

(End of definition for **spmQ-zero-denominator** and **spmQ-not-integer**.)

11.32.3 Definition of keys for \spmQ

Now we add the keys `join-word`, `use-spnZ`, `wrap-parens`, `read-parens` and `print-dfrac`.

```

join-word 2458 \keys_define:nn { spintenc / spmQ }
use-spnZ   2459 {
wrap-parens 2460   join-word           .choices:nn =
read-parens 2461   { entero , enteros , y }
print-dfrac 2462   {
unknown     2463     \int_case:nn { \l_keys_choice_int }
2464     {
2465       {1} { \str_set:Nn \l__spintenc_spmQ_join_word_str { entero } }
2466       {2} { \str_set:Nn \l__spintenc_spmQ_join_word_str { enteros } }
2467       {3} { \str_set:Nn \l__spintenc_spmQ_join_word_str { y } }
2468     }
2469   },
2470   join-word           .initial:n = y,
2471   join-word           .value_required:n = true,
2472   join-word/unknown   .code:n =
2473     \msg_error:nneee { spintenc } { unknown-choice }
2474     { join-word } { entero, ~enteros, ~y } { \exp_not:n {#1} },
2475   use-spnZ             .bool_set:N = \l__spintenc_spmQ_use_spnZ_bool,
2476   use-spnZ             .initial:n = true,
2477   use-spnZ             .value_required:n = true,
2478   print-dfrac          .bool_set:N = \l__spintenc_spmQ_print_dfrac_bool,
2479   print-dfrac          .initial:n = false,
2480   print-dfrac          .value_required:n = true,
2481   wrap-parens          .bool_set:N = \l__spintenc_spmQ_wrap_parens_bool,
2482   wrap-parens          .initial:n = false,
2483   wrap-parens          .value_required:n = true,
2484   read-parens          .bool_set:N = \l__spintenc_spmQ_read_parens_bool,
2485   read-parens          .initial:n = true,
2486   read-parens          .value_required:n = true,
2487   unknown              .code:n =
2488     {
2489       \str_set:Nn \l__spintenc_command_name_str { spmQ }
2490       \__spintenc_unknown_keys_cmd:n {#1}
2491     },
2492 }

```

(End of definition for `join-word` and others.)

11.32.4 Internal functions for \spmQ

`\__spintenc_spmQ_set_wrap_parens_size:`

The function `\__spintenc_spmQ_set_wrap_parens_size:` captures the current mathematical style using the primitive `\mathstyle` from LuaTeX and sets the variables `\l__spintenc_spmQ_wrap_parens_l_tl` and `\l__spintenc_spmQ_wrap_parens_r_tl` with the appropriate size for the parentheses. Display styles (0 and 1) receive `\Bigl\lparen` and `\Bigr\rparen`; text styles (2 and 3) receive `\bigl\lparen` and `\bigr\rparen`; any other style falls back to `\lparen` and `\rparen`.

If the key `print-dfrac` is active, the size is always forced to `\Bigl\lparen`/`\Bigr\rparen` regardless of the current mathematical style, since `\dfrac` always produces a display-height fraction.

```

2493 \cs_new_protected:Nn \__spintenc_spmQ_set_wrap_parens_size:
2494 {
2495   \int_case:nnF { \mathstyle }
2496   {
2497     { 0 }
2498     {
2499       \tl_set:Nn \l__spintenc_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2500       \tl_set:Nn \l__spintenc_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2501     }
2502     { 1 }
2503     {
2504       \tl_set:Nn \l__spintenc_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2505       \tl_set:Nn \l__spintenc_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2506     }
2507     { 2 }
2508     {
2509       \tl_set:Nn \l__spintenc_spmQ_wrap_parens_l_tl { \bigl \lparen }
2510       \tl_set:Nn \l__spintenc_spmQ_wrap_parens_r_tl { \bigr \rparen }
2511     }
2512     { 3 }
2513     {
2514       \tl_set:Nn \l__spintenc_spmQ_wrap_parens_l_tl { \bigl \lparen }

```

```

2515         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2516     }
2517 }
2518 {
2519     \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \lparen }
2520     \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \rparen }
2521 }

```

If the key `print-dfrac` is active, we must reset the variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` to `\Bigl/\Bigr` unconditionally.

```

2522     \bool_if:NT \l__spintent_spmQ_print_dfrac_bool
2523     {
2524         \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2525         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2526     }
2527 }

```

(End of definition for `\__spintent_spmQ_set_wrap_parens_size:`)

```

\__spintent_spmQ_start_wrap_parens:
\__spintent_spmQ_stop_wrap_parens:

```

The functions `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:` open and close the parentheses around the “*mixed number*” when the key `wrap-parens` is active. They first call `\__spintent_spmQ_set_wrap_parens_size:` to determine the correct size for the current mathematical style, then print the appropriate parenthesis stored in `\l__spintent_spmQ_wrap_parens_l_tl` or `\l__spintent_spmQ_wrap_parens_r_tl`.

If the variable `\l__spintent_spmQ_read_parens_bool`, controlled by the key `read-parens`, is *false*, each parenthesis is wrapped inside `\MathMLintent{:silent}` so that NVDA/MathCAT does not verbalize them.

```

2528 \cs_new_protected:Nn \__spintent_spmQ_start_wrap_parens:
2529 {
2530     \bool_if:NT \l__spintent_spmQ_wrap_parens_bool
2531     {
2532         \__spintent_spmQ_set_wrap_parens_size:
2533         \bool_if:NTF \l__spintent_spmQ_read_parens_bool
2534         { \l__spintent_spmQ_wrap_parens_l_tl }
2535         { \MathMLintent { :silent } { \l__spintent_spmQ_wrap_parens_l_tl } }
2536     }
2537 }
2538 \cs_new_protected:Nn \__spintent_spmQ_stop_wrap_parens:
2539 {
2540     \bool_if:NT \l__spintent_spmQ_wrap_parens_bool
2541     {
2542         \__spintent_spmQ_set_wrap_parens_size:
2543         \bool_if:NTF \l__spintent_spmQ_read_parens_bool
2544         { \l__spintent_spmQ_wrap_parens_r_tl }
2545         { \MathMLintent { :silent } { \l__spintent_spmQ_wrap_parens_r_tl } }
2546     }
2547 }

```

(End of definition for `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:`)

```
\__spintent_spmQ_set_join_word:
```

The function `\__spintent_spmQ_set_join_word:` injects the grammatical connector between the integer part and the fraction by dispatching on the value stored in `\l__spintent_spmQ_join_word_str`, set by the key `join-word`. In all three cases the visual output is the Unicode invisible separator U+2063 via `\c__spintent_invisible_sep_tl`, which produces a `<mi>` node in the MathML structure; what differs is the `:intent` text attached to it via `\MathMLintent`, which NVDA/MathCAT will verbalize as «*entero*», «*enteros*» or «*y*» respectively.

```

2548 \cs_new_protected:Nn \__spintent_spmQ_set_join_word:
2549 {
2550     \str_case:Vn \l__spintent_spmQ_join_word_str
2551     {
2552         { entero } { \MathMLintent { _entero } { \c__spintent_invisible_sep_int } }
2553         { enteros } { \MathMLintent { _enteros } { \c__spintent_invisible_sep_int } }
2554         { y } { \MathMLintent { _y- } { \c__spintent_invisible_sep_int } }
2555     }
2556 }

```

(End of definition for `\__spintent_spmQ_set_join_word:`)

```
\__spintent_spmQ_print_scale_int:
```

The function `\__spintent_spmQ_print_scale_int:` prints the integer part of the “*mixed number*” scaled to match the visual height of the accompanying fraction. Before calling the Lua function `\__spintent_luafun_spmQ_scale_int` it temporarily disables `luamml` by saving the current value of `\l__luamml_flag_int` into `\l__spintent_saved_luamml_flag_int` and setting it to 0 (skip completely). This prevents `luamml` from generating a

spurious `<math>` wrapper around the `sub_box` node that holds the scaled integer, which is processed in isolation before `\frac` has expanded. Once the Lua function returns, the original flag value is restored via `\l__spintenc_saved_luaamml_flag_int`.

The Lua function receives the integer digits from `\l__spintenc_luaaset_part_int_tl` and the fraction command name (`\frac` or `\dfrac`, determined by the key `print-dfrac`) as its two arguments, and inserts the appropriately scaled `sub_box` node directly into the current horizontal list.

```

2557 \cs_new_protected:Nn \__spintenc_spmQ_print_scale_int:
2558 {
2559   \int_set_eq:Nc \l__spintenc_saved_luaamml_flag_int { l__luaamml_flag_int }
2560   \int_set:Nn \l__luaamml_flag_int { 0 }
2561   \bool_if:NTF \l__spintenc_spmQ_print_dfrac_bool
2562     { \__spintenc_luafun_spmQ_scale_int:Vn \l__spintenc_luaaset_part_int_tl { dfrac } }
2563     { \__spintenc_luafun_spmQ_scale_int:Vn \l__spintenc_luaaset_part_int_tl { frac } }
2564   \int_set_eq:cN { l__luaamml_flag_int } \l__spintenc_saved_luaamml_flag_int
2565 }

```

(End of definition for `\__spintenc_spmQ_print_scale_int:`)

`\__spintenc_spmQ_print_scale_int:n`

The function `\__spintenc_spmQ_print_scale_int:n` takes the integer part of the “mixed number” passed in ‘#1’. It first calls `\__spintenc_luafun_clean_split_and_set:n` on ‘#1’ to parse and populate the shared Lua variables, then checks the status of `\l__spintenc_luaaset_has_integer_str`.

If the status is `true`, it inspects `\l__spintenc_luaaset_sign_tl`. When the sign is *empty*, it delegates directly to `\__spintenc_spmQ_print_scale_int:` to render the scaled integer. When a sign is present, it sets `\l__spintenc_spmQ_intent_read_sign_str` to either `_más:prefix($e)` or `_menos:prefix($e)` and wraps the sign glyph together with a MathML argument (`$e`) containing the scaled integer inside a `\MathMLintent`, so that NVDA/MathCAT verbalizes the sign as a prefix before the integer value. An `\__spintenc_invisible_sep:` call between the sign and the `MathMLarg` forces the opening of a new `<mrow>` in the MathML structure, which is required for the intent tree to be well-formed.

If the status is `false`, the `spmQ-not-integer` “error” is raised immediately.

```

2566 \cs_new_protected:Nn \__spintenc_spmQ_print_scale_int:n
2567 {
2568   \__spintenc_luafun_clean_split_and_set:n { \exp_not:n { #1 } }
2569   \str_if_eq:VnTF \l__spintenc_luaaset_has_integer_str { true }
2570     {
2571       \tl_if_empty:NTF \l__spintenc_luaaset_sign_tl
2572         {
2573           \__spintenc_spmQ_print_scale_int:
2574         }
2575         {
2576           \str_if_eq:VnTF \l__spintenc_luaaset_sign_tl { + }
2577             {
2578               \str_set:Nn
2579                 \l__spintenc_spmQ_intent_read_sign_str { _más:prefix($e) }
2580             }
2581             {
2582               \str_set:Nn
2583                 \l__spintenc_spmQ_intent_read_sign_str { _menos:prefix($e) }
2584             }
2585           \MathMLintent { \l__spintenc_spmQ_intent_read_sign_str }
2586             {
2587               \l__spintenc_luaaset_sign_tl \c__spintenc_invisible_sep_int % \__spintenc_invisible_s
2588               \MathMLarg { e } { \__spintenc_spmQ_print_scale_int: }
2589             }
2590         }
2591     }
2592     { \msg_error:nnn { spintenc } { spmQ-not-integer } { #1 } }
2593 }

```

(End of definition for `\__spintenc_spmQ_print_scale_int:n`)

`\__spintenc_spmQ_print_spmZ_dfrac:nn`

The function `\__spintenc_spmQ_print_spmZ_dfrac:nn` typesets the fractional part of the “mixed number”. It takes the numerator in ‘#1’ and the denominator in ‘#2’. It first calls `\__spintenc_luafun_clean_split_and_set:` on ‘#1’ to validate that the numerator is an integer; if `\l__spintenc_luaaset_has_integer_str` is `false`, the `spmQ-not-integer` “error” is raised immediately.

If the validation passes, the function dispatches on the two boolean variables `\l__spintenc_spmQ_use_spmZ_bool` and `\l__spintenc_spmQ_print_dfrac_bool`, controlled respectively by the keys `use-spmZ` and `print-dfrac`. When `use-spmZ` is `true`, both the numerator and denominator are passed through `\spnZ` to gain MathML integer semantics; otherwise, the raw token lists ‘#1’ and ‘#2’ are used directly. In both

branches, the `fraction` command is selected dynamically via `\use:c`: if `print-dfrac` is `true`, `dfrac` is assembled; otherwise, `frac` is used.

```

2594 \cs_new_protected:Nn \__spintent_spmQ_print_spnZ_dfrac:nn
2595 {
2596   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2597   \str_if_eq:VnT \l__spintent_luaaset_has_integer_str { false }
2598   {
2599     \msg_error:nnn { spintent } { spmQ-not-integer } { #1 }
2600   }
2601   \bool_if:NTF \l__spintent_spmQ_use_spnZ_bool
2602   {
2603     \use:c { \bool_if:NT \l__spintent_spmQ_print_dfrac_bool { d } frac }
2604     { \spnZ { #1 } } { \spnZ { #2 } }
2605   }
2606   {
2607     \use:c { \bool_if:NT \l__spintent_spmQ_print_dfrac_bool { d } frac }
2608     { #1 } { #2 }
2609   }
2610 }

```

(End of definition for `\__spintent_spmQ_print_spnZ_dfrac:nn`.)

`\__spintent_spmQ_print:nnn`

The function `\__spintent_spmQ_print:nnn` is the main rendering orchestrator for `\spmQ`. It takes three arguments: the integer part `#1`, the numerator `#2`, and the denominator `#3`. It opens the optional parentheses via `\__spintent_spmQ_start_wrap_parens:`, then calls `\__spintent_spmQ_print_scale_int:n` to scale and render the integer part. Next, it injects the grammatical connector between the integer and the fraction via `\__spintent_spmQ_set_join_word:`. Then it calls `\__spintent_spmQ_print_spnZ_dfrac:nn` to typeset the fraction using the keys `print-dfrac` and `use-spnZ`. Finally, it closes the optional parentheses via `\__spintent_spmQ_stop_wrap_parens:`.

```

2611 \cs_new_protected:Nn \__spintent_spmQ_print:nnn
2612 {
2613   \__spintent_spmQ_start_wrap_parens:
2614   \__spintent_spmQ_print_scale_int:n { #1 }
2615   \__spintent_spmQ_set_join_word:
2616   \__spintent_spmQ_print_spnZ_dfrac:nn { #2 } { #3 }
2617   \__spintent_spmQ_stop_wrap_parens:
2618 }

```

(End of definition for `\__spintent_spmQ_print:nnn`.)

`\__spintent_spmQ_exec:nnnn`

The internal function `\__spintent_spmQ_exec:nnnn` takes four arguments: the optional `<keys>` `#1`, the integer `#2`, the numerator `#3`, and the denominator `#4`. First sets the optional `<keys>` from `#1`, then processes the denominator input in `#4` using `\__spintent_luafun_clean_split_and_set:n` to validate it is a non-zero integer.

It checks the status of `\l__spintent_luaaset_has_integer_str`. If `true` and the integer part is exactly zero, it issues the `spmQ-zero-denominator` “error”. Otherwise, whether the status is `false` or the integer part is *non-zero*, it calls `\__spintent_spmQ_print:nnn` to format the “mixed number”.

```

2619 \cs_new_protected:Nn \__spintent_spmQ_exec:nnnn
2620 {
2621   \keys_set:nn { spintent / spmQ } { #1 }
2622   \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2623   \str_if_eq:VnTF \l__spintent_luaaset_has_integer_str { true }
2624   {
2625     \int_compare:nNnTF { \l__spintent_luaaset_part_int_tl } = { 0 }
2626     { \msg_error:nn { spintent } { spmQ-zero-denominator } }
2627     { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2628   }
2629   { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2630 }

```

(End of definition for `\__spintent_spmQ_exec:nnnn`.)

`\spmQ`

The command `\spmQ` is the public interface for formatting “mixed numbers”. It first checks if it is being used in *math mode*, issuing an “error” if not. Inside a local group, it calls `\__spintent_spmQ_exec:nnnn` to process the optional `<keys>` and the `{<arguments>}`.

```

2631 \NewDocumentCommand \spmQ { O{} m m m }
2632 {
2633   \mode_if_math:TF
2634   {
2635     \group_begin:

```

```

2636         \l__spinttent_spmQ_exec:nnnn { #1 } { #2 } { #3 } { #4 }
2637     \group_end:
2638 }
2639 { \msg_error:nnn { spinttent } { need-math-mode } { \spmQ } }
2640 }

```

(End of definition for \spmQ. This function is documented on page 16.)

11.33 The commands of the geo family

The commands described in this section provide a semantic interface for geometric objects in Spanish mathematical contexts. They all delegate parsing to `spinttent.lua`, sharing the variables `\l__spinttent_geo_luaset_error_str`, `\l__spinttent_geo_luaset_intent_str`, `\l__spinttent_geo_luaset_print_tl`, `\l__spinttent_geo_luaset_sub_str` and `\l__spinttent_geo_luaset_tipo_str` — constructing a valid MathCAT intent — and a single point is always annotated through `\l__spinttent_mathmmlarg_point:nnn`. Every command also activates `\l__spinttent_scope_attr_int` within its own group, so the shared queue never claims content typeset outside a geo command in the same formula.

11.33.1 Definition of shared variables

Internal variables shared across all commands of the family to avoid repeated calls into Lua.

```

\l__spinttent_geo_luaset_error_str
\l__spinttent_geo_luaset_intent_str
\l__spinttent_geo_luaset_print_tl
\l__spinttent_geo_luaset_is_name_str
\l__spinttent_geo_luaset_poly_sym_str
\l__spinttent_geo_luaset_angulo_case_str
\l__spinttent_geo_luaset_concept_str
\l__spinttent_geo_luaset_letra_word_str
\l__spinttent_geo_luaset_sub_spoken_str
\l__spinttent_geo_luaset_nombre_es_num_str
\l__spinttent_geo_luaset_letra_str
\l__spinttent_geo_luaset_sub_str
\l__spinttent_geo_luaset_tipo_str
2641 \str_new:N \l__spinttent_geo_luaset_error_str
2642 \str_new:N \l__spinttent_geo_luaset_intent_str
2643 \tl_new:N \l__spinttent_geo_luaset_print_tl
2644 \clist_new:N \l__spinttent_geo_luaset_points_clist
2645 \str_new:N \l__spinttent_geo_luaset_is_name_str
2646 \str_new:N \l__spinttent_geo_luaset_poly_sym_str
2647 \str_new:N \l__spinttent_geo_luaset_angulo_case_str
2648 \str_new:N \l__spinttent_geo_luaset_concept_str
2649 \str_new:N \l__spinttent_geo_luaset_letra_word_str
2650 \str_new:N \l__spinttent_geo_luaset_sub_spoken_str
2651 \str_new:N \l__spinttent_geo_luaset_nombre_es_num_str
2652 \str_new:N \l__spinttent_geo_luaset_letra_str
2653 \str_new:N \l__spinttent_geo_luaset_sub_str
2654 \str_new:N \l__spinttent_geo_luaset_tipo_str

```

(End of definition for \l__spinttent_geo_luaset_error_str and others.)

11.34 The command \spcoord

The user command `\spcoord` typesets an ordered pair (x, y) , with the comma (or semicolon) between them carrying its own `:infix` intent, and each side wrapped with `:prefix($vp)/:prefix($vn)` when it starts with $+/-$, so the sign is always read as *más/menos* — never *positivo/negativo*, which is not how MathCAT reads a bare sign in this position.

11.34.1 Definition of variables for \spcoord

Variables set by the function `\l__spinttent_luafun_spcoord_parse_and_set:n`. The string variable `\l__spinttent_spcoord_luaset_sep_str` is `,` or `;`, whichever the argument actually used. For the abscissa and ordinate values, the variables `\l__spinttent_spcoord_luaset_ #1 _sign_str` is `+`, `-`, or empty; the variables `\l__spinttent_spcoord_luaset_ #1 _type_str` is `number` or `raw`; the variables `\l__spinttent_spcoord_luaset_ #1 _value_str` is the value with the sign already removed; and the variables `\l__spinttent_spcoord_luaset_ #1 _has_frac_str` is `none`, `frac`, or `dfrac`.

```

2655 \str_new:N \l__spinttent_spcoord_luaset_error_str
2656 \str_new:N \l__spinttent_spcoord_luaset_sep_str
2657 \clist_map_inline:nn { x , y }
2658 {
2659     \str_new:c { \l__spinttent_spcoord_luaset_ #1 _sign_str }
2660     \str_new:c { \l__spinttent_spcoord_luaset_ #1 _type_str }
2661     \str_new:c { \l__spinttent_spcoord_luaset_ #1 _value_str }
2662     \str_new:c { \l__spinttent_spcoord_luaset_ #1 _has_frac_str }
2663 }

```

(End of definition for \l__spinttent_spcoord_luaset_error_str and others.)

```

\l__spinttent_spcoord_raw_arg_tl
\l__spinttent_spcoord_intent_str
\l__spinttent_spcoord_wrap_parens_l_tl
\l__spinttent_spcoord_wrap_parens_r_tl

```

The variable `\l__spinttent_spcoord_raw_arg_tl` holds the `{(argument)}` exactly as given. The variable `\l__spinttent_spcoord_intent_str` holds the `:infix` word for the separator `_coma:infix` or `_punto-y-coma:infix`. The variables `\l__spinttent_spcoord_wrap_parens_l_tl`/`\l__spinttent_spcoord_wrap_parens_r_tl` hold the opening/closing delimiter, sized by the function `\l__spinttent_spcoord_set_wrap_parens_size:.`

```

2664 \tl_new:N \l__spinttent_spcoord_raw_arg_tl
2665 \str_new:N \l__spinttent_spcoord_intent_str
2666 \tl_new:N \l__spinttent_spcoord_wrap_parens_l_tl
2667 \tl_new:N \l__spinttent_spcoord_wrap_parens_r_tl

```

(End of definition for \l__spinttent_spcoord_raw_arg_tl and others.)

11.34.2 Definition of error messages for \spcoord

spcoord-invalid-arg

```

2668 \msg_new:nnn { spintrent } { spcoord-invalid-arg }
2669 {
2670   Invalid~argument~'#1'~for~\token_to_str:N \spcoord.
2671 }

```

(End of definition for spcoord-invalid-arg.)

11.34.3 Definition of keys for \spcoord

silent-cmd

We define the key `silent-cmd`.

```

2672 \keys_define:nn { spintrent / spcoord }
2673 {
2674   silent-cmd .bool_set:N = \l__spintrent_spcoord_silent_cmd_bool,
2675   silent-cmd .initial:n = false,
2676   silent-cmd .value_required:n = true,
2677   unknown    .code:n =
2678     {
2679       \str_set:Nn \l__spintrent_command_name_str { spcoord }
2680       \__spintrent_unknown_keys_cmd:n {#1}
2681     },
2682 }

```

(End of definition for silent-cmd.)

11.34.4 Internal functions for \spcoord

__spintrent_spcoord_set_wrap_parens_size:

With neither side carrying a fraction, the parentheses stay unsized. Otherwise, `\mathstyle` picks `\Bigl/\Bigr` for display, `\bigl/\bigr` for text. When either side is `dfrac`, the parentheses are forced to `\Bigl/\Bigr` regardless of `\mathstyle`, since `\dfrac` always renders at display height.

```

2683 \cs_new_protected:Nn \__spintrent_spcoord_set_wrap_parens_size:
2684 {
2685   \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \lparen }
2686   \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \rparen }
2687   \bool_if:nT
2688     {
2689       \bool_lazy_or_p:nn
2690         { \bool_not_p:n { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_x_has_frac_str { none } } }
2691         { \bool_not_p:n { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_y_has_frac_str { none } } }
2692     }
2693     {
2694       \int_case:nnF { \mathstyle }
2695       {
2696         { 0 }
2697         {
2698           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2699           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2700         }
2701         { 1 }
2702         {
2703           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2704           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2705         }
2706         { 2 }
2707         {
2708           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2709           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2710         }
2711         { 3 }
2712         {
2713           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2714           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2715         }
2716       }
2717     }
2718   \bool_if:nT
2719     {
2720       \bool_lazy_or_p:nn
2721         { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_x_has_frac_str { dfraction } }
2722         { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_y_has_frac_str { dfraction } }
2723     }
2724     {

```

```

2725         \tl_set:Nn \l__spintenc_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2726         \tl_set:Nn \l__spintenc_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2727     }
2728 }
2729 }

```

(End of definition for __spintenc_spcoord_set_wrap_parens_size:.)

__spintenc_spcoord_value:n

With `\l__spintenc_spcoord_luaset_ #1 _type_str` equal to `number`, calls `\spnum` on `\l__spintenc_spcoord_value:n`. Otherwise, typesets it as given — a `\frac/\dfrac` or a letter expression, untouched. Receives `x` or `y` in ‘#1’.

```

2730 \cs_new_protected:Nn \__spintenc_spcoord_value:n
2731 {
2732     \str_if_eq:eeTF
2733     { \str_use:c { \l__spintenc_spcoord_luaset_ #1 _type_str } } { number }
2734     {
2735         \exp_args:Ne \spnum { \str_use:c { \l__spintenc_spcoord_luaset_ #1 _value_str } }
2736     }
2737     { \str_use:c { \l__spintenc_spcoord_luaset_ #1 _value_str } }
2738 }

```

(End of definition for __spintenc_spcoord_value:n.)

__spintenc_spcoord_print:n

With `\l__spintenc_spcoord_luaset_ #1 _sign_str` equal to `+` or `-`, wraps the sign and `\__spintenc_spcoord_value:n` together with `_más:prefix($vp)/_menos:prefix($vn)`, referencing the value through `read` as `más/menos`. With no sign, delegates directly to `\__spintenc_spcoord_value:n`. Receives `x` or `y` in ‘#1’.

```

2739 \cs_new_protected:Nn \__spintenc_spcoord_print:n
2740 {
2741     \str_case:enF { \str_use:c { \l__spintenc_spcoord_luaset_ #1 _sign_str } }
2742     {
2743         { + }
2744         {
2745             \MathMLintent { _más:prefix ($vp) }
2746             {
2747                 \str_use:c { \l__spintenc_spcoord_luaset_ #1 _sign_str }
2748                 \MathMLarg{ vp } { \__spintenc_spcoord_value:n { #1 } }
2749             }
2750         }
2751         { - }
2752         {
2753             \MathMLintent { _menos :prefix($vn) }
2754             {
2755                 \str_use:c { \l__spintenc_spcoord_luaset_ #1 _sign_str }
2756                 \MathMLarg{ vn } { \__spintenc_spcoord_value:n { #1 } }
2757             }
2758         }
2759     }
2760     { \__spintenc_spcoord_value:n { #1 } }
2761 }

```

(End of definition for __spintenc_spcoord_print:n.)

__spintenc_spcoord_intent_infix:

Sets `\l__spintenc_spcoord_intent_str` to `_coma:infix` or `_punto-y-coma:infix`, per `\l__spintenc_spcoord_intent:n`, then wraps both sides — each referenced through with it, `$x/$y` standing for `\__spintenc_spcoord_print:n`’s own output.

```

2762 \cs_new_protected:Nn \__spintenc_spcoord_intent_infix:
2763 {
2764     \str_if_eq:VnTF \l__spintenc_spcoord_luaset_sep_str { ; }
2765     { \str_set:Nn \l__spintenc_spcoord_intent_str { _punto-y-coma:infix } }
2766     { \str_set:Nn \l__spintenc_spcoord_intent_str { _coma:infix } }
2767     \MathMLintent{ \l__spintenc_spcoord_intent_str ($x, $y)}
2768     {
2769         \MathMLarg{x}{ \__spintenc_spcoord_print:n { x } }
2770         \MathMLintent { :silent } { \l__spintenc_spcoord_luaset_sep_str }
2771         \MathMLarg{y}{ \__spintenc_spcoord_print:n { y } }
2772     }
2773 }

```

(End of definition for __spintenc_spcoord_intent_infix:.)

`\__spintent_spcoord_mathml:` With `silent-cmd=true`, everything — parentheses and argument alike — is wrapped in a single `_-`, reading nothing at all: `\l__spintent_spcoord_raw_arg_tl` is typeset exactly as given, untouched by `\__spintent_spcoord_print:n/\__spintent_spcoord_intent_infix:`. Otherwise, the parentheses are silenced individually and `\__spintent_spcoord_intent_infix:` provides the reading in between.

```

2774 \cs_new_protected:Nn \__spintent_spcoord_mathml:
2775 {
2776   \bool_if:NTF \l__spintent_spcoord_silent_cmd_bool
2777   {
2778     \MathMLintent { _- }
2779     {
2780       \l__spintent_spcoord_wrap_parens_l_tl
2781       \l__spintent_spcoord_raw_arg_tl
2782       \l__spintent_spcoord_wrap_parens_r_tl
2783     }
2784   }
2785   {
2786     \MathMLintent { :silent } { \l__spintent_spcoord_wrap_parens_l_tl }
2787     \__spintent_spcoord_intent_infix:
2788     \MathMLintent { :silent } { \l__spintent_spcoord_wrap_parens_r_tl }
2789   }
2790 }

```

(End of definition for `\__spintent_spcoord_mathml:`)

`\__spintent_spcoord_exec:nn` Sets the *(keys)* in ‘#1’ and `\l__spintent_spcoord_raw_arg_tl` to ‘#2’. Calls `\__spintent_luafun_spcoord_parse_and_set:n` either way — even with `silent-cmd=true`, since `\__spintent_spcoord_set_wrap_parens_size:` still needs `\l__spintent_spcoord_luaset_ #1 _has_frac_str` — but with `silent-cmd=true`, forces `\l__spintent_spcoord_luaset_error_str` back to `false` rather than reporting it: the raw `{(argument)}` is typeset regardless of whether it parsed. Otherwise, reports `spcoord-invalid-arg` on error. Either way, sizes the parentheses, then delegates to `\__spintent_spcoord_mathml:` unless there was an error.

```

2791 \cs_new_protected:Nn \__spintent_spcoord_exec:nn
2792 {
2793   \keys_set:nn { spintent / spcoord } { #1 }
2794   \tl_set:Nn \l__spintent_spcoord_raw_arg_tl { #2 }
2795   \bool_if:NTF \l__spintent_spcoord_silent_cmd_bool
2796   {
2797     \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n { #2 } }
2798     \str_set:Nn \l__spintent_spcoord_luaset_error_str { false }
2799   }
2800   {
2801     \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n { #2 } }
2802     \str_if_eq:VnT \l__spintent_spcoord_luaset_error_str { true }
2803     { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n { #2 } } }
2804   }
2805   \__spintent_spcoord_set_wrap_parens_size:
2806   \str_if_eq:VnF \l__spintent_spcoord_luaset_error_str { true }
2807   { \__spintent_spcoord_mathml: }
2808 }

```

(End of definition for `\__spintent_spcoord_exec:nn`)

`\spcoord` The public command enforces *math mode*, opens a local group, and delegates to `\__spintent_spcoord_exec:nn`.

```

2809 \NewDocumentCommand \spcoord { 0{} m }
2810 {
2811   \mode_if_math:TF
2812   {
2813     \group_begin:
2814     \attribute \l__spintent_scope_attr_int = 1
2815     \__spintent_spcoord_exec:nn { #1 } { #2 }
2816     \group_end:
2817   }
2818   { \msg_error:nnn { spintent } { need-math-mode } { \spcoord } }
2819 }

```

(End of definition for `\spcoord`. This function is documented on page 17.)

11.35 The function `\__spintent_mathmlarg_point:nnn`

Scratch variables for `\__spintent_mathmlarg_point:nnn`: the reference name (#1), and the calling command's own `read-cmd/silent-cmd` state, looked up by name (#2).

```
\__spintent_mathmlarg_point_ref_tl
\__spintent_mathmlarg_point_mode_str
\__spintent_mathmlarg_point_kv_tl
\__spintent_mathmlarg_point_silent_bool
```

```
2820 \tl_new:N \__spintent_mathmlarg_point_ref_tl
2821 \str_new:N \__spintent_mathmlarg_point_mode_str
2822 \tl_new:N \__spintent_mathmlarg_point_kv_tl
2823 \bool_new:N \__spintent_mathmlarg_point_silent_bool
```

(End of definition for `\__spintent_mathmlarg_point_ref_tl` and others.)

```
\__spintent_mathmlarg_point:nnn
\__spintent_mathmlarg_point:nnV
```

Annotates one point (#3) with `arg` and, when applicable, a literal `intent`, without wrapping it in its own `<mrow>`. Concept `intent` is passed empty on purpose: the caller applies it externally, via a native `\MathMLintent`.

With `\__spintent_geo_luaset_tipo_str` equal to `sub`, for `read-cmd=school` the letter alone is tagged through `\__spintent_luafun_mrow_intent_arg:nn`, and the subscript is appended separately, silenced. For `read-cmd=mathcat`, letter and subscript are typeset together, tagged as a whole through a native `\MathMLarg`.

```
2824 \cs_new_protected:Nn \__spintent_mathmlarg_point:nnn
2825 {
2826   \tl_set:Nn \__spintent_mathmlarg_point_ref_tl {#1}
2827   \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#3} }
2828   \bool_set_eq:Nc \__spintent_mathmlarg_point_silent_bool { \__spintent_ #2 _silent_cmd_bool }
2829   \str_set_eq:Nc \__spintent_mathmlarg_point_mode_str { \__spintent_ #2 _mode_str }
2830   \__spintent_invisible_sep:
2831   \str_case:VnF \__spintent_geo_luaset_tipo_str
2832   {
2833     { sub }
2834     {
2835       \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
2836       {
2837         \tl_set:Ne \__spintent_mathmlarg_point_kv_tl
2838         { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_lua
2839         \__spintent_luafun_mrow_intent_arg:nn
2840         { } { \__spintent_mathmlarg_point_kv_tl }
2841         {
2842           \__spintent_geo_luaset_letra_str
2843           \c_math_subscript_token
2844           { \MathMLintent { :silent } { \__spintent_geo_luaset_sub_str } }
2845         }
2846       }
2847     }
2848     {
2849       \__spintent_inner_sep:
2850       \MathMLarg { \__spintent_mathmlarg_point_ref_tl }
2851       {
2852         {
2853           \__spintent_geo_luaset_letra_str
2854           \c_math_subscript_token { \__spintent_geo_luaset_sub_str }
2855         }
2856       }
2857     }
2858   }
2859 }
```

With `\__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the letter with a literal `intent` and appends the mark unsilenced (a prime is not spoken on the base, so nothing to repeat); `read-cmd=mathcat` tags letter and mark together through a native `\MathMLarg`.

```
2860 { prima }
2861 {
2862   \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
2863   {
2864     \tl_set:Ne \__spintent_mathmlarg_point_kv_tl
2865     { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_lua
2866     \__spintent_luafun_mrow_intent_arg:nn
2867     { } { \__spintent_mathmlarg_point_kv_tl }
2868     { \__spintent_geo_luaset_letra_str \__spintent_geo_luaset_sub_str }
2869   }
2870   {
2871     {
2872       \__spintent_inner_sep:
```

```

2873         \MathMLarg { \l__spintent_mathmlarg_point_ref_tl }
2874         {
2875             { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
2876         }
2877     }
2878 }
2879 }
2880 }

```

Otherwise (no subscript, no prime), the same distinction applies to the combined `\l__spintent_geo_luaset_print_tl` as a whole — the shared queue for `school` (or when `silent-cmd` is `true`), a native `\MathMLarg` for `mathcat`, never mixed within the same formula, since the queue does not recognize a native `\MathMLarg` claim as already taken.

```

2881 {
2882     \bool_if:NTF \l__spintent_mathmlarg_point_silent_bool
2883     {
2884         \tl_set:Nx \l__spintent_mathmlarg_point_kv_tl
2885         { arg = "\l__spintent_mathmlarg_point_ref_tl", intent = "-_" }
2886         \__spintent_luafun_mrow_intent_arg:nn
2887         { } % debe ir vacío
2888         { \l__spintent_mathmlarg_point_kv_tl }
2889         { \l__spintent_geo_luaset_print_tl }
2890     }
2891     {
2892         \str_if_eq:VnTF \l__spintent_mathmlarg_point_mode_str { school }
2893         {
2894             \tl_set:Nx \l__spintent_mathmlarg_point_kv_tl
2895             { arg = "\l__spintent_mathmlarg_point_ref_tl", intent = "- \l__spintent_geo_luaset_print_tl" }
2896             \__spintent_luafun_mrow_intent_arg:nn
2897             { } % debe ir vacío
2898             { \l__spintent_mathmlarg_point_kv_tl }
2899             { \l__spintent_geo_luaset_print_tl }
2900         }
2901         {
2902             \MathMLarg { \l__spintent_mathmlarg_point_ref_tl }
2903             { \l__spintent_geo_luaset_print_tl }
2904         }
2905     }
2906 }
2907 }
2908 \cs_generate_variant:Nn \__spintent_mathmlarg_point:nnn { nnV }

```

(End of definition for `\l__spintent_mathmlarg_point:nnn`.)

11.36 The command `\spPoint`

The user-level command `\spPoint` provides a semantic interface for a single geometric point. It delegates argument validation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.36.1 Definition of variables for `\spPoint`

Internal variables holding the values set by `read-cmd` and `prefix`.

```

\l__spintent_spPoint_mode_str
\l__spintent_spPoint_prefix_tl
\l__spintent_spPoint_intent_arg_tl
2909 \str_new:N \l__spintent_spPoint_mode_str
2910 \tl_new:N \l__spintent_spPoint_prefix_tl
2911 \tl_new:N \l__spintent_spPoint_intent_arg_tl

```

(End of definition for `\l__spintent_spPoint_mode_str`, `\l__spintent_spPoint_prefix_tl`, and `\l__spintent_spPoint_intent_arg_tl`.)

11.36.2 Definition of error messages for `\spPoint`

`spPoint-invalid-point`

The “error” `spPoint-invalid-point` is raised when the argument passed in ‘`#1`’ is not a single uppercase letter with an optional subscript or prime modifier.

```

2912 \msg_new:nnnn { spintent } { spPoint-invalid-point }
2913 {
2914     Invalid~point~format~in~\token_to_str:N \spPoint:~'#1'.
2915 }
2916 {
2917     The~argument~must~be~a~single~uppercase~letter~with~an~optional~
2918     subscript~(\_{...})~or~prime~(').\ \
2919 }

```

(End of definition for `spPoint-invalid-point`.)

11.36.3 Definition of keys for \spPoint

We define the *(keys)* `read-cmd`, `prefix`, `silent-cmd` and `concept`.

```

2920 \keys_define:nn { spintenz / spPoint }
2921 {
2922   read-cmd          .choices:n = { school , mathcat }
2923   {
2924     \int_case:nn { \l_keys_choice_int }
2925     {
2926       {1} { \str_set:Nn \l__spintenz_spPoint_mode_str { school } }
2927       {2} { \str_set:Nn \l__spintenz_spPoint_mode_str { mathcat } }
2928     }
2929   },
2930   read-cmd          .initial:n = school,
2931   read-cmd          .value_required:n = true,
2932   read-cmd / unknown .code:n =
2933     \msg_error:nneee { spintenz } { unknown-choice }
2934     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
2935   prefix            .code:n =
2936     { \__spintenz_sanitize_affix:Nn \l__spintenz_spPoint_prefix_tl {#1} },
2937   prefix            .initial:n = ,
2938   prefix            .value_required:n = true,
2939   silent-cmd        .bool_set:N = \l__spintenz_spPoint_silent_cmd_bool,
2940   silent-cmd        .initial:n = false,
2941   silent-cmd        .value_required:n = true,
2942   concept           .bool_set:N = \l__spintenz_spPoint_concept_bool,
2943   concept           .initial:n = true,
2944   concept           .value_required:n = true,
2945   unknown           .code:n =
2946     {
2947       \str_set:Nn \l__spintenz_command_name_str { spPoint }
2948       \__spintenz_unknown_keys_cmd:n {#1}
2949     },
2950 }
```

(End of definition for `read-cmd` and others.)

11.36.4 Internal functions for \spPoint

__spintenz_spPoint_print:n

The internal function `\__spintenz_spPoint_print:n` processes the *{(argument)}* passed in ‘#1’. Calls `\__spintenz_luafun_geo_point_parse_and_set:n`, reporting `spPoint-invalid-point` on error. If `silent-cmd` is `true` or `concept` is `false`, sets `\l__spintenz_spPoint_intent_arg_tl` to `\_($pt)` — forced further to `\_` — when `silent-cmd` is `true` and `read-cmd` is not `school`, since `\__spintenz_spPoint_mathml:n` then tags through a native `\MathMLarg` that needs the concept silenced too. Otherwise sets it to `punto:prefix($pt)`, prefixed with `prefix` when given. Finally calls `\__spintenz_spPoint_mathml:n`.

```

2951 \cs_new_protected:Nn \__spintenz_spPoint_print:n
2952 {
2953   \__spintenz_luafun_geo_point_parse_and_set:n { \exp_not:n {#1} }
2954   \str_if_eq:VnT \l__spintenz_geo_luasets_error_str { true }
2955   {
2956     \msg_error:nne { spintenz } { spPoint-invalid-point } { \exp_not:n {#1} }
2957   }
2958   \bool_if:nTF
2959   {
2960     \bool_lazy_or:pnn
2961     { \l__spintenz_spPoint_silent_cmd_bool }
2962     { ! \l__spintenz_spPoint_concept_bool }
2963   }
2964   {
2965     \tl_set:Nn \l__spintenz_spPoint_intent_arg_tl { \_($pt) }
2966     \bool_if:NT \l__spintenz_spPoint_silent_cmd_bool
2967     {
2968       \str_if_eq:VnF \l__spintenz_spPoint_mode_str { school }
2969       {
2970         \tl_set:Nn \l__spintenz_spPoint_intent_arg_tl { \_ - }
2971       }
2972     }
2973   }
2974   {
2975     \tl_set:Ne \l__spintenz_spPoint_intent_arg_tl
2976     {
2977       \tl_if_empty:NF \l__spintenz_spPoint_prefix_tl
```

```

2978         {
2979             \tl_use:N \__spintent_spPoint_prefix_tl -
2980         }
2981         punto:prefix($pt)
2982     }
2983 }
2984 \__spintent_spPoint_mathml:n { #1 }
2985 }

```

(End of definition for `\__spintent_spPoint_print:n`.)

`\__spintent_spPoint_mathml:n` The function `\__spintent_spPoint_mathml:n` calls `\__spintent_invisible_sep:` first, then wraps `\__spintent_mathmlarg_point:nnn` — which parses ‘#1’, tags it `arg="pt"`, and silences a real subscript or prime on its own — inside a native `\MathMLintent` carrying `\__spintent_spPoint_intent_arg_tl` as the concept `intent`, with `\__spintent_inner_sep:` first inside that group.

```

2986 \cs_new_protected:Nn \__spintent_spPoint_mathml:n
2987 {
2988     \__spintent_invisible_sep:
2989     \MathMLintent { \__spintent_spPoint_intent_arg_tl }
2990     {
2991         \__spintent_inner_sep:
2992         \__spintent_mathmlarg_point:nnn { pt } { spPoint } { #1 }
2993     }
2994 }

```

(End of definition for `\__spintent_spPoint_mathml:n`.)

`\__spintent_spPoint_exec:nnn` The function `\__spintent_spPoint_exec:nnn` receives the optional `<keys>` in ‘#1’ and the point `{<argument>}` in ‘#2’, delegating to `\__spintent_spPoint_print:n`. With ‘#3’ given (the optional coordinate), silences the gap between the point and the coordinate first (otherwise read as an implicit product), then temporarily aliases `\__spintent_spcoord_silent_cmd_bool` to `\__spintent_spPoint_silent_cmd_bool` and calls `\__spintent_luafun_spcoord_parse_and_set:n` directly, reporting `spcoord-invalid-arg` on error unless silenced, then `\__spintent_spcoord_set_wrap_parens_size:` and `\__spintent_spcoord_math`

```

2995 \cs_new_protected:Nn \__spintent_spPoint_exec:nnn
2996 {
2997     \keys_set:nn { spintent / spPoint } { #1 }
2998     \__spintent_spPoint_print:n { #2 }
2999     \tl_if_novalue:nF {#3}
3000     {
3001         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % silencia el "por"
3002         \bool_set_eq:NN \__spintent_spcoord_silent_cmd_bool \__spintent_spPoint_silent_cmd_bool
3003         \tl_set:Nn \__spintent_spcoord_raw_arg_tl {#3}
3004         \bool_if:NTF \__spintent_spcoord_silent_cmd_bool
3005         {
3006             \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3007             \str_set:Nn \__spintent_spcoord_luaaset_error_str { false }
3008         }
3009         {
3010             \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3011             \str_if_eq:VnT \__spintent_spcoord_luaaset_error_str { true }
3012             { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n {#3} } }
3013         }
3014         \__spintent_spcoord_set_wrap_parens_size:
3015         \str_if_eq:VnF \__spintent_spcoord_luaaset_error_str { true }
3016         { \__spintent_spcoord_mathml: }
3017     }
3018 }

```

(End of definition for `\__spintent_spPoint_exec:nnn`.)

`\spPoint` The public command `\spPoint` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spPoint_exec:nnn`.

```

3019 \NewDocumentCommand \spPoint { 0{} m d() }
3020 {
3021     \mode_if_math:TF
3022     {
3023         \group_begin:
3024         \attribute \__spintent_scope_attr_int = 1
3025         \__spintent_spPoint_exec:nnn { #1 } { #2 } { #3 }
3026         \group_end:
3027     }

```

```

3028     { \msg_error:nnn { spintent } { need-math-mode } { \spPoint } }
3029   }

```

(End of definition for `\spPoint`. This function is documented on page 18.)

11.37 The command `\spvec`

The user command `\spvec` supports two types of notation for the input $\langle argument \rangle$: a pair of points (with `\overrightarrow` applied), or a one-letter name (with `\vec` applied instead). Both notations are automatically detected by the Lua module. An optional coordinate, in parentheses, may follow the $\langle argument \rangle$.

11.37.1 Definition of variables for `\spvec`

The variable `\l__spinttent_spvec_mode_str` stores the value set by the key `read-cmd` and the variable `\l__spinttent_spvec_prefix_tl` stores the value set by the key `prefix`.

```

3030 \str_new:N \l__spinttent_spvec_mode_str
3031 \tl_new:N \l__spinttent_spvec_prefix_tl
3032 \tl_new:N \l__spinttent_spvec_intent_arg_tl
3033 \tl_new:N \l__spinttent_spvec_kv_tl
3034 \seq_new:N \l__spinttent_spvec_pt_seq
3035 \tl_new:N \l__spinttent_spvec_refs_tl
3036 \int_new:N \l__spinttent_spvec_pt_idx_int

```

(End of definition for `\l__spinttent_spvec_mode_str` and others.)

11.37.2 Definition of error messages for `\spvec`

`spvec-invalid-arg` The “error” `spvec-invalid-arg` is raised when the argument fails to parse, when a one-letter name carries a prime, or when the point list is not exactly two points.

```

3037 \msg_new:nnn { spintent } { spvec-invalid-arg }
3038 {
3039   Invalid~argument~'#1'~for~\token_to_str:N \spvec.
3040 }

```

(End of definition for `spvec-invalid-arg`.)

11.37.3 Definition of keys for `\spvec`

We define the *(keys)* `silent-cmd`, `concept`, `autofit`, `read-cmd` and `prefix`.

```

3041 \keys_define:nn { spintent / spvec }
3042 {
3043   silent-cmd .bool_set:N = \l__spinttent_spvec_silent_cmd_bool,
3044   silent-cmd .initial:n = false,
3045   silent-cmd .value_required:n = true,
3046   concept .bool_set:N = \l__spinttent_spvec_concept_bool,
3047   concept .initial:n = true,
3048   concept .value_required:n = true,
3049   autofit .bool_set:N = \l__spinttent_spvec_autofit_bool,
3050   autofit .initial:n = false,
3051   autofit .value_required:n = true,
3052   read-cmd .choices:nn = { school , mathcat }
3053   {
3054     \int_case:nn { \l_keys_choice_int }
3055     {
3056       {1} { \str_set:Nn \l__spinttent_spvec_mode_str { school } }
3057       {2} { \str_set:Nn \l__spinttent_spvec_mode_str { mathcat } }
3058     }
3059   },
3060   read-cmd .initial:n = school,
3061   read-cmd .value_required:n = true,
3062   read-cmd / unknown .code:n =
3063     \msg_error:nneee { spintent } { unknown-choice }
3064     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3065   prefix .code:n =
3066     { \__spinttent_sanitizize_affix:Nn \l__spinttent_spvec_prefix_tl {##1} },
3067   prefix .initial:n = { },
3068   prefix .value_required:n = true,
3069   unknown .code:n =
3070     {
3071       \str_set:Nn \l__spinttent_command_name_str { spvec }
3072       \__spinttent_unknown_keys_cmd:n {##1}
3073     },
3074 }

```

(End of definition for `silent-cmd` and others.)

11.37.4 Internal functions for \spvec

__spinttent_spvec_print:n

The internal function `\__spinttent_spvec_print:n` processes the $\langle argument \rangle$ passed in ‘#1’. Calls `\__spinttent_luafun_geo_parse_and_set:n`, reporting `spvec-invalid-arg` on error, on a proper name with `\l__spinttent_geo_luaset_tipo_str` equal to `prima` (a one-letter vector cannot carry a prime), or on a point list that is not exactly two points. Otherwise calls `\__spinttent_spvec_mathml:n`.

```

3075 \cs_new_protected:Nn \__spinttent_spvec_print:n
3076 {
3077   \__spinttent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3078   \str_if_eq:VnT \l__spinttent_geo_luaset_is_name_str { true }
3079   {
3080     \str_if_eq:VnT \l__spinttent_geo_luaset_tipo_str { prima }
3081     { \str_set:Nn \l__spinttent_geo_luaset_error_str { true } }
3082   }
3083   \str_if_eq:VnT \l__spinttent_geo_luaset_is_name_str { false }
3084   {
3085     \int_compare:nT { \clist_count:N \l__spinttent_geo_luaset_points_clist != 2 }
3086     { \str_set:Nn \l__spinttent_geo_luaset_error_str { true } }
3087   }
3088   \str_if_eq:VnTF \l__spinttent_geo_luaset_error_str { true }
3089   { \msg_error:nne { spinttent } { spvec-invalid-arg } { \exp_not:n {#1} } }
3090   { \__spinttent_spvec_mathml:n { #1 } }
3091 }

```

(End of definition for `\__spinttent_spvec_print:n`.)

__spinttent_spvec_mathml:n

The function `\__spinttent_spvec_mathml:n` branches on `\l__spinttent_geo_luaset_is_name_str`. For a proper name (a single letter, no prime), the concept `intent` is `_( $\$pt$ )` when `silent-cmd` is `true` or `concept` is `false` — forced further to `_-` when `silent-cmd` is `true` and `read-cmd` is not `school`; otherwise `vector:prefix( $\$pt$ )`. With `\l__spinttent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags `\vec{letra}` through `\__spinttent_luafun_mrow_intent_arg:nn` with a literal `kv`, the subscript appended and silenced; `read-cmd=mathcat` tags `\vec{...}` through a native `\MathMLarg` instead, the subscript silenced only under `silent-cmd`. `autofit` inserts a `\vphantom{b}` inside `\vec{...}` to normalize its height. Otherwise (no subscript), the same `kv` logic wraps `\vec{print\_tl}` as a whole.

```

3092 \cs_new_protected:Nn \__spinttent_spvec_mathml:n
3093 {
3094   \str_if_eq:VnTF \l__spinttent_geo_luaset_is_name_str { true }
3095   {
3096     \bool_if:nTF
3097     {
3098       \bool_lazy_or_p:nn
3099       { \l__spinttent_spvec_silent_cmd_bool }
3100       { ! \l__spinttent_spvec_concept_bool }
3101     }
3102     {
3103       \tl_set:Nn \l__spinttent_spvec_intent_arg_tl { _( $\$pt$ ) }
3104       \bool_if:NT \l__spinttent_spvec_silent_cmd_bool
3105       {
3106         \str_if_eq:VnF \l__spinttent_spvec_mode_str { school }
3107         { \tl_set:Nn \l__spinttent_spvec_intent_arg_tl { _- } }
3108       }
3109     }
3110     {
3111       \tl_set:Nn \l__spinttent_spvec_intent_arg_tl { vector:prefix( $\$pt$ ) }
3112     }
3113   }
3114   \str_case:VnF \l__spinttent_geo_luaset_tipo_str
3115   {
3116     { sub }
3117     {
3118       \str_if_eq:VnTF \l__spinttent_spvec_mode_str { school }
3119       {
3120         \bool_if:NTF \l__spinttent_spvec_silent_cmd_bool
3121         { \tl_set:Nn \l__spinttent_spvec_kv_tl { arg = "pt", intent = "_-" } }
3122         {
3123           \tl_set:Ne \l__spinttent_spvec_kv_tl
3124           { arg = "pt", intent = "_ \l__spinttent_geo_luaset_intent_str -" }
3125         }
3126         \__spinttent_invisible_sep:
3127         \MathMLintent { \l__spinttent_spvec_intent_arg_tl }
3128         {
3129           \__spinttent_inner_sep:

```

```

3129         \__spintent_luafun_mrow_intent_arg:nn
3130         { } { \__spintent_sptvec_kv_tl }
3131     {
3132         \vec
3133         {
3134             \bool_if:NT \__spintent_sptvec_autofit_bool
3135             { \vphantom{ b } }
3136             \__spintent_geo_luaset_letra_str
3137         }
3138         \c_math_subscript_token
3139         { \MathMLint { :silent } { \__spintent_geo_luaset_sub_str } }
3140     }
3141 }
3142 }
3143 {
3144     \__spintent_invisible_sep:
3145     \MathMLint { \__spintent_sptvec_intent_arg_tl }
3146     {
3147         \__spintent_inner_sep:
3148         \vec
3149         {
3150             \bool_if:NT \__spintent_sptvec_autofit_bool
3151             { \vphantom{ b } }
3152             \MathMLarg{ pt } { \__spintent_geo_luaset_letra_str }
3153         }
3154         \bool_if:NT \__spintent_sptvec_silent_cmd_bool
3155         { \c_math_subscript_token { \__spintent_geo_luaset_sub_str } }
3156     }
3157     \bool_if:NF \__spintent_sptvec_silent_cmd_bool
3158     { \c_math_subscript_token { \__spintent_geo_luaset_sub_str } }
3159 }
3160 }
3161 }
3162 {
3163     \bool_if:NTF \__spintent_sptvec_silent_cmd_bool
3164     { \tl_set:Nn \__spintent_sptvec_kv_tl { arg = "pt", intent = "_" } }
3165     {
3166         \tl_set:Nn \__spintent_sptvec_kv_tl
3167         { arg = "pt", intent = "_" \__spintent_geo_luaset_intent_str -" }
3168     }
3169     \__spintent_invisible_sep:
3170     \MathMLint { \__spintent_sptvec_intent_arg_tl }
3171     {
3172         \__spintent_inner_sep:
3173         \__spintent_luafun_mrow_intent_arg:nn
3174         { } { \__spintent_sptvec_kv_tl }
3175     }
3176     \vec
3177     {
3178         \bool_if:NT \__spintent_sptvec_autofit_bool
3179         { \vphantom{ b } }
3180         \__spintent_geo_luaset_print_tl
3181     }
3182 }
3183 }
3184 }
3185 }

```

For a pair of points, `\__spintent_geo_luaset_points_clist` is split into `\__spintent_sptvec_pt_seq` and walked to build `\__spintent_sptvec_refs_tl`, the $\$pts1$, $\$pts2$ reference list. The concept intent is `_` (refs) or vector: `prefix(refs)`, same conditions as above, applied with a native `\MathMLint` wrapping `\overrightarrow`. The sequence is walked again, calling `\__spintent_mathmlarg_point:nnn` once per point.

```

3186     {
3187         \seq_set_from_clist:NN \__spintent_sptvec_pt_seq \__spintent_geo_luaset_points_clist
3188         \tl_clear:N \__spintent_sptvec_refs_tl
3189         \int_zero:N \__spintent_sptvec_pt_idx_int
3190         \seq_map_inline:Nn \__spintent_sptvec_pt_seq
3191         {
3192             \int_incr:N \__spintent_sptvec_pt_idx_int
3193             \tl_if_empty:NF \__spintent_sptvec_refs_tl

```

```

3194         { \tl_put_right:Nn \__spintent_svec_refs_tl { , } }
3195     \tl_put_right:Ne \__spintent_svec_refs_tl { $ pts \int_use:N \__spintent_svec_pt_
3196 }
3197 \bool_if:nTF
3198 {
3199     \bool_lazy_or_p:nn
3200     { \__spintent_svec_silent_cmd_bool }
3201     { ! \__spintent_svec_concept_bool }
3202 }
3203 {
3204     \tl_set:Ne \__spintent_svec_intent_arg_tl { _ ( \__spintent_svec_refs_tl ) }
3205 }
3206 {
3207     \tl_set:Ne \__spintent_svec_intent_arg_tl
3208     {
3209         \tl_if_empty:NF \__spintent_svec_prefix_tl
3210         { \tl_use:N \__spintent_svec_prefix_tl - }
3211         vector:prefix( \__spintent_svec_refs_tl )
3212     }
3213 }
3214 \__spintent_invisible_sep:
3215 \MathMLintent { \__spintent_svec_intent_arg_tl }
3216 {
3217     \__spintent_inner_sep:
3218     \overrightarrow
3219     {
3220         \int_zero:N \__spintent_svec_pt_idx_int
3221         \seq_map_inline:Nn \__spintent_svec_pt_seq
3222         {
3223             \int_incr:N \__spintent_svec_pt_idx_int
3224             \__spintent_mathmlarg_point:nnn
3225             { pts \int_use:N \__spintent_svec_pt_idx_int }
3226             { svec }
3227             { ##1 }
3228         }
3229     }
3230 }
3231 }
3232 }

```

(End of definition for `\__spintent_svec_mathml:n`.)

`\__spintent_svec_exec:nnn` The function `\__spintent_svec_exec:nnn` receives the optional *keys* in ‘#1’ and the $\{ \langle argument \rangle \}$ in ‘#2’, delegating to `\__spintent_svec_print:n`. With ‘#3’ given (the optional coordinate), aliases `\__spintent_spcord_silent_cmd_bool` to `\__spintent_svec_silent_cmd_bool` — reverted when the calling group closes — and calls `\__spintent_luafun_spcord_parse_and_set:n` directly, reporting `spcord-invalid-arg` on error unless silenced, then `\__spintent_spcord_set_wrap_parens_size:` and `\__spintent_spcord_mathml:` — the same rendering functions `\spcord` itself uses.

```

3233 \cs_new_protected:Nn \__spintent_svec_exec:nnn
3234 {
3235     \keys_set:nn { spintent / svec } { #1 }
3236     \__spintent_svec_print:n { #2 }
3237     \tl_if_novalue:nF {#3}
3238     {
3239         \equal % notación de vector =( , )
3240         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % previene lectura de intervalo
3241         \bool_set_eq:NN \__spintent_spcord_silent_cmd_bool \__spintent_svec_silent_cmd_bool
3242         \tl_set:Nn \__spintent_spcord_raw_arg_tl {#3}
3243         \bool_if:NTF \__spintent_svec_silent_cmd_bool
3244         {
3245             \__spintent_luafun_spcord_parse_and_set:n { \exp_not:n {#3} }
3246             \str_set:Nn \__spintent_spcord_luaset_error_str { false }
3247         }
3248         {
3249             \__spintent_luafun_spcord_parse_and_set:n { \exp_not:n {#3} }
3250             \str_if_eq:VnT \__spintent_spcord_luaset_error_str { true }
3251             { \msg_error:nnn { spintent } { spcord-invalid-arg } { \exp_not:n {#3} } }
3252         }
3253         \__spintent_spcord_set_wrap_parens_size:
3254         \str_if_eq:VnF \__spintent_spcord_luaset_error_str { true }
3255         { \__spintent_spcord_mathml: }

```

```

3256     }
3257 }

```

(End of definition for `\__spintent_svec_exec:nnn`.)

\spvec The public command `\spvec` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_svec_exec:nnn`.

```

3258 \NewDocumentCommand \spvec { O{} m d() }
3259 {
3260   \mode_if_math:TF
3261   {
3262     \group_begin:
3263     \attribute \__spintent_scope_attr_int = 1
3264     \__spintent_svec_exec:nnn { #1 } { #2 } { #3 }
3265     \group_end:
3266   }
3267   { \msg_error:nnn { spintent } { need-math-mode } { \spvec } }
3268 }

```

(End of definition for `\spvec`. This function is documented on page 18.)

11.38 The command `\sprecta`

The user command `\sprecta` supports two types of notation for the input $\langle argument \rangle$: when the argument contains two points (separated by an optional comma), `\overlefttrightarrow` is applied; when the argument is a single point (without comma) or a lowercase letter, it is formatted without any visual decorator. Both notations are automatically detected by the Lua module.

11.38.1 Definition of variables for `\sprecta`

The variable `\__spintent_sprecta_mode_str` store the values set by the key `read-cmd` and the variable `\__spintent_sprayo_prefix_tl` store the value set by the key `prefix`.

```

3269 \str_new:N \__spintent_sprecta_mode_str
3270 \tl_new:N \__spintent_sprecta_prefix_tl
3271 \seq_new:N \__spintent_sprecta_pt_seq
3272 \tl_new:N \__spintent_sprecta_refs_tl
3273 \tl_new:N \__spintent_sprecta_intent_arg_tl
3274 \tl_new:N \__spintent_sprecta_kv_tl
3275 \int_new:N \__spintent_sprecta_pt_idx_int

```

(End of definition for `\__spintent_sprecta_mode_str` and others.)

11.38.2 Definition of error messages for `\sprecta`

The “error” `sprecta-invalid-points` is raised when the argument matches neither the two-point notation nor the name notation.

```

3276 \msg_new:nnnn { spintent } { sprecta-invalid-points }
3277 {
3278   Invalid~format~in~\token_to_str:N \sprecta:~'#1'~\msg_line_context:.
3279 }
3280 {
3281   See~the~documentation~of~\token_to_str:N \sprecta~for~the~accepted~argument~forms.
3282 }

```

(End of definition for `sprecta-invalid-points`.)

11.38.3 Definition of keys for `\sprecta`

`read-cmd` We define the $\langle keys \rangle$ `read-cmd`, `prefix` and `silent-cmd`.

```

3283 \keys_define:nn { spintent / sprecta }
3284 {
3285   read-cmd .choices:nn = { school , mathcat }
3286   {
3287     \int_case:nn { \l_keys_choice_int }
3288     {
3289       {1} { \str_set:Nn \__spintent_sprecta_mode_str { school } }
3290       {2} { \str_set:Nn \__spintent_sprecta_mode_str { mathcat } }
3291     }
3292   },
3293   read-cmd .initial:n = school,
3294   read-cmd .value_required:n = true,
3295   read-cmd / unknown .code:n =
3296     \msg_error:nnee { spintent } { unknown-choice }
3297     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3298   prefix .code:n =

```

```

3299     { \__spintent_sanitiz_e_affix:Nn \__spintent_sprecta_prefix_tl {#1} },
3300     prefix                .initial:n = ,
3301     prefix                .value_required:n = true,
3302     silent-cmd            .bool_set:N = \__spintent_sprecta_silent_cmd_bool,
3303     silent-cmd            .initial:n = false,
3304     silent-cmd            .value_required:n = true,
3305     concept               .bool_set:N = \__spintent_sprecta_concept_bool,
3306     concept               .initial:n = true,
3307     concept               .value_required:n = true,
3308     unknown               .code:n =
3309     {
3310         \str_set:Nn \__spintent_command_name_str { sprecta }
3311         \__spintent_unknown_keys_cmd:n {#1}
3312     },
3313 }

```

(End of definition for read-cmd and others.)

11.38.4 Internal functions for \sprecta

__spintent_sprecta_print:n

The internal function `\__spintent_sprecta_print:n` processes the `{\argument}` passed in ‘#1’. Calls `\__spintent_luafun_geo_parse_and_set:n`, reporting sprecta-invalid-points on error, then `\__spintent_`

```

3314 \cs_new_protected:Nn \__spintent_sprecta_print:n
3315 {
3316     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3317     \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
3318     {
3319         \msg_error:nne { spintent } { sprecta-invalid-points } { \exp_not:n {#1} }
3320     }
3321     \__spintent_sprecta_mathml:n { #1 }
3322 }

```

(End of definition for __spintent_sprecta_print:n.)

__spintent_sprecta_mathml:n

The function `\__spintent_sprecta_mathml:n` branches on `\__spintent_geo_luaset_is_name_str`. For a proper name (m, l, ...), the concept `intent` is `:silent($pt)` when `silent-cmd` is `true` or `concept` is `false`; otherwise `recta:prefix($pt)`, prefixed with `prefix` when given.

With `\__spintent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags the base through `\__spintent_luafun_mrow_intent_arg:nn` with a literal kv, the subscript appended and silenced; `read-cmd=mathcat` tags base and subscript together through a native `\MathMLarg` instead.

```

3323 \cs_new_protected:Nn \__spintent_sprecta_mathml:n
3324 {
3325     \str_if_eq:VnTF \__spintent_geo_luaset_is_name_str { true }
3326     {
3327         \__spintent_invisible_sep:
3328         \bool_if:nTF
3329         {
3330             \bool_lazy_or_p:nn
3331             { \__spintent_sprecta_silent_cmd_bool }
3332             { ! \__spintent_sprecta_concept_bool }
3333         }
3334         {
3335             \tl_set:Nn \__spintent_sprecta_intent_arg_tl { :silent($pt) }
3336         }
3337         {
3338             \tl_set:Ne \__spintent_sprecta_intent_arg_tl
3339             {
3340                 \tl_if_empty:NF \__spintent_sprecta_prefix_tl
3341                 { \tl_use:N \__spintent_sprecta_prefix_tl - }
3342                 recta:prefix($pt)
3343             }
3344         }
3345     }
3346     \str_case:VnF \__spintent_geo_luaset_tipo_str
3347     {
3348         { sub }
3349         {
3350             \str_if_eq:VnTF \__spintent_sprecta_mode_str { school }
3351             {
3352                 \tl_set:Ne \__spintent_sprecta_kv_tl
3353                 { arg = "pt", intent = "- \__spintent_geo_luaset_intent_str -" }
3354                 \MathMLintent { \__spintent_sprecta_intent_arg_tl }
3355             }

```

```

3355         \l__spintent_inner_sep:
3356         \l__spintent_luafun_mrow_intent_arg:nn
3357         { } { \l__spintent_sprecta_kv_tl }
3358         {
3359             \l__spintent_geo_luaset_letra_str
3360             \c_math_subscript_token
3361             { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
3362         }
3363     }
3364 }

```

For `read-cmd=mathcat`, the same sub case tags base and subscript together through a native `\MathMLarg` instead — never mixed with the shared queue, since the queue does not recognize a native `\MathMLarg` claim as already taken.

```

3365     {
3366         \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3367         {
3368             {
3369                 \l__spintent_inner_sep:
3370                 \MathMLarg { pt }
3371                 {
3372                     {
3373                         \l__spintent_geo_luaset_letra_str
3374                         \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
3375                     }
3376                 }
3377             }
3378         }
3379     }
3380 }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the base with the literal kv and appends the mark, unsilenced (a prime is not spelled out on the base, so nothing to repeat); `read-cmd=mathcat` tags base and mark together through a native `\MathMLarg`.

```

3381     { prima }
3382     {
3383         \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
3384         {
3385             \tl_set:Ne \l__spintent_sprecta_kv_tl
3386             { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3387             \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3388             {
3389                 \l__spintent_inner_sep:
3390                 \l__spintent_luafun_mrow_intent_arg:nn
3391                 { } { \l__spintent_sprecta_kv_tl }
3392                 { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
3393             }
3394         }
3395     }
3396     \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3397     {
3398         {
3399             \l__spintent_inner_sep:
3400             \MathMLarg { pt }
3401             {
3402                 { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
3403             }
3404         }
3405     }
3406 }
3407 }
3408 }

```

Otherwise (no subscript, no prime), the same kv logic wraps the combined `\l__spintent_geo_luaset_print_tl` as a whole — the shared queue for `school`, its own `arg` alone for `mathcat`.

```

3409     {
3410         \bool_if:NTF \l__spintent_sprecta_silent_cmd_bool
3411         {
3412             \tl_set:Nn \l__spintent_sprecta_kv_tl { arg = "pt", intent = "-_" }
3413         }
3414         {
3415             \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }

```

```

3416         {
3417             \tl_set:Nc \l__spintent_sprecta_kv_tl
3418                 { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3419         }
3420         {
3421             \tl_set:Nn \l__spintent_sprecta_kv_tl { arg = "pt" }
3422         }
3423     }
3424     \__spintent_luafun_mrow_intent_arg:nn
3425     { \l__spintent_sprecta_intent_arg_tl }
3426     { \l__spintent_sprecta_kv_tl }
3427     { \l__spintent_geo_luaset_print_tl }
3428 }
3429 }

```

For a list of points, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_sprecta_pt_seq` and walked to build `\l__spintent_sprecta_refs_tl`, the `$pt1, $pt2, ...` reference list. The concept `intent` is `_(refs)` or `recta:prefix(refs)`, same conditions as above, applied with a native `\MathMLintent` wrapping `\overleftrightharrow`. The sequence is walked again, calling `\__spintent_mathmlarg` once per point.

```

3430 {
3431     \seq_set_from_clist:NN \l__spintent_sprecta_pt_seq \l__spintent_geo_luaset_points_clist
3432     \tl_clear:N \l__spintent_sprecta_refs_tl
3433     \int_zero:N \l__spintent_sprecta_pt_idx_int
3434     \seq_map_inline:Nn \l__spintent_sprecta_pt_seq
3435     {
3436         \int_incr:N \l__spintent_sprecta_pt_idx_int
3437         \tl_if_empty:NF \l__spintent_sprecta_refs_tl
3438         { \tl_put_right:Nn \l__spintent_sprecta_refs_tl { , } }
3439         \tl_put_right:Nc \l__spintent_sprecta_refs_tl { $ pt \int_use:N \l__spintent_sprecta_
3440     }
3441     \bool_if:nTF
3442     {
3443         \bool_lazy_or_p:nn
3444         { \l__spintent_sprecta_silent_cmd_bool }
3445         { ! \l__spintent_sprecta_concept_bool }
3446     }
3447     {
3448         \tl_set:Nc \l__spintent_sprecta_intent_arg_tl { _ ( \l__spintent_sprecta_refs_tl ) }
3449     }
3450     {
3451         \tl_set:Nc \l__spintent_sprecta_intent_arg_tl
3452         {
3453             \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
3454             { \tl_use:N \l__spintent_sprecta_prefix_tl - }
3455             recta:prefix( \l__spintent_sprecta_refs_tl )
3456         }
3457     }
3458     \__spintent_invisible_sep:
3459     \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3460     {
3461         \__spintent_inner_sep:
3462         \overleftrightharrow
3463         {
3464             \int_zero:N \l__spintent_sprecta_pt_idx_int
3465             \seq_map_inline:Nn \l__spintent_sprecta_pt_seq
3466             {
3467                 \int_incr:N \l__spintent_sprecta_pt_idx_int
3468                 \__spintent_mathmlarg_point:nnn
3469                 { pt \int_use:N \l__spintent_sprecta_pt_idx_int }
3470                 { sprecta }
3471                 { ##1 }
3472             }
3473         }
3474     }
3475 }
3476 }

```

(End of definition for `\__spintent_sprecta_mathml:n`)

`\__spintent_sprecta_exec:nn` The function `\__spintent_sprecta_exec:nn` receives the optional `(keys)` passed in ‘#1’ and the `{(argu-`

`ment\}` passed in ‘#2’ delegating the rendering of the latter to `\__spintent_sprecta_print:n`.

```

3477 \cs_new_protected:Nn \__spintent_sprecta_exec:nn
3478 {
3479   \keys_set:nn { spintent / sprecta } { #1 }
3480   \__spintent_sprecta_print:n { #2 }
3481 }

```

(End of definition for `\__spintent_sprecta_exec:nn`.)

`\sprecta` The public command `\sprecta` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sprecta_exec:nn`.

```

3482 \NewDocumentCommand \sprecta { 0{} m }
3483 {
3484   \mode_if_math:TF
3485   {
3486     \group_begin:
3487     \attribute \__spintent_scope_attr_int = 1
3488     \__spintent_sprecta_exec:nn { #1 } { #2 }
3489     \group_end:
3490   }
3491   { \msg_error:nnn { spintent } { need-math-mode } { \sprecta } }
3492 }

```

(End of definition for `\sprecta`. This function is documented on page 19.)

11.39 The command `\sprayo`

The user command `\sprayo` provides a semantic interface for a ray. The key `read-sty` selects the verbalized term, *rayo* or *semirrecta*, both rendered identically with `\overrightarrow`.

11.39.1 Definition of variables for `\sprayo`

The variable `\__spintent_sprayo_mode_str` store the values set by the key `read-cmd`; the variable `\__spintent_sprayo_prefix_tl` store the value set by the key `prefix` and the variable `\__spintent_sprayo_read_sty_str` stores the value set by the key `read-sty`.

🍷 The variables defined here are assigned using `.choices:n` or `.code:n` within `\keys_define:nn`.

```

3493 \str_new:N \__spintent_sprayo_mode_str
3494 \tl_new:N \__spintent_sprayo_prefix_tl
3495 \str_new:N \__spintent_sprayo_read_sty_str

```

(End of definition for `\__spintent_sprayo_mode_str`, `\__spintent_sprayo_prefix_tl`, and `\__spintent_sprayo_read_sty_str`.)

11.39.2 Definition of error messages for `\sprayo`

sprayo-invalid-points

Raised when the argument is not a comma-separated pair of uppercase letters.

```

3496 \msg_new:nnnn { spintent } { sprayo-invalid-points }
3497 {
3498   Invalid~point~format~in~\token_to_str:N \sprayo:~'#1'~\msg_line_context:.
3499 }
3500 {
3501   See~the~documentation~of~\token_to_str:N \sprayo~for~the~accepted~point~formats.
3502 }

```

(End of definition for `sprayo-invalid-points`.)

11.39.3 Definition of keys for `\sprayo`

read-cmd
prefix
silent-cmd
read-sty
unknown

We define the `(keys)` `read-cmd`, `prefix`, `silent-cmd`, `concept` and `read-sty`.

```

3503 \keys_define:nn { spintent / sprayo }
3504 {
3505   read-cmd .choices:nn = { school , mathcat }
3506   {
3507     \int_case:nn { \l_keys_choice_int }
3508     {
3509       {1} { \str_set:Nn \__spintent_sprayo_mode_str { school } }
3510       {2} { \str_set:Nn \__spintent_sprayo_mode_str { mathcat } }
3511     }
3512   },
3513   read-cmd .initial:n = school,
3514   read-cmd .value_required:n = true,
3515   read-cmd / unknown .code:n =
3516     \msg_error:nnee { spintent } { unknown-choice }
3517     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3518   prefix .code:n =

```

```

3519     { \__spintent_sanitizе_affix:Nn \l__spintent_sprayo_prefix_tl {#1} },
3520     prefix                .initial:n = ,
3521     prefix                .value_required:n = true,
3522     silent-cmd            .bool_set:N = \l__spintent_sprayo_silent_cmd_bool,
3523     silent-cmd            .initial:n = false,
3524     silent-cmd            .value_required:n = true,
3525     concept               .bool_set:N = \l__spintent_sprayo_concept_bool,
3526     concept               .initial:n = true,
3527     concept               .value_required:n = true,
3528     read-sty              .choices:nn = { rayo , semirrecta }
3529     {
3530         \int_case:nn { \l_keys_choice_int }
3531         {
3532             {1} { \str_set:Nn \l__spintent_sprayo_read_sty_str { rayo } }
3533             {2} { \str_set:Nn \l__spintent_sprayo_read_sty_str { semirrecta } }
3534         }
3535     },
3536     read-sty              .initial:n = rayo,
3537     read-sty              .value_required:n = true,
3538     read-sty / unknown    .code:n =
3539         \msg_error:nneee { spintent } { unknown-choice }
3540         { read-sty } { rayo,~semirrecta } { \exp_not:n {#1} },
3541     unknown               .code:n =
3542     {
3543         \str_set:Nn \l__spintent_command_name_str { sprayo }
3544         \__spintent_unknown_keys_cmd:n {#1}
3545     },
3546 }

```

(End of definition for read-cmd and others.)

11.39.4 Internal functions for \sprayo

Auxiliary variables used by __spintent_sprayo_mathml: to split \l__spintent_geo_luaset_points_clist and build the \$pts1, \$pts2, ... reference list.

```

\l__spintent_sprayo_pt_seq
\l__spintent_sprayo_refs_tl
\l__spintent_sprayo_intent_arg_tl
\l__spintent_sprayo_pt_idx_int
3547 \seq_new:N \l__spintent_sprayo_pt_seq
3548 \tl_new:N \l__spintent_sprayo_refs_tl
3549 \tl_new:N \l__spintent_sprayo_intent_arg_tl
3550 \int_new:N \l__spintent_sprayo_pt_idx_int

```

(End of definition for \l__spintent_sprayo_pt_seq and others.)

__spintent_sprayo_print:n The internal function __spintent_sprayo_print:n process the {<argument>} passed in ‘#1’. It calls __spintent_luafun_geo_parse_and_set:n to process the {<argument>}, returning an “error” if the input is invalid, then calls __spintent_sprayo_mathml:. Unlike \sprecta, \sprayo has no proper-name argument: it always takes a list of points, so no intent is built here.

```

3551 \cs_new_protected:Nn \__spintent_sprayo_print:n
3552 {
3553     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3554     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
3555     {
3556         \msg_error:nne { spintent } { sprayo-invalid-points } { \exp_not:n {#1} }
3557     }
3558     \__spintent_sprayo_mathml:
3559 }

```

(End of definition for __spintent_sprayo_print:n.)

__spintent_sprayo_mathml: The function __spintent_sprayo_mathml: splits \l__spintent_geo_luaset_points_clist (set by Lua, one raw point per element) into \l__spintent_sprayo_pt_seq and walks it once to build \l__spintent_sprayo_refs_tl, the comma-separated \$pts1, \$pts2, ... reference list, naming each point by position.

The concept intent is _(refs) when silent-cmd is true or concept is false; otherwise it is \l__spintent_sprayo_read_sty_str:prefix(refs) — the concept word itself (rayo or semirrecta) comes from the read-sty key, not a literal, since \sprayo verbalizes under two names — prefixed with prefix when given. It is applied with a native \MathMLintent wrapping \overrightarrow, preceded by __spintent_inner_sep: (a second sibling so the sole <mrow> does not collapse into <math>). The sequence is walked a second time, calling __spintent_mathmlarg_point:nnn once per point with its ptsn reference name and silent-cmd/read-cmd looked up under sprayo: it alone decides arg/literal intent per point, so concept plays no role at that level.

```

3560 \cs_new_protected:Nn \__spintent_sprayo_mathml:

```

```

3561 {
3562   \seq_set_from_clist:NN \l__spintent_sprayo_pt_seq \l__spintent_geo_luaset_points_clist
3563   \tl_clear:N \l__spintent_sprayo_refs_tl
3564   \int_zero:N \l__spintent_sprayo_pt_idx_int
3565   \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
3566   {
3567     \int_incr:N \l__spintent_sprayo_pt_idx_int
3568     \tl_if_empty:NF \l__spintent_sprayo_refs_tl
3569     { \tl_put_right:Nn \l__spintent_sprayo_refs_tl { , } }
3570     \tl_put_right:Ne \l__spintent_sprayo_refs_tl { $ pts \int_use:N \l__spintent_sprayo_pt_idx_int }
3571   }
3572   \bool_if:nTF
3573   {
3574     \bool_lazy_or_p:nn
3575     { \l__spintent_sprayo_silent_cmd_bool }
3576     { ! \l__spintent_sprayo_concept_bool }
3577   }
3578   {
3579     \tl_set:Ne \l__spintent_sprayo_intent_arg_tl { _ ( \l__spintent_sprayo_refs_tl ) }
3580   }
3581   {
3582     \tl_set:Ne \l__spintent_sprayo_intent_arg_tl
3583     {
3584       \tl_if_empty:NF \l__spintent_sprayo_prefix_tl
3585       { \tl_use:N \l__spintent_sprayo_prefix_tl - }
3586       \l__spintent_sprayo_read_sty_str :prefix( \l__spintent_sprayo_refs_tl )
3587     }
3588   }
3589   \__spintent_invisible_sep:
3590   \MathMLintent { \l__spintent_sprayo_intent_arg_tl }
3591   {
3592     \__spintent_inner_sep:
3593     \overrightarrow
3594     {
3595       \int_zero:N \l__spintent_sprayo_pt_idx_int
3596       \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
3597       {
3598         \int_incr:N \l__spintent_sprayo_pt_idx_int
3599         \__spintent_mathmlarg_point:nnn
3600         { pts \int_use:N \l__spintent_sprayo_pt_idx_int }
3601         { sprayo }
3602         { ##1 }
3603       }
3604     }
3605   }
3606 }

```

(End of definition for __spintent_sprayo_mathml:.)

__spintent_sprayo_exec:nn The function __spintent_sprayo_exec:nn receives the optional $\langle keys \rangle$ passed in ‘#1’ and the $\{\langle argument \rangle\}$ passed in ‘#2’ delegating the rendering of the latter to __spintent_sprayo_print:n.

```

3607 \cs_new_protected:Nn \__spintent_sprayo_exec:nn
3608 {
3609   \keys_set:nn { spintent / sprayo } { #1 }
3610   \__spintent_sprayo_print:n { #2 }
3611 }

```

(End of definition for __spintent_sprayo_exec:nn.)

\sprayo The public command \sprayo enforces *math mode*, opens a local group, and delegates execution to __spintent_sprayo_exec:nn.

```

3612 \NewDocumentCommand \sprayo { 0{ } m }
3613 {
3614   \mode_if_math:TF
3615   {
3616     \group_begin:
3617     \attribute \l__spintent_scope_attr_int = 1
3618     \__spintent_sprayo_exec:nn { #1 } { #2 }
3619     \group_end:
3620   }
3621   { \msg_error:nnn { spintent } { need-math-mode } { \sprayo } }
3622 }

```

(End of definition for `\sprayo`. This function is documented on page 19.)

11.40 The commands `\spside` and `\spmside`

The user commands `\spside` and `\spmside` provide a semantic interface for a side, segment or trace determined by two points. The key `read-sty` selects the verbalized term — `lado`, `segmento`, or `trazo`, the latter used in straightedge-and-compass constructions — and the key `read-arg` overrides that term entirely with a free-form string. Additionally, the command `\spmside` provides the key `print-m`, controlling whether `\operatorname{m}` is typeset before the argument.

11.40.1 Definition of variables for `\spside` and `\spmside`

Each pair of variables is declared once for both `\spside` and `\spmside` via `\clist_map_inline:nn`. The variables `\l__spinttent_spside_mode_str` and `\l__spinttent_spside_mode_str` store the values set by the key `read-cmd`; the variables `\l__spinttent_spside_prefix_tl` and `\l__spinttent_spmside_prefix_tl` store the values set by the key `prefix`.

The variables `\l__spinttent_spside_read_sty_str` and `\l__spinttent_spmside_read_sty_str` stores the value set by the key `read-sty`.

The variables `\l__spinttent_spside_read_arg_tl` and `\l__spinttent_spmside_read_arg_tl` stores the value set by the key `read-arg`.

- The variables are defined here as they are assigned using `.choices:nn` or `.code:n` rather than a direct property within `\keys_define:nn`.

```
3623 \clist_map_inline:nn { side , mside }
3624 {
3625   \str_new:c { l__spinttent_sp #1 _mode_str      }
3626   \tl_new:c { l__spinttent_sp #1 _prefix_tl     }
3627   \str_new:c { l__spinttent_sp #1 _read_sty_str }
3628   \tl_new:c { l__spinttent_sp #1 _read_arg_tl  }
3629 }
```

(End of definition for `\l__spinttent_spside_mode_str` and others.)

11.40.2 Definition of error messages for `\spside` and `\spmside`

We define an error message for the commands `\spside` and `\spmside`. The “error” is triggered if the user includes a invalid “*point*” in the `{\argument}` passed in ‘`#1`’ to the command.

```
3630 \clist_map_inline:nn { side , mside }
3631 {
3632   \msg_new:nnnn { spinttent } { sp #1 -invalid-points }
3633   {
3634     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
3635   }
3636   {
3637     See~the~documentation~of~##1~for~the~accepted~point~formats.
3638   }
3639 }
```

(End of definition for `spside-invalid-points` and `spmside-invalid-points`.)

11.40.3 Definition of keys for `\spside` and `\spmside`

We define the shared `\keys` `read-cmd`, `prefix`, `silent-cmd`, `concept`, `read-sty`, `read-arg` and `print-m`.

```
read-cmd 3640 \clist_map_inline:nn { side , mside }
prefix   3641 {
silent-cmd 3642   \keys_define:nn { spinttent / sp #1 }
concept    3643   {
read-sty   3644     read-cmd .choices:nn = { school , mathcat }
read-arg   3645     {
print-m    3646       \int_case:nn { \l_keys_choice_int }
unknown    3647       {
3648         {1} { \str_set:cn { l__spinttent_sp #1 _mode_str } { school } }
3649         {2} { \str_set:cn { l__spinttent_sp #1 _mode_str } { mathcat } }
3650       }
3651     },
3652     read-cmd .initial:n = school,
3653     read-cmd .value_required:n = true,
3654     read-cmd / unknown .code:n =
3655       \msg_error:nnee { spinttent } { unknown-choice }
3656       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3657     prefix .code:n =
3658       { \__spinttent_sanitizize_affix:cn { l__spinttent_sp #1 _prefix_tl } {##1} },
3659     prefix .initial:n = ,
3660     prefix .value_required:n = true,
```

```

3661     silent-cmd          .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
3662     silent-cmd          .initial:n  = false,
3663     silent-cmd          .value_required:n = true,
3664     concept             .bool_set:c = { l__spintent_sp #1 _concept_bool },
3665     concept             .initial:n  = true,
3666     concept             .value_required:n = true,
3667     read-sty            .choices:nn = { lado , segmento , trazo }
3668     {
3669         \int_case:nn { \l_keys_choice_int }
3670         {
3671             {1} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { lado      } }
3672             {2} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { segmento } }
3673             {3} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { trazo     } }
3674         }
3675     },
3676     read-sty            .initial:n  = lado,
3677     read-sty            .value_required:n = true,
3678     read-sty / unknown .code:n =
3679         \msg_error:nnee { spintent } { unknown-choice }
3680         { read-sty } { lado,~segmento,~trazo } { \exp_not:n {##1} },
3681     read-arg            .code:n =
3682     {
3683         \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1}
3684     },
3685     read-arg            .initial:n  = ,
3686     read-arg            .value_required:n = true,
3687     print-m             .bool_set:c = { l__spintent_sp #1 _print_m_bool },
3688     print-m             .initial:n  = false,
3689     print-m             .value_required:n = true,
3690     unknown             .code:n =
3691     {
3692         \str_set:Nn \l__spintent_command_name_str { sp #1 }
3693         \__spintent_unknown_keys_cmd:n {##1}
3694     },
3695 }
3696 }

```

• The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spside`.

(End of definition for `read-cmd` and others.)

`\l__spintent_spside_intent_tl` `\l__spintent_spside_pt_seq` `\l__spintent_spside_refs_tl` `\l__spintent_spside_intent_arg_tl` `\l__spintent_spside_pt_idx_int` `\l__spintent_spside_intent_tl` holds the concept word (`read-arg` or `read-sty`). The rest are scratch variables shared by both commands (the function itself is shared) to split `\l__spintent_geo_luaset_points_clist` and build the `$pts1`, `$pts2`, ... reference list.

```

3697 \tl_new:N \l__spintent_spside_intent_tl
3698 \seq_new:N \l__spintent_spside_pt_seq
3699 \tl_new:N \l__spintent_spside_refs_tl
3700 \tl_new:N \l__spintent_spside_intent_arg_tl
3701 \int_new:N \l__spintent_spside_pt_idx_int

```

(End of definition for `\l__spintent_spside_intent_tl` and others.)

`\__spintent_spside_process:nn` The internal function `\__spintent_spside_process:nn` receives the short name of the command (side or mside) in ‘#1’ and the ‘{*argument*}’ in ‘#2’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘#2’, returning an “error” if invalid, then builds `\l__spintent_spside_intent_tl`. When `read-arg` is empty, it is set from `read-sty`, prefixed with ‘medida-del-’ for `\spmside`. When `read-arg` is given, it is used as is, and ‘medida-del-’ is prepended only if `concept` is `true`: with `concept=false`, `read-arg` is the verbatim escape hatch for a full user-written phrase (e.g. `read-arg={medida de la hipotenusa}`), gender and article left to the user. No `intent` is built here: it needs the reference list, which `\__spintent_spside_math` builds after splitting the points.

```

3702 \cs_new_protected:Nn \__spintent_spside_process:nn
3703 {
3704     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
3705     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
3706     {
3707         \msg_error:nnee { spintent } { sp #1 -invalid-points }
3708         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3709     }
3710     \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
3711     {
3712         \tl_set:Ne \l__spintent_spside_intent_tl { \str_use:c { l__spintent_sp #1 _read_sty_str }
3713         \str_if_eq:eeT {#1} { mside }

```

```

3714         {
3715             \tl_put_left:Nn \l__spintent_spside_intent_tl { medida-del- }
3716         }
3717     }
3718     {
3719         \tl_set:Ne \l__spintent_spside_intent_tl { \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
3720         \bool_if:cT { l__spintent_sp #1 _concept_bool }
3721         {
3722             \str_if_eq:eeT {#1} { mside }
3723             {
3724                 \tl_put_left:Nn \l__spintent_spside_intent_tl { medida-del- }
3725             }
3726         }
3727     }
3728     \__spintent_spside_mathml:n { #1 }
3729 }

```

(End of definition for `\__spintent_spside_process:nn`.)

`\__spintent_spside_mathml:n`

The function `\__spintent_spside_mathml:n` receives the short command name in ‘#1’. It splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spside_pt_seq` and walks it once to build `\l__spintent_spside_refs_tl`, the comma-separated `$pts1,$pts2,...` reference list, naming each point by position.

```

3730 \cs_new_protected:Nn \__spintent_spside_mathml:n
3731 {
3732     \seq_set_from_clist:NN \l__spintent_spside_pt_seq \l__spintent_geo_luaset_points_clist
3733     \tl_clear:N \l__spintent_spside_refs_tl
3734     \int_zero:N \l__spintent_spside_pt_idx_int
3735     \seq_map_inline:Nn \l__spintent_spside_pt_seq
3736     {
3737         \int_incr:N \l__spintent_spside_pt_idx_int
3738         \tl_if_empty:NF \l__spintent_spside_refs_tl
3739         { \tl_put_right:Nn \l__spintent_spside_refs_tl { , } }
3740         \tl_put_right:Ne \l__spintent_spside_refs_tl { $ pts \int_use:N \l__spintent_spside_pt_idx_int
3741     }

```

The concept `intent` is `_ (refs)` when `silent-cmd` is `true`; otherwise, if `read-arg` is empty, it is `_ (refs)` when `concept` is `false` or `\l__spintent_spside_intent_tl:prefix(refs)` when `true`; if `read-arg` is given, it is always `\l__spintent_spside_intent_tl:prefix(refs)` — concept never silences a user-given `read-arg` (see `\__spintent_spside_process:nn`). The `:prefix(...)` form is prefixed with `prefix` when given.

```

3742     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
3743     {
3744         \tl_set:Ne \l__spintent_spside_intent_arg_tl { _ ( \l__spintent_spside_refs_tl ) }
3745     }
3746     {
3747         \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
3748         {
3749             \bool_if:cTF { l__spintent_sp #1 _concept_bool }
3750             {
3751                 \tl_set:Ne \l__spintent_spside_intent_arg_tl
3752                 {
3753                     \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3754                     { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
3755                     \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
3756                 }
3757             }
3758             {
3759                 \tl_set:Ne \l__spintent_spside_intent_arg_tl { _ ( \l__spintent_spside_refs_tl ) }
3760             }
3761         }
3762         {
3763             \tl_set:Ne \l__spintent_spside_intent_arg_tl
3764             {
3765                 \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3766                 { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
3767                 \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
3768             }
3769         }
3770     }

```

Applied with a native `\MathMLintent`, preceded by `\__spintent_inner_sep`: (a second sibling so the sole `<mrow>` does not collapse into `<math>`. Inside, for `#1=side` the points are always wrapped in `\overline`; for `#1=mside`, when `print-m` is `true` a silent `\operatorname{m}` precedes the `\overline`-wrapped points; when `false`, the points are typeset bare, with no `\overline` at all — the recursive lookup in the shared `mathml_filter` callback finds each point either way, whether nested inside `\overline` or sitting directly in the `\MathMLintent` group. In every case the sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptsn` reference name and `silent-cmd/read-cmd` looked up under `sp#1`: it alone decides `arg`/literal `intent` per point, so `concept` plays no role at that level.

```

3771 \__spintent_invisible_sep: % (luamml issues #26)
3772 \MathMLintent { \__spintent_spside_intent_arg_tl }
3773 {
3774   \__spintent_inner_sep:
3775   \str_if_eq:eeTF {#1} { side }
3776   {
3777     \overline { \__spintent_spside_points_loop:n {#1} }
3778   }
3779   {
3780     \bool_if:cTF { l__spintent_sp #1 _print_m_bool }
3781     {
3782       \__spintent_luafun_mrow_intent_arg:nn
3783       { } { intent = ":silent" }
3784       { \operatorname { m } }
3785       \overline { \__spintent_spside_points_loop:n {#1} }
3786     }
3787     {
3788       \__spintent_spside_points_loop:n {#1}
3789     }
3790   }
3791 }
3792 }

```

(End of definition for `\__spintent_spside_mathml:n`.)

`\__spintent_spside_points_loop:n`

Walks `\__spintent_spside_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per point. Factored out of `\__spintent_spside_mathml:n` since it is needed identically in three places there (`side`, `mside` with and without `print-m`); it takes the short command name in `#1` again since a separate macro does not inherit its caller's own `#1`.

```

3793 \cs_new_protected:Nn \__spintent_spside_points_loop:n
3794 {
3795   \int_zero:N \__spintent_spside_pt_idx_int
3796   \seq_map_inline:Nn \__spintent_spside_pt_seq
3797   {
3798     \int_incr:N \__spintent_spside_pt_idx_int
3799     \__spintent_mathmlarg_point:nnn
3800     { pts \int_use:N \__spintent_spside_pt_idx_int }
3801     { sp #1 }
3802     { ##1 }
3803   }
3804 }

```

(End of definition for `\__spintent_spside_points_loop:n`.)

`\__spintent_spside_exec:nnn`

The function `\__spintent_spside_exec:nnn` receives the *short name* of the command passed in `#1`, the optional `<keys>` passed in `#2` and the `<argument>` passed in `#3`, delegating the rendering to `\__spintent_spside_process:nn`.

```

3805 \cs_new_protected:Nn \__spintent_spside_exec:nnn
3806 {
3807   \keys_set:nn { spintent / sp #1 } { #2 }
3808   \__spintent_spside_process:nn { #1 } { #3 }
3809 }

```

(End of definition for `\__spintent_spside_exec:nnn`.)

`\spside`
`\spmside`

Both public commands `\spside` and `\spmside` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spside_exec:nnn`.

```

3810 \NewDocumentCommand \spside { O{} m }
3811 {
3812   \mode_if_math:TF
3813   {

```

```

3814     \group_begin:
3815         \attribute \l__spintent_scope_attr_int = 1
3816         \__spintent_spside_exec:nnn { side } { #1 } { #2 }
3817     \group_end:
3818 }
3819 { \msg_error:nnn { spintent } { need-math-mode } { \spside } }
3820 }
3821 \NewDocumentCommand \spmside { 0{} m }
3822 {
3823     \mode_if_math:TF
3824     {
3825         \group_begin:
3826             \attribute \l__spintent_scope_attr_int = 1
3827             \__spintent_spside_exec:nnn { mside } { #1 } { #2 }
3828         \group_end:
3829     }
3830     { \msg_error:nnn { spintent } { need-math-mode } { \spmside } }
3831 }

```

(End of definition for `\spside` and `\spmside`. These functions are documented on page 20.)

11.41 The commands `\spangle` and `\spmangle`

The user commands `\spangle` and `\spmangle`, provide a semantic interface for angles, following the same generic pattern as `\spside/\spmside`. The key `print-m` chooses between the `\angle/\rightanglesqr` pair, typeset together with an explicit `\operatorname{m}`, and the `\measuredangle/\measuredrightangle` pair, which already conveys the notion of measure on its own; the key `recto` independently chooses the right-angle variant of whichever pair is active.

11.41.1 Definition of variables for `\spangle` and `\spmangle`

```

\l__spintent_spangle_mode_str
\l__spintent_spmangle_mode_str
\l__spintent_spangle_prefix_tl
\l__spintent_spmangle_prefix_tl

```

Each pair of variables is declared once for both `\spangle` and `\spmangle` via `\clist_map_inline:nn`. The variables `\l__spintent_spangle_mode_str` and `\l__spintent_spmangle_mode_str` store the values set by the key `read-cmd`; the variables `\l__spintent_spangle_prefix_tl` and `\l__spintent_spmangle_prefix_tl` store the values set by the key `prefix`.

• The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```

3832 \clist_map_inline:nn { angle , mangle }
3833 {
3834     \str_new:c { l__spintent_sp #1 _mode_str }
3835     \tl_new:c { l__spintent_sp #1 _prefix_tl }
3836 }

```

(End of definition for `\l__spintent_spangle_mode_str` and others.)

11.41.2 Definition of error messages for `\spangle` and `\spmangle`

```

spangle-invalid-arg
spmangle-invalid-arg

```

We define an “error” message for the commands `\spangle` and `\spmangle`. The “error” is triggered if the user includes a `invalid` {`\langle argument \rangle`} passed in ‘`#1`’ to the command.

```

3837 \clist_map_inline:nn { angle , mangle }
3838 {
3839     \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
3840     {
3841         Invalid~format~in~##1:~'##2'~\msg_line_context:.
3842     }
3843     {
3844         See~the~documentation~of~##1~for~the~accepted~argument~forms.
3845     }
3846 }

```

(End of definition for `spangle-invalid-arg` and `spmangle-invalid-arg`.)

11.41.3 Definition of keys for `\spangle` and `\spmangle`

```

read-cmd
prefix
recto
print-m
silent-cmd
concept
unknown

```

We define the shared `\keys` `read-cmd`, `prefix`, `recto`, `print-m`, `silent-cmd` and `concept`,

```

3847 \clist_map_inline:nn { angle , mangle }
3848 {
3849     \keys_define:nn { spintent / sp #1 }
3850     {
3851         read-cmd .choices:nn = { school , mathcat }
3852         {
3853             \int_case:nn { \l_keys_choice_int }
3854             {
3855                 {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
3856                 {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }

```

```

3857     }
3858   },
3859   read-cmd      .initial:n = school,
3860   read-cmd      .value_required:n = true,
3861   read-cmd / unknown .code:n =
3862     \msg_error:nneee { spintent } { unknown-choice }
3863     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3864   prefix        .code:n =
3865     { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
3866   prefix        .initial:n = ,
3867   prefix        .value_required:n = true,
3868   recto         .bool_set:c = { l__spintent_sp #1 _recto_bool },
3869   recto         .initial:n = false,
3870   recto         .value_required:n = true,
3871   print-m       .bool_set:c = { l__spintent_sp #1 _print_m_bool },
3872   print-m       .initial:n = false,
3873   print-m       .value_required:n = true,
3874   silent-cmd    .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
3875   silent-cmd    .initial:n = false,
3876   silent-cmd    .value_required:n = true,
3877   concept       .bool_set:c = { l__spintent_sp #1 _concept_bool },
3878   concept       .initial:n = true,
3879   concept       .value_required:n = true,
3880   unknown       .code:n =
3881     {
3882       \str_set:Nn \l__spintent_command_name_str { sp #1 }
3883       \__spintent_unknown_keys_cmd:n {##1}
3884     },
3885   }
3886 }

```

🔗 The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spangle`.

(End of definition for `read-cmd` and others.)

11.41.4 Internal functions for `\spangle` and `\spmangle`

`\__spintent_spangle_sym:n`

The internal function `\__spintent_spangle_sym:n` dispatches on `#1`, `print-m` and `recto` to select one of four symbols: `\angle/\rightanglesqr` when `print-m` is true; `\measuredangle/\measuredrightangle` otherwise. When `#1` is `angle`, only `recto` applies, choosing between `\angle` and `\rightangle`.

```

3887 \cs_new_protected:Nn \__spintent_spangle_sym:n
3888 {
3889   \str_if_eq:eeTF {#1} { mangle }
3890   {
3891     \bool_if:cTF { l__spintent_sp #1 _print_m_bool }
3892     {
3893       \bool_if:cTF { l__spintent_sp #1 _recto_bool }
3894       { \__spintent_right_angle_sqr: } { \angle }
3895     }
3896     {
3897       \bool_if:cTF { l__spintent_sp #1 _recto_bool }
3898       { \measuredrightangle } { \measuredangle }
3899     }
3900   }
3901   {
3902     \bool_if:cTF { l__spintent_sp #1 _recto_bool }
3903     { \__spintent_right_angle_sqr: } { \angle }
3904   }
3905 }

```

(End of definition for `\__spintent_spangle_sym:n`.)

`\l__spintent_spangle_intent_tl`

`\l__spintent_spangle_intent_tl` holds the concept word (ángulo or medida-del-ángulo, ‘-recto’- suffixed when `recto` is true). The rest are scratch variables shared by both commands to split `\l__spintent_geo_lua` and build the reference list — used only in the tres-puntos and un-punto cases (see `\__spintent_spangle_mathml:`

`\l__spintent_spangle_pt_seq`
`\l__spintent_spangle_refs_tl`
`\l__spintent_spangle_intent_arg_tl`
`\l__spintent_spangle_pt_idx_int`
`\l__spintent_spangle_print_m_now_bool`

```

3906 \tl_new:N \l__spintent_spangle_intent_tl
3907 \seq_new:N \l__spintent_spangle_pt_seq
3908 \tl_new:N \l__spintent_spangle_refs_tl
3909 \tl_new:N \l__spintent_spangle_intent_arg_tl
3910 \int_new:N \l__spintent_spangle_pt_idx_int
3911 \str_new:N \l__spintent_spangle_kv_tl
3912 \bool_new:N \l__spintent_spangle_print_m_now_bool

```

(End of definition for `\__spintent_spangle_intent_tl` and others.)

`\__spintent_spangle_process:nn`

The internal function `\__spintent_spangle_process:nn` receives the short name of the command (`angle` or `mangle`) in ‘#1’ and the $\{\langle argument \rangle\}$ in ‘#2’. Calls `\__spintent_luafun_geo_angulo_parse_and_set:n`, reporting `sp#1-invalid-arg` on error. For a proper name that is a single letter (not numeric), also reports the same error when that letter is lowercase — `angle` names take an uppercase letter, number, or Greek letter, never a lowercase Latin one; the length check keeps a Greek letter command such as `\alpha` out of this comparison. Then sets `\__spintent_spangle_intent_tl` to `ángulo` (or `medida-del-ángulo` for `\spmangle`), suffixed with `-recto` when `recto` is `true`, and calls `\__spintent_spangle_mathml:n`.

```

3913 \cs_new_protected:Nn \__spintent_spangle_process:nn
3914 {
3915   \__spintent_luafun_geo_angulo_parse_and_set:n { \exp_not:n {#2} }
3916   \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
3917   {
3918     \msg_error:nnee { spintent } { sp #1 -invalid-arg }
3919     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3920   }
3921   \str_if_eq:VnT \__spintent_geo_luaset_angulo_case_str { nombre }
3922   {
3923     \str_if_eq:VnF \__spintent_geo_luaset_nombre_es_num_str { true }
3924     {
3925       \int_compare:nT { \str_count:N \__spintent_geo_luaset_letra_str = 1 }
3926       {
3927         \str_if_eq:eeF { \__spintent_geo_luaset_letra_str } { \str_uppercase:n { \__spintent_geo_luaset_letra_str } }
3928         {
3929           \msg_error:nnee { spintent } { sp #1 -invalid-arg }
3930           { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3931         }
3932       }
3933     }
3934   }
3935   \str_if_eq:eeTF {#1} { mangle }
3936   {
3937     \tl_set:Nn \__spintent_spangle_intent_tl { medida-del-ángulo }
3938   }
3939   {
3940     \tl_set:Nn \__spintent_spangle_intent_tl { ángulo }
3941   }
3942   \bool_if:cT { l__spintent_sp #1 _recto_bool }
3943   {
3944     \tl_put_right:Nn \__spintent_spangle_intent_tl { -recto }
3945   }
3946   \__spintent_spangle_mathml:n {#1}
3947 }

```

(End of definition for `\__spintent_spangle_process:nn`.)

`\__spintent_spangle_mathml:n`

The function `\__spintent_spangle_mathml:n` branches on `\__spintent_geo_luaset_angulo_case_str`. For `nombre`, the symbol (`\__spintent_spangle_sym:n`) is tagged through `\__spintent_luafun_mrow_intent_arg_tl` on its own, with `\__spintent_spangle_intent_tl:prefix` (no $\$$ -reference: `nombre` never refers to more than one thing) — `_` — when `silent-cmd` is `true` or `concept` is `false`. With `silent-cmd` `true`, the name that follows is silenced as a whole.

```

3948 \cs_new_protected:Nn \__spintent_spangle_mathml:n
3949 {
3950   \__spintent_invisible_sep: % (luamml issues #26)
3951   \str_if_eq:VnTF \__spintent_geo_luaset_angulo_case_str { nombre }
3952   {
3953     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
3954     {
3955       \tl_set:Nn \__spintent_spangle_intent_arg_tl { _- }
3956     }
3957     {
3958       \bool_if:cTF { l__spintent_sp #1 _concept_bool }
3959       {
3960         \tl_set:Nn \__spintent_spangle_intent_arg_tl
3961         {
3962           \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3963           { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
3964           \__spintent_spangle_intent_tl :prefix
3965         }

```

```

3966         }
3967         {
3968             \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
3969         }
3970     }
3971     \tl_set:Ne \l__spintent_spangle_kv_tl
3972     { intent = "\l__spintent_spangle_intent_arg_tl" }
3973     \__spintent_luafun_mrow_intent_arg:nn
3974     { } { \l__spintent_spangle_kv_tl }
3975     { \__spintent_spangle_sym:n {#1} }
3976     \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
3977     {
3978         \__spintent_luafun_mrow_intent_arg:nn { } { intent = "_-" }
3979         { \l__spintent_geo_luaset_print_tl }
3980     }

```

Otherwise, a numeric name is tagged as a whole, through `\__spintent_luafun_mrow_intent_arg:nn`, with a literal `intent` built from `\l__spintent_geo_luaset_intent_str`.

```

3981     {
3982         \str_if_eq:VnTF \l__spintent_geo_luaset_nombre_es_num_str { true }
3983         {
3984             \tl_set:Ne \l__spintent_spangle_kv_tl
3985             { intent = "_ \l__spintent_geo_luaset_intent_str -" }
3986             \__spintent_luafun_mrow_intent_arg:nn
3987             { } { \l__spintent_spangle_kv_tl }
3988             { \l__spintent_geo_luaset_print_tl }
3989         }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags the letter through the same function, with a literal `intent`, and the subscript is appended and silenced; `read-cmd=mathcat` leaves both letter and subscript untouched, for MathCAT to read the whole `<msub>` natively (no `arg` is set in either case: there is no `$`-reference to resolve).

```

3990     {
3991         \str_case:VnF \l__spintent_geo_luaset_tipo_str
3992         {
3993             { sub }
3994             {
3995                 \str_if_eq:eeTF { \use:c { \l__spintent_sp #1 _mode_str } } { school }
3996                 {
3997                     \tl_set:Ne \l__spintent_spangle_kv_tl
3998                     { intent = "_ \l__spintent_geo_luaset_intent_str -" }
3999                     \__spintent_luafun_mrow_intent_arg:nn
4000                     { } { \l__spintent_spangle_kv_tl }
4001                     {
4002                         \l__spintent_geo_luaset_letra_str
4003                         \c_math_subscript_token
4004                         { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
4005                     }
4006                 }
4007             }
4008             \l__spintent_geo_luaset_letra_str
4009             \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
4010         }
4011     }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the prime is always read — spoken as part of the literal for `read-cmd=school`, native for `read-cmd=mathcat` — since a prime is not a repeated word to silence.

```

4012     { prima }
4013     {
4014         \str_if_eq:eeTF { \use:c { \l__spintent_sp #1 _mode_str } } { school }
4015         {
4016             \tl_set:Ne \l__spintent_spangle_kv_tl
4017             { intent = "_ \l__spintent_geo_luaset_intent_str -" }
4018             \__spintent_luafun_mrow_intent_arg:nn
4019             { } { \l__spintent_spangle_kv_tl }
4020             { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
4021         }
4022         {
4023             \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str
4024         }
4025     }

```

```

4026         }
4027         { \l__spintent_geo_luaset_print_tl }
4028     }
4029 }
4030 }

```

For tres-puntos/un-punto, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_spangle_pt` and walked to build `\l__spintent_spangle_refs_tl`, the `$pt1, $pt2, ...` reference list. A single native `\MathMLintent` wraps the symbol and the points together, with `\l__spintent_spangle_intent_tl:prefix(refs — —` when `silent-cmd` is `true`, `_(refs)` when `concept` is `false` alone. For `\spmangle` with `print-m true`, a silent `\operatorname{m}` precedes `\l__spintent_spangle_sym:n` inside the same `\MathMLintent`. The sequence is walked again, calling `\l__spintent_mathmlarg_point:nnn` once per point.

```

4031 {
4032   \seq_set_from_clist:NN \l__spintent_spangle_pt_seq \l__spintent_geo_luaset_points_clist
4033   \tl_clear:N \l__spintent_spangle_refs_tl
4034   \int_zero:N \l__spintent_spangle_pt_idx_int
4035   \seq_map_inline:Nn \l__spintent_spangle_pt_seq
4036   {
4037     \int_incr:N \l__spintent_spangle_pt_idx_int
4038     \tl_if_empty:NF \l__spintent_spangle_refs_tl
4039     { \tl_put_right:Nn \l__spintent_spangle_refs_tl { , } }
4040     \tl_put_right:Ne \l__spintent_spangle_refs_tl { $ pt \int_use:N \l__spintent_spangle_pt
4041   }
4042   \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
4043   {
4044     \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
4045   }
4046   {
4047     \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4048     {
4049       \tl_set:Ne \l__spintent_spangle_intent_arg_tl
4050       {
4051         \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4052         { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4053         \l__spintent_spangle_intent_tl :prefix( \l__spintent_spangle_refs_tl )
4054       }
4055     }
4056     {
4057       \tl_set:Ne \l__spintent_spangle_intent_arg_tl { _( \l__spintent_spangle_refs_tl )
4058     }
4059   }
4060   \bool_set_false:N \l__spintent_spangle_print_m_now_bool
4061   \str_if_eq:eeT { #1 } { mangle }
4062   {
4063     \bool_if:cT { \l__spintent_sp #1 _print_m_bool }
4064     { \bool_set_true:N \l__spintent_spangle_print_m_now_bool }
4065   }
4066   \MathMLintent { \l__spintent_spangle_intent_arg_tl }
4067   {
4068     \l__spintent_inner_sep:
4069     \bool_if:NT \l__spintent_spangle_print_m_now_bool
4070     {
4071       \l__spintent_luafun_mrow_intent_arg:nn
4072       { } { intent = ":silent" }
4073       { \operatorname { m } } }
4074     }
4075     \l__spintent_luafun_mrow_intent_arg:nn
4076     { } { intent = ":silent" }
4077     { \l__spintent_spangle_sym:n { #1 } }
4078     \int_zero:N \l__spintent_spangle_pt_idx_int
4079     \seq_map_inline:Nn \l__spintent_spangle_pt_seq
4080     {
4081       \int_incr:N \l__spintent_spangle_pt_idx_int
4082       \l__spintent_mathmlarg_point:nnn
4083       { pt \int_use:N \l__spintent_spangle_pt_idx_int }
4084       { sp#1 }
4085       { ##1 }
4086     }
4087   }
4088 }
4089 }

```

(End of definition for `\__spintent_spangle_mathml:n`.)

`\__spintent_spangle_exec:nnn`

The function `\__spintent_spangle_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *{argument}* passed in ‘#3’, delegating the rendering to `\__spintent_spangle_process:nn`.

```
4090 \cs_new_protected:Nn \__spintent_spangle_exec:nnn
4091 {
4092   \keys_set:nn { spintent / sp #1 } { #2 }
4093   \__spintent_spangle_process:nn { #1 } { #3 }
4094 }
```

(End of definition for `\__spintent_spangle_exec:nnn`.)

`\spangle`

`\spmangle`

Both public commands `\spangle` and `\spmangle` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spangle_exec:nnn`.

```
4095 \NewDocumentCommand \spangle { 0{} m }
4096 {
4097   \mode_if_math:TF
4098   {
4099     \group_begin:
4100     \attribute \__spintent_scope_attr_int = 1
4101     \__spintent_spangle_exec:nnn { angle } { #1 } { #2 }
4102     \group_end:
4103   }
4104   { \msg_error:nnn { spintent } { need-math-mode } { \spangle } }
4105 }
4106 \NewDocumentCommand \spmangle { 0{} m }
4107 {
4108   \mode_if_math:TF
4109   {
4110     \group_begin:
4111     \attribute \__spintent_scope_attr_int = 1
4112     \__spintent_spangle_exec:nnn { mangle } { #1 } { #2 }
4113     \group_end:
4114   }
4115   { \msg_error:nnn { spintent } { need-math-mode } { \spmangle } }
4116 }
```

(End of definition for `\spangle` and `\spmangle`. These functions are documented on page 21.)

11.42 The commands `\sparc` and `\spmarc`

The user commands `\sparc` and `\spmarc` provide a semantic interface for arcs. Unlike `\spmside` and `\spmangle`, `\spmarc` has no `print-m` key: `\operatorname{m}` is always present, since $m\widehat{AB}$ is the only standard notation for the measure of an arc.

11.42.1 Definition of variables for `\sparc` and `\spmarc`

`\__spintent_sparc_mode_str`

`\__spintent_spmarc_mode_str`

`\__spintent_sparc_prefix_tl`

`\__spintent_spmarc_prefix_tl`

Each pair of variables is declared once for both `\sparc` and `\spmarc` via `\clist_map_inline:nn`. The variables `\__spintent_sparc_mode_str` and `\__spintent_spmarc_mode_str` store the values set by the key `read-cmd`; the variables `\__spintent_sparc_prefix_tl` and `\__spintent_spmarc_prefix_tl` store the values set by the key `prefix`.

```
4117 \clist_map_inline:nn { arc , marc }
4118 {
4119   \str_new:c { \__spintent_sp #1 _mode_str }
4120   \tl_new:c { \__spintent_sp #1 _prefix_tl }
4121 }
```

(End of definition for `\__spintent_sparc_mode_str` and others.)

11.42.2 Definition of error messages for `\sparc` and `\spmarc`

`sparc-invalid-points`

`spmarc-invalid-points`

We define an “error” message for the commands `\sparc` and `\spmarc`. The “error” is triggered if the user includes a invalid *{argument}* passed in ‘#1’ to the command.

```
4122 \clist_map_inline:nn { arc , marc }
4123 {
4124   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
4125   {
4126     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
4127   }
4128   {
4129     See~the~documentation~of~##1~for~the~accepted~point~formats.
4130   }
4131 }
```

(End of definition for `sparc-invalid-points` and `spmarc-invalid-points`.)

11.42.3 Definition of keys for \sparc and \spmarc

We define the shared *(keys)* `read-cmd`, `prefix`, `silent-cmd` and `concept`.

```

read-cmd 4132 \clist_map_inline:nn { arc , marc }
prefix   4133 {
silent-cmd 4134 \keys_define:nn { spintent / sp #1 }
concept   4135 {
unknown   4136   read-cmd      .choices:nn = { school , mathcat }
          4137   {
          4138     \int_case:nn { \l_keys_choice_int }
          4139     {
          4140       {1} { \str_set:cn { __spintent_sp #1 _mode_str } { school } }
          4141       {2} { \str_set:cn { __spintent_sp #1 _mode_str } { mathcat } }
          4142     }
          4143   },
          4144   read-cmd      .initial:n = school,
          4145   read-cmd      .value_required:n = true,
          4146   read-cmd / unknown .code:n =
          4147     \msg_error:nneee { spintent } { unknown-choice }
          4148     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
          4149   prefix      .code:n =
          4150     { __spintent_sanitize_affix:cn { __spintent_sp #1 _prefix_tl } {##1} },
          4151   prefix      .initial:n = ,
          4152   prefix      .value_required:n = true,
          4153   silent-cmd   .bool_set:c = { __spintent_sp #1 _silent_cmd_bool },
          4154   silent-cmd   .initial:n = false,
          4155   silent-cmd   .value_required:n = true,
          4156   concept      .bool_set:c = { __spintent_sp #1 _concept_bool },
          4157   concept      .initial:n = true,
          4158   concept      .value_required:n = true,
          4159   unknown      .code:n =
          4160     {
          4161       \str_set:Nn \l__spintent_command_name_str { sp #1 }
          4162       __spintent_unknown_keys_cmd:n {##1}
          4163     },
          4164   }
          4165 }

```

(End of definition for `read-cmd` and others.)

11.42.4 Internal functions for \sparc and \spmarc

`\__spintent_sparc_wide_arc:n` The function `\__spintent_sparc_wide_arc:n` acts as a fallback when the mathematical font used in the document does not provide the command `\widearc`. The definition of this function is a direct adaptation of a response given by Clea F. Rees (@cfr) to a query in the chat room of TeX-SX.

```

4166 \cs_new_protected_nopar:Nn \__spintent_sparc_wide_arc:n
4167 {
4168   \Umathaccent 7 \symoperators "023DC \scan_stop: {#1}
4169 }

```

(End of definition for `\__spintent_sparc_wide_arc:n`)

`\__spintent_sparc_print_arc:n` The function `\__spintent_sparc_print_arc:n` applies the command `\widearc` to the argument passed to `\sparc` or `\spmarc`. If the mathematical font set in the document does not provide the command `\widearc`, we will use the built-in function `\__spintent_sparc_wide_arc:n`.

```

4170 \cs_new_protected:Nn \__spintent_sparc_print_arc:n
4171 {
4172   \cs_if_exist:NTF \widearc
4173   { \widearc {#1} }
4174   { \__spintent_sparc_wide_arc:n {#1} }
4175 }

```

(End of definition for `\__spintent_sparc_print_arc:n`)

`\__spintent_sparc_intent_tl` `\__spintent_sparc_pt_seq` `\__spintent_sparc_refs_tl` `\__spintent_sparc_intent_arg_tl` `\__spintent_sparc_pt_idx_int` `\__spintent_sparc_intent_tl` holds the concept word (arco or medida-del-arco). The rest are scratch variables to split `\__spintent_geo_luaset_points_clist` and build the `$pt1`, `$pt2`, ... reference list used in the concept intent.

```

4176 \tl_new:N \__spintent_sparc_intent_tl
4177 \seq_new:N \__spintent_sparc_pt_seq
4178 \tl_new:N \__spintent_sparc_refs_tl
4179 \tl_new:N \__spintent_sparc_intent_arg_tl
4180 \int_new:N \__spintent_sparc_pt_idx_int

```

(End of definition for `\l__spintent_sparc_intent_tl` and others.)

`\__spintent_sparc_process:nn`

The internal function `\__spintent_sparc_process:nn` receives the short name of the command (arc or marc) in ‘#1’ and the $\{(\textit{argument})\}$ in ‘#2’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘#2’, returning an “error” if invalid, then sets `\l__spintent_sparc_intent_tl` to arco (or medida-del-arco for `\spmarc`). No `intent` is built here: it needs the reference list, which `\__spintent_sparc_mathml:n` builds after splitting the points.

```

4181 \cs_new_protected:Nn \__spintent_sparc_process:nn
4182 {
4183   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
4184   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
4185   {
4186     \msg_error:nnee { spintent } { sp #1 -invalid-points }
4187     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4188   }
4189   \str_if_eq:eeTF {#1} { marc }
4190   { \tl_set:Nn \l__spintent_sparc_intent_tl { medida-del-arco } }
4191   { \tl_set:Nn \l__spintent_sparc_intent_tl { arco } }
4192   \__spintent_sparc_mathml:n {#1}
4193 }

```

(End of definition for `\__spintent_sparc_process:nn`.)

`\__spintent_sparc_mathml:n`

The function `\__spintent_sparc_mathml:n` receives the short command name in ‘#1’. It splits `\l__spintent_geo_` into `\l__spintent_sparc_pt_seq` and walks it once to build `\l__spintent_sparc_refs_tl`, the comma-separated $\$pt1, \$pt2, \dots$ reference list, naming each point by position.

```

4194 \cs_new_protected:Nn \__spintent_sparc_mathml:n
4195 {
4196   \seq_set_from_clist:NN \l__spintent_sparc_pt_seq \l__spintent_geo_luaset_points_clist
4197
4198   \tl_clear:N \l__spintent_sparc_refs_tl
4199   \int_zero:N \l__spintent_sparc_pt_idx_int
4200   \seq_map_inline:Nn \l__spintent_sparc_pt_seq
4201   {
4202     \int_incr:N \l__spintent_sparc_pt_idx_int
4203     \tl_if_empty:NF \l__spintent_sparc_refs_tl
4204     { \tl_put_right:Nn \l__spintent_sparc_refs_tl { , } }
4205     \tl_put_right:Ne \l__spintent_sparc_refs_tl { $ pt \int_use:N \l__spintent_sparc_pt_idx_int }
4206   }

```

The concept `intent` is `_` - when `silent-cmd` is `true` or `concept` is `false`; otherwise `\l__spintent_sparc_intent_` prefixed with `prefix` when given.

```

4207   \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
4208   {
4209     \tl_set:Nn \l__spintent_sparc_intent_arg_tl { _- }
4210   }
4211   {
4212     \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4213     {
4214       \tl_set:Ne \l__spintent_sparc_intent_arg_tl
4215       {
4216         \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4217         { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4218         \l__spintent_sparc_intent_tl :prefix( \l__spintent_sparc_refs_tl )
4219       }
4220     }
4221     {
4222       \tl_set:Nn \l__spintent_sparc_intent_arg_tl { _- }
4223     }
4224   }

```

It is applied with a single native `\MathMLintent` wrapping `\__spintent_sparc_print_arc:n` directly (same shape as `\__spintent_spside_mathml:n`), preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, luamml issue #26). A silent `\operatorname{m}` precedes it for `\spmarc` (always, no `print-m` key here). The sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptn` reference name, looked up under `sp#1`.

```

4225   \__spintent_invisible_sep: % need by MathCAT (bug in luamml issues #26)
4226   \MathMLintent { \l__spintent_sparc_intent_arg_tl }
4227   {
4228     \__spintent_inner_sep:
4229     \str_if_eq:eeT {#1} { marc }

```

```

4230     {
4231         \__spintent_luafun_mrow_intent_arg:nn
4232         { } { intent = ":silent" }
4233         { \operatorname { m } }
4234     }
4235     \__spintent_sparc_print_arc:n
4236     {
4237         \int_zero:N \__spintent_sparc_pt_idx_int
4238         \seq_map_inline:Nn \__spintent_sparc_pt_seq
4239         {
4240             \int_incr:N \__spintent_sparc_pt_idx_int
4241             \__spintent_mathmlarg_point:nnn
4242             { pt \int_use:N \__spintent_sparc_pt_idx_int }
4243             { sp #1 }
4244             { ##1 }
4245         }
4246     }
4247 }
4248 }

```

(End of definition for __spintent_sparc_mathml:n.)

__spintent_sparc_exec:nnn

The function `\__spintent_sparc_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *argument* passed in ‘#3’, delegating the rendering to `\__spintent_sparc_process:nn`.

```

4249 \cs_new_protected:Nn \__spintent_sparc_exec:nnn
4250 {
4251     \keys_set:nn { spintent / sp #1 } { #2 }
4252     \__spintent_sparc_process:nn { #1 } { #3 }
4253 }

```

(End of definition for __spintent_sparc_exec:nnn.)

\sparc
\spmarc

Both public commands `\sparc` and `\spmarc` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sparc_exec:nnn`.

```

4254 \NewDocumentCommand \sparc { O{} m }
4255 {
4256     \mode_if_math:TF
4257     {
4258         \group_begin:
4259         \attribute \__spintent_scope_attr_int = 1
4260         \__spintent_sparc_exec:nnn { arc } { #1 } { #2 }
4261         \group_end:
4262     }
4263     { \msg_error:nnn { spintent } { need-math-mode } { \sparc } }
4264 }
4265 \NewDocumentCommand \spmarc { O{} m }
4266 {
4267     \mode_if_math:TF
4268     {
4269         \group_begin:
4270         \attribute \__spintent_scope_attr_int = 1
4271         \__spintent_sparc_exec:nnn { marc } { #1 } { #2 }
4272         \group_end:
4273     }
4274     { \msg_error:nnn { spintent } { need-math-mode } { \spmarc } }
4275 }

```

(End of definition for \sparc and \spmarc. These functions are documented on page 23.)

11.43 The command \spPoly

The user command `\spPoly` typesets a polygon defined by its vertices. The figure type and its visual symbol are inferred from the vertex count: three points produce a triangle, four a square, five or more a generic polygon with no symbol. It never sends a boolean to Lua: `\__spintent_geo_luaseta_intent_str` (the full *intent*, with the figure name).

11.43.1 Definition of variables for \spPoly

The variable `\__spintent_spPoly_mode_str` stores the value set by the key `read-cmd`; the variable `\__spintent_spPoly_prefix_tl` stores the value set by the key `prefix`.

```

4276 \str_new:N \__spintent_spPoly_mode_str
4277 \tl_new:N \__spintent_spPoly_prefix_tl

```

(End of definition for __spintent_spPoly_mode_str and __spintent_spPoly_prefix_tl.)

11.43.2 Definition of error messages for \spPoly

spPoly-invalid-points

Raised when the argument is not a comma-separated list of at least three uppercase letters.

```

4278 \msg_new:nnnn { spintent } { spPoly-invalid-points }
4279 {
4280   Invalid~point~format~in~\token_to_str:N \spPoly:~'#1'~\msg_line_context:.
4281 }
4282 {
4283   See~the~documentation~of~\token_to_str:N \spPoly~for~the~accepted~point~formats.
4284 }

```

(End of definition for spPoly-invalid-points.)

11.43.3 Definition of keys for \spPoly

read-cmd

We define the *(keys)* read-cmd, prefix, print-sym, silent-cmd and concept.

prefix

print-sym

silent-cmd

concept

unknown

```

4285 \keys_define:nn { spintent / spPoly }
4286 {
4287   read-cmd          .choices:n = { school , mathcat }
4288   {
4289     \int_case:nn { \l_keys_choice_int }
4290     {
4291       {1} { \str_set:Nn \l__spintent_spPoly_mode_str { school } }
4292       {2} { \str_set:Nn \l__spintent_spPoly_mode_str { mathcat } }
4293     }
4294   },
4295   read-cmd          .initial:n = school,
4296   read-cmd          .value_required:n = true,
4297   read-cmd / unknown .code:n =
4298     \msg_error:nneee { spintent } { unknown-choice }
4299     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
4300   prefix            .code:n =
4301     { \__spintent_sanitize_affix:Nn \l__spintent_spPoly_prefix_tl {#1} },
4302   prefix            .initial:n = ,
4303   prefix            .value_required:n = true,
4304   print-sym         .bool_set:N = \l__spintent_spPoly_print_sym_bool,
4305   print-sym         .initial:n = true,
4306   print-sym         .value_required:n = true,
4307   silent-cmd        .bool_set:N = \l__spintent_spPoly_silent_cmd_bool,
4308   silent-cmd        .initial:n = false,
4309   silent-cmd        .value_required:n = true,
4310   concept           .bool_set:N = \l__spintent_spPoly_concept_bool,
4311   concept           .initial:n = true,
4312   concept           .value_required:n = true,
4313   unknown           .code:n =
4314     {
4315       \str_set:Nn \l__spintent_command_name_str { spPoly }
4316       \__spintent_unknown_keys_cmd:n {#1}
4317     },
4318 }

```

(End of definition for read-cmd and others.)

11.43.4 Internal functions for \spPoly

\l__spintent_spPoly_pt_seq

\l__spintent_spPoly_refs_tl

\l__spintent_spPoly_intent_arg_tl

\l__spintent_spPoly_pt_idx_int

Scratch variables to split \l__spintent_geo_luaset_points_clist and build the \$pt1, \$pt2, ... reference list, and to hold the final concept intent once built.

```

4319 \seq_new:N \l__spintent_spPoly_pt_seq
4320 \tl_new:N \l__spintent_spPoly_refs_tl
4321 \tl_new:N \l__spintent_spPoly_intent_arg_tl
4322 \int_new:N \l__spintent_spPoly_pt_idx_int

```

(End of definition for \l__spintent_spPoly_pt_seq and others.)

__spintent_spPoly_sym:

Unchanged: reads \l__spintent_geo_luaset_poly_sym_str and typesets the corresponding symbol.

```

4323 \cs_new_protected:Nn \__spintent_spPoly_sym:
4324 {
4325   \str_case:Vn \l__spintent_geo_luaset_poly_sym_str
4326   {
4327     { triangle } { \triangle }
4328     { square } { \mdlgwhtsquare }
4329   }
4330 }

```

(End of definition for `\__spintent_spPoly_sym:`.)

`\__spintent_spPoly_build_refs:` Splits `\__spintent_geo_luaset_points_clist` into `\__spintent_spPoly_pt_seq` and walks it once to build `\__spintent_spPoly_refs_tl`, the comma-separated `$pt1, $pt2, ...` reference list, naming each vertex by position.

```

4331 \cs_new_protected:Nn \__spintent_spPoly_build_refs:
4332 {
4333   \seq_set_from_clist:NN \__spintent_spPoly_pt_seq \__spintent_geo_luaset_points_clist
4334   \tl_clear:N \__spintent_spPoly_refs_tl
4335   \int_zero:N \__spintent_spPoly_pt_idx_int
4336   \seq_map_inline:Nn \__spintent_spPoly_pt_seq
4337   {
4338     \int_incr:N \__spintent_spPoly_pt_idx_int
4339     \tl_if_empty:NF \__spintent_spPoly_refs_tl
4340     { \tl_put_right:Nn \__spintent_spPoly_refs_tl { , } }
4341     \tl_put_right:Ne \__spintent_spPoly_refs_tl { $ pt \int_use:N \__spintent_spPoly_pt_idx.
4342   }
4343 }
```

(End of definition for `\__spintent_spPoly_build_refs:`.)

`\__spintent_spPoly_build_concept:` Builds `\__spintent_spPoly_intent_arg_tl`, independently of `print-sym: _`(refs) when `silent-cmd` is `true` or `concept` is `false`; otherwise `\__spintent_geo_luaset_concept_str:prefix(refs)`, prefixed with `prefix` when given. Must be called after `\__spintent_spPoly_build_refs:`.

```

4344 \cs_new_protected:Nn \__spintent_spPoly_build_concept:
4345 {
4346   \bool_if:NTF \__spintent_spPoly_silent_cmd_bool
4347   {
4348     \tl_set:Ne \__spintent_spPoly_intent_arg_tl { _ ( \__spintent_spPoly_refs_tl ) }
4349   }
4350   {
4351     \bool_if:NTF \__spintent_spPoly_concept_bool
4352     {
4353       \tl_set:Ne \__spintent_spPoly_intent_arg_tl
4354       {
4355         \tl_if_empty:NF \__spintent_spPoly_prefix_tl
4356         { \tl_use:N \__spintent_spPoly_prefix_tl - }
4357         \__spintent_geo_luaset_concept_str :prefix( \__spintent_spPoly_refs_tl )
4358       }
4359     }
4360     {
4361       \tl_set:Ne \__spintent_spPoly_intent_arg_tl { _ ( \__spintent_spPoly_refs_tl ) }
4362     }
4363   }
4364 }
```

(End of definition for `\__spintent_spPoly_build_concept:`.)

`\__spintent_spPoly_points_loop:` Walks `\__spintent_spPoly_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per vertex with its `ptn` reference name, looked up under `spPoly:` it alone decides `arg/literal intent` per vertex — replacing the single combined `\MathMLarg{pts}{...}` that lost `arg` on any vertex with a subscript or prime (the same `annotate_attribute/sub-sup` limitation already found and worked around for `\spPoint`).

```

4365 \cs_new_protected:Nn \__spintent_spPoly_points_loop:
4366 {
4367   \int_zero:N \__spintent_spPoly_pt_idx_int
4368   \seq_map_inline:Nn \__spintent_spPoly_pt_seq
4369   {
4370     \int_incr:N \__spintent_spPoly_pt_idx_int
4371     \__spintent_mathmlarg_point:nnn
4372     { pt \int_use:N \__spintent_spPoly_pt_idx_int }
4373     { spPoly }
4374     { ##1 }
4375   }
4376 }
```

(End of definition for `\__spintent_spPoly_points_loop:`.)

`\__spintent_spPoly_process:n` Calls `\__spintent_luafun_geo_poly_parse_and_set:n`, reports `spPoly-invalid-points` on failure, then delegates to `\__spintent_spPoly_mathml:`. No `intent` is built here: it needs the reference list, which `\__spintent_spPoly_mathml:` builds after splitting the vertices.

```

4377 \cs_new_protected:Nn \__spintent_spPoly_process:n
```

```

4378 {
4379   \__spintent_luafun_geo_poly_parse_and_set:n { \exp_not:n {#1} }
4380   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
4381   {
4382     \msg_error:nne { spintent } { spPoly-invalid-points }
4383     { \exp_not:n {#1} }
4384   }
4385   \__spintent_spPoly_mathml:
4386 }

```

(End of definition for __spintent_spPoly_process:n.)

__spintent_spPoly_mathml: Builds the reference list and the concept intent(__spintent_spPoly_build_refs:, __spintent_spPoly_build_refs:) then wraps everything in a single native \MathMLintent, preceded by __spintent_inner_sep: (a second sibling so the sole <mrow> does not collapse into <math>, luamml issue #26).

print-sym and concept/silent-cmd are independent: the former only decides whether the glyph (__spintent_spPoly_sym:, silenced with __spintent_luafun_mrow_intent_arg:nn) is attempted — it is skipped anyway when \l__spintent_geo_luaset_poly_sym_str is empty (no symbol defined for that vertex count, e.g. pentagon and above) — and never affects whether the concept word itself is spoken.

```

4387 \cs_new_protected:Nn \__spintent_spPoly_mathml:
4388 {
4389   \__spintent_invisible_sep: % (luamml issues #26)
4390   \__spintent_spPoly_build_refs:
4391   \__spintent_spPoly_build_concept:
4392   \MathMLintent { \l__spintent_spPoly_intent_arg_tl }
4393   {
4394     \__spintent_inner_sep:
4395     \bool_if:NT \l__spintent_spPoly_print_sym_bool
4396     {
4397       \str_if_empty:NF \l__spintent_geo_luaset_poly_sym_str
4398       {
4399         \__spintent_luafun_mrow_intent_arg:nn
4400         { } { intent = ":silent" }
4401         { \__spintent_spPoly_sym: }
4402       }
4403     }
4404     \__spintent_spPoly_points_loop:
4405   }
4406 }

```

(End of definition for __spintent_spPoly_mathml:.)

__spintent_spPoly_exec:nn The function __spintent_spPoly_exec:nn receives the optional keys in #1 and the argument in #2, delegating the rendering to __spintent_spPoly_process:n.

```

4407 \cs_new_protected:Nn \__spintent_spPoly_exec:nn
4408 {
4409   \keys_set:nn { spintent / spPoly } { #1 }
4410   \__spintent_spPoly_process:n { #2 }
4411 }

```

(End of definition for __spintent_spPoly_exec:nn.)

\spPoly The public command \spPoly enforces *math mode*, opens a local group, and delegates execution to __spintent_spPoly_exec:nn.

```

4412 \NewDocumentCommand \spPoly { 0{} m }
4413 {
4414   \mode_if_math:TF
4415   {
4416     \group_begin:
4417     \attribute \l__spintent_scope_attr_int = 1
4418     \__spintent_spPoly_exec:nn { #1 } { #2 }
4419     \group_end:
4420   }
4421   { \msg_error:nnn { spintent } { need-math-mode } { \spPoly } }
4422 }

```

(End of definition for \spPoly. This function is documented on page 24.)

11.44 The commands `\spAfig` and `\spPfig`

The user commands `\spAfig` and `\spPfig` typeset the area and the perimeter of a polygonal figure, using the delimited-optional argument syntax `d()` already established by `\spmcd/\spmcm`. The key `print-sym` is never sent to Lua: the symbol type is returned unconditionally in `\l__spintent_geo_luaset_poly_sym_str`, and `expl3` decides whether to wrap it in `\MathMLintent{(:silent)}` — the same mechanism already used by `\spPoly`.

11.44.1 Definition of variables for `\spAfig` and `\spPfig`

```
\l__spintent_geo_final_print_tl
```

Scratch variables used by `\__spintent_spfig_process:nn` to assemble the visual and the `intent`. `\l__spintent_geo_final_print_tl` holds the final visual placed under `\mathcal`’s subscript. The final literal `intent` and the concept forwarded when `read-cmd` is `mathcat`.

```
4423 \tl_new:N \l__spintent_geo_final_print_tl
```

(End of definition for `\l__spintent_geo_final_print_tl`.)

```
\l__spintent_spAfig_mode_str
```

```
\l__spintent_spPfig_mode_str
```

```
\l__spintent_spAfig_print_sym_str
```

```
\l__spintent_spPfig_print_sym_str
```

```
\l__spintent_spAfig_prefix_tl
```

```
\l__spintent_spPfig_prefix_tl
```

```
\l__spintent_spAfig_read_arg_tl
```

```
\l__spintent_spPfig_read_arg_tl
```

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `prefix` and `read-arg`, respectively. The value set by `read-arg` overrides the figure name inferred from the vertex count, independently of `print-sym`.

```
4424 \clist_map_inline:nn { Afig , Pfig }
```

```
4425 {
```

```
4426   \str_new:c { \l__spintent_sp #1 _mode_str }
```

```
4427   \str_new:c { \l__spintent_sp #1 _print_sym_str }
```

```
4428   \tl_new:c { \l__spintent_sp #1 _prefix_tl }
```

```
4429   \tl_new:c { \l__spintent_sp #1 _read_arg_tl }
```

```
4430 }
```

(End of definition for `\l__spintent_spAfig_mode_str` and others.)

11.44.2 Definition of error messages for `\spAfig` and `\spPfig`

```
spAfig-invalid-points
```

```
spPfig-invalid-points
```

```
spAfig-invalid-arg
```

```
spPfig-invalid-arg
```

```
spAfig-sym-letra-conflict
```

```
spPfig-sym-letra-conflict
```

```
spAfig-sym-cardinality-conflict
```

```
spPfig-sym-cardinality-conflict
```

```
spAfig-read-arg-key
```

```
spPfig-read-arg-key
```

All eight messages are generated by the same map, using the same `##1/##2` doubling convention as `\spside/\spmside`. `sp #1-invalid-points` is currently unused by `\__spintent_spfig_process:nn`, which raises only `sp #1-invalid-arg` for any parse failure, regardless of whether a comma was present. `sp #1-sym-letra-conflict` and `sp #1-sym-cardinality-conflict` are raised when `print-sym` is forced to a value incompatible with the actual argument.

```
4431 \clist_map_inline:nn { Afig , Pfig }
```

```
4432 {
```

```
4433   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
```

```
4434   {
```

```
4435     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
```

```
4436   }
```

```
4437   {
```

```
4438     See~the~documentation~of~##1~for~the~accepted~point~formats.
```

```
4439   }
```

```
4440   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
```

```
4441   {
```

```
4442     Invalid~argument~format~in~##1:~'##2'~\msg_line_context:.
```

```
4443   }
```

```
4444   {
```

```
4445     See~the~documentation~of~##1~for~the~accepted~argument~forms.
```

```
4446   }
```

```
4447   \msg_new:nnnn { spintent } { sp #1 -sym-letra-conflict }
```

```
4448   {
```

```
4449     print-sym~cannot~be~combined~with~a~lettered~argument~in~'##1'~\msg_line_context:.
```

```
4450   }
```

```
4451   {
```

```
4452     See~the~documentation~of~##1~for~details~on~print-sym.
```

```
4453   }
```

```
4454   \msg_new:nnnn { spintent } { sp #1 -sym-cardinality-conflict }
```

```
4455   {
```

```
4456     print-sym~does~not~match~the~vertex~count~in~'##1'~\msg_line_context:.
```

```
4457   }
```

```
4458   {
```

```
4459     See~the~documentation~of~##1~for~details~on~print-sym.
```

```
4460   }
```

```
4461   \msg_new:nnnn { spintent } { sp #1 -read-arg-key }
```

```
4462   {
```

```
4463     read-arg~has~no~effect~with~a~single~argument~in~'##1'~\msg_line_context:.
```

```
4464   }
```

```
4465   {
```

```
4466     See~the~documentation~of~##1~for~details~on~read-arg.
```

```

4467     }
4468 }

```

(End of definition for `spAfig-invalid-points` and others.)

11.44.3 Definition of keys for `\spAfig` and `\spPfig`

The key `print-sym` chooses the visual symbol: `auto` (default) infers it from the vertex count (three points give `\triangle`, four give `\mdlgwhtsquare`, five or more give none); `triangle/square` force one, requiring the vertex count to match; `none` omits it, even with vertices. With a bare number/letter argument, `auto` and `none` behave the same (no symbol); `triangle/square` nest the argument under the forced symbol instead, and cannot be combined with a table letter (F/P/B/L). The key `read-arg` overrides the spoken figure name for real vertices, independently of `print-sym`. The key `read-sub` controls whether the bare number/letter argument is read with the word «sub» or as a plain number.

```

read-cmd  \clist_map_inline:nn { Afig , Pfig }
prefix    {
print-sym  \keys_define:nn { spintent / sp #1 }
read-arg   {
read-sub   {
silent-cmd {
concept    {
unknown    \clist_map_inline:nn { Afig , Pfig }
            {
4471         \keys_define:nn { spintent / sp #1 }
4472         {
4473             read-cmd          .choices:nn = { school , mathcat }
4474             {
4475                 \int_case:nn { \l_keys_choice_int }
4476                 {
4477                     {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
4478                     {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
4479                 }
4480             },
4481             read-cmd          .initial:n = school,
4482             read-cmd          .value_required:n = true,
4483             read-cmd / unknown .code:n =
4484                 \msg_error:nneee { spintent } { unknown-choice }
4485                 { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4486             prefix           .code:n =
4487                 { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
4488             prefix           .initial:n = ,
4489             prefix           .value_required:n = true,
4490             print-sym         .choices:nn = { auto , triangle , square , none }
4491             {
4492                 \int_case:nn { \l_keys_choice_int }
4493                 {
4494                     {1} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { auto } }
4495                     {2} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { triangle } }
4496                     {3} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { square } }
4497                     {4} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { none } }
4498                 }
4499             },
4500             print-sym         .initial:n = auto,
4501             print-sym         .value_required:n = true,
4502             print-sym / unknown .code:n =
4503                 \msg_error:nneee { spintent } { unknown-choice }
4504                 { print-sym } { auto,~triangle,~square,~none } { \exp_not:n {##1} },
4505             read-arg          .code:n =
4506                 { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
4507             read-arg          .initial:n = ,
4508             read-arg          .value_required:n = true,
4509             read-sub          .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
4510             read-sub          .initial:n = false,
4511             read-sub          .value_required:n = true,
4512             silent-cmd        .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
4513             silent-cmd        .initial:n = false,
4514             silent-cmd        .value_required:n = true,
4515             concept           .bool_set:c = { l__spintent_sp #1 _concept_bool },
4516             concept           .initial:n = true,
4517             concept           .value_required:n = true,
4518             unknown           .code:n =
4519                 {
4520                     \str_set:Nn \l__spintent_command_name_str { sp #1 }
4521                     \__spintent_unknown_keys_cmd:n {##1}
4522                 },
4523             }
4524         }

```

(End of definition for `read-cmd` and others.)

11.44.4 Internal functions for \spAfig and \spPfig

__spinttent_spfig_sym:

The function `\__spinttent_spfig_sym:` reads `\l__spinttent_geo_print_sym_str` and typesets the corresponding symbol: `\triangle` for the value `triangle`, `\mdlgwhtsquare` for the value `square`.

```
4525 \cs_new_protected:Nn \__spinttent_spfig_sym:
4526 {
4527   \str_case:Vn \l__spinttent_geo_print_sym_str
4528   {
4529     { triangle } { \triangle }
4530     { square } { \mdlgwhtsquare }
4531   }
4532 }
```

(End of definition for `\__spinttent_spfig_sym:`.)

\l__spinttent_spfig_op_tl
\l__spinttent_spfig_intent_tl
\l__spinttent_spfig_symbol_tl
\l__spinttent_spfig_pt_seq
\l__spinttent_spfig_refs_tl
\l__spinttent_spfig_intent_arg_tl
\l__spinttent_spfig_pt_idx_int
\l__spinttent_geo_print_sym_str

`\l__spinttent_spfig_op_tl` holds `área` or `perímetro`. `\l__spinttent_spfig_intent_tl` holds the concept word for the vertices case (`read-arg` or the auto-detected figure name). The rest are scratch variables to split `\l__spinttent_geo_luaset_points_clist` and build the `$pt1, $pt2, ...` reference list.

```
4533 \tl_new:N \l__spinttent_spfig_op_tl
4534 \tl_new:N \l__spinttent_spfig_intent_tl
4535 \tl_new:N \l__spinttent_spfig_desc_tl
4536 \tl_new:N \l__spinttent_spfig_symbol_tl
4537 \seq_new:N \l__spinttent_spfig_pt_seq
4538 \tl_new:N \l__spinttent_spfig_refs_tl
4539 \tl_new:N \l__spinttent_spfig_intent_arg_tl
4540 \int_new:N \l__spinttent_spfig_pt_idx_int
4541 \str_new:N \l__spinttent_geo_print_sym_str
```

(End of definition for `\l__spinttent_spfig_op_tl` and others.)

__spinttent_spfig_process:nn

Receives the short command name (Afig or Pfig) in ‘#1’ and the `{\argument}` in ‘#2’. Calls `\__spinttent_luafun_geo` or `\__spinttent_luafun_geo_pfig_parse_and_set:n`, reports `sp#1-invalid-arg` on failure, and `sp#1-read-arg` when `read-arg` is given for a bare number/letter argument (no vertices), where it has no effect. Sets `\l__spinttent_spfig_op_tl` to `área/perímetro`, then delegates to `\__spinttent_spfig_mathml:n`.

```
4542 \cs_new_protected:Nn \__spinttent_spfig_process:nn
4543 {
4544   \str_if_eq:eeTF {#1} { Afig }
4545   { \__spinttent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
4546   { \__spinttent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
4547   \str_if_eq:VnT \l__spinttent_geo_luaset_error_str { true }
4548   {
4549     \msg_error:nnee { spinttent } { sp #1 -invalid-arg }
4550     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4551   }
4552   \clist_if_empty:NT \l__spinttent_geo_luaset_points_clist
4553   {
4554     \tl_if_empty:cF { \l__spinttent_sp #1 _read_arg_tl }
4555     {
4556       \msg_error:nne { spinttent } { sp #1 -read-arg-key }
4557       { \c_backslash_str sp #1 }
4558     }
4559   }
4560   \str_if_eq:eeTF {#1} { Afig }
4561   { \tl_set:Nn \l__spinttent_spfig_op_tl { área } }
4562   { \tl_set:Nn \l__spinttent_spfig_op_tl { perímetro } }
4563   \str_set:Ne \l__spinttent_geo_print_sym_str { \str_use:c { \l__spinttent_sp #1 _print_sym_str } }
4564   \__spinttent_spfig_mathml:n {#1}
4565 }
```

(End of definition for `\__spinttent_spfig_process:nn`.)

__spinttent_spfig_build_simple:n

Handles the argument-less, table-letter and bare number/letter cases (no vertices). Builds `\l__spinttent_geo_final_print` (the visual placed under `\mathcal`’s subscript) and `\l__spinttent_spfig_desc_tl` (the descriptive part, without `área/perímetro`). Receives the short command name in ‘#1’.

With an empty argument, both are cleared for `print-sym` auto/ `none`; a forced `print-sym` instead places the symbol alone, with its own name as the description.

```
4566 \cs_new_protected:Nn \__spinttent_spfig_build_simple:n
4567 {
4568   \tl_if_empty:nTF { \l__spinttent_geo_luaset_print_tl }
4569   {
4570     \str_case:VnF \l__spinttent_geo_print_sym_str
```

```

4571     {
4572     { auto }
4573     {
4574         \tl_clear:N \l__spintent_geo_final_print_tl
4575         \tl_clear:N \l__spintent_spfig_desc_tl
4576     }
4577     { none }
4578     {
4579         \tl_clear:N \l__spintent_geo_final_print_tl
4580         \tl_clear:N \l__spintent_spfig_desc_tl
4581     }
4582     }
4583     {
4584         \tl_set:Nn \l__spintent_geo_final_print_tl { \__spintent_spfig_sym: }
4585         \tl_set:Ne \l__spintent_spfig_desc_tl
4586         {
4587             \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
4588             { triángulo } { cuadrado }
4589         }
4590     }
4591 }

```

With a bare number/letter (empty `\l__spintent_geo_luaset_letra_word_str`), the value itself becomes the visual for `print-sym` `auto/none`, or its own subscript under a forced symbol otherwise; the description is the spoken value, prefixed with «sub-» when `read-sub` is `true` (further prefixed with the symbol's own name for a forced `print-sym`).

```

4592     {
4593         \str_if_empty:NTF \l__spintent_geo_luaset_letra_word_str
4594         {
4595             \str_case:VnF \l__spintent_geo_print_sym_str
4596             {
4597                 { auto } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_p
4598                 { none } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_p
4599             }
4600             {
4601                 \tl_set:Nn \l__spintent_geo_final_print_tl
4602                 {
4603                     \__spintent_spfig_sym:
4604                     \c_math_subscript_token { \l__spintent_geo_luaset_print_tl }
4605                 }
4606             }
4607             \str_case:VnF \l__spintent_geo_print_sym_str
4608             {
4609                 { auto }
4610                 {
4611                     \tl_set:Ne \l__spintent_spfig_desc_tl
4612                     {
4613                         \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4614                         \l__spintent_geo_luaset_sub_spoken_str
4615                     }
4616                 }
4617                 { none }
4618                 {
4619                     \tl_set:Ne \l__spintent_spfig_desc_tl
4620                     {
4621                         \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4622                         \l__spintent_geo_luaset_sub_spoken_str
4623                     }
4624                 }
4625             }
4626             {
4627                 \tl_set:Ne \l__spintent_spfig_desc_tl
4628                 {
4629                     \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
4630                     { triángulo } { cuadrado }
4631                 }
4632                 -
4633                 \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4634                 \l__spintent_geo_luaset_sub_spoken_str
4635             }
4636         }

```

With a table letter (F/P/B/L/T, non-empty `\l__spintent_geo_luaset_letra_word_str`), a forced `print-sym` raises `sp#1-sym-letra-conflict` — a table letter is never nested under a symbol. The visual is the value as-is; the description is its own word, connected as `\l__spintent_geo_luaset_letra_conector_str` requires, with the same spoken subscript as above when present.

```

4637         {
4638             \str_case:VnF \l__spintent_geo_print_sym_str
4639             {
4640                 { auto } { }
4641                 { none } { }
4642             }
4643             {
4644                 \msg_error:nne { spintent } { sp #1 -sym-letra-conflict }
4645                 { \c_backslash_str sp #1 }
4646             }
4647             \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }
4648             \tl_set:Ne \l__spintent_spfig_desc_tl
4649             {
4650                 \str_if_empty:NF \l__spintent_geo_luaset_letra_conector_str
4651                 { \l__spintent_geo_luaset_letra_conector_str - }
4652                 \l__spintent_geo_luaset_letra_word_str
4653                 \str_if_empty:NF \l__spintent_geo_luaset_sub_spoken_str
4654                 {
4655                     -
4656                     \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub - }
4657                     \l__spintent_geo_luaset_sub_spoken_str
4658                 }
4659             }
4660         }
4661     }

```

The variable `\l__spintent_spfig_intent_tl` is always the `_word-` shape: `\l__spintent_spfig_op_tl` alone when `\l__spintent_spfig_desc_tl` is empty, both joined by a dash otherwise. `silent-cmd` overrides `\l__spintent_spfig_intent_arg_tl` with `_-`; with `silent-cmd=false` and `concept false`, it becomes `_l__spintent_spfig_desc_tl-` (the descriptive part alone, without *área/perímetro*); with `concept true` it is `\l__spintent_spfig_intent_tl`.

```

4662     \tl_set:Ne \l__spintent_spfig_intent_tl
4663     {
4664         \tl_if_empty:NTF \l__spintent_spfig_desc_tl
4665         { _ \l__spintent_spfig_op_tl - }
4666         { _ \l__spintent_spfig_op_tl - \l__spintent_spfig_desc_tl - }
4667     }
4668     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
4669     { \tl_set:Nn \l__spintent_spfig_intent_arg_tl { _- } }
4670     {
4671         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
4672         { \tl_set:Ne \l__spintent_spfig_intent_arg_tl { \l__spintent_spfig_intent_tl } }
4673         { \tl_set:Ne \l__spintent_spfig_intent_arg_tl { _ \l__spintent_spfig_desc_tl - } }
4674     }
4675 }

```

(End of definition for `\l__spintent_spfig_build_simple:n`.)

`\l__spintent_spfig_build_refs:`

Splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spfig_pt_seq` and walks it once to build `\l__spintent_spfig_refs_tl`, the comma-separated `$pt1,$pt2,...` reference list, naming each vertex by position.

```

4676 \cs_new_protected:Nn \l__spintent_spfig_build_refs:
4677 {
4678     \seq_set_from_clist:NN \l__spintent_spfig_pt_seq \l__spintent_geo_luaset_points_clist
4679     \tl_clear:N \l__spintent_spfig_refs_tl
4680     \int_zero:N \l__spintent_spfig_pt_idx_int
4681     \seq_map_inline:Nn \l__spintent_spfig_pt_seq
4682     {
4683         \int_incr:N \l__spintent_spfig_pt_idx_int
4684         \tl_if_empty:NF \l__spintent_spfig_refs_tl
4685         { \tl_put_right:Nn \l__spintent_spfig_refs_tl { , } }
4686         \tl_put_right:Ne \l__spintent_spfig_refs_tl { $ pt \int_use:N \l__spintent_spfig_pt_idx_int
4687     }
4688 }

```

(End of definition for `\l__spintent_spfig_build_refs:`.)

__spintent_spfig_build_concept_vertices:n

Handles the vertices case. Receives the short command name in ‘#1’. Must be called after `\__spintent_spfig_build_concept_vertices:n` `read-arg`, when given, overrides the auto-detected figure name in `\__spintent_spfig_intent_tl`: with `concept true` it is still prefixed with `área-del-/perímetro-del-`, with `concept false` it is used verbatim. With `read-arg` empty, `\__spintent_spfig_intent_tl` is simply `\__spintent_geo_luaset_concept_str`.

```

4689 \cs_new_protected:Nn \__spintent_spfig_build_concept_vertices:n
4690 {
4691   \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
4692   { \tl_set:Ne \__spintent_spfig_intent_tl { \__spintent_geo_luaset_concept_str } }
4693   {
4694     \bool_if:cTF { \__spintent_sp #1 _concept_bool }
4695     {
4696       \tl_set:Ne \__spintent_spfig_intent_tl
4697       {
4698         \__spintent_spfig_op_tl -del-
4699         \tl_use:c { \__spintent_sp #1 _read_arg_tl }
4700       }
4701     }
4702     {
4703       \tl_set:Ne \__spintent_spfig_intent_tl { \tl_use:c { \__spintent_sp #1 _read_arg_tl } }
4704     }
4705   }

```

Determines `\__spintent_spfig_symbol_tl` (empty when none applies): for `print-sym=auto`, the symbol matching `\__spintent_geo_luaset_poly_sym_str` (`\triangle/ \mdlgwhtsquare`), or none when that is empty; `print-sym=none` always clears it; a forced `print-sym` requires it to match the actual vertex count, reporting `sp#1-sym-cardinality-conflict` otherwise.

```

4706   \str_case:VnF \__spintent_geo_print_sym_str
4707   {
4708     { auto }
4709     {
4710       \str_case:Vn \__spintent_geo_luaset_poly_sym_str
4711       {
4712         { triangle } { \tl_set:Nn \__spintent_spfig_symbol_tl { \triangle } }
4713         { square } { \tl_set:Nn \__spintent_spfig_symbol_tl { \mdlgwhtsquare } }
4714       }
4715       \str_if_empty:NT \__spintent_geo_luaset_poly_sym_str
4716       { \tl_clear:N \__spintent_spfig_symbol_tl }
4717     }
4718     { none }
4719     { \tl_clear:N \__spintent_spfig_symbol_tl }
4720   }
4721   {
4722     \str_if_eq:VVTF \__spintent_geo_print_sym_str \__spintent_geo_luaset_poly_sym_str
4723     { \tl_set:Nn \__spintent_spfig_symbol_tl { \__spintent_spfig_sym: } }
4724     {
4725       \msg_error:nne { spintent } { sp #1 -sym-cardinality-conflict }
4726       { \c_backslash_str sp #1 }
4727     }
4728   }

```

`silent-cmd` sets `\__spintent_spfig_intent_arg_tl` to `_(refs)`. Otherwise, with `read-arg` empty, `concept` chooses between `\__spintent_spfig_intent_tl:prefix(refs)` (prefixed with `prefix` when given) and `_(refs)` alone; with `read-arg` given, it is always `\__spintent_spfig_intent_tl:prefix(refs)` — `read-arg` is never silenced by `concept` alone, only by `silent-cmd`.

```

4729   \bool_if:cTF { \__spintent_sp #1 _silent_cmd_bool }
4730   {
4731     \tl_set:Ne \__spintent_spfig_intent_arg_tl { _ ( \__spintent_spfig_refs_tl ) }
4732   }
4733   {
4734     \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
4735     {
4736       \bool_if:cTF { \__spintent_sp #1 _concept_bool }
4737       {
4738         \tl_set:Ne \__spintent_spfig_intent_arg_tl
4739         {
4740           \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
4741           { \tl_use:c { \__spintent_sp #1 _prefix_tl } - }
4742           \__spintent_spfig_intent_tl :prefix( \__spintent_spfig_refs_tl )
4743         }
4744       }
4745     }

```

```

4746         \tl_set:Ne \l__spinttent_spfig_intent_arg_tl { _ ( \l__spinttent_spfig_refs_tl ) }
4747     }
4748 }
4749 {
4750     \tl_set:Ne \l__spinttent_spfig_intent_arg_tl
4751     {
4752         \tl_if_empty:cF { \l__spinttent_sp #1 _prefix_tl }
4753         { \tl_use:c { \l__spinttent_sp #1 _prefix_tl } - }
4754         \l__spinttent_spfig_intent_tl :prefix( \l__spinttent_spfig_refs_tl )
4755     }
4756 }
4757 }
4758 }

```

(End of definition for `\l__spinttent_spfig_build_concept_vertices:n`.)

`\l__spinttent_spfig_points_loop:n` Walks `\l__spinttent_spfig_pt_seq` a second time, calling `\l__spinttent_mathmlarg_point:nnn` once per vertex with its `pt` n reference name, looked up under `sp` $\#1$. Receives the short command name in ‘ $\#1$ ’.

```

4759 \cs_new_protected:Nn \l__spinttent_spfig_points_loop:n
4760 {
4761     \int_zero:N \l__spinttent_spfig_pt_idx_int
4762     \seq_map_inline:Nn \l__spinttent_spfig_pt_seq
4763     {
4764         \int_incr:N \l__spinttent_spfig_pt_idx_int
4765         \l__spinttent_mathmlarg_point:nnn
4766         { pt \int_use:N \l__spinttent_spfig_pt_idx_int }
4767         { sp #1 }
4768         { ##1 }
4769     }
4770 }

```

(End of definition for `\l__spinttent_spfig_points_loop:n`.)

`\l__spinttent_spfig_mathml:n` Receives the short command name in ‘ $\#1$ ’. Branches on whether `\l__spinttent_geo_luaset_points_clist` is empty.

Without vertices, `\l__spinttent_spfig_build_simple:n` builds the content; a `\MathMLintent` wraps it only when `concept` is `true` — with `concept=false` the content is typeset directly, with no intent at all. With vertices, `\l__spinttent_spfig_build_refs:` then `\l__spinttent_spfig_build_concept_vertices:n` build the reference list and the concept intent, applied with a single native `\MathMLintent`. Inside, `\l__spinttent_mathcal:n{⟨ $A$  |  $P$ ⟩}` is silenced with `\l__spinttent_luafun_mrow_intent_arg:nn` (it would otherwise be claimed by the queue meant for the symbol/vertices, the same ordering issue already found and fixed for `\spangle`’s own symbol); `\l__spinttent_spfig_symbol_tl` is silenced the same way when non-empty; the sequence is then walked by `\l__spinttent_spfig_points_loop:n`. All of this sits nested inside a subscript (`\c_math_subscript_token`), reached by the shared `mathml_filter` callback’s recursive lookup through a noad’s sub field, not just its nucleus.

`\l__spinttent_invisible_sep:` precedes both branches (a second sibling so the sole `<mrow>` does not collapse into `<math>`).

```

4771 \cs_new_protected:Nn \l__spinttent_spfig_mathml:n
4772 {
4773     \l__spinttent_invisible_sep: % (luamml issues #26)
4774     \clist_if_empty:NTF \l__spinttent_geo_luaset_points_clist
4775     {
4776         \l__spinttent_spfig_build_simple:n {#1}
4777         \MathMLintent { \l__spinttent_spfig_intent_arg_tl }
4778         {
4779             \l__spinttent_inner_sep:
4780             \str_if_eq:eeTF {#1} { Afig } { \l__spinttent_mathcal:n { A } } { \l__spinttent_mathcal:n
4781             \tl_if_empty:NF \l__spinttent_geo_final_print_tl
4782             {
4783                 \c_math_subscript_token { \l__spinttent_geo_final_print_tl }
4784             }
4785         }
4786     }
4787     {
4788         \l__spinttent_spfig_build_refs:
4789         \l__spinttent_spfig_build_concept_vertices:n {#1}
4790         \MathMLintent { \l__spinttent_spfig_intent_arg_tl }
4791         {
4792             \l__spinttent_inner_sep:
4793             \l__spinttent_luafun_mrow_intent_arg:nn

```

```

4794         { } { intent = ":silent" }
4795     {
4796         \str_if_eq:eeTF {#1} { Afig } { \__spintent_mathcal:n { A } } { \__spintent_mathcal
4797     }
4798     \c_math_subscript_token
4799     {
4800         \tl_if_empty:NF \__spintent_spfig_symbol_tl
4801         {
4802             \__spintent_luafun_mrow_intent_arg:nn
4803             { } { intent = ":silent" }
4804             { \tl_use:N \__spintent_spfig_symbol_tl }
4805         }
4806         \__spintent_spfig_points_loop:n {#1}
4807     }
4808 }
4809 }
4810 }

```

(End of definition for `\__spintent_spfig_mathml:n`.)

`\__spintent_spfig_exec:nnn` The function `\__spintent_spfig_exec:nnn` receives the short name of the command in `#1`, the optional keys in `#2`, and the argument in `#3`, delegating the rendering to `\__spintent_spfig_process:nn`.

```

4811 \cs_new_protected:Nn \__spintent_spfig_exec:nnn
4812 {
4813     \keys_set:nn { spintent / sp #1 } { #2 }
4814     \__spintent_spfig_process:nn { #1 } { #3 }
4815 }

```

(End of definition for `\__spintent_spfig_exec:nnn`.)

`\spAfig` Both public commands `\spAfig` and `\spPfig` enforce math mode, open a local group, and delegate execution to `\__spintent_spfig_exec:nnn`.

`\spPfig`

```

4816 \NewDocumentCommand \spAfig { 0{} d() }
4817 {
4818     \mode_if_math:TF
4819     {
4820         \group_begin:
4821         \attribute \__spintent_scope_attr_int = 1
4822         \tl_if_novalue:nTF {#2}
4823         { \__spintent_spfig_exec:nnn { Afig } { #1 } { } }
4824         { \__spintent_spfig_exec:nnn { Afig } { #1 } { #2 } }
4825         \group_end:
4826     }
4827     { \msg_error:nnn { spintent } { need-math-mode } { \spAfig } }
4828 }
4829 \NewDocumentCommand \spPfig { 0{} d() }
4830 {
4831     \mode_if_math:TF
4832     {
4833         \group_begin:
4834         \attribute \__spintent_scope_attr_int = 1
4835         \tl_if_novalue:nTF {#2}
4836         { \__spintent_spfig_exec:nnn { Pfig } { #1 } { } }
4837         { \__spintent_spfig_exec:nnn { Pfig } { #1 } { #2 } }
4838         \group_end:
4839     }
4840     { \msg_error:nnn { spintent } { need-math-mode } { \spPfig } }
4841 }

```

(End of definition for `\spAfig` and `\spPfig`. These functions are documented on page 25.)

11.45 The commands `\sprad` and `\spdiam`

The user-level commands `\sprad` and `\spdiam` provide a semantic interface for the radius and diameter of a circle. They delegate argument validation to the Lua module `spintent.lua`, thereby constructing a valid intent for MathCAT.

11.45.1 Definition of variables for \sprad and \spdiam

Each pair of variables is declared once for both \sprad and \spdiam via \clist_map_inline:nn.

```
\l__spintent_sprad_mode_str
\l__spintent_spdiam_mode_str
\l__spintent_sprad_upper_case_bool
\l__spintent_spdiam_upper_case_bool
\l__spintent_sprad_sub_index_str
\l__spintent_spdiam_sub_index_str
4842 \clist_map_inline:nn { rad , diam }
4843 {
4844     \str_new:c { l__spintent_sp #1 _mode_str }
4845     \str_new:c { l__spintent_sp #1 _sub_index_str }
4846     \tl_new:c { l__spintent_sp #1 _read_arg_tl }
4847 }
```

(End of definition for \l__spintent_sprad_mode_str and others.)

```
\l__spintent_sp_rad_diam_letra_tl
\l__spintent_sp_rad_diam_word_tl
```

Scratch variables used by the function __spintent_sp_rad_diam_print:n.

```
4848 \tl_new:N \l__spintent_sp_rad_diam_letra_tl
4849 \tl_new:N \l__spintent_sp_rad_diam_word_tl
```

(End of definition for \l__spintent_sp_rad_diam_letra_tl and \l__spintent_sp_rad_diam_word_tl.)

11.45.2 Definition of keys for \sprad and \spdiam

We define the shared *(keys)* read-cmd, upper-case, sub-index, read-sub, read-arg, concept, silent-cmd and print-sym.

```
4850 \clist_map_inline:nn { rad , diam }
4851 {
4852     \keys_define:nn { spintent / sp #1 }
4853     {
4854         read-cmd .choices:nn = { school , mathcat }
4855         {
4856             \int_case:nn { \l_keys_choice_int }
4857             {
4858                 {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
4859                 {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
4860             }
4861         },
4862         read-cmd .initial:n = school,
4863         read-cmd .value_required:n = true,
4864         read-cmd / unknown .code:n =
4865             \msg_error:nneee { spintent } { unknown-choice }
4866             { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4867         upper-case .bool_set:c = { l__spintent_sp #1 _upper_case_bool },
4868         upper-case .initial:n = false,
4869         upper-case .value_required:n = true,
4870         sub-index .code:n =
4871             { \str_set:cn { l__spintent_sp #1 _sub_index_str } {##1} },
4872         sub-index .initial:n = ,
4873         sub-index .value_required:n = true,
4874         read-sub .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
4875         read-sub .initial:n = false,
4876         read-sub .value_required:n = true,
4877         read-arg .code:n =
4878             { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
4879         read-arg .initial:n = ,
4880         read-arg .value_required:n = true,
4881         concept .bool_set:c = { l__spintent_sp #1 _concept_bool },
4882         concept .initial:n = true,
4883         concept .value_required:n = true,
4884         silent-cmd .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
4885         silent-cmd .initial:n = false,
4886         silent-cmd .value_required:n = true,
4887         print-sym .bool_set:c = { l__spintent_sp #1 _print_sym_bool },
4888         print-sym .initial:n = false,
4889         print-sym .value_required:n = true,
4890         unknown .code:n =
4891             {
4892                 \str_set:Nn \l__spintent_command_name_str { sp #1 }
4893                 \__spintent_unknown_keys_cmd:n {##1}
4894             },
4895     }
4896 }
```

🟢 The keys `print-sym` are defined for both commands for simplicity, but has no effect on \sprad.

(End of definition for read-cmd and others.)

11.45.3 Internal functions for \sprad and \spdiam

_spintent_sp_rad_diam_print:n

The function `\_spintent_sp_rad_diam_print:n` sets `\l__spintent_sp_rad_diam_letra_tl` to the bare letter — `r/R` for `\sprad`, `d/D` for `\spdiam` — per `upper-case`, and, for `\spdiam` with `print-sym true`, silently typesets `\diameter` first. Receives the short command name in ‘`#1`’ (rad/diam).

```

4897 \cs_new_protected:Nn \_spintent_sp_rad_diam_print:n
4898 {
4899   \bool_if:cTF { \l__spintent_sp #1_upper_case_bool }
4900   {
4901     \str_if_eq:eeTF {#1} { rad }
4902     { \tl_set:Nn \l__spintent_sp_rad_diam_letra_tl { R } }
4903     { \tl_set:Nn \l__spintent_sp_rad_diam_letra_tl { D } }
4904   }
4905   {
4906     \str_if_eq:eeTF {#1} { rad }
4907     { \tl_set:Nn \l__spintent_sp_rad_diam_letra_tl { r } }
4908     { \tl_set:Nn \l__spintent_sp_rad_diam_letra_tl { d } }
4909   }
4910   \_spintent_invisible_sep:
4911   \str_if_eq:eeT {#1} { diam }
4912   {
4913     \bool_if:NT \l__spintent_spdiam_print_sym_bool
4914     { \MathMLintent { :silent } { \diameter } }
4915   }

```

With `silent-cmd true`, the intent is `_` — the letter alone through the shared queue when there is no `sub-index`, or letter and subscript together in one native `\MathMLintent` (subscript silenced) when there is.

```

4916   \bool_if:cTF { \l__spintent_sp #1_silent_cmd_bool }
4917   {
4918     \str_if_empty:cTF { \l__spintent_sp #1_sub_index_str }
4919     {
4920       \_spintent_luafun_mrow_intent_arg:nn
4921       { } { intent = "_" }
4922       { \l__spintent_sp_rad_diam_letra_tl }
4923     }
4924     {
4925       \MathMLintent { _- }
4926       {
4927         \_spintent_inner_sep:
4928         \l__spintent_sp_rad_diam_letra_tl
4929         \c_math_subscript_token
4930         {
4931           \MathMLintent { :silent }
4932           { \str_use:c { \l__spintent_sp #1_sub_index_str } }
4933         }
4934       }
4935     }
4936   }

```

Otherwise, with `concept true` (the default), it verbalizes `radio`/`diámetro` (`read-arg` replaces this word when given), with `sub-index`, if present, appended right after (`read-sub` choosing `sub-⟨value⟩` or `⟨value⟩` alone) — the letter itself is never read here. With no `sub-index`, the shared queue tags the letter alone; with one, letter and subscript are wrapped together in one native `\MathMLintent` for `read-cmd=school` (subscript silenced, already spoken), or the shared queue tags the letter alone with the subscript left bare for `read-cmd=mathcat`.

```

4937   {
4938     \bool_if:cTF { \l__spintent_sp #1_concept_bool }
4939     {
4940       \tl_if_empty:cTF { \l__spintent_sp #1_read_arg_tl }
4941       {
4942         \str_if_eq:eeTF {#1} { rad }
4943         { \tl_set:Nn \l__spintent_sp_rad_diam_word_tl { radio } }
4944         { \tl_set:Nn \l__spintent_sp_rad_diam_word_tl { diámetro } }
4945       }
4946       { \tl_set:Ne \l__spintent_sp_rad_diam_word_tl { \tl_use:c { \l__spintent_sp #1_read_arg_tl } } }
4947       \str_if_empty:cTF { \l__spintent_sp #1_sub_index_str }
4948       {
4949         \_spintent_luafun_mrow_intent_arg:nn
4950         { } { intent = "_" \l__spintent_sp_rad_diam_word_tl - :prefix" }
4951         { \l__spintent_sp_rad_diam_letra_tl }
4952       }
4953     }

```

```

4954 \str_if_eq:eeTF
4955 { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
4956 {
4957   \MathMLintent
4958   {
4959     _ \l__spintent_sp_rad_diam_word_tl -
4960     \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
4961     \str_use:c { l__spintent_sp #1 _sub_index_str }
4962     :prefix
4963   }
4964   {
4965     \__spintent_inner_sep:
4966     \l__spintent_sp_rad_diam_letra_tl
4967     \c_math_subscript_token
4968     {
4969       \MathMLintent { :silent }
4970       { \str_use:c { l__spintent_sp #1 _sub_index_str } }
4971     }
4972   }
4973 }
4974 {
4975   \__spintent_luafun_mrow_intent_arg:nn
4976   { } { intent = "_ \l__spintent_sp_rad_diam_word_tl - :prefix" }
4977   { \l__spintent_sp_rad_diam_letra_tl }
4978   \c_math_subscript_token
4979   { \str_use:c { l__spintent_sp #1 _sub_index_str } }
4980 }
4981 }
4982 }

```

With `concept false`, the letter itself is read instead — but only when `read-cmd` is `school`, built the same way from the letter (with `sub-index`, letter and subscript wrapped together, subscript silenced). With `read-cmd mathcat`, the intent is left empty: MathCAT applies its own rules to the letter and its subscript, unmarked by `us`.

```

4983 {
4984   \str_if_empty:cTF { l__spintent_sp #1 _sub_index_str }
4985   {
4986     \str_if_eq:eeTF
4987     { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
4988     {
4989       \__spintent_luafun_mrow_intent_arg:nn
4990       { } { intent = "_ \l__spintent_sp_rad_diam_letra_tl - :prefix" }
4991       { \l__spintent_sp_rad_diam_letra_tl }
4992     }
4993     { \l__spintent_sp_rad_diam_letra_tl }
4994   }
4995   {
4996     \str_if_eq:eeTF
4997     { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
4998     {
4999       \MathMLintent
5000       {
5001         _ \l__spintent_sp_rad_diam_letra_tl -
5002         \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5003         \str_use:c { l__spintent_sp #1 _sub_index_str }
5004         :prefix
5005       }
5006       {
5007         \__spintent_inner_sep:
5008         \l__spintent_sp_rad_diam_letra_tl
5009         \c_math_subscript_token
5010         {
5011           \MathMLintent { :silent }
5012           { \str_use:c { l__spintent_sp #1 _sub_index_str } }
5013         }
5014       }
5015     }
5016   }
5017   \l__spintent_sp_rad_diam_letra_tl
5018   \c_math_subscript_token
5019   { \str_use:c { l__spintent_sp #1 _sub_index_str } }

```

```

5020         }
5021     }
5022 }
5023 }
5024 }

```

(End of definition for `\__spintent_sp_rad_diam_print:n`.)

`\__spintent_sp_rad_diam_exec:nn` The function `\__spintent_sp_rad_diam_exec:nn` receives the *short name* of the command passed in ‘#1’ and the optional *(keys)* passed in ‘#2’ then delegating the rendering to `\__spintent_sp_rad_diam_print:n`.

```

5025 \cs_new_protected:Nn \__spintent_sp_rad_diam_exec:nn
5026 {
5027     \keys_set:nn { spintent / sp #1 } {#2}
5028     \__spintent_sp_rad_diam_print:n {#1}
5029 }

```

(End of definition for `\__spintent_sp_rad_diam_exec:nn`.)

`\sprad` Both public commands `\sprad` and `\spdiam` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sp_rad_diam_exec:nn`.

```

5030 \NewDocumentCommand \sprad { O{} }
5031 {
5032     \mode_if_math:TF
5033     {
5034         \group_begin:
5035         \__spintent_sp_rad_diam_exec:nn { rad } {#1}
5036         \group_end:
5037     }
5038     { \msg_error:nnn { spintent } { need-math-mode } { \sprad } }
5039 }
5040 \NewDocumentCommand \spdiam { O{} }
5041 {
5042     \mode_if_math:TF
5043     {
5044         \group_begin:
5045         \__spintent_sp_rad_diam_exec:nn { diam } {#1}
5046         \group_end:
5047     }
5048     { \msg_error:nnn { spintent } { need-math-mode } { \spdiam } }
5049 }

```

(End of definition for `\sprad` and `\spdiam`. These functions are documented on page ??.)

11.46 The command `\spcirc`

The user command `\spcirc` provides a semantic interface for a circle or a circumference, using the delimited-optional argument syntax `d()` already established by `\spmcd/\spmcm` and `\spAfig/\spPfig`. The argument holds a required center point and an optional radius, separated by a comma. The radius is never required to declare its own kind via a key: its shape alone decides whether it is a pure number, a measure (number + unit), a bare letter, or a two-letter segment — reusing, respectively, `\spnum`, `\spunit`, and `\spside/\spmside` directly as sub-commands, rather than reimplementing any of their parsing.

11.46.1 Definition of variables for `\spcirc`

`\__spintent_geo_luaset_circ_radio_type_str` Shared outputs of `\__spintent_luafun_geo_circ_parse_and_set:n`. `\__spintent_geo_luaset_circ_radio_type_str` holds one of *number*, *measure*, *label*, *segment*, or an empty string when no radius was given. `\__spintent_geo_luaset_circ_radio_raw_str` holds the corresponding raw text, ready to be forwarded to the sub-command that renders it.

```

5050 \str_new:N \__spintent_geo_luaset_circ_radio_type_str
5051 \str_new:N \__spintent_geo_luaset_circ_radio_raw_str

```

(End of definition for `\__spintent_geo_luaset_circ_radio_type_str` and `\__spintent_geo_luaset_circ_radio_raw_str`.)

`\__spintent_spcirc_mode_str` The variable `\__spintent_spcirc_mode_str` stores the value set by the key `read-cmd`; the variable `\__spintent_spcirc_prefix_tl` stores the value set by the key `prefix`. The variable `\__spintent_spcirc_print_sym_str` stores the value set by the key `print-sym` (usual or *odot*). The variable `\__spintent_spcirc_read_sty_str` stores the value set by the key `read-sty` (*circun* or *circle*). The variable `\__spintent_spcirc_radio_sty_str` stores the value set by the key `radio-sty`, only meaningful when the radius is a segment. The variable `\__spintent_spcirc_sub_index_str` stores the value set by the key `sub-index`. The variable `\__spintent_spcirc_articulo_str` is set internally by `\__spintent_spcirc_word_and_article:` to *la* or *el*, matching the grammatical gender of the chosen word.

```

5052 \str_new:N \__spintent_spcirc_mode_str
5053 \tl_new:N \__spintent_spcirc_prefix_tl

```

```

5054 \str_new:N \__spintent_spcirc_print_sym_str
5055 \str_new:N \__spintent_spcirc_read_sty_str
5056 \str_new:N \__spintent_spcirc_radio_sty_str
5057 \str_new:N \__spintent_spcirc_sub_index_str
5058 \str_new:N \__spintent_spcirc_articulo_str
5059 \tl_new:N \__spintent_spcirc_read_arg_tl

```

(End of definition for `\__spintent_spcirc_mode_str` and others.)

`\__spintent_spcirc_word_tl`

Scratch variables used by `\__spintent_spcirc_process:n` to assemble the chosen word (circunferencia/círculo or the `read-arg` override) and the final literal `intent` string.

```

5060 \tl_new:N \__spintent_spcirc_word_tl

```

(End of definition for `\__spintent_spcirc_word_tl`.)

11.46.2 Definition of error messages for `\spcirc`

`spcirc-invalid-arg`

Raised when the argument does not match a single center point, optionally followed by a comma and a valid radius shape.

```

5061 \msg_new:nnnn { spintent } { spcirc-invalid-arg }
5062 {
5063   Invalid~argument~format~in~\token_to_str:N \spcirc:~'!#1'~\msg_line_context:.
5064 }
5065 {
5066   See~the~documentation~of~\token_to_str:N \spcirc~for~the~accepted~argument~forms.
5067 }

```

(End of definition for `spcirc-invalid-arg`.)

11.46.3 Definition of keys for `\spcirc`

read-cmd
prefix
print-sym
read-sty
diameter
radio-sty
sub-index
read-sub
read-arg
silent-cmd
concept
spside
spmside
unknown

The key `print-sym` chooses the visual symbol: usual for a plain C, `odot` (default) for `\odot`. The key `read-sty` chooses the verbalized word and its article: `circun` (default) for «la circunferencia», `circle` for «el círculo». The key `radio-sty` chooses, only when the radius is a two-letter segment, whether it is rendered as the *object* (side, via `\spside`) or its *measure* (mside, default, via `\spmside`). The key `sub-index` sets a subscript on the symbol; the key `read-sub` controls whether it is read with the word «sub» or as a plain number, matching the same convention as `\spAfig`.

```

5068 \keys_define:nn { spintent / spcirc }
5069 {
5070   read-cmd      .choices:nn = { school , mathcat }
5071   {
5072     \int_case:nn { \l_keys_choice_int }
5073     {
5074       {1} { \str_set:Nn \__spintent_spcirc_mode_str { school } }
5075       {2} { \str_set:Nn \__spintent_spcirc_mode_str { mathcat } }
5076     }
5077   },
5078   read-cmd      .initial:n = school,
5079   read-cmd      .value_required:n = true,
5080   prefix        .code:n =
5081   { \__spintent_sanitize_affix:Nn \__spintent_spcirc_prefix_tl {#1} },
5082   prefix        .initial:n = ,
5083   prefix        .value_required:n = true,
5084   print-sym     .choices:nn = { usual , odot }
5085   {
5086     \int_case:nn { \l_keys_choice_int }
5087     {
5088       {1} { \str_set:Nn \__spintent_spcirc_print_sym_str { usual } }
5089       {2} { \str_set:Nn \__spintent_spcirc_print_sym_str { odot } }
5090     }
5091   },
5092   print-sym     .initial:n = odot,
5093   print-sym     .value_required:n = true,
5094   read-sty      .choices:nn = { circun , circle }
5095   {
5096     \int_case:nn { \l_keys_choice_int }
5097     {
5098       {1} { \str_set:Nn \__spintent_spcirc_read_sty_str { circun } }
5099       {2} { \str_set:Nn \__spintent_spcirc_read_sty_str { circle } }
5100     }
5101   },
5102   read-sty      .initial:n = circun,
5103   read-sty      .value_required:n = true,

```

```

5104     diameter      .bool_set:N = \l__spintenc_spcirc_diameter_bool,
5105     diameter      .initial:n = false,
5106     diameter      .value_required:n = true,
5107     radio-sty      .choices:nn = { side , mside }
5108     {
5109         \int_case:nn { \l_keys_choice_int }
5110         {
5111             {1} { \str_set:Nn \l__spintenc_spcirc_radio_sty_str { side } }
5112             {2} { \str_set:Nn \l__spintenc_spcirc_radio_sty_str { mside } }
5113         }
5114     },
5115     radio-sty      .initial:n = mside,
5116     radio-sty      .value_required:n = true,
5117     sub-index      .code:n =
5118     { \str_set:Nn \l__spintenc_spcirc_sub_index_str {#1} },
5119     sub-index      .initial:n = ,
5120     sub-index      .value_required:n = true,
5121     read-sub       .bool_set:N = \l__spintenc_spcirc_read_sub_bool,
5122     read-sub       .initial:n = false,
5123     read-sub       .value_required:n = true,
5124     read-arg       .code:n =
5125     { \__spintenc_sanitizc_affix:Nn \l__spintenc_spcirc_read_arg_tl {#1} },
5126     read-arg       .initial:n = ,
5127     read-arg       .value_required:n = true,
5128     silent-cmd     .bool_set:N = \l__spintenc_spcirc_silent_cmd_bool,
5129     silent-cmd     .initial:n = false,
5130     silent-cmd     .value_required:n = true,
5131     concept        .bool_set:N = \l__spintenc_spcirc_concept_bool,
5132     concept        .initial:n = true,
5133     concept        .value_required:n = true,
5134     spside         .meta:nn = { spintenc / spside } { #1 },
5135     spside         .value_required:n = true,
5136     spmside        .meta:nn = { spintenc / spmside } { #1 },
5137     spmside        .value_required:n = true,
5138     unknown        .code:n =
5139     {
5140         \str_set:Nn \l__spintenc_command_name_str { spcirc }
5141         \__spintenc_unknown_keys_cmd:n {#1}
5142     },
5143 }

```

(End of definition for read-cmd and others.)

11.46.4 Internal functions for \spcirc

`\l__spintenc_spcirc_has_arg_bool` `\l__spintenc_spcirc_intent_tl` `\l__spintenc_spcirc_intent_arg_tl` `\l__spintenc_spcirc_connector_tl` `\l__spintenc_spcirc_connector_arg_tl` `\l__spintenc_spcirc_radio_arg_tl`
`\l__spintenc_spcirc_has_arg_bool` records whether the `d()` argument was given at all (`\spcirc` with no (...) at all has neither a center nor a radius). `\l__spintenc_spcirc_intent_tl` holds the word (circunferencia or círculo, with an optional `sub-index` suffix) paired with the symbol. `\l__spintenc_spcirc_connector_tl` holds de-centro-en, paired with the (center) group; the center is read through `\__spintenc_mathmlarg_point:nnn`, referenced as `$pt`, not concatenated into either word — it can be subscripted or primed (e.g. O_{1} , as in the Euler circle) without breaking anything. The `_arg_tl` variables hold each word's final wrapped form; a third, `\l__spintenc_spcirc_radio_arg_tl`, is spoken directly on the , delimiter after the center, replacing its own reading rather than pairing with a group.

```

5144 \bool_new:N \l__spintenc_spcirc_has_arg_bool
5145 \tl_new:N \l__spintenc_spcirc_intent_tl
5146 \tl_new:N \l__spintenc_spcirc_intent_arg_tl
5147 \tl_new:N \l__spintenc_spcirc_connector_tl
5148 \tl_new:N \l__spintenc_spcirc_connector_arg_tl
5149 \tl_new:N \l__spintenc_spcirc_radio_arg_tl

```

(End of definition for \l__spintenc_spcirc_has_arg_bool and others.)

`\__spintenc_spcirc_sym:` Typesets `\bigodot` or the plain letter C, according to `print-sym`.

```

5150 \cs_new_protected:Nn \__spintenc_spcirc_sym:
5151 {
5152     \str_if_eq:VnTF \l__spintenc_spcirc_print_sym_str { odot }
5153     {
5154         \group_begin:
5155         \textstyle\bigodot
5156         \group_end:
5157     }
5158     { C }
5159 }

```

(End of definition for `\__spintent_spcirc_sym:`)

`\__spintent_spcirc_word_and_article:` Sets `\l__spintent_spcirc_word_tl` to `read-arg` or `circunferencia/círculo` from `read-sty`, and `\l__spintent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word.

```
5160 \cs_new_protected:Nn \__spintent_spcirc_word_and_article:
5161 {
5162   \tl_if_empty:NTF \l__spintent_spcirc_read_arg_tl
5163   {
5164     \str_if_eq:VnTF \l__spintent_spcirc_read_sty_str { circun }
5165     {
5166       \tl_set:Nn \l__spintent_spcirc_word_tl { circunferencia }
5167       \str_set:Nn \l__spintent_spcirc_articulo_str { la }
5168     }
5169     {
5170       \tl_set:Nn \l__spintent_spcirc_word_tl { círculo }
5171       \str_set:Nn \l__spintent_spcirc_articulo_str { el }
5172     }
5173   }
5174   {
5175     \tl_set:Nn \l__spintent_spcirc_word_tl { \l__spintent_spcirc_read_arg_tl }
5176     \str_if_eq:VnTF \l__spintent_spcirc_read_sty_str { circun }
5177     { \str_set:Nn \l__spintent_spcirc_articulo_str { la } }
5178     { \str_set:Nn \l__spintent_spcirc_articulo_str { el } }
5179   }
5180 }
```

(End of definition for `\__spintent_spcirc_word_and_article:`)

`\__spintent_spcirc_radio_print:` Dispatches on `\l__spintent_geo_luaset_circ_radio_type_str`, reusing `\spnum/\spunit/\spside/\spmside` directly rather than reimplementing any parsing.

```
5181 \cs_new_protected:Nn \__spintent_spcirc_radio_print:
5182 {
5183   \str_case:Vn \l__spintent_geo_luaset_circ_radio_type_str
5184   {
5185     { number } { \exp_args:Ne \spnum { \l__spintent_geo_luaset_circ_radio_raw_str } }
5186     { measure } { \exp_args:Ne \spunit { \l__spintent_geo_luaset_circ_radio_raw_str } }
5187     { label } { \l__spintent_geo_luaset_circ_radio_raw_str }
5188     { segment }
5189     {
5190       \str_if_eq:VnTF \l__spintent_spcirc_radio_sty_str { side }
5191       { \exp_args:Ne \spside { \l__spintent_geo_luaset_circ_radio_raw_str } }
5192       { \exp_args:Ne \spmside { \l__spintent_geo_luaset_circ_radio_raw_str } }
5193     }
5194   }
5195 }
```

(End of definition for `\__spintent_spcirc_radio_print:`)

`\__spintent_spcirc_build_concept:` Builds two words: `\l__spintent_spcirc_intent_tl` (via `\__spintent_spcirc_word_and_article:`, plus an optional `sub-index` suffix), paired with the symbol, and `\l__spintent_spcirc_connector_tl` (`de-centro-en`), paired with the `(center)` group — built even when `\l__spintent_spcirc_has_arg_bool` is `false`, though `\__spintent_spcirc_mathml:` never uses it in that case. Neither carries the center's own text: it is read separately, by reference, through `\__spintent_mathmlarg_point:nnn` — so a subscripted or primed center (e.g. O_{1}), as in the Euler circle) is never an issue.

```
5196 \cs_new_protected:Nn \__spintent_spcirc_build_concept:
5197 {
5198   \__spintent_spcirc_word_and_article:
5199   \tl_set:Ne \l__spintent_spcirc_intent_tl { \l__spintent_spcirc_word_tl }
5200   \str_if_empty:NF \l__spintent_spcirc_sub_index_str
5201   {
5202     \tl_put_right:Ne \l__spintent_spcirc_intent_tl
5203     {
5204       -
5205       \bool_if:NT \l__spintent_spcirc_read_sub_bool { sub - }
5206       \l__spintent_spcirc_sub_index_str
5207     }
5208   }
5209   \tl_set:Nn \l__spintent_spcirc_connector_tl { de-centro-en }
```

`silent-cmd true` overrides `\__spintent_spcirc_intent_arg_tl`/`\__spintent_spcirc_radio_arg_tl` with `_-` and `\__spintent_spcirc_connector_arg_tl` with `_($pt)` (the center is always read), regardless of `read-arg`.

```

5210 \bool_if:NTF \__spintent_spcirc_silent_cmd_bool
5211 {
5212   \tl_set:Nn \__spintent_spcirc_intent_arg_tl { _- }
5213   \tl_set:Ne \__spintent_spcirc_connector_arg_tl { _($ pt) }
5214   \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _- }
5215 }

```

With `concept true`, `\__spintent_spcirc_intent_arg_tl` is `\__spintent_spcirc_intent_tl:prefix` (bare — the group it wraps has nothing to reference, only a silenced symbol), prefixed with `prefix` when given; `\__spintent_spcirc_connector_arg_tl` is `\__spintent_spcirc_connector_tl:prefix($pt)`; and `\__spintent_spcirc_radio_arg_tl` — spoken directly on the `,` delimiter, replacing its own reading rather than pairing with any group — is the literal `_y-radio-`, or `_y-diámetro-` when `diameter` is `true`, read as its own phrase right before `\__spintent_spcirc_radio_print:`'s independent reading follows as the next sibling.

```

5216 {
5217   \bool_if:NTF \__spintent_spcirc_concept_bool
5218   {
5219     \tl_set:Ne \__spintent_spcirc_intent_arg_tl
5220     {
5221       \tl_if_empty:NF \__spintent_spcirc_prefix_tl
5222       { \tl_use:N \__spintent_spcirc_prefix_tl - }
5223       \__spintent_spcirc_intent_tl :prefix
5224     }
5225     \tl_set:Ne \__spintent_spcirc_connector_arg_tl
5226     { \__spintent_spcirc_connector_tl :prefix( $ pt ) }
5227     \bool_if:NTF \__spintent_spcirc_diameter_bool
5228     { \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
5229     { \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _y-radio- } }
5230   }

```

With `concept false` and `read-arg` empty, the same `_/_($pt)` silencing applies as `silent-cmd`; with `read-arg` given, none of the three is silenced instead — the connector and `y-radio/y-diámetro` words are fixed structural pieces, not customized by `read-arg`, but once a real word is being spoken there is nothing left to silence around it (same convention as `\__spintent_spside_process:nn`).

```

5231 {
5232   \tl_if_empty:NTF \__spintent_spcirc_read_arg_tl
5233   {
5234     \tl_set:Nn \__spintent_spcirc_intent_arg_tl { _- }
5235     \tl_set:Ne \__spintent_spcirc_connector_arg_tl { _($ pt) }
5236     \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _- }
5237   }
5238   {
5239     \tl_set:Ne \__spintent_spcirc_intent_arg_tl { \__spintent_spcirc_intent_tl }
5240     \tl_set:Ne \__spintent_spcirc_connector_arg_tl
5241     { \__spintent_spcirc_connector_tl :prefix( $ pt ) }
5242     \bool_if:NTF \__spintent_spcirc_diameter_bool
5243     { \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
5244     { \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _y-radio- } }
5245   }
5246 }
5247 }
5248 }

```

(End of definition for `\__spintent_spcirc_build_concept:`)

`\__spintent_spcirc_process:n`

Receives the `d()` argument in `#1`, an explicit empty group when omitted (`\spcirc` itself converts `-NoValue-` to `{ }` before calling `\__spintent_spcirc_exec:nn`) — `\__spintent_spcirc_has_arg_bool` set to `false` in that case, and `\__spintent_luafun_geo_circ_parse_and_set:n` never called at all — there is nothing to parse. Otherwise calls it, reports `spcirc-invalid-arg` on failure, then delegates to `\__spintent_spcirc_mathml:`, which calls `\__spintent_spcirc_build_concept:` itself.

```

5249 \cs_new_protected:Nn \__spintent_spcirc_process:n
5250 {
5251   \tl_if_empty:NTF {#1}
5252   {
5253     \bool_set_false:N \__spintent_spcirc_has_arg_bool
5254   }
5255   {
5256     \bool_set_true:N \__spintent_spcirc_has_arg_bool

```

```

5257     \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#1} }
5258     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
5259     {
5260         \msg_error:nne { spintent } { spcirc-invalid-arg } { \exp_not:n {#1} }
5261     }
5262 }
5263 \__spintent_spcirc_mathml:
5264 }

```

(End of definition for `\__spintent_spcirc_process:n`.)

`\__spintent_spcirc_mathml:` Calls `\__spintent_spcirc_build_concept:`, then wraps the symbol (`\l__spintent_spcirc_intent_arg_tl`) in its own native `\MathMLintent` — this alone is the whole result when `\l__spintent_spcirc_has_arg_bool` is `false` (`\spcirc` with no `(...)` at all: no center, no radius).

Otherwise, the `(center)` group (`\l__spintent_spcirc_connector_arg_tl`) follows as a separate sibling, never nested inside the symbol's own `\MathMLintent` — or MathCAT falls back to its own structural reading of the underlying `<mover>`/`<msub>` instead, the same problem already found for `\spangle`'s symbol and `\sparc`'s accent. When a radius is present, the `,` delimiter after the center carries `\l__spintent_spcirc_radio_arg_tl` directly (replacing its own reading, not paired with any group), and `\__spintent_spcirc_radio_print:` follows as an ordinary sibling right after — wrapping it in another `\MathMLintent` would lose the reading of whatever it produces (itself a native `\MathMLintent`, from `\spnum`/`\spunit`, nested inside another one does not compose).

The symbol and, when `sub-index` is given, its subscript number are each silenced with `\__spintent_luafun_mrow_intent_arg:nn` *separately* — one queue entry per leaf. The two are reached differently by the shared `mathml_filter` call-back's recursive lookup (the symbol via `nucleus`, the subscript via `sub`), so a single entry covering both leaves nothing registered for the second one, which the recursion then assigns whatever comes next in the queue instead — shifting every claim after it by one. The `(` delimiter is silenced the same way, but *without* the usual `{...}` around the character: `luamml` only keeps a delimiter's own math class (giving `<mo>` instead of `<mi>`, needed for MathCAT to read the group at all) when it is not itself sitting inside a brace group.

The final `)` — following `\__spintent_spcirc_radio_print:`, whose own output the shared queue does not reliably skip over — is typeset as `\rparen` inside its own native `\MathMLintent{:silent}` instead of a bare `)`: `\rparen` keeps its own math class through the wrapping, unlike a bare delimiter character, and this sits outside the shared queue entirely, so nothing in between can claim an entry meant for it.

The center itself is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt`.

```

5265 \cs_new_protected:Nn \__spintent_spcirc_mathml:
5266 {
5267     \__spintent_invisible_sep: % (luamml issues #26)
5268     \__spintent_spcirc_build_concept:
5269     \MathMLintent { \l__spintent_spcirc_intent_arg_tl }
5270     {
5271         \__spintent_inner_sep:
5272         \__spintent_luafun_mrow_intent_arg:nn
5273         { } { intent = ":silent" }
5274         { \__spintent_spcirc_sym: }
5275         \str_if_empty:NF \l__spintent_spcirc_sub_index_str
5276         {
5277             \c_math_subscript_token
5278             {
5279                 \__spintent_luafun_mrow_intent_arg:nn
5280                 { } { intent = ":silent" }
5281                 { \l__spintent_spcirc_sub_index_str }
5282             }
5283         }
5284     }
5285     \bool_if:NT \l__spintent_spcirc_has_arg_bool
5286     {
5287         \str_if_empty:NTF \l__spintent_geo_luaset_circ_radio_type_str
5288         {
5289             \MathMLintent { \l__spintent_spcirc_connector_arg_tl }
5290             {
5291                 \__spintent_inner_sep:
5292                 \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } (
5293                 \__spintent_mathmlarg_point:nnV { pt } { spcirc } \l__spintent_geo_luaset_points:nn
5294                 \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } )
5295             }
5296         }
5297         {
5298             \MathMLintent { \l__spintent_spcirc_connector_arg_tl }
5299             {

```

```

5300         \__spinttent_inner_sep:
5301         \__spinttent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } (
5302         \__spinttent_mathmlarg_point:nnV { pt } { spcirc } \__spinttent_geo_luaset_points_
5303     }
5304     \MathMLintent { \__spinttent_spcirc_radio_arg_tl } { , }
5305     \__spinttent_spcirc_radio_print:
5306     \MathMLintent { :silent } { \rparen }
5307 }
5308 }
5309 }

```

(End of definition for `\__spinttent_spcirc_mathml:`.)

`\__spinttent_spcirc_exec:nn` The function `\__spinttent_spcirc_exec:nn` receives the optional keys in #1 and the argument in #2, delegating the rendering to `\__spinttent_spcirc_print:n`.

```

5310 \cs_new_protected:Nn \__spinttent_spcirc_exec:nn
5311 {
5312     \keys_set:nn { spinttent / spcirc } { #1 }
5313     \__spinttent_spcirc_process:n { #2 }
5314 }

```

(End of definition for `\__spinttent_spcirc_exec:nn`.)

`\spcirc` The public command `\spcirc` enforces math mode, opens a local group, and delegates execution to `\__spinttent_spcirc_exec:nn`. It follows the same `d()`-argument pattern as `\spmcld`/`\spmcm` and `\spAfig`/`\spPfig`; an omitted radius forwards an explicit empty group rather than the `-NoValue-` marker.

```

5315 \NewDocumentCommand \spcirc { 0{ } d() }
5316 {
5317     \mode_if_math:TF
5318     {
5319         \group_begin:
5320         \attribute \__spinttent_scope_attr_int = 1
5321         \tl_if_novalue:nTF {#2}
5322         { \__spinttent_spcirc_exec:nn { #1 } { } }
5323         { \__spinttent_spcirc_exec:nn { #1 } { #2 } }
5324     } \group_end:
5325 }
5326 { \msg_error:nnn { spinttent } { need-math-mode } { \spcirc } }
5327 }

```

(End of definition for `\spcirc`. This function is documented on page ??.)

11.47 The commands `\spAcirc` and `\spPcirc`

The user commands `\spAcirc` and `\spPcirc` typeset the area and the perimeter of a circle or a circumference. The argument follows `\spcirc`'s own point-and-radius grammar for a real center (with an optional radius); when no comma is present and no valid center is found, it falls back to a bare number/letter shortcut, reusing the same Lua entry point as `\spAfig`/`\spPfig`. A table letter matched by that fallback is rejected, since it has no meaning for a circle.

11.47.1 Definition of variables for `\spAcirc` and `\spPcirc`

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `read-sty`, `radio-sty`, `sub-index`, `prefix` and `read-arg`, respectively.

```

5328 \clist_map_inline:nn { Acirc , Pcirc }
5329 {
5330     \str_new:c { \__spinttent_sp #1 _mode_str }
5331     \str_new:c { \__spinttent_sp #1 _print_sym_str }
5332     \str_new:c { \__spinttent_sp #1 _read_sty_str }
5333     \str_new:c { \__spinttent_sp #1 _radio_sty_str }
5334     \str_new:c { \__spinttent_sp #1 _sub_index_str }
5335     \tl_new:c { \__spinttent_sp #1 _prefix_tl }
5336     \tl_new:c { \__spinttent_sp #1 _read_arg_tl }
5337 }

```

(End of definition for `\__spinttent_spAcirc_mode_str` and others.)

The variable `\__spinttent_geo_circfig_mode_str` stores whether the argument was resolved as a real circle (`circle`) or as the bare number/letter shortcut (`bare`).

```

5338 \str_new:N \__spinttent_geo_circfig_mode_str

```

(End of definition for `\__spinttent_geo_circfig_mode_str`.)

11.47.2 Definition of error messages for \spAcirc and \spPcirc

spAcirc-invalid-arg
spPcirc-invalid-arg

The error message shown when the argument does not match any of the accepted forms: empty, a center point (optionally followed by a comma and a radius), or a bare number/letter.

```

5339 \clist_map_inline:nn { Acirc , Pcirc }
5340 {
5341   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
5342   {
5343     Invalid~argument~in~##1:~'##2'~\msg_line_context:.
5344   }
5345   {
5346     See~the~documentation~of~##1~for~the~accepted~argument.
5347   }

```

(End of definition for spAcirc-invalid-arg and spPcirc-invalid-arg.)

11.47.3 Definition of keys for \spAcirc and \spPcirc

read-cmd
prefix
print-sym
read-sty
concept
sub-index
read-sub
read-arg
silent-cmd
unknown

We define the shared (*keys*) read-cmd, prefix, print-sym, read-sty, concept, sub-index, read-sub, read-arg and silent-cmd.

```

5348 \keys_define:nn { spintent / sp #1 }
5349 {
5350   read-cmd .choices:nn = { school , mathcat }
5351   {
5352     \int_case:nn { \l_keys_choice_int }
5353     {
5354       {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
5355       {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
5356     }
5357   },
5358   read-cmd .initial:n = school,
5359   read-cmd .value_required:n = true,
5360   read-cmd / unknown .code:n =
5361     \msg_error:nneee { spintent } { unknown-choice }
5362     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
5363   prefix .code:n =
5364     { \__spintent_sanitizet_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
5365   prefix .initial:n = ,
5366   prefix .value_required:n = true,
5367   print-sym .choices:nn = { usual , odot }
5368   {
5369     \int_case:nn { \l_keys_choice_int }
5370     {
5371       {1} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { usual } }
5372       {2} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { odot } }
5373     }
5374   },
5375   print-sym .initial:n = odot,
5376   print-sym .value_required:n = true,
5377   print-sym / unknown .code:n =
5378     \msg_error:nneee { spintent } { unknown-choice }
5379     { print-sym } { usual,~odot } { \exp_not:n {##1} },
5380   read-sty .choices:nn = { circun , circle }
5381   {
5382     \int_case:nn { \l_keys_choice_int }
5383     {
5384       {1} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { circun } }
5385       {2} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { circle } }
5386     }
5387   },
5388   read-sty .initial:n = circun,
5389   read-sty .value_required:n = true,
5390   concept .bool_set:c = { l__spintent_sp #1 _concept_bool },
5391   concept .initial:n = true,
5392   concept .value_required:n = true,
5393   sub-index .code:n =
5394     { \str_set:cn { l__spintent_sp #1 _sub_index_str } {##1} },
5395   sub-index .initial:n = ,
5396   sub-index .value_required:n = true,
5397   read-sub .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
5398   read-sub .initial:n = false,
5399   read-sub .value_required:n = true,

```

```

5400     read-arg          .code:n =
5401     { \__spintent_sanitiz_affix:cn { \__spintent_sp #1 _read_arg_tl } {##1} },
5402     read-arg          .initial:n = ,
5403     read-arg          .value_required:n = true,
5404     silent-cmd        .bool_set:c = { \__spintent_sp #1 _silent_cmd_bool },
5405     silent-cmd        .initial:n = false,
5406     silent-cmd        .value_required:n = true,
5407     unknown           .code:n =
5408     {
5409         \str_set:Nn \__spintent_command_name_str { sp #1 }
5410         \__spintent_unknown_keys_cmd:n {##1}
5411     },
5412 }
5413 }

```

(End of definition for `read-cmd` and others.)

`\__spintent_spcircfig_short_bool` `\__spintent_spcircfig_op_tl` `\__spintent_spcircfig_intent_tl` `\__spintent_spcircfig_intent_arg_tl` `\__spintent_spcircfig_short_bool` records whether the argument resolved as the short number/letter form (via `\__spintent_luafun_geo_afig_parse_and_set:n/\__spintent_luafun_geo_pfig_parse_and_set:n`) the same entry point `\spAfig/\spPfig` use) rather than a real circle. `\__spintent_spcircfig_op_tl` holds *área* or *perímetro*. `\__spintent_spcircfig_intent_tl` holds the full spoken phrase: `\__spintent_spcircfig_intent_arg_tl` (via `\__spintent_spcircfig_del_word:n`: the word, contracted with `del-/de-la-`, or `read-arg`), an optional `sub-index` suffix, and, when a center is given, `-con-centro-en` — the center itself is never concatenated into it: it is read separately, by reference, through `\__spintent_mathmlarg_point:nnn`, so a subscripted or primed center (e.g. $O_{\{1\}}$, as in the Euler circle) is never an issue.

```

5414 \bool_new:N \__spintent_spcircfig_short_bool
5415 \tl_new:N \__spintent_spcircfig_op_tl
5416 \tl_new:N \__spintent_spcircfig_del_word_tl
5417 \tl_new:N \__spintent_spcircfig_intent_tl
5418 \tl_new:N \__spintent_spcircfig_intent_arg_tl

```

(End of definition for `\__spintent_spcircfig_short_bool` and others.)

`\__spintent_spcircfig_sym:n` Typesets `\bigodot` or the plain letter C, according to `print-sym`. Receives the short command name in ‘`#1`’.

```

5419 \cs_new_protected:Nn \__spintent_spcircfig_sym:n
5420 {
5421     \str_if_eq:vnTF { \__spintent_sp #1 _print_sym_str } { odot }
5422     { \bigodot }
5423     { C }
5424 }

```

(End of definition for `\__spintent_spcircfig_sym:n`.)

`\__spintent_spcircfig_word_and_article:n` Sets `\__spintent_spcirc_word_tl` to `read-arg` or `circunferencia/círculo` from `read-sty`, and `\__spintent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word. Receives the short command name in ‘`#1`’.

```

5425 \cs_new_protected:Nn \__spintent_spcircfig_word_and_article:n
5426 {
5427     \str_if_eq:vnTF { \__spintent_sp #1 _read_sty_str } { circun }
5428     { \str_set:Nn \__spintent_spcirc_articulo_str { la } }
5429     { \str_set:Nn \__spintent_spcirc_articulo_str { el } }
5430     \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
5431     {
5432         \str_if_eq:vnTF { \__spintent_sp #1 _read_sty_str } { circun }
5433         { \tl_set:Nn \__spintent_spcirc_word_tl { circunferencia } }
5434         { \tl_set:Nn \__spintent_spcirc_word_tl { círculo } }
5435     }
5436     {
5437         \tl_set:Ne \__spintent_spcirc_word_tl
5438         {
5439             \tl_use:c { \__spintent_sp #1 _read_arg_tl }
5440         }
5441     }
5442 }

```

(End of definition for `\__spintent_spcircfig_word_and_article:n`.)

`\__spintent_spcircfig_del_word:n` Sets `\__spintent_spcircfig_del_word_tl` to `\__spintent_spcircfig_word_and_article:n`’s word, prefixed with `del-` or `de-la-` — contracted, not `de-el-`, since a screen reader does not contract that on its own. Receives the short command name in ‘`#1`’.

```

5443 \cs_new_protected:Nn \__spintent_spcircfig_del_word:n

```

```

5444 {
5445   \__spintent_spcircfig_word_and_article:n {#1}
5446   \str_if_eq:VnTF \__spintent_spcirc_articulo_str { el }
5447   {
5448     \tl_set:Ne \__spintent_spcircfig_del_word_tl { del- \__spintent_spcirc_word_tl }
5449   }
5450   {
5451     \tl_set:Ne \__spintent_spcircfig_del_word_tl { de-la- \__spintent_spcirc_word_tl }
5452   }
5453 }

```

(End of definition for __spintent_spcircfig_del_word:n.)

__spintent_spcircfig_build_short:n

Handles the short number/letter case: builds __spintent_spcircfig_intent_arg_tl as __spintent_spcircfig_intent_arg_tl (e.g. *ber word*), wrapped in bare :prefix (no reference — there is nothing to point to), or _- when silent-cmd is true or concept is false. Receives the short command name in ‘#1’.

```

5454 \cs_new_protected:Nn \__spintent_spcircfig_build_short:n
5455 {
5456   \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
5457   {
5458     \tl_set:Nn \__spintent_spcircfig_intent_arg_tl { _- }
5459   }
5460   {
5461     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5462     {
5463       \__spintent_spcircfig_del_word:n {#1}
5464       \tl_set:Ne \__spintent_spcircfig_intent_arg_tl
5465       {
5466         \__spintent_spcircfig_op_tl - \__spintent_spcircfig_del_word_tl -
5467         \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5468         \__spintent_geo_luaset_sub_spoken_str :prefix
5469       }
5470     }
5471     {
5472       \tl_set:Nn \__spintent_spcircfig_intent_arg_tl { _- }
5473     }
5474   }
5475 }

```

(End of definition for __spintent_spcircfig_build_short:n.)

__spintent_spcircfig_finish_intent:nn

Appends an optional sub-index suffix and, when #2 is non-empty, -con-centro-en to __spintent_spcircfig_intent_arg_tl — shared by both branches of __spintent_spcircfig_build_concept:nn, after either one has set the word itself. Receives the short command name in ‘#1’ and the argument in ‘#2’.

```

5476 \cs_new_protected:Nn \__spintent_spcircfig_finish_intent:nn
5477 {
5478   \str_if_empty:cF { l__spintent_sp #1 _sub_index_str }
5479   {
5480     \tl_put_right:Ne \__spintent_spcircfig_intent_tl
5481     {
5482       -
5483       \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5484       \str_use:c { l__spintent_sp #1 _sub_index_str }
5485     }
5486   }
5487   \tl_if_empty:nF {#2}
5488   {
5489     \tl_put_right:Nn \__spintent_spcircfig_intent_tl { -con-centro-en }
5490   }
5491 }

```

(End of definition for __spintent_spcircfig_finish_intent:nn.)

__spintent_spcircfig_build_concept:nn

Builds __spintent_spcircfig_intent_arg_tl. _- when silent-cmd is true, or when false and concept is false with read-arg empty (,(\$pt) instead when #2 is non-empty — the center is still read either way). Otherwise __spintent_spcircfig_intent_tl:prefix — with (\$pt) appended when #2 is non-empty — prefixed with prefix when given, where __spintent_spcircfig_intent_tl is: with concept true, __spintent_spcircfig_op_tl- __spintent_spcircfig_del_word_tl (via __spintent_spcircfig_del_word:n: the word, contracted with del-/de-la-, or read-arg the same way); with false and read-arg given, read-arg verbatim instead, with __spintent_spcircfig_op_tl dropped entirely — área/ perímetro is itself the concept concept toggles, not the -con-centro-en that

follows, which always stays (it identifies which figure, not the concept). No automatic article either — the word (and any article it needs) is the user’s to write, same convention as `\__spintent_spcirc_process:n`. Either way, an optional `sub-index` suffix and, when `#2` is non-empty, `-con-centro-en` follow. Receives the short command name in ‘`#1`’ and the argument in ‘`#2`’.

```

5492 \cs_new_protected:Nn \__spintent_spcircfig_build_concept:nn
5493 {
5494   \bool_if:cTF { \__spintent_sp #1 _silent_cmd_bool }
5495   {
5496     \tl_if_empty:nTF {#2}
5497     { \tl_set:Nn \__spintent_spcircfig_intent_arg_tl { _- } }
5498     { \tl_set:Ne \__spintent_spcircfig_intent_arg_tl { _ ( $ pt ) } }
5499   }
5500   {
5501     \bool_if:cTF { \__spintent_sp #1 _concept_bool }
5502     {
5503       \__spintent_spcircfig_del_word:n {#1}
5504       \tl_set:Ne \__spintent_spcircfig_intent_tl
5505       {
5506         \__spintent_spcircfig_op_tl - \__spintent_spcircfig_del_word_tl
5507       }
5508       \__spintent_spcircfig_finish_intent:nn {#1} {#2}
5509       \tl_set:Ne \__spintent_spcircfig_intent_arg_tl
5510       {
5511         \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
5512         {
5513           \tl_use:c { \__spintent_sp #1 _prefix_tl } -
5514         }
5515         \__spintent_spcircfig_intent_tl :prefix
5516         \tl_if_empty:nF {#2} { ( $ pt ) }
5517       }
5518     }
5519     {
5520       \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
5521       {
5522         \tl_if_empty:nTF {#2}
5523         { \tl_set:Nn \__spintent_spcircfig_intent_arg_tl { _- } }
5524         { \tl_set:Ne \__spintent_spcircfig_intent_arg_tl { _ ( $ pt ) } }
5525       }
5526       {
5527         \tl_set:Ne \__spintent_spcircfig_intent_tl
5528         { \tl_use:c { \__spintent_sp #1 _read_arg_tl } }
5529         \__spintent_spcircfig_finish_intent:nn {#1} {#2}
5530         \tl_set:Ne \__spintent_spcircfig_intent_arg_tl
5531         {
5532           \__spintent_spcircfig_intent_tl :prefix
5533           \tl_if_empty:nF {#2} { ( $ pt ) }
5534         }
5535       }
5536     }
5537   }
5538 }

```

(End of definition for `\__spintent_spcircfig_build_concept:nn`.)

`\__spintent_spcircfig_process:nn`

Receives the short command name in ‘`#1`’ and the argument in ‘`#2`’. Sets `\__spintent_spcircfig_op_tl` to `área/perímetro`. With `#2` empty, `\__spintent_spcircfig_short_bool` is set `false` and nothing else is parsed. With a comma in `#2`, reports `sp#1-invalid-arg` immediately — `\@@_luafun_geo_circ_parse_and_set:` (shared with `\spcirc`) still accepts a second, comma-separated part as a generic `label`-type radius, matching almost any text, so it would never itself report an error for a comma-containing argument here, where a comma is never meaningful at all. Without a comma, calls `\__spintent_luafun_geo_circ_parse_and_set:n`; on error, retries through `\__spintent_luafun_geo_afig_parse_and_set:n/\__spintent_luafun_geo_pfig_parse_and_set:n` (the same entry point `\spAfig/\spPfig` use) for the short number/letter form, rejecting a table-letter match (not meaningful for a circle) back to an error. Reports `sp#1-invalid-arg` on failure either way, then delegates to `\__spintent_spcircfig_build_short:n/\__spintent_spcircfig_build_concept:nn` and `\__spintent_spcircfig_mathml:nn`.

- Unlike `\spcirc`, there is no radius/diameter here at all: a formula’s left-hand side (`A_0`) names the figure, it does not carry the values needed to compute it — those belong on the right-hand side, elsewhere in the document.

```

5539 \cs_new_protected:Nn \__spintent_spcircfig_process:nn
5540 {
5541   \str_if_eq:eeTF {#1} { Acirc }

```

```

5542 { \tl_set:Nn \__spintnt_spcircfig_op_tl { área } }
5543 { \tl_set:Nn \__spintnt_spcircfig_op_tl { perímetro } }
5544 \tl_if_empty:nTF {#2}
5545 {
5546   \bool_set_false:N \__spintnt_spcircfig_short_bool
5547 }
5548 {
5549   \bool_set_false:N \__spintnt_spcircfig_short_bool
5550   \tl_if_in:nnTF {#2} { , }
5551   {
5552     \msg_error:nnee { spintnt } { sp #1 -invalid-arg }
5553     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5554   }
5555   {
5556     \__spintnt_luafun_geo_circ_parse_and_set:n { \exp_not:n {#2} }
5557     \str_if_eq:VnT \__spintnt_geo_luaset_error_str { true }
5558     {
5559       \str_if_eq:eeTF {#1} { Acirc }
5560       { \__spintnt_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
5561       { \__spintnt_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
5562       \str_if_eq:VnF \__spintnt_geo_luaset_error_str { true }
5563       {
5564         \str_if_empty:NTF \__spintnt_geo_luaset_letra_word_str
5565         { \bool_set_true:N \__spintnt_spcircfig_short_bool }
5566         { \str_set:Nn \__spintnt_geo_luaset_error_str { true } }
5567       }
5568     }
5569     \bool_if:NF \__spintnt_spcircfig_short_bool
5570     {
5571       \str_if_eq:VnT \__spintnt_geo_luaset_error_str { true }
5572       {
5573         \msg_error:nnee { spintnt } { sp #1 -invalid-arg }
5574         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5575       }
5576     }
5577   }
5578 }
5579 \bool_if:NTF \__spintnt_spcircfig_short_bool
5580 { \__spintnt_spcircfig_build_short:n {#1} }
5581 { \__spintnt_spcircfig_build_concept:nn {#1} {#2} }
5582 \__spintnt_spcircfig_mathml:nn { #1 } { #2 }
5583 }

```

(End of definition for `\__spintnt_spcircfig_process:nn`.)

`\__spintnt_spcircfig_mathml:nn`

Renders `\__spintnt_spcircfig_intent_arg_tl`, already built by `\__spintnt_spcircfig_process:nn`. Three cases: the short case prints the symbol with the short value as its own subscript; an empty `#2` prints the symbol alone; a real center prints the symbol (plus optional `sub-index` subscript) and the center between parentheses. In every case a single native `\MathMLintent` wraps `\__spintnt_mathcal:n{⟨A|P⟩}` and the whole subscript together — splitting the symbol and the center into separate `\MathMLintent` siblings inside the same subscript was tried and reads as `<msub>`'s own native structural narration instead. The center is read through `\__spintnt_mathmlarg_point:nnn`, referenced as `$pt` (same convention as `\spcirc`). Receives the short command name in `'#1'` and the argument in `'#2'`.

```

5584 \cs_new_protected:Nn \__spintnt_spcircfig_mathml:nn
5585 {
5586   \__spintnt_invisible_sep: % (luamml issues #26)
5587   \MathMLintent { \__spintnt_spcircfig_intent_arg_tl }
5588   {
5589     \__spintnt_inner_sep:
5590     \__spintnt_luafun_mrow_intent_arg:nn
5591     { } { intent = "silent" }
5592   }
5593   \str_if_eq:eeTF {#1} { Acirc } { \__spintnt_mathcal:n { A } } { \__spintnt_mathcal:n {
5594   }
5595   \bool_if:NTF \__spintnt_spcircfig_short_bool
5596   {
5597     \c_math_subscript_token
5598     {
5599       \__spintnt_spcircfig_sym:n {#1}
5600       \c_math_subscript_token { \__spintnt_geo_luaset_print_tl }
5601     }

```

```

5602     }
5603     {
5604         \c_math_subscript_token
5605         {
5606             \MathMLint { :silent } { \__spintenc_spcircfig_sym:n {#1} }
5607             \str_if_empty:cF { \__spintenc_sp #1 _sub_index_str }
5608             {
5609                 \c_math_subscript_token
5610                 {
5611                     \MathMLint { :silent }
5612                     { \str_use:c { \__spintenc_sp #1 _sub_index_str } }
5613                 }
5614             }
5615             \tl_if_empty:nF {#2}
5616             {
5617                 \MathMLint { :silent } { \lparen }
5618                 \__spintenc_mathmlarg_point:nnV { pt } { sp #1 } \__spintenc_geo_luaseta_poin
5619                 \MathMLint { :silent } { \rparen }
5620             }
5621         }
5622     }
5623 }
5624 }

```

(End of definition for `\__spintenc_spcircfig_mathml:nn`.)

`\__spintenc_spcircfig_exec:nnn`

The function `\__spintenc_spcircfig_exec:nnn` receives the short name of the command in #1, the optional keys in #2, and the argument in #3, delegating the rendering to `\__spintenc_spcircfig_process:nn`.

```

5625 \cs_new_protected:Nn \__spintenc_spcircfig_exec:nnn
5626 {
5627     \keys_set:nn { spintenc / sp #1 } { #2 }
5628     \__spintenc_spcircfig_process:nn { #1 } { #3 }
5629 }

```

(End of definition for `\__spintenc_spcircfig_exec:nnn`.)

`\spAcirc`
`\spPcirc`

Both public commands `\spAcirc` and `\spPcirc` enforce math mode, open a local group, and delegate execution to `\__spintenc_spcircfig_exec:nnn`.

```

5630 \NewDocumentCommand \spAcirc { 0{} d() }
5631 {
5632     \mode_if_math:TF
5633     {
5634         \group_begin:
5635         \attribute \__spintenc_scope_attr_int = 1
5636         \tl_if_novalue:nTF {#2}
5637         { \__spintenc_spcircfig_exec:nnn { Acirc } { #1 } { } }
5638         { \__spintenc_spcircfig_exec:nnn { Acirc } { #1 } { #2 } }
5639         \group_end:
5640     }
5641     { \msg_error:nnn { spintenc } { need-math-mode } { \spAcirc } }
5642 }
5643 \NewDocumentCommand \spPcirc { 0{} d() }
5644 {
5645     \mode_if_math:TF
5646     {
5647         \group_begin:
5648         \attribute \__spintenc_scope_attr_int = 1
5649         \tl_if_novalue:nTF {#2}
5650         { \__spintenc_spcircfig_exec:nnn { Pcirc } { #1 } { } }
5651         { \__spintenc_spcircfig_exec:nnn { Pcirc } { #1 } { #2 } }
5652         \group_end:
5653     }
5654     { \msg_error:nnn { spintenc } { need-math-mode } { \spPcirc } }
5655 }

```

(End of definition for `\spAcirc` and `\spPcirc`. These functions are documented on page ??.)

11.48 Finish package

Finish package implementation.

```

5656 \file_input_stop:
5657 \endpackage

```

12 Índice de la Implementación

The italic numbers denote the pages where the corresponding entry is described, the numbers underlined and all others indicate the line on which they are implemented in the package code.

A	
\angle	113
autofit	<u>3041</u>
C	
cifras-sep	<u>284</u> , <u>569</u> , <u>842</u> , <u>1788</u> , <u>1859</u> , <u>2380</u>
Document class:	
l3doc	32
cmd-key-unknown	<u>203</u>
cmd-key-value-unknown	<u>203</u>
Commands provide by spintent :	
\spmoney	50
\spnum	40
\spsiglo	59, 62
\spunit	45
concept <u>1950</u> , <u>2120</u> , <u>3041</u> , <u>3640</u> , <u>3847</u> , <u>4132</u> , <u>4285</u> , <u>4469</u> , <u>4850</u> ,	
<u>5068</u> , <u>5348</u>	
currency	<u>842</u>
currency-no-subunits	<u>811</u>
D	
diameter	<u>5068</u>
dolar-math	<u>842</u>
I	
inline-numeric-denominator	<u>547</u>
invalid-angle-format	<u>2330</u>
invalid-century-format	<u>1231</u>
invalid-currency	<u>811</u>
invalid-date-format	<u>1461</u>
invalid-time-format	<u>1382</u>
J	
join-word	<u>2458</u>
K	
Keys for \spAcirc provide by spintent :	
concept	142
prefix	141, 142
print-sym	141–143
radio-sty	141
read-arg	141, 142
read-cmd	141, 142
read-sty	141, 142
read-sub	142
silent-cmd	142
sub-index	141, 142
Keys for \spAfig provide by spintent :	
prefix	124
print-sym	124, 125
read-arg	124–126
read-cmd	124
read-sub	125
Keys for \spangle provide by spintent :	
concept	112
prefix	112
print-m	112
read-cmd	112
recto	112
silent-cmd	112
Keys for \sparc provide by spintent :	
concept	118
prefix	117, 118
read-cmd	117, 118
silent-cmd	118
Keys for \spcirc provide by spintent :	
prefix	135
print-sym	135–137
radio-sty	135, 136
read-arg	136
read-cmd	135
read-sty	135, 136
read-sub	136
sub-index	135, 136
Keys for \spcoord provide by spintent :	
silent-cmd	90
Keys for \spdiam provide by spintent :	
concept	132
read-arg	132
read-cmd	132
read-sub	132
silent-cmd	132
sub-index	132
upper-case	132
Keys for \spdiv provide by spintent :	
cifras-sep	70
notation-sym	70
read-parens	70
read-period-sep	70
read-sign	70
read-sym	70
set-sym	70
wrap-parens	70
Keys for \spdsym provide by spintent :	
prefix	68
read-sym	68
set-sym	68
suffix	68
Keys for \spmangle provide by spintent :	
concept	112
prefix	112
print-m	112
read-cmd	112
recto	112
silent-cmd	112
Keys for \spmarc provide by spintent :	
concept	118
prefix	117, 118
read-cmd	117, 118
silent-cmd	118
Keys for \spmcid provide by spintent :	
concept	74
prefix	74
read-cmd	74
result	74
silent-cmd	74
upper-case	74
Keys for \spmcn provide by spintent :	
concept	74
prefix	74

read-cmd	74	radio-sty	141
result	74	read-arg	141, 142
silent-cmd	74	read-cmd	141, 142
upper-case	74	read-sty	141, 142
Keys for \spmoney provide by spintent:		read-sub	142
cifras-sep	51	silent-cmd	142
currency	51	sub-index	141, 142
dolar-math	51	Keys for \spPfig provide by spintent:	
sub-units	51	prefix	124
Keys for \spmQ provide by spintent:		print-sym	124, 125
join-word	84–86	read-arg	124–126
print-dfrac	85–88	read-cmd	124
read-parens	85, 86	read-sub	125
use-spnZ	85, 87, 88	Keys for \spPoint provide by spintent:	
wrap-parens	84–86	concept	95
Keys for \spmside provide by spintent:		prefix	94, 95
concept	108	read-cmd	94, 95
prefix	108	silent-cmd	95
print-m	108	Keys for \spPoly provide by spintent:	
print-sym	132	concept	121
read-arg	108	prefix	120, 121
read-cmd	108, 111	print-sym	121
read-sty	108	read-cmd	120, 121
silent-cmd	108, 111	silent-cmd	121
Keys for \spmsym provide by spintent:		Keys for \sprad provide by spintent:	
not-print	69	concept	132
prefix	69	read-arg	132
read-sym	69	read-cmd	132
set-sym	69	read-sub	132
suffix	69	silent-cmd	132
Keys for \spm\ provide by spintent:		sub-index	132
cifras-sep	72	upper-case	132
notation-sym	72	Keys for \sprayo provide by spintent:	
read-parens	72	concept	105
read-period-sep	72	prefix	105
read-sign	72	read-cmd	105, 106
read-sym	72	read-sty	105
set-sym	72	silent-cmd	105, 106
wrap-parens	72	Keys for \sprazon provide by spintent:	
Keys for \spnD provide by spintent:		silent-cmd	80
concept	77	use-dfrac	80
prefix	77	Keys for \sprecta provide by spintent:	
result-num	77	prefix	101
result	77	read-cmd	101
silent-cmd	77	silent-cmd	101
Keys for \spnM provide by spintent:		Keys for \spshort provide by spintent:	
concept	77	read-arg	57
prefix	77	small-caps	57
result-num	77	Keys for \spside provide by spintent:	
result	77	concept	108
silent-cmd	77	prefix	108
Keys for \spnum provide by spintent:		read-arg	108
cifras-sep	39–41	read-cmd	108, 111
notation-sym	40	read-sty	108
read-period-sep	40, 42	silent-cmd	108, 111
read-sign	40	Keys for \spsiglo provide by spintent:	
Keys for \spnZ provide by spintent:		print-era	60, 61
cifras-sep	83	Keys for \spunit provide by spintent:	
read-parens	83	cifras-sep	46
read-sign	83	notation-sym	46
wrap-parens	83	print-cdot	46
Keys for \spPcirc provide by spintent:		read-cdot	46
concept	142	read-period-sep	46
prefix	141, 142	read-slash	46
print-sym	141–143		

Keys for \spusep provide by **spintent**:

silent	65
size	65

Keys for \spvec provide by **spintent**:

autofit	97
concept	97
prefix	97
read-cmd	97
silent-cmd	97

M

mathcal-fallback	80
\MathMLintenc	116
mc-invalid-arguments	1943
md-invalid-argument	2146

Lua module provide by **spintent**:

spintenc.lua	32, 33, 35, 38-40, 45, 49-52, 56, 57, 59, 61, 62, 64, 73, 77, 81, 84, 89, 94, 131
--------------	---

N

not-print	1714
notation-sym	284, 569, 1788, 1859

P

Packages :

expl3	38, 40
lua-unicode-math	32
luamml	35, 84, 86
tagpdf	56, 61
unicode-math	32, 66
prefix	1644, 1714, 1950, 2120, 2920, 3041, 3283, 3503, 3640, 3847, 4132, 4285, 4469, 5068, 5348
print-dfrac	2458
print-era	1239
print-m	3640, 3847
print-sym	4285, 4469, 4850, 5068, 5348

R

radio-sty	5068
read-arg	1109, 3640, 4469, 4850, 5068, 5348
read-cdot	569
read-cmd	1950, 2920, 3041, 3283, 3503, 3640, 3847, 4132, 4285, 4469, 4850, 5068, 5348
read-parens	1788, 1859, 2380, 2458
read-period-sep	284, 569, 1788, 1859
read-sign	284, 1788, 1859, 2380
read-slash	569
read-sty	3503, 3640, 5068, 5348
read-sub	4469, 4850, 5068, 5348
read-sym	1644, 1714, 1788, 1859
recto	3847
result	1950, 2120
result-num	2120
right-angle-sqr-fallback	80
\rightangle	113

S

seq commands:

\seq_set_from_clist:Nn	62
set-sym	1644, 1714, 1788, 1859
\setspintenc	3, 36, 191
silent	1509
silent-cmd	1950, 2120, 2260, 2672, 2920, 3041, 3283, 3503, 3640, 3847, 4132, 4285, 4469, 4850, 5068, 5348

size	1509
small-caps	1109
\spAcirc	141, 5630
spAcirc-invalid-arg	5339
\spAfig	25, 124, 4816
spAfig-invalid-arg	4431
spAfig-invalid-points	4431
spAfig-read-arg-key	4431
spAfig-sym-cardinality-conflict	4431
spAfig-sym-letra-conflict	4431
\spang	23, 81, 2370
\spangle	21, 112, 4095
spangle-invalid-arg	3837
\sparc	23, 117, 4254
sparc-invalid-points	4122
\spcirc	135, 5315
spcirc-invalid-arg	5061
\spcoord	17, 89, 2809
spcoord-invalid-arg	2668
\spdate	11, 64, 1485
\spdiam	131, 5030
\spdiv	13, 70, 1849
spdiv-empty-argument	1808
\spdsym	12, 68, 1691
spintenc internal commands:	

__spintenc_collapse_hyphens:N	128
\l__spintenc_command_name_str	203
\l__spintenc_geo_circfig_mode_str	5338
\l__spintenc_geo_final_print_tl	4423
\l__spintenc_geo_luaaset_angulo_case_str	2641
\l__spintenc_geo_luaaset_circ_radio_raw_str	5050
\l__spintenc_geo_luaaset_circ_radio_type_str	5050
\l__spintenc_geo_luaaset_concept_str	2641
\l__spintenc_geo_luaaset_error_str	2641
\l__spintenc_geo_luaaset_intent_str	2641
\l__spintenc_geo_luaaset_letra_str	2641
\l__spintenc_geo_luaaset_letra_word_str	2641
\l__spintenc_geo_luaaset_nombre_es_num_str	2641
\l__spintenc_geo_luaaset_points_clist\l__spintenc_geo_luaaset_is_name_str	2641
\l__spintenc_geo_luaaset_poly_sym_str	2641
\l__spintenc_geo_luaaset_print_tl	2641
\l__spintenc_geo_luaaset_sub_spoken_str	2641
\l__spintenc_geo_luaaset_sub_str	2641
\l__spintenc_geo_luaaset_tipo_str	2641
\l__spintenc_geo_print_sym_str	4533
__spintenc_inner_sep:	120
__spintenc_invisible_sep:	116
\c__spintenc_invisible_sep_int	124
\c__spintenc_invisible_sep_tl	86
__spintenc_luafun_clean_split_and_set:n	76, 241
__spintenc_luafun_mrow_intent_arg:nn	116
__spintenc_luafun_set_debug_ghost:n	70
__spintenc_luafun_spang_parse:n	82
__spintenc_luafun_spdate_parse:n	64
__spintenc_luafun_spmoney_lookup_data:n	76
__spintenc_luafun_spmQ_scale_int:nn	76

<code>__spintrent_luafun_sptime_parse:n</code>	63	<code>__spintrent_sp_rad_diam_print:n</code>	4897
<code>__spintrent_luafun_spunit_lookup_alias:n</code> 76		<code>\l__spintrent_sp_rad_diam_word_tl</code>	4848
<code>\l__spintrent_luaset_arg_above_tl</code>	260	<code>\l__spintrent_spAcirc_mode_str</code>	5328
<code>\l__spintrent_luaset_arg_below_tl</code>	260	<code>\l__spintrent_spAcirc_prefix_tl</code>	5328
<code>\l__spintrent_luaset_cal_fam_ok_str</code>	67	<code>\l__spintrent_spAcirc_print_sym_str</code>	5328
<code>\l__spintrent_luaset_dec_is_all_zeros_-</code> <code>str</code>	253	<code>\l__spintrent_spAcirc_radio_sty_str</code>	5328
<code>\l__spintrent_luaset_dec_sep_tl</code>	241	<code>\l__spintrent_spAcirc_read_arg_tl</code>	5328
<code>\l__spintrent_luaset_decimal_period_str</code> 253		<code>\l__spintrent_spAcirc_read_sty_str</code>	5328
<code>\l__spintrent_luaset_denom_has_number_-</code> <code>str</code>	260	<code>\l__spintrent_spAcirc_sub_index_str</code>	5328
<code>\l__spintrent_luaset_exponent_tl</code>	253	<code>\l__spintrent_sPAfig_mode_str</code>	4424
<code>\l__spintrent_luaset_format_part_dec_str</code> 248		<code>\l__spintrent_sPAfig_prefix_tl</code>	4424
<code>\l__spintrent_luaset_format_part_int_str</code> 248		<code>\l__spintrent_sPAfig_print_sym_str</code>	4424
<code>\l__spintrent_luaset_has_exponent_str</code>	253	<code>\l__spintrent_sPAfig_read_arg_tl</code>	4424
<code>\l__spintrent_luaset_has_integer_str</code>	87, 88, 241	<code>\l__spintrent_spang_luaset_error_str</code>	2326
<code>\l__spintrent_luaset_has_natural_str</code>	241	<code>\l__spintrent_spang_luaset_grado_str</code>	2326
<code>\l__spintrent_luaset_has_units_str</code>	260	<code>\l__spintrent_spang_luaset_minuto_str</code>	2326
<code>\l__spintrent_luaset_millions_str</code>	253	<code>\l__spintrent_spang_luaset_segundo_str</code>	2326
<code>\l__spintrent_luaset_not_decimal_not_-</code> <code>period_str</code>	253	<code>__spintrent_spang_process:n</code>	2335
<code>\l__spintrent_luaset_not_decimal_period_-</code> <code>str</code>	253	<code>__spintrent_spangle_exec:nnn</code>	4090
<code>\l__spintrent_luaset_only_part_dec_str</code>	248	<code>\l__spintrent_spangle_intent_arg_tl</code>	3906
<code>\l__spintrent_luaset_only_part_int_str</code>	248	<code>\l__spintrent_spangle_intent_tl</code>	3906
<code>\l__spintrent_luaset_only_part_period_-</code> <code>str</code>	248	<code>__spintrent_spangle_mathml:n</code>	3948
<code>\l__spintrent_luaset_part_dec_tl</code>	241	<code>\l__spintrent_spangle_mode_str</code>	3832
<code>\l__spintrent_luaset_part_int_tl</code>	87, 241	<code>\l__spintrent_spangle_prefix_tl</code>	3832
<code>\l__spintrent_luaset_part_period_tl</code>	241	<code>\l__spintrent_spangle_print_m_now_bool</code>	3906
<code>\l__spintrent_luaset_sign_tl</code>	87, 241	<code>__spintrent_spangle_process:nn</code>	3913
<code>\l__spintrent_luaset_status_str</code>	260	<code>\l__spintrent_spangle_pt_idx_int</code>	3906
<code>\l__spintrent_luaset_sym_fam_ok_str</code>	67	<code>\l__spintrent_spangle_pt_seq</code>	3906
<code>__spintrent_mathcal:n</code>	94	<code>\l__spintrent_spangle_refs_tl</code>	3906
<code>__spintrent_mathmlarg_point:nnn</code>	2824	<code>__spintrent_spangle_sym:n</code>	3887
<code>\l__spintrent_mathmlarg_point_kv_tl</code>	2820	<code>__spintrent_sparc_exec:nnn</code>	4249
<code>\l__spintrent_mathmlarg_point_mode_str</code>	2820	<code>\l__spintrent_sparc_intent_arg_tl</code>	4176
<code>\l__spintrent_mathmlarg_point_ref_tl</code>	2820	<code>\l__spintrent_sparc_intent_tl</code>	4176
<code>\l__spintrent_mathmlarg_point_silent_-</code> <code>bool</code>	2820	<code>__spintrent_sparc_mathml:n</code>	4194
<code>\l__spintrent_mc_args_seq</code>	1930	<code>\l__spintrent_sparc_mode_str</code>	4117
<code>\l__spintrent_mc_luaset_all_numeric_str</code> 1930		<code>\l__spintrent_sparc_prefix_tl</code>	4117
<code>\l__spintrent_mc_luaset_types_str</code>	1930	<code>__spintrent_sparc_print_arc:n</code>	4170
<code>__spintrent_mc_process_args:nn</code>	1991	<code>__spintrent_sparc_process:nn</code>	4181
<code>\l__spintrent_mc_raw_args_seq</code>	1930	<code>\l__spintrent_sparc_pt_idx_int</code>	4176
<code>\l__spintrent_mc_types_seq_{uuu}\l__spintrent_-</code> <code>spmcd_print_result_bool</code>	1930	<code>\l__spintrent_sparc_pt_seq</code>	4176
<code>\l__spintrent_md_out_seq</code>	2112	<code>\l__spintrent_sparc_refs_tl</code>	4176
<code>__spintrent_print_spunit_hierarchy:</code>	670	<code>__spintrent_sparc_wide_arc:n</code>	4166
<code>__spintrent_right_angle_sqr:</code>	107	<code>\l__spintrent_spcirc_articulo_str</code>	5052
<code>__spintrent_sanitize_affix:Nn</code>	136	<code>__spintrent_spcirc_build_concept:</code>	5196
<code>\l__spintrent_saved_luamml_flag_int</code>	86, 87, 2439	<code>\l__spintrent_spcirc_connector_arg_tl</code>	5144
<code>\l__spintrent_scope_attr_int</code>	73	<code>\l__spintrent_spcirc_connector_tl</code>	5144
<code>\l__spintrent_scope_luaset_attr_str</code>	73	<code>__spintrent_spcirc_exec:nn</code>	5310
<code>\l__spintrent_setspintrent_input_arg_cmd_-</code> <code>tl</code>	154	<code>\l__spintrent_spcirc_has_arg_bool</code>	5144
<code>__spintrent_setspintrent_parse_args:nn</code>	174	<code>\l__spintrent_spcirc_intent_arg_tl</code>	5144
<code>__spintrent_setspintrent_set_keys:nn</code>	163	<code>\l__spintrent_spcirc_intent_tl</code>	5144
<code>\l__spintrent_smparc_prefix_tl</code>	4117	<code>__spintrent_spcirc_mathml:</code>	5265
<code>__spintrent_sp_rad_diam_exec:nn</code>	5025	<code>\l__spintrent_spcirc_mode_str</code>	5052
<code>\l__spintrent_sp_rad_diam_letra_tl</code>	4848	<code>\l__spintrent_spcirc_prefix_tl</code>	5052
		<code>\l__spintrent_spcirc_print_sym_str</code>	5052
		<code>__spintrent_spcirc_process:n</code>	5249
		<code>\l__spintrent_spcirc_radio_arg_tl</code>	5144
		<code>__spintrent_spcirc_radio_print:</code>	5181
		<code>\l__spintrent_spcirc_radio_sty_str</code>	5052
		<code>\l__spintrent_spcirc_read_sty_str</code>	5052
		<code>\l__spintrent_spcirc_sub_index_str</code>	5052
		<code>__spintrent_spcirc_sym:</code>	5150

__spintent_spcirc_word_and_article: ..	5160	__spintent_spfig_op_tl	4533
__spintent_spcirc_word_tl	5060	__spintent_spfig_points_loop:n	4759
__spintent_spcircfig_build_concept:nn	5492	__spintent_spfig_process:nn	4542
__spintent_spcircfig_build_short:n ...	5454	__spintent_spfig_pt_idx_int	4533
__spintent_spcircfig_del_word:n	5443	__spintent_spfig_pt_seq	4533
__spintent_spcircfig_exec:nnn	5625	__spintent_spfig_refs_tl	4533
__spintent_spcircfig_finish_intent:nn	5476	__spintent_spfig_sym:	4525
__spintent_spcircfig_intent_arg_tl ..	5414	__spintent_spfig_symbol_tl	4533
__spintent_spcircfig_intent_tl	5414	__spintent_spmangle_mode_str	3832
__spintent_spcircfig_mathml:nn	5584	__spintent_spmangle_prefix_tl	3832
__spintent_spcircfig_op_tl	5414	__spintent_spmarc_mode_str	4117
__spintent_spcircfig_process:nn	5539	__spintent_spmc_process:nnnn	2026
__spintent_spcircfig_short_bool	5414	__spintent_spmcd_prefix_tl	1930
__spintent_spcircfig_sym:n	5419	__spintent_spmcd_print_upper_bool ...	1930
__spintent_spcircfig_word_and- article:n	5425	__spintent_spmcd_read_terse_bool ...	1930
__spintent_spcord_exec:nn	2791	__spintent_spmcd_suffix_tl	1930
__spintent_spcord_intent_infix:	2762	__spintent_spmcm_prefix_tl	1930
__spintent_spcord_intent_str	2664	__spintent_spmcm_print_result_bool ..	1930
__spintent_spcord_luaset_error_str .	2655	__spintent_spmcm_print_upper_bool ...	1930
__spintent_spcord_luaset_sep_str ...	2655	__spintent_spmcm_read_terse_bool ...	1930
__spintent_spcord_luaset_x_has_frac- str	2655	__spintent_spmcm_spmcd_luaset_error- str	1930
__spintent_spcord_luaset_x_sign_str	2655	__spintent_spmcm_suffix_tl	1930
__spintent_spcord_luaset_x_type_str	2655	__spintent_spmoney_build_tree:	1021
__spintent_spcord_luaset_x_value- str	2655	__spintent_spmoney_check_sub_units: ..	974
__spintent_spcord_luaset_y_has_frac- str	2655	__spintent_spmoney_extracted_curr_str	803
__spintent_spcord_luaset_y_sign_str	2655	__spintent_spmoney_extracted_num_tl .	803
__spintent_spcord_luaset_y_type_str	2655	__spintent_spmoney_intent_int:	939
__spintent_spcord_luaset_y_value- str	2655	__spintent_spmoney_intent_money_str .	803
__spintent_spcord_mathml:	2774	__spintent_spmoney_intent_sign_str ..	803
__spintent_spcord_print:n	2739	__spintent_spmoney_intent_sub_unit- str	803
__spintent_spcord_raw_arg_tl	2664	__spintent_spmoney_luaset_conde_str .	793
__spintent_spcord_set_wrap_parens- size:	2683	__spintent_spmoney_luaset_currency_arg- str	793
__spintent_spcord_value:n	2730	__spintent_spmoney_luaset_plur_str ..	793
__spintent_spcord_wrap_parens_l_tl .	2664	__spintent_spmoney_luaset_position- str	793
__spintent_spcord_wrap_parens_r_tl .	2664	__spintent_spmoney_luaset_print_iso- str	793
__spintent_spdate_luaset_error_str ..	1459	__spintent_spmoney_luaset_sing_str ..	793
__spintent_spdate_luaset_output_str .	1459	__spintent_spmoney_luaset_status_str	793
__spintent_spdate_process:n	65, 1470	__spintent_spmoney_luaset_subplur_str	793
__spintent_spdiam_mode_str	4842	__spintent_spmoney_luaset_subsing_str	793
__spintent_spdiam_sub_index_str	4842	__spintent_spmoney_luaset_symbol_str	793
__spintent_spdiam_upper_case_bool ...	4842	__spintent_spmoney_normalize_key:n ...	828
__spintent_spdiv_process:nnn	1834	__spintent_spmoney_parse_and_route:n .	1062
__spintent_spdiv_render_op:nn	1813	__spintent_spmoney_print_dec_sub- unit:	1021
__spintent_spdiv_render_parens:nn ...	1822	__spintent_spmoney_print_int:	910
__spintent_spdsym_prefix_tl	1640	__spintent_spmoney_print_sep_con: ...	1021
__spintent_spdsym_read_sym_tl	1640	__spintent_spmoney_print_symbol:	876
__spintent_spdsym_suffix_tl	1640	__spintent_spmoney_render_layout: ...	889
__spintent_spdsym_symbol_tl	1640	__spintent_spmoney_render_number_node:	901
__spintent_spfig_build_concept- vertices:n	4689	__spintent_spmoney_render_only_integer- node:	901
__spintent_spfig_build_refs:	4676	__spintent_spmoney_res_main_voice_str	803
__spintent_spfig_build_simple:n	4566	__spintent_spmoney_set_and_print_sub- units:	974
__spintent_spfig_exec:nnn	4811	__spintent_spmoney_set_currency:n ...	828
__spintent_spfig_intent_arg_tl	4533	__spintent_spmoney_sub_units_bool ...	803
__spintent_spfig_intent_tl	4533	__spintent_spmoney_subnum_int	803
__spintent_spfig_mathml:n	4771		

<code>__spintent_spmoney_validate_money: ...</code>	860	<code>__spintent_spnun_render_dec: ...</code>	337
<code>__spintent_spmQ_exec:nnnn ...</code>	2619	<code>__spintent_spnun_render_decimal_sep: .</code>	391
<code>\l__spintent_spmQ_join_word_str ...</code>	86, 2439	<code>__spintent_spnun_render_int: ...</code>	337
<code>__spintent_spmQ_print:nnn ...</code>	2611	<code>__spintent_spnun_render_only_period: .</code>	354
<code>\l__spintent_spmQ_print_dfrac_bool ...</code>	87	<code>__spintent_spnun_render_part:n ...</code>	337
<code>__spintent_spmQ_print_scale_int: ...</code>	2557	<code>__spintent_spnun_render_period_sep: ..</code>	354
<code>__spintent_spmQ_print_scale_int:n ...</code>	2566	<code>__spintent_spnz_process:nn ...</code>	2413
<code>__spintent_spmQ_print_spnZ_dfrac:nn ..</code>	2594	<code>__spintent_spnz_render_parens: ...</code>	2401
<code>__spintent_spmQ_set_join_word: ...</code>	2548	<code>\l__spintent_spPcirc_mode_str ...</code>	5328
<code>__spintent_spmQ_set_wrap_parens_size: ..</code>	2493	<code>\l__spintent_spPcirc_prefix_tl ...</code>	5328
<code>__spintent_spmQ_start_wrap_parens: ...</code>	2528	<code>\l__spintent_spPcirc_print_sym_str ...</code>	5328
<code>__spintent_spmQ_stop_wrap_parens: ...</code>	2528	<code>\l__spintent_spPcirc_radio_sty_str ...</code>	5328
<code>\l__spintent_spmQ_use_spnZ_bool ...</code>	87	<code>\l__spintent_spPcirc_read_arg_tl ...</code>	5328
<code>\l__spintent_spm_side_mode_str ...</code>	3623	<code>\l__spintent_spPcirc_read_sty_str ...</code>	5328
<code>\l__spintent_spm_side_prefix_tl ...</code>	3623	<code>\l__spintent_spPcirc_sub_index_str ...</code>	5328
<code>\l__spintent_spm_side_read_arg_tl ...</code>	3623	<code>\l__spintent_spPfig_mode_str ...</code>	4424
<code>\l__spintent_spm_side_read_sty_str ...</code>	3623	<code>\l__spintent_spPfig_prefix_tl ...</code>	4424
<code>\l__spintent_spm_sym_prefix_tl ...</code>	1710	<code>\l__spintent_spPfig_print_sym_str ...</code>	4424
<code>\l__spintent_spm_sym_read_sym_tl ...</code>	1710	<code>\l__spintent_spPfig_read_arg_tl ...</code>	4424
<code>\l__spintent_spm_sym_suffix_tl ...</code>	1710	<code>__spintent_spPoint_exec:nnn ...</code>	2995
<code>\l__spintent_spm_sym_symbol_tl ...</code>	1710	<code>\l__spintent_spPoint_intent_arg_tl ...</code>	2909
<code>__spintent_spmul_process:nnn ...</code>	1905	<code>__spintent_spPoint_mathml:n ...</code>	2986
<code>__spintent_spmul_render_op:nn ...</code>	1884	<code>\l__spintent_spPoint_mode_str ...</code>	2909
<code>__spintent_spmul_render_parens:nn ...</code>	1893	<code>\l__spintent_spPoint_prefix_tl ...</code>	2909
<code>\l__spintent_spnD_luaset_is_truncated_-</code>		<code>__spintent_spPoint_print:n ...</code>	2951
<code>str ...</code>	2112	<code>__spintent_spPoly_build_concept: ...</code>	4344
<code>\l__spintent_spnD_luaset_result_tl ...</code>	2112	<code>__spintent_spPoly_build_refs: ...</code>	4331
<code>\l__spintent_spnD_prefix_tl ...</code>	2112	<code>__spintent_spPoly_exec:nn ...</code>	4407
<code>\l__spintent_spnD_print_result_num_tl .</code>	2112	<code>\l__spintent_spPoly_intent_arg_tl ...</code>	4319
<code>\l__spintent_spnM_luaset_is_truncated_-</code>		<code>__spintent_spPoly_mathml: ...</code>	4387
<code>str ...</code>	2112	<code>\l__spintent_spPoly_mode_str ...</code>	4276
<code>\l__spintent_spnM_luaset_result_tl ...</code>	2112	<code>__spintent_spPoly_points_loop: ...</code>	4365
<code>\l__spintent_spnM_prefix_tl ...</code>	2112	<code>\l__spintent_spPoly_prefix_tl ...</code>	4276
<code>\l__spintent_spnM_print_result_num_tl .</code>	2112	<code>__spintent_spPoly_process:n ...</code>	4377
<code>__spintent_spnM_spnD_exec:nnnn ...</code>	2192	<code>\l__spintent_spPoly_pt_idx_int ...</code>	4319
<code>__spintent_spnM_spnD_intent:nn ...</code>	2156	<code>\l__spintent_spPoly_pt_seq ...</code>	4319
<code>__spintent_spnM_spnD_process:nnnn ...</code>	2151	<code>\l__spintent_spPoly_refs_tl ...</code>	4319
<code>__spintent_spnM_spnD_result:n ...</code>	2173	<code>__spintent_spPoly_sym: ...</code>	4323
<code>__spintent_spnM_spnD_silent:nn ...</code>	2165	<code>\l__spintent_sprad_mode_str ...</code>	4842
<code>\l__spintent_spnmd_luaset_is_number_-</code>		<code>\l__spintent_sprad_sub_index_str ...</code>	4842
<code>str ...</code>	2112	<code>\l__spintent_sprad_upper_case_bool ...</code>	4842
<code>\l__spintent_spnQ_intent_read_sign_str</code>	2439	<code>__spintent_sprayo_exec:nn ...</code>	3607
<code>\l__spintent_spnQ_read_parens_bool ...</code>	86	<code>\l__spintent_sprayo_intent_arg_tl ...</code>	3547
<code>\l__spintent_spnQ_wrap_parens_l_tl ..</code>	85, 86, 2439	<code>__spintent_sprayo_mathml: ...</code>	3560
<code>\l__spintent_spnQ_wrap_parens_r_tl ..</code>	85, 86, 2439	<code>\l__spintent_sprayo_mode_str ...</code>	3493
<code>\l__spintent_spnun__notation_sym_tl ...</code>	40	<code>\l__spintent_sprayo_prefix_tl ...</code>	3493
<code>\l__spintent_spnun_intent_read_dec_sep_-</code>		<code>__spintent_sprayo_print:n ...</code>	3551
<code>str ...</code>	40, 42, 279	<code>\l__spintent_sprayo_pt_idx_int ...</code>	3547
<code>\l__spintent_spnun_intent_read_sign_str</code>	40, 279	<code>\l__spintent_sprayo_pt_seq ...</code>	3547
<code>\l__spintent_spnun_notation_sym_tl ...</code>	279	<code>\l__spintent_sprayo_read_sty_str ...</code>	3493
<code>__spintent_spnun_print_integer: ...</code>	446	<code>\l__spintent_sprayo_refs_tl ...</code>	3547
<code>__spintent_spnun_print_num_start: ...</code>	446	<code>__spintent_sprazon_exec:nnn ...</code>	2274
<code>__spintent_spnun_print_number_core: ..</code>	446	<code>__spintent_spread_exec:nn ...</code>	1621
<code>__spintent_spnun_print_part_dec: ...</code>	425	<code>\l__spintent_spread_intent_tl ...</code>	1620
<code>__spintent_spnun_print_part_period: ..</code>	425	<code>__spintent_sprecta_exec:nn ...</code>	3477
<code>__spintent_spnun_print_period_bar: ...</code>	354	<code>\l__spintent_sprecta_intent_arg_tl ...</code>	3269
<code>\l__spintent_spnun_read_period_sep_str .</code>	40, 279	<code>\l__spintent_sprecta_kv_tl ...</code>	3269
<code>\l__spintent_spnun_read_terse_bool .</code>	40, 279	<code>__spintent_sprecta_mathml:n ...</code>	3323
		<code>\l__spintent_sprecta_mode_str ...</code>	3269
		<code>\l__spintent_sprecta_prefix_tl ...</code>	3269
		<code>__spintent_sprecta_print:n ...</code>	3314

<code>\l__spintent_sprecta_pt_idx_int</code>	3269	<code>\l__spintent_spunit_luaset_is-</code>	
<code>\l__spintent_sprecta_pt_seq</code>	3269	<code>sexagesimal_str</code>	533
<code>\l__spintent_sprecta_refs_tl</code>	3269	<code>\l__spintent_spunit_luaset_lookup_status-</code>	
<code>__spintent_spshort_execute_layout:</code> . . .	1170	<code>str</code>	533
<code>\l__spintent_spshort_luaset_base_str</code> . .	1087	<code>\l__spintent_spunit_luaset_read_str</code> . .	533
<code>\l__spintent_spshort_luaset_expanded-</code>		<code>\l__spintent_spunit_luaset_status_str</code> .	533
<code>str</code>	1087	<code>\l__spintent_spunit_mult_seq</code>	540
<code>\l__spintent_spshort_luaset_layout_str</code>	1087	<code>__spintent_spunit_process_base_exp:n</code> .	611
<code>\l__spintent_spshort_luaset_output_str</code>	1087	<code>__spintent_spunit_process_mult:n</code> . . .	663
<code>\l__spintent_spshort_luaset_status_str</code>	1087	<code>\l__spintent_spunit_read_cdot_bool</code> . .	540
<code>\l__spintent_spshort_luaset_suffix_str</code>	1087	<code>\l__spintent_spunit_read_slash_str</code> . .	540
<code>__spintent_spshort_print:nn</code>	1123	<code>__spintent_spunit_safe_exec:n</code>	682
<code>__spintent_spshort_process_and-</code>		<code>\l__spintent_spunit_unit_name_tl</code>	540
<code>render:n</code>	1135	<code>\l__spintent_spusep_arg_tl</code>	65, 66, 1506
<code>__spintent_spshort_render_bypass:n</code> . .	1146	<code>__spintent_spusep_convert:n</code>	1568
<code>__spintent_spshort_render_error:n</code> . .	1146	<code>\l__spintent_spusep_left_tl</code>	65, 66, 1506
<code>__spintent_spshort_render_fallback:</code> . .	1146	<code>\l__spintent_spusep_right_tl</code>	65, 66, 1506
<code>__spintent_spshort_render_success:</code> . .	1146	<code>__spintent_spusep_set_parens:nn</code>	1557
<code>__spintent_spshort_tag_span:nn</code>	1096	<code>\l__spintent_spusep_silent_bool</code>	66
<code>__spintent_spside_exec:nnn</code>	3805	<code>__spintent_spvec_exec:nnn</code>	3233
<code>\l__spintent_spside_intent_arg_tl</code>	3697	<code>\l__spintent_spvec_intent_arg_tl</code>	3030
<code>\l__spintent_spside_intent_tl</code>	3697	<code>\l__spintent_spvec_kv_tl</code>	3030
<code>__spintent_spside_mathml:n</code>	3730	<code>__spintent_spvec_mathml:n</code>	3092
<code>\l__spintent_spside_mode_str</code>	3623	<code>\l__spintent_spvec_mode_str</code>	3030
<code>__spintent_spside_points_loop:n</code>	3793	<code>\l__spintent_spvec_prefix_tl</code>	3030
<code>\l__spintent_spside_prefix_tl</code>	3623	<code>__spintent_spvec_print:n</code>	3075
<code>__spintent_spside_process:nn</code>	3702	<code>\l__spintent_spvec_pt_idx_int</code>	3030
<code>\l__spintent_spside_pt_idx_int</code>	3697	<code>\l__spintent_spvec_pt_seq</code>	3030
<code>\l__spintent_spside_pt_seq</code>	3697	<code>\l__spintent_spvec_refs_tl</code>	3030
<code>\l__spintent_spside_read_arg_tl</code>	3623	<code>__spintent_unknown_keys:n</code>	230
<code>\l__spintent_spside_read_sty_str</code>	3623	<code>__spintent_unknown_keys:nn</code>	230
<code>\l__spintent_spside_refs_tl</code>	3697	<code>spintent.lua</code>	69
<code>\l__spintent_spsiglo_luaset_arabic_str</code>	1227	<code>\spmangle</code>	22, 112, 4095
<code>\l__spintent_spsiglo_luaset_error_str</code> .	1227	<code>spmangle-invalid-arg</code>	3837
<code>\l__spintent_spsiglo_luaset_roman_min-</code>		<code>\spmarc</code>	23, 117, 4254
<code>str</code>	1227	<code>spmarc-invalid-points</code>	4122
<code>\l__spintent_spsiglo_print_era_str</code> . .	1227	<code>\spmcd</code>	15, 73, 2092
<code>__spintent_spsiglo_print_math:n</code>	1311	<code>\spmcm</code>	15, 73, 2092
<code>__spintent_spsiglo_print_text:n</code>	1311	<code>\spmoney</code>	6, 50, 1076
<code>__spintent_spsiglo_render_simple:nn</code> . .	1262	<code>\spmQ</code>	16, 84, 2631
<code>__spintent_spsiglo_render_with-</code>		<code>spmQ-not-integer</code>	2444
<code>era:nnn</code>	1262	<code>spmQ-zero-denominator</code>	2444
<code>__spintent_spsiglo_tag_span:nn</code>	1298	<code>\spmside</code>	21, 108, 3810
<code>__spintent_sptime_build_seconds:nn</code> . .	1391	<code>spmside</code>	5068
<code>__spintent_sptime_build_tree:</code>	64, 1409	<code>spmside-invalid-points</code>	3630
<code>\l__spintent_sptime_luaset_base_str</code> . .	1377	<code>\spmsym</code>	12, 69, 1763
<code>\l__spintent_sptime_luaset_error_str</code> . .	1377	<code>\spmul</code>	13, 72, 1920
<code>\l__spintent_sptime_luaset_has_seconds-</code>		<code>spmul-empty-argument</code>	1879
<code>str</code>	1377	<code>\spnD</code>	14, 77, 2240
<code>\l__spintent_sptime_luaset_is_fraction-</code>		<code>\spnewunit</code>	6, 49, 761
<code>str</code>	1377	<code>spnewunit-duplicate</code>	740
<code>\l__spintent_sptime_luaset_seconds_str</code>	1377	<code>\spnM</code>	14, 77, 2240
<code>\l__spintent_spunit_exp_seq</code>	540	<code>\spnum</code>	3, 40, 41, 515
<code>\l__spintent_spunit_exponent_tl</code>	540	<code>spnum-has-units</code>	501
<code>__spintent_spunit_format_canonical:</code> . .	603	<code>spnum-no-digits</code>	501
<code>\l__spintent_spunit_is_mult_bool</code>	540	<code>\spnZ</code>	16, 82, 2429
<code>\l__spintent_spunit_luaset_canonical-</code>		<code>spnz-not-integer</code>	2396
<code>str</code>	533	<code>\spPcirc</code>	141, 5630
<code>\l__spintent_spunit_luaset_compact_exp-</code>		<code>spPcirc-invalid-arg</code>	5339
<code>str</code>	533	<code>\spPfig</code>	26, 124, 4816
<code>\l__spintent_spunit_luaset_is_compact-</code>		<code>spPfig-invalid-arg</code>	4431
<code>str</code>	533		

spPfig-invalid-points [4431](#)
spPfig-read-arg-key [4431](#)
spPfig-sym-cardinality-conflict [4431](#)
spPfig-sym-letra-conflict [4431](#)
\spPoint [18](#), [94](#), [3019](#)
spPoint-invalid-point [2912](#)
\spPoly [24](#), [120](#), [4412](#)
spPoly-invalid-points [4278](#)
\spProp [81](#), [2310](#)
\sprad [131](#), [5030](#)
\sprayo [19](#), [105](#), [3612](#)
sprayo-invalid-points [3496](#)
\sprazon [80](#), [2300](#)
\spread [11](#), [67](#), [1630](#)
spread-empty-args [1617](#)
\sprecta [19](#), [101](#), [3482](#)
sprecta-invalid-points [3276](#)
\spshort [7](#), [56](#), [1214](#)
\spside [20](#), [108](#), [3810](#)
spside [5068](#)
spside-invalid-points [3630](#)
\spsiglo [11](#), [59](#), [1364](#)
\sptime [11](#), [62](#), [1441](#)
\spunit [4](#), [45](#), [46](#), [48](#), [728](#)
spunit-invalid-unit [547](#)
spunit-no-units [547](#)
\spunitalias [6](#), [49](#), [775](#)

spunitalias-duplicate [740](#)
spunitalias-invalid-expr [740](#)
\spusep [12](#), [65](#), [1606](#)
spusep-empty-arg [1495](#)
spusep-invalid-arg [1495](#)
\spvec [18](#), [97](#), [3258](#)
spvec-invalid-arg [3037](#)
str commands:
 \str_if_eq:nnTF [62](#)
 \str_lowercase:n [62](#)
 \str_uppercase:n [62](#)
sub-index [4850](#), [5068](#), [5348](#)
sub-units [842](#)
suffix [1644](#), [1714](#)

U

unknown [1509](#), [1644](#), [1714](#), [1950](#), [2120](#), [2260](#), [2458](#), [2920](#), [3041](#),
 [3283](#), [3503](#), [3640](#), [3847](#), [4132](#), [4285](#), [4469](#), [4850](#), [5068](#),
 [5348](#)
unknown-abbreviation [1093](#)
unknown-choice [203](#)
unknown-command-setup [154](#)
upper-case [1950](#), [4850](#)
use-dfrac [2260](#)
use-spnZ [2458](#)

W

wrap-parens [1788](#), [1859](#), [2380](#), [2458](#)